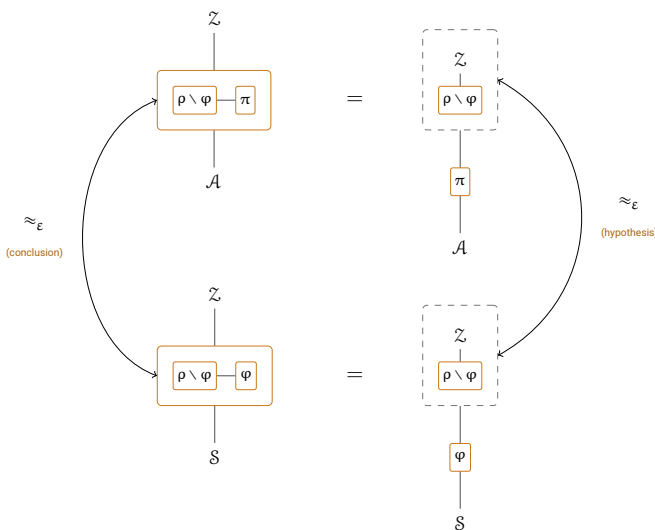


UC برای بازی بازان

UC for Gamers



Claude Opus 5 (1M context)

Pooya Farshim

هیچ کس

2 شهریور 1405

هوش مصنوعی:

پرامپت ها:

بازیبنا:

تاریخ:

ترکیب‌پذیری جهانی را، هر جا که نوشته می‌شود، به آمیزه‌ای از نوشتار و کد می‌نویسند؛ و همین است که پژوهش UC را سخت‌ابطال می‌کند و آزمودن یا وارسی‌صوری‌اش را سخت‌تر. ما، به سبکِ کدمحورِ امنیتِ ویژگی‌محور، زبانی دقیق برای برنامه‌نویسی کارکردهای UC می‌نشانیم: هویت و کنترل دسترسی داده‌اند، پوشانه‌ها آن‌ها را اعمال می‌کنند، فساد دفتری است که هر واسط می‌خواندش، و یک اجرا هندلری است که تنها یک ژتون را دست‌به‌دست می‌کند. آن‌گاه تقلید را روی رده‌های صریح ماشین و با کران‌های عینی تعریف می‌کنیم – بی‌پارامترِ امنیت، بی‌مجان‌بی – و جعبه‌ابزارِ ترکیب از پی آن می‌آید: جانشانی، ترکیبِ موازی، و قضیهٔ تمامیت برای حریفِ بدل – هر سه نتیجهٔ یک لمِ جذب، لمی که پروتکل یا حریف را به درون محیط می‌برد و بازگروه‌بندیِ دقیقِ یک اجرای واحد است، نه مقایسهٔ دو اجرا – و تراییبی مفت، از خودِ متر. ویژگی‌های بازی‌محور نیز، چنان که خواهیم دید، خودشان کارکردند و در امتدادِ تقلید منتقل می‌شوند. یک کارکردِ امضا، یک ساعتِ جهانی و یک شبکهٔ با تأخیرِ کراندار سبک را در کار نشان می‌دهند. امیدمان این است که این کار به به‌کارگیریِ روش‌های صوری در UC کمک کند و سرانجام به اثباتی صوری از UC برسد.

فهرست مطالب

3	مقدمه
6	1 کارکردها و سیستم‌ها
13	2 نگهبان‌ها، خموش‌کننده‌ها و واسطه‌گرها
16	3 مدل اجرا
16	3.1 فساد
18	3.2 نشت
19	3.3 محیط
20	3.4 حریف
21	3.5 واسطه کامل
23	3.6 اجراها
25	4 تقلید و ترکیب UC
25	4.1 جانشانی زیرسیستم
27	4.2 تقلید و جذب
45	4.3 ترکیب موازی
51	4.4 تمامیت حریف بدل
58	4.5 امنیت عینی
61	5 ویژگی‌ها
76	6 مسدودگرها
82	7 جابه‌جایی و تغییرنام
103	8 هزینه‌های عینی
109	9 فراخوان‌های پاسخگو
112	10 زبان هسته
116	11 همسان تا خرابی
123	12 پاک‌سازی
125	13 تصادف
125	13.1 کارکرد
126	13.2 ویژگی‌ها
129	14 ذخیره‌سازی
129	14.1 کارکرد
130	14.2 ویژگی‌ها

134	امضاهای دیجیتال	15
134	کارکرد	15.1
136	ویژگی‌ها	15.2
141	زیرساختِ کلید عمومی	16
141	کارکرد	16.1
143	ویژگی‌ها	16.2
147	ساعتِ جهانی	17
147	کارکرد	17.1
150	ویژگی‌ها	17.2
156	Network -Delayed Δ	18
156	کارکرد	18.1
158	ویژگی‌ها	18.2
163	Channel Authenticated -Delayed Δ	19
163	کارکرد	19.1
166	ویژگی‌ها	19.2
170	تحقیقِ کانالِ اصالت‌دار	20
183	Dive Deeper A : $\mathcal{F}_{\text{Sig}}$	21
183	طرح‌های امضا و بازی‌هایشان	21.1
184	π_{Sig} protocol The	21.2
187	عکس قضیه، و آنچه ویژگی‌ها به آن نمی‌رسند	21.3
191	lacks $\mathcal{F}_{\text{Sig}}$ that π_{Sig} of property A	21.4
193	Issues Pending	
196	Bibliography	

مقدمه

ترکیب‌پذیری جهانی را، تقریباً هر جا که نوشته می‌شود، به آمیزه‌ای از نوشتار و کد می‌نویسند. واسطه‌های یک کارکرد کنند؛ اینکه چه کسی می‌تواند آن‌ها را فرا بخواند، یک جمله است. مدل فساد یک بند است؛ و اینکه واسط یک طرف فاسد در واقع چه برمی‌گرداند، به خواننده واگذار می‌شود. هر یک از این شکاف‌ها کوچک است، و داور انسانی بی‌آنکه بفهمد پُرش می‌کند. ماشین نمی‌تواند، و همین است که اثبات UC را سخت‌ابطال می‌کند و واریسیِ صوری‌اش را سخت‌تر؛ بسیاری از آنچه باید واریسی شود، هرگز نوشته نشده بود.

این مقاله آن بخش‌ها را به‌صورت کد می‌نویسد. هویت و کنترل دسترسی داده‌هایی می‌شوند که کارکرد در میدان‌های نام‌دار حمل می‌کند. پوشانه‌ها اعمال‌شان می‌کنند: نگهبان تصمیم می‌گیرد چه فراخوان‌هایی به هسته می‌رسند، خموش‌کننده تصمیم می‌گیرد هسته چه ادعایی می‌تواند بکند، و واسطه‌گر فراخوان یک طرف فاسد را به حریف می‌سپارد. فساد دفتری است که هر واسط می‌خواندش، دو بار، در نقاطی که خود کد نامشان می‌برد. یک اجرا هندلری است که تنها یک ژتون را دست‌به‌دست می‌کند. هیچ چیز اینجا مدلی تازه نیست — همان مدل کانتی [7] است، با چند قراردادش که صریح شده‌اند — اما بیان آن‌ها به کد و نه به نوشتار، همان چیزی است که می‌گذارد باقی مقاله اثبات شود و نه استدلال.

یک لم. آنگاه تقلید روی رده‌های صریح ماشین تعریف می‌شود، با کران‌های عینی: بی‌پارامتر امنیتی، بی‌مجان‌بی، کرانی که یک عدد است. آنچه از پی می‌آید جعبه‌ابزار ترکیب است، و شکل آن ادعای اصلی ساختاری این مقاله است. جانشانی، ترکیب موازی و تمامیت حریف بدل، هر سه، نتیجه یک لم جذابند، لمی که پروتکل یا حریف را به درون محیط می‌برد. جذب مقایسه دو اجرا نیست، بازگروه‌بندی دقیق یک اجراست: همان اجرا، با کمانک‌گذاری دیگر، پس مزیت دو سو یک عدد است و نه دو عدد با کرانی مشترک. تریایی استثناست و از گونه دیگری است، چون بر نامساوی مثلثی روی متر تکیه دارد و نه بر هیچ بازگروه‌بندی؛ به همین سبب همراه متر بیان می‌شود، پیش از ماشین‌آلاتی که آن سه دیگر لازم دارند.

بازی‌ها کارکردند. یک ویژگی به سبک بازی‌محور بلر و راگوی [5] — جعل‌ناپذیری، زنده‌بودن، اصالت — چنان که پیدا می‌شود، خودش هم به‌صورت کارکرد بیان‌شدنی است، با مبارزطلب بازی به‌عنوان ماشینی معمولی و حکمش به‌عنوان واسطی معمولی.

ویژگی‌هایی که این‌گونه نوشته شوند در امتداد تقلید منتقل می‌شوند: اگر پروتکلی شیئی ایده‌آل را تقلید کند، ویژگی‌های آن شیء به آن فرود می‌آیند، زیر فرض‌هایی که مقاله بیان‌شان می‌کند و در هر کاربرد واری‌شان می‌کند و مفروض‌شان نمی‌گیرد.

نخست چه بخوانیم. دستگاه از آنچه شمار قضیه‌ها نشان می‌دهد بزرگ‌تر است، و همه آن باربر نیست. پنج تعریف بیشتر بار مقاله را حمل می‌کنند: یک کارکرد و پنج میدان‌ش؛ یک اجرا و اینکه اشغال یک هویت چه معنایی دارد؛ یک دسته، که چند ماشین را چون یک اشغال‌گر می‌راند و جذب از آن ساخته شده است؛ خود جذب؛ و تناظر بازگروه‌بندی که دو اجرا را ماشین‌به‌ماشین جفت می‌کند. خواننده‌ای که این پنج را داشته باشد تیرک مقاله را دارد، و نتایج ترکیب به‌ترتیب از آن‌ها می‌آیند. کارکردهای عینی – یک طرح امضا، یک ساعت جهانی و یک شبکه با تأخیر کراندار به‌پیروی کاتز، مائورر، تاکمان و زیکاس [11] و بادرچر، مائورر، چودی و زیکاس [3]، و یک کانال اصالت‌دار که روی آن‌ها محقق می‌شود – در پایان می‌آیند و هستند تا سبک را در کار نشان دهند، نه چون چیزی پیش از آن‌ها به آن‌ها نیاز داشته باشد.

این به چه کار می‌آید. پژوهشگر UC باید قراردادهای اینجا را همان قراردادهایی بیاید که خودش به کار می‌برد، اما نوشته شده. مخاطب دوم آن کسی است که این دقت افزوده برای اوست. هر کس که ابزاری روی UC می‌سازد – یک نوع‌سنج برای کد کارکرد، یک رمزگذاری در دستیار اثبات، یک داربست آزمون – مشخصه‌ای لازم دارد که پرسش‌هایی را حل کند که ارائه غیررسمی می‌تواند به خواننده خود واگذارد، و حل همان‌ها بیشتر کاری است که این مقاله می‌کند: نگهبان چه فراخوانی را می‌پذیرد، هسته خموش‌شده چه ادعایی می‌تواند بکند، دفتر فساد کی خوانده می‌شود. اینکه این اندازه برای حمل اثباتی ماشین‌واری‌شده از UC بس باشد یا نه، اینجا برقرار نشده است، و ما ادعایش نمی‌کنیم. همین است که چرا این چارچوب این‌گونه نوشته شده است.

کارکردها و سیستم‌ها

ما در چارچوب ترکیب‌پذیری جهانی کانتی [7] کار می‌کنیم، که چند قراردادش را در زیر صریح می‌کنیم.

شناسه فرایند نام یک نمونه در حال اجرای پروتکل است و سه بخش دارد:

$$\text{pid} = (F, s, i),$$

یک نام کارکرد F ، یک شناسه نشست s ، و یک شماره نمونه i . نام می‌گوید چه کدی اجرا می‌شود، نشست می‌گوید نمونه به کدام اجرای پروتکل تعلق دارد، و شماره چند نمونه از یک نام را در یک نشست از هم جدا می‌کند. برای سه بخش id.pid می‌نویسیم id.F ، id.i و id.s ، و به همین شکل $\mathcal{F}.F$ ، $\mathcal{F}.s$ ، $\mathcal{F}.i$ برای بخش‌های $\mathcal{F}.pid$ و $\mathcal{F}.pid = \text{id.pid}$ به تنهایی یعنی هر سه یکی‌اند.

نام‌ها عضو $\{0, 1\}^* \cup \{A, Z, C\}$ و شناسه‌های طرف عضو $\{0, 1\}^* \cup \{A, Z\}$ اند، که در آن A ، Z و C برای حریف، محیط و دفتر فساد رزرو شده‌اند. چون رزروند، هیچ‌یک از این سه عضو $\{0, 1\}^*$ نیست، پس میدانی که اعلام شده $\{0, 1\}^*$ را در بر دارد هیچ‌یک از آن‌ها را در بر ندارد؛ و C نام هیچ طرفی نیست، چون دفتر ماشینی است که طرف‌ها فرایش می‌خوانند، نه ماشینی که طرفی آن را براند. هر سه به صورت یک نمونه واحد در یک نشست واحد اجرا می‌شوند، که ثابتاً نشست 0 و نمونه 0 است، پس شناسه‌های فرایندشان $(A, 0, 0)$ ، $(Z, 0, 0)$ و $(C, 0, 0)$ است. ما در تمام کتاب این‌ها را به همان نام خالی A ، Z و C کوتاه می‌کنیم، چون آن دو مؤلفه دیگر خبری حمل نمی‌کنند.

آزمون‌ها تقریباً همیشه فقط روی نام‌اند. هر جا می‌نویسیم $\text{id.F} = A$ مقصودمان مؤلفه نام است، هرگز کل سه‌گانه؛ و همین برای Z ، برای C ، و برای هر کارکردی که در آزمون عضویت نامش می‌آید.

دو تا از میدان‌های زیر نام‌هایی از این دست را جمع می‌کنند، و چیزهای متفاوتی جمع می‌کنند. مجموعه طرف‌ها شناسه‌های طرف را در بر دارد، عضوهای $\{0, 1\}^* \cup \{A, Z\}$ ؛ و مجموعه فراخواننده‌ها کل شناسه‌های فرایند را، چون اینکه چه کسی می‌تواند فرا بخواند پرسشی است در باب نمونه‌ها و نه فقط در باب کد. پس مجموعه فراخواننده‌ها را با این کوتاه‌نویسی‌ها می‌نویسیم:

$$A := \{\text{pid} : \text{pid.F} = A\}, \quad Z := \{\text{pid} : \text{pid.F} = Z\},$$

$$\text{Std} := \{\text{pid} : \text{pid.F} \in \{0, 1\}^*\}$$

برای شناسه‌های فرایند حریف، محیط، و هر چیز استاندارد. هر جا نمادگذاری قدیمی‌تر " Z و A " را می‌پذیرفت، اکنون $A \cup Z$ را می‌پذیرد، که از آن تنها $(A, 0, 0)$ و $(Z, 0, 0)$ هرگز رخ می‌دهند و اجرا از هر یک یکی فراهم می‌کند؛ و هر جا " $\{0, 1\}^*$ " را می‌پذیرفت، اکنون Std را می‌پذیرد، و آنجا هر نشست و هر نمونه‌ای واقعاً رخ می‌دهد. مجموعه‌های طرف همان $[A, Z]$ خالی را نگه می‌دارند، چون آن‌ها شناسه طرف‌اند و نه شناسه فرایند، پس اعلامی که پیش‌ازاین $\mathbf{N} := \mathbf{P}$ خوانده می‌شد باید دو شقه شود.

کارکرد هم به همین شکل نام می‌گیرد، با مؤلفه نامش که به صورت زیرنویس نوشته می‌شود: $\mathcal{F}_{\text{Sig}}$ کارکردی است با $\text{Sig} = \mathcal{F}_{\text{Sig}}.F$ ، که نشست و نمونه‌اش ناگفته مانده‌اند، پس این نمادگذاری به جای شناسه فرایند (Sig, s, i) می‌ایستد با s و i نامعین. هر جا نمونه خاصی مقصود باشد آن‌ها را جداگانه می‌دهیم؛ هر جا فقط کد مهم باشد، هیچ نوشته نمی‌شوند.

با ناگفته گذاشتنشان چیز کمی از دست می‌رود، چون کد خود کارکرد به سختی هرگز آن‌ها را می‌خواند. محلی یا جهانی، هسته برای طرفی که به آن خدمت می‌دهد و حالتی که نگه می‌دارد نوشته می‌شود؛ اینکه به کدام نشست تعلق دارد، و کدام نمونه از نام خودش است، کار اجراست و نه کار او. در این مقاله s تنها در سه جا رخ می‌دهد – سطر 3 از *Guard*، پوشانه‌هایی که آن را فرا می‌خوانند، و آزمون ایستای تعریف 1.3 – و i هرگز خوانده نمی‌شود. شماره نمونه جای خود را از راهی دیگر به دست می‌آورد؛ همان است که می‌گذارد دو نمونه از یک نام در یک نشست، آن شناسه‌های فرایند متمایزی را حمل کنند که تعریف 1.2 می‌خواهد. فصل 7 این مشاهده را به یک حکم بدل می‌کند: تغییرنام نشست‌ها و نمونه‌ها یک یکرختی اجراهاست، و تقلیدی که در یک شناسه فرایند اثبات شود در هر شناسه دیگری برقرار است، زیر یک شرط بهداشتی در باب اینکه چه کسی را می‌توان از میان نشست‌ها فرا خواند، شرطی که هر کارکرد اینجا برآورده‌اش می‌کند.

تعریف 1.1 (هویت‌ها و کارکردها). یک هویت زوج (id, P) از یک شناسه فرایند و یک شناسه طرف است، و یک مجموعه هویت هر مجموعه $\mathcal{I}$ از این‌هاست – شناسه فرایند سه‌گانه است، و دیگر مجموعه‌ای از زوج‌های نام نیست. یک کارکرد $\mathcal{F}$ این پنج میدان را حمل می‌کند

$\mathcal{F}.\text{pid}$	یک شناسه فرایند
$\mathcal{F}.\mathbf{P}$	مجموعه طرف‌های خدمت‌گیرنده
$\mathcal{F}.\mathbf{N}$	شناسه‌های فرایند فراخوانده‌ای که می‌پذیرد
$\mathcal{F}.\mathbf{U}$	کارکردهایی که به کار می‌برد
$\mathcal{F}.\text{par}$	هر پارامتر دیگر
	واسطه‌هایی که وجود دارند
	چه کسی می‌تواند آن واسطه‌ها را فرا بخواند
	چه چیزی باید باشد تا او بتواند بدود
	کد جز موارد بالا چه می‌خواند

و این مجموعه هویت را اشغال می‌کند

$$\text{IDs}(\mathcal{F}) := \{(\mathcal{F}.\text{pid}, P) : P \in \mathcal{F}.\mathbf{P}\}.$$

مجموعه‌های هویت دو چیز می‌گویند: اینکه گردابه‌ای از کارکردها چه هویت‌هایی را اشغال می‌کند، و اینکه یک ماشین چه هویت‌هایی را می‌تواند ادعا کند. این پنج میدان در سطر پارامتر هر جعبه زیر تکرار می‌شوند، بی نام صاحبشان، چون سر جعبه آن را می‌دهد. تنها دو تای اول $IDs(\mathcal{F})$ را تعیین می‌کنند: شناسه فرایند بین همه هویت‌هایش مشترک است و طرف‌های خدمت‌گیرنده مؤلفه دوم را می‌دهند. میدان چهارم وابستگی را ثبت می‌کند. هر درایه $\mathcal{F}.U$ زوجی است $(\mathcal{F}', R)$: $\mathcal{F}'$ نام کارکردی است که باید حاضر باشد تا $\mathcal{F}$ بدود، و R محمولی است که می‌گوید $\mathcal{F}$ از هر چه آنجا یافت شود چه می‌خواهد. نام‌بردن از یک زیروال به‌تنهایی به‌ندرت بس است — زیروالی که به هیچ‌یک از طرف‌های فراخواننده‌اش خدمت نمی‌داد، یا هیچ‌یک از فراخواننده‌هایش را نمی‌پذیرفت، بی‌مصرف بود — پس می‌نویسیم

$$\text{serves}(\mathcal{F}, \mathcal{G}) : \iff \mathcal{G}.P \supseteq \mathcal{F}.P \wedge \mathcal{F}.pid \in \mathcal{G}.N$$

برای آن خواسته‌ای که بیش از همه پیش می‌آید: $\mathcal{G}$ به هر طرفی که $\mathcal{F}$ خدمت می‌دهد خدمت می‌دهد، و $\mathcal{F}$ را در میان فراخواننده‌هایش می‌پذیرد. بی عطف دوم، **Guard** فراخوان از $\mathcal{F}$ به $\mathcal{G}$ را رد می‌کرد.

آنچه **U** ثبت می‌کند همان است که $\mathcal{F}$ نامش را می‌برد. فراخوان‌هایی که پوشانه‌اش به‌جای او می‌گذارد — به $\mathcal{C}$ در سطرهای 2 و 6 از بخش 3.5، و به $\mathcal{A}$ از طریق **Mediate** — بین هر کارکرد استاندارد مشترکند و کنار گذاشته می‌شوند، و این هیچ هزینه‌ای ندارد: هر دو هدف به $\{0, 1\}^* \cup \{A, Z\}$ خدمت می‌دهند و **Std** را می‌پذیرند، پس $\text{serves}(\mathcal{F}, \mathcal{C})$ و $\text{serves}(\mathcal{F}, \mathcal{A})$ برای هر $\mathcal{F}$ استاندارد، هر میدانی هم که خودش داشته باشد، برقرارند و درایه‌ای برای هیچ‌یک هرگز نمی‌توانست شکست بخورد.

تعریف 1.2 (سیستم‌ها). یک کارکرد استاندارد است اگر $\mathcal{F}.F \notin \{A, Z, C\}$ و هر یک از هسته‌هایش چنان که در بخش 3.5 آمده پوشانده شده باشد — جز **Leak**، که چنان که در بخش 3.2 آمده پوشانده می‌شود، و **Initialize**، که یک رویه است و هیچ پوشانده نمی‌شود. یک سیستم (یا جهان) π مجموعه‌ای متناهی از کارکردهای استاندارد است که شناسه‌های فرایند متمایز حمل می‌کنند: اگر $\mathcal{F}, \mathcal{F}' \in \pi$ و $\mathcal{F} \neq \mathcal{F}'$ آن‌گاه $\mathcal{F}.pid \neq \mathcal{F}'.pid$. سیستم هر چه اعضایش اشغال کنند اشغال می‌کند،

$$IDs(\pi) := \bigcup_{\mathcal{F} \in \pi} IDs(\mathcal{F}) = \{(\mathcal{F}.pid, P) : \mathcal{F} \in \pi, P \in \mathcal{F}.P\}.$$

سیستم همان است که طراح پروتکل می‌نویسدش — ما پروتکل و سیستم را به‌جای هم به کار می‌بریم، و ایده‌آل و واقعی نام سبک‌های نوشتن یکی از آن‌ها (در زیر)، نه دو رده جدا. سه ماشین هیچ بخشی از آن نیستند: محیط $\mathbb{Z}$ ، حریف $\mathcal{A}$ ، و دفتر فساد $\mathcal{C}$ از بخش 3.1. هیچ‌یک استاندارد نیست، پس بنا بر تعریف 1.2 هیچ سیستمی هیچ‌یک از آن‌ها را در بر ندارد و $IDs(\mathcal{A})$ ، $IDs(\mathbb{Z})$ و $IDs(\mathcal{C})$ بیرون هر سیستمی می‌مانند؛ به‌جایش اجرای بخش 3.6 هر سه را فراهم می‌کند. برای $\mathcal{A}$ و $\mathbb{Z}$ همین است که می‌گذارد

هر یک هویت خودش را ادعا کند و در همان حال روی سیستم زیر آزمون خموش باشد. برای $\mathcal{C}$ همین است که دفتر را از عملیات مجموعه‌ای بخش 4.1 و فصل 6 برکنار نگه می‌دارد: هیچ مسدودگری برایش ساخته نمی‌شود، هیچ جانشانای جایش را نمی‌برد، و هر مقایسه روی یک و همان دفتر می‌دود. رزروکردن آن سه نام آن‌ها را از هر $\mathcal{F}.N$ که به صورت **Std** اعلام شده هم بیرون نگه می‌دارد، پس هیچ کارکردی دفتر را در میان فراخواننده‌هایش نمی‌پذیرد – و این درست است، چون دفتر تنها فراخوانده است. تمایز شناسه‌های فرایند کار واقعی می‌کند. سیستم با آنچه در هر شناسه‌ای که به کار می‌برد می‌نشانند تعیین می‌شود، پس عضویت را تنها شناسه حل می‌کند. و هر هویت آن شناسه را در مؤلفه اولش ثبت می‌کند، پس اعضای متمایز $IDs(\mathcal{F}) \cap IDs(\mathcal{F}') = \emptyset$ دارند: اجتماع بالا مجزاست، و هیچ هویتی از یک سیستم دو بار اشغال نمی‌شود.

تعریف 1.3 (سیستم‌های خوش‌ساخت). بنویسیم $\pi^+ := \pi \cup \{Z, A, \mathcal{C}\}$ برای یک سیستم به همراه آن سه ماشینی که هر اجرا فراهم می‌کند. سیستم π خوش‌ساخت است، و می‌نویسیم $Wf(\pi)$ ، اگر

$$\begin{aligned} \forall \mathcal{F} \in \pi^+ \quad \forall (\mathcal{F}', R) \in \mathcal{F}.U \quad \exists g \in \pi^+ : \\ g.F = \mathcal{F}'.F \quad \wedge \quad (g.s = \mathcal{F}.s \vee \text{global}(g)) \\ \wedge \quad R(\mathcal{F}, g). \end{aligned}$$

سه چیز در باب این. شرط‌های روی g درست همان‌هایی هستند که **Guard** هنگام گذاشتن فراخوان تحمیل می‌کند – نام پذیرفته‌شده، نشست جور، مگر آنکه فراخوانده مشترک باشد، و میدان‌های برانزده – پس $Wf(\pi)$ درست همین را می‌گوید که فراخوان‌هایی که اعضای π برای گذاشتنشان ساخته شده‌اند می‌توانند موفق شوند. جورشدن با نام است، نه با خود کارکرد، و همین است که می‌گذارد یک وابستگی از جانشانی زیرسیستم جان سالم ببرد: درایه‌ای که نام φ_q را می‌برد با هر چه در آن نام بنشیند برآورده می‌شود، ایده‌آل یا واقعی، تا وقتی باقی شرط‌ها را برآورد. و نیمه‌خواسته میدان‌های فراخواننده و فراخوانده را به هم گره می‌زند؛ Wf بی آن تقریباً هیچ نمی‌گفت. هر دو سور روی π^+ دامنه دارند، و هر بزرگ‌شدن جای خود را به‌دست می‌آورد: در سمت راست، درایه‌ای که نام یکی از آن سه ماشین فراهم‌شده را می‌برد اصلاً برآورده‌شدنی می‌شود، چون هیچ‌یک عضو هیچ سیستمی نیست؛ و در سمت چپ، درایه‌های خود آن‌ها واری می‌شود – بی‌درنگ همین‌جا، چون $\text{serves}(Z, A)$ و $\text{serves}(A, Z)$ برقرارند و $\mathcal{C}.U$ تهی است. آنچه Wf به‌تنهایی نمی‌کند، جان‌بردن از ساخت‌هایی است که در پی می‌آیند: افزودن عضو می‌تواند تعهد بیاورد، و جانشانی زیرسیستم هر خواسته را در اشغال‌گر تازه از نو می‌سجد. گزاره 6.4 (فصل 6) اولی را حل می‌کند و قضیه 4.4 دومی را.

شرط‌ها، یک‌جا. خوش‌ساختی یکی از چند شرط ایستایی است که این مقاله بر سیستم‌ها، هسته‌ها و ماشین‌ها می‌گذارد، و هر یک همان‌جا که نخست لازم می‌شود

تعریف می‌شود – که خواننده را بی هیچ راهی می‌گذارد تا ببیند کدام نتیجه کدام شرط را می‌خواند. این جدول یک بار همه را جمع می‌کند. هیچ‌یک از آن‌ها در طول اجرا خوانده نمی‌شود: Exec میدان می‌خواند، نه شرط، چنان که گزاره 7.7 به‌طور خاص برای U ثبت می‌کند.

شرط	چه چیزی را مقید می‌کند	چه کسی می‌خواندش
$wf(\pi)$ ، تعریف 1.3	یک سیستم: هر درایه U شاهدهی دارد – فراخوان‌هایی که اعضایش برای گذاشتنشان ساخته شده‌اند می‌توانند موفق شوند	قضیه 4.4، گزاره 6.4، تعریف 5.1
جانشانی خوش‌ساخت، تعریف 4.2	سیستم ورودی در برابر آنچه می‌ماند: هیچ شناسه فرایند مشترکی	گزاره 4.3، قضیه 4.4، گزاره 4.16، قضیه 4.20، قضیه 4.24، گزاره 5.10
سازگار، تعریف 6.1	دو سیستم: شناسه فرایند مشترک یکسان خدمت می‌دهد و یکسان می‌پذیرد	قضیه 6.3، ادعای دوم
نشست‌یکنوا، تعریف 7.2	یک سیستم یا محیط: اینکه چه کسی می‌تواند به عضوی برسد به نشست او وابسته نیست	لم 7.9، گزاره 7.10، نتیجه 7.12، نتیجه 7.13
تنگ در I، نکته 6.5	یک سیستم: اعضای بیرون I فراخوانده‌ها را با شناسه فرایند سنجاق می‌کنند	تعریف 5.1
هیچ فراخوان پاسخگو نمی‌گذارد، تعریف 9.2	هسته‌های یک سیستم	قضیه 4.29، لم 4.18 (نیمه) واگذارنده، گزاره 5.9، نتیجه 5.11
بی‌اعتنا به رد، تعریف 4.28	ماشینی در جایگاه حریف	همان چهار
هویت‌اتمی، تعریف 7.4	همه کد؛ قراردادی جاری و نه آزمونی به‌ازای هر سیستم	لم 7.9، گزاره 7.15
نشست خوش‌ساخت، بخش 3.2	یک تحقق	هیچ‌کس: ثبت شده، به کار نرفته

دو چیز که این جدول آشکار می‌کند. دو سطر اول یک نام دارند و دو شرط متفاوتند: wf در باب یک سیستم به‌تنهایی است، و جانشانی خوش‌ساخت در باب اینکه سیستم ورودی کجا فرود می‌آید. و wf و نشست‌یکنوایی دوگان یکدیگرند – فراخوان‌هایی که عضوی برای گذاشتنشان ساخته شده، در برابر اینکه چه کسی می‌تواند به او برسد – که وسوسه می‌کند آن‌ها را در یک شرط تا کنیم. جدا نگه داشته شده‌اند چون هیچ نتیجه‌ای هر دو را لازم ندارد، و چون یک تعریف در هر حال نمی‌توانست هر دو را حمل

کند: از بازی تعریف 5.1 خواسته می‌شود که خوش‌ساخت باشد و عماداً نشست‌یکنوا نیست، چنان که نکته 5.6 توضیح می‌دهد.

می‌گوییم $\mathcal{F}$ محلی است اگر $\mathcal{F}.N = \{\text{pid} : \text{pid}.F = F^1\}$ برای نام والد (فراخواننده) معین F^1 . (این را به خود کارکرد وامی‌گذاریم که بگوید آیا $\mathbf{A}$ و $\mathbf{Z}$ می‌توانند فرایش بخوانند یا نه.) مفهوم‌های میانی ممکن‌اند: برپاسازی‌ای که از دو پروتکل نام‌دار در دسترس است $\{\text{pid} : \text{pid}.F \in \{F_1, F_2\}\}$ را می‌پذیرد؛ و یک $\mathbf{N}$ می‌تواند کل شناسه‌های فرایند را سنجاق کند و نه نام‌ها را، و نمونه‌های اشغال‌شده فراخواننده‌هایش را رو نویسد – نکته 6.5 می‌گوید چرا اعضای خصوصی یک تحقق باید چنین کنند. در سر دیگر طیف می‌گوییم $\mathcal{F}$ جهانی است، به معنای برپاسازی مشترک UC تعمیم‌یافته [8]، اگر

$$\text{global}(\mathcal{F}) : \Longleftrightarrow \text{Std} \subseteq \mathcal{F}.N \vee \mathcal{F}.F \in \{A, Z, C\},$$

که یک مفهوم است و نه دو: عطف اول حالت معمولی است، هر کارکرد استاندارد می‌تواند فرایش بخواند، و عطف دوم آن سه ماشینی را می‌پوشاند که اجرا فراهم می‌کند – و هر چه در جایگاهشان بایستد – که به دلیلی دیگر جهانی‌اند و هر جا global آزموده شود همان‌گونه با آن‌ها رفتار می‌شود.

این تمایز همان چیزی است که مؤلفه نشست برای آن است. کارکرد محلی بخشی از یک اجرا از یک پروتکل است، پس فراخوان تنها از نشست خودش می‌تواند به آن برسد: سطر 3 از **Guard** می‌پرسد $\text{id}.s = \text{id}.s$. شماره نمونه آزموده نمی‌شود، پس یک نشست می‌تواند هر چند نمونه از یک نام که بخواد در بر داشته باشد و آن‌ها می‌توانند یکدیگر را فرا بخوانند. کارکرد جهانی بین اجراها مشترک است – یک نشست از آن، در دسترس هر نشستی – پس آن آزمون برای او بخشیده می‌شود، و همه نشست‌های $\mathcal{F}$ می‌توانند یک نمونه مشترک را فرا بخوانند. کارکرد ایده‌آل جهانی را با G در آغاز نام می‌دهیم و واقعی جهانی را با γ در آغاز، چنان که در $\mathcal{F}_{\text{Sig}}$ ؛ و $\mathcal{G}$ خوش‌نویس همان می‌ماند که بوده، کارکرد دومی در کنار $\mathcal{F}$. آن سه ماشینی که اجرا فراهم می‌کند جهانی‌اند: A و C همین حالا با بند اول، و Z با بند دوم، که آنجاست چون $\mathbf{Z}.N = \mathbf{A} \cup \mathbf{Z}$ عماداً تنگ است و با این حال محیط باید از هر نشستی در دسترس باشد.

مشخصه واقعی در برابر ایده‌آل. کارکردهای ایده‌آل و پروتکل‌های واقعی از چند جهت متفاوتند. اول، کارکردهای ایده‌آل وضع فساد طرف‌های موجود در مجموعه طرف‌هایشان را می‌دانند. دوم، می‌توانند اجرا را به شبیه‌ساز بسپارند و می‌توانند واسطه‌های ویژه شبیه‌ساز عرضه کنند (مثلاً $\mathcal{F}_{\text{Diffuse}}$ را ببینید). سرانجام، می‌توانند حالت را بین طرف‌ها مشترک کنند – و همین اشتراک نکته اصلی است: نگهبان فصل 2 هر فراخوان را به یک طرف گره می‌زند، پس طرف‌ها تنها از راه کارکردهایی با هم سخن می‌گویند که حالت را بینشان مشترک می‌کنند، و پروتکل واقعی با هم‌تاهایش از راه زیرروال‌های ایده‌آل سخن می‌گوید و از هیچ راه دیگری. اینجا و در زیر، $\mathcal{F}_{\text{IO}}$ و $\mathcal{F}_{\text{Diffuse}}$ نام کارکردهایی است که نمی‌دهیمشان: یک کانال پخش و یک بازپخش ورودی-خروجی. هیچ چیز این مقاله به کد آن‌ها بند نیست. سرچشمه تصادف $\mathcal{F}_{\text{Rand}}$ و انبار $\mathcal{F}_{\text{Store}}$ داده شده‌اند، در فصل‌های 13 و 14؛ آن‌ها کوچک‌ترین مثال‌های این مقاله‌اند و همان جایی هستند که خواننده ناآشنا با این سبک باید از آنجا شروع کند.

واسطه‌های هسته پروتکل‌های واقعی خوش‌مشخصه هیچ‌یک از این ویژگی‌ها را ندارند، جز آنچه در بلوک‌های ساختمانی ایده‌آل‌شده و پوشانه‌های کنترل دسترسی مستتر است؛ و تحقق‌های واقعی مشخص نمی‌کنند که نشأت هنگام فساد چگونه حساب می‌شود، چون آن محتوای نوارهای کار است.¹

مقدار آغازین. Initialize یک رویه است و نه یک واسطه: یک بار می‌دود، وقتی کارکرد نخستین بار فعال می‌شود، و متغیرهایی را که باقی‌کد به کار می‌برد برپا می‌کند. هیچ فراخوانی نمی‌گذارد – اجرای بخش 3.6 هر ماشین را پیش از نصب توزیع‌گرش مقدار آغازین می‌دهد، پس چیزی نبود که فراخوانی را خدمت کند – هیچ فراخوانده‌ای نمی‌تواند به آن برسد، هیچ شناسه فراخوانده ادعایی نمی‌گیرد، و کارکردی که هیچ اعلام نکند تهی آن را می‌گیرد. پروتکل‌های واقعی هیچ مقدارده جهانی از این دست حمل نمی‌کنند، تنها مقدارده ویژه یک طرف در یک فرایند، که `Initialize(pid, P)` نوشته می‌شود و در آن نمادگذاری را بدرفتاری می‌کنیم: آن‌ها خصوصی‌اند، نه واسطه‌هایی که هر کس بتواند خطابشان کند.

¹ رهیافتی منظم‌تر ذخیره‌سازی را به‌صورت کارکردی ایده‌آل مدل می‌کرد که پاک‌کردن امن عرضه می‌کند؛ ما از آن لایه افزوده می‌پرهیزیم تا نمادگذاری سبک بماند.

نگهبان‌ها، خموش‌کننده‌ها و واسطه‌گرها

این فصل سه تکه ماشین‌آلات را تعیین می‌کند. محمول **Guard** فراخوان‌هایی را که به یک ماشین می‌رسند مقید می‌کند؛ پوشانۀ **Silence** فراخوان‌هایی را که او می‌گذارد؛ و پوشانۀ **Mediate** فراخوانی را که طرفِ فاسد نمی‌تواند خودش پاسخش دهد به حریف می‌سپارد. واسطۀ کامل بخش 3.5 ترکیبِ همین سه است.

بنویسیم Op برای واسطۀ هستهٔ عمومی $\mathcal{F}$. ما تنها هسته‌ها را مشخص می‌کنیم؛ پوشانۀ زیر هر یک را به واسطی کامل بدل می‌کند، و فراخواننده به **FullOp** می‌رسد، هرگز به Op . آزمون‌های کنترلِ دسترسی در محمول **Guard** جمع شده‌اند، که کارکرد نیست بلکه الگوریتم است: هویتِ هدف id ، هویتِ فراخوانندۀ ادعایی id' و کارکرد $\mathcal{F}$ را می‌گیرد و یک بولی برمی‌گرداند. دو قراردادِ ارائه. جعبه‌ها دو وزن دارند: پُر نشانۀ کارکرد است، که هویت اشغال می‌کند، و تُهی نشانۀ محمول یا پوشانه یا رویه، که هیچ اشغال نمی‌کند. و فراخوان چنین نوشته می‌شود: $id' \text{ as } id.Op(in)$ که در آن id' هویتی است که در گذاشتن آن ادعا شده، و چنین دریافت می‌شود: **from** $id.Op(in)$ که در آن id' هویتی است که فراخوان زیر نام آن ادعا شده بود. اِعمال در پوشانه نشسته است، همان جا که سطر 1 آن را **require** می‌کند، پس فراخوانی که او ردش کند هرگز به هسته نمی‌رسد. سطر 1 آزمون می‌کند که واسط وجود دارد؛ سطر 2، که نام فراخواننده پذیرفته است و برای طرفِ درست عمل می‌کند؛ سطر 3، که فراخواننده و فراخوانده در یک نشستند، مگر آنکه فراخوانده بین نشست‌ها مشترک باشد.

محمول Guard

$:Guard_T(id, id')$

1: **return** $id.pid = \mathcal{F}.pid \wedge id.P \in \mathcal{F}.P$

/ دارند وجود که واسط‌هایی

2: $\wedge id'.pid \in \mathcal{F}.N \wedge id'.P = id.P$

/ پذیرفته فراخواننده‌های

3: $\wedge (id'.s = id.s \vee global(\mathcal{F}))$

/ باشد مشترک آنکه مگر نشست، یک

Guard فراخوان‌هایی را که به ماشین می‌رسند مقید می‌کند؛ پوشانۀ **Silence** در زیر، فراخوان‌هایی را که او می‌گذارد. این پوشانه یک مجموعهٔ هویت $\mathcal{I}$ و یک برنامه می‌گیرد، برنامه را می‌دواند، و هر فراخوانی که صادر کند را می‌رباید. فراخوانی که هویتی

در $\mathcal{J}$ ادعا کند با rej رد می‌شود، بی‌آنکه هرگز گذاشته شود؛ هر فراخوانی دیگری گذاشته می‌شود و پاسخش پس داده می‌شود. پس $\mathcal{J}$ درست همان جایی است که برنامه پوشانده‌شده در آن خموش است، و می‌تواند به‌عنوان هر چیز بیرون آن سخن بگوید. دامنه، همان برنامه پوشانده‌شده است و نه فراتر از آن. فراخوانی که او می‌گذارد ژتون را به ماشینی دیگر می‌سپارد، و آنچه آن ماشین می‌گوید با پوشانه خودش حکم می‌شود، نه با این یکی. اگر جز این بود، کارکرد زیرروال‌هایش را تراپا خموش می‌کرد و هیچ‌یک نمی‌توانست به‌عنوان خودش سخن بگوید. هر کاربرد Silence در زیر همین شکل را دارد: واسطی کامل که هسته زیر خدمتش را می‌پوشاند.

خواندن این به‌عنوان هندلر، به معنای اثرهای جبری [4, 12]، به‌کار می‌آید. فراخوانی که برنامه صادر می‌کند عملی از اثر محیطی است، و k همان ادامه‌ای است که چشم‌به‌راه پاسخ آن است. پوشانه Silence تصمیم می‌گیرد با چه چیزی از سر گرفته شود: rej برای ادعای خموش‌شده، و در غیر این صورت پاسخ فراخوانی که واقعاً گذاشته شد. بند return نتیجه خود برنامه را دست‌نخورده رد می‌کند، پس خموش‌سازی آنچه برنامه می‌تواند بگوید را تغییر می‌دهد و هیچ‌چیز دیگر را. این قیاس تنها شکل پوشانه را تعیین می‌کند.

پوشانه Silence

مجموعه هویت $\mathcal{J}$ ، برنامه $\text{id} \cdot \text{Op}(\text{in})$ از id'

[:Silence\(\$\mathcal{J}\$, \$\text{id} \cdot \text{Op}\(\text{in}\)\$ from \$\text{id}'\$ \)](#)

:1	handle $\text{id} \cdot \text{Op}(\text{in})$ from id' with	
:2	return out	$\mapsto \text{out}$
:3	$\text{id}'' \cdot \text{FullOp}(\text{in}'')$ as id''' ; k	$\mapsto k(\text{rej})$
:4		$\mapsto k(\text{id}'' \cdot \text{FullOp}(\text{in}'')$ as $\text{id}''')$

اگر $\text{id}''' \in \mathcal{J}$

وگرنه

با داشتن Silence می‌توانیم بگوییم بر فراخوانی که طرف فاسد نمی‌تواند خودش پاسخش دهد چه می‌گذرد. هر واسط کامل زیر چنین فراخوانی را دو بار به حریف می‌سپارد، در راه ورود و در راه خروج، و هر دو بار در همان یک شکل: پوشانه Mediate . این پوشانه یک مجموعه فساد $\mathcal{C}$ ، عمل $\text{id} \cdot \text{Op}$ که در حال خدمت است، مقدار x که باید سپرده شود، و فراخواننده ادعایی id' عملی که به‌جایش می‌ایستد را می‌گیرد. اینکه پاسخ از کجا می‌آید در درون خودش تعیین شده است: هر عملی که باشد، پاسخ‌های طرف فاسد از $(A, \text{id} \cdot P)$ می‌آیند. یک چیز در باب آن آرگومانی معمولی نیست: هم require و هم Mediate return از عملی برمی‌گردند که در $\mathcal{C}$ آن ایستاده‌اند، و نه فقط از خود آن سطر. وقتی آزمون Mediate شکست بخورد او هیچ نمی‌کند و آن عمل ادامه می‌دهد.

این قلاب یک فراخوان معمولی روی واسط کامل A است، که هویت id واسط در حال خدمت را ادعا می‌کند. پس با Guard_A روبه‌رو می‌شود، و از آن می‌گذرد چون طرف ادعایی $\text{id} \cdot P$ همان طرف هدف است؛ و حریفی که پاسخ می‌دهد در بند خموش‌کننده خود A است، که تنها $(A, \text{id} \cdot P)$ را می‌پذیرد. پس حریف در حال پاسخ‌دادن برای $\text{id} \cdot P$ می‌تواند به‌عنوان خودش در $\text{id} \cdot P$ سخن بگوید و به‌عنوان هیچ‌چیز دیگر، و Mediate به خموش‌سازی خودش نیازی ندارد.

پوشانۀ فساد Mediate

مجموعۀ فساد C ، عمل $id.Op$ ، مقدار x ، فراخوانندۀ ادعایی id'

$:Mediate(C, id.Op, x, id')$

```

:1 if ( $id'.F \neq A \wedge id.P \in C$ ) then
:2   return ( $A, id.P$ ).FullA( $id.Op, x$ ) as id

```


مدل اجرا

3.1 فساد

فساد واسطی نیست که پروتکل‌های واقعی محققش کنند (پروتکل واقعی وقتی «دستور» بگیرد که فاسد شود چه می‌کند؟)، اما برای تقلید باید وجود داشته باشد. می‌توانیم فرض کنیم پروتکل‌های واقعی همیشه فساد را رد می‌کنند، اما آن‌گاه هرگز به‌عنوان فاسد ثبت نمی‌شوند.

ما فساد را به‌صورت کارکردی جهانی به نام $\mathcal{C}$ مدل می‌کنیم. چند ماشین واسطی کاملشان یک‌جا داده شده و از هسته به‌دست نیامده – محیط، حریف، و عمل نشسته بخش 3.2 – اما $\mathcal{C}$ تنها ماشینی است که نه **Silence** را به کار می‌برد و نه **Mediate**، و گزارش وضع دلایلش است: آن پوشانه در هر فراخوان وضع فساد را می‌خواند، که همان چیزی است که این واسط پیاده‌اش می‌کند، پس پوشاندنش دوری بود. پس هر واسط زیر تنها آزمون‌هایی را که لازم دارد به‌دست حمل می‌کند، و **FullStatus** به‌طور خاص همیشه مجموعه فاسدها را برمی‌گرداند، بی‌آنکه نخست بپرسد آیا آن طرف فاسد است. نشسته خود $\mathcal{C}$ ثابت است، پس آزمون فساد که بخش 3.2 بر هر عمل نشسته دیگری تحمیل می‌کند اینجا چیزی برای محافظت نداشت؛ با این حال نگهبان نگه داشته می‌شود، تا حریف دفتر را در همان طرفی بخواند که برایش عمل می‌کند و در هیچ طرف دیگر.

خواندن دفتر. $\mathcal{C}$ حالت خود $\mathcal{C}$ است، پس هیچ ماشین دیگری رونوشتی از آن ندارد. هر جا $\mathcal{C}$ در کد کارکردی جز خود $\mathcal{C}$ ظاهر می‌شود، فهم مطلب این است که نخست واکنشی شده است، با

$\mathcal{C} \leftarrow (C, \text{id.P}).\text{FullStatus}() \text{ as id,}$

و ما آن سطر را هر جا که تنها چیزی را تکرار می‌کرد که کد پیرامون خودش روشن می‌کند حذف می‌کنیم. در درون $\mathcal{C}$ چیزی برای واکنشی نیست: $\mathcal{C}$ مستقیم خوانده و نوشته می‌شود، و Corrupt در جا به‌روزش می‌کند.

نکته 3.1 (چرا **FullStatus** واسطه‌گری نمی‌شود). اگر مثل عمل‌های دیگر پوشانده می‌شد، **FullStatus** درست همان جا که مهم است بی‌مصرف می‌بود. فرض کنید

دَفتَرِ فسادِ $\mathcal{C}$

$$\text{par} := \perp, \quad \mathbf{U} := \emptyset, \quad \mathbf{N} := \text{Std} \cup \mathbf{A} \cup \mathbf{Z}, \quad \mathbf{P} := \{0, 1\}^* \cup \{\mathbf{A}, \mathbf{Z}\}, \quad \text{pid} := \mathcal{C}$$

:Initialize()

1: $\mathbf{C} \leftarrow \emptyset$ id' از $\text{id.FullCorrupt}()$ 2: **require** $\text{Guard}_e(\text{id}, \text{id}')$ 3: **require** $\text{id}.F \in \{\mathbf{A}, \mathbf{Z}\}$ 4: **require** $\text{id}.P \in \{0, 1\}^*$ 5: $\mathbf{C} \leftarrow \mathbf{C} \cup \{\text{id}.P\}$ 6: **return** ok id' از $\text{id.FullStatus}()$ 7: **require** $\text{Guard}_e(\text{id}, \text{id}')$ 8: **return** C id' از $\text{id.FullLeak}()$ 9: **require** $\text{Guard}_e(\text{id}, \text{id}')$ 10: **require** $\text{id}.F = \mathbf{A}$ 11: **return** $\perp$

/ نمی‌دارد نگه رازی هیچ دفتر ثابت: /

$\mathbb{Z}$ به عنوان id' دست دراز کند تا وضع فسادِ $\text{id}.P$ را بپرسد. اگر $\text{id}.P$ فاسد باشد، دروازه فراخوان را رد می‌کند و **Mediate** آن را به حریف می‌سپارد – پس پاسخ «آیا $\text{id}.P$ فاسد است؟» هر چیزی است که حریف بگوید، و می‌تواند بگوید نه. و چون هر فراخوانده‌ای دفتر را می‌خواند تا تصمیم بگیرد با طرف فاسد چگونه رفتار کند، پاسخی که حریف مهارش کند ماشین‌آلات واسطه‌گری را به میل خود خاموش می‌کرد.

پس **FullStatus** هیچ واسطه‌گری و هیچ خاموش‌سازی نمی‌کند: پس از نگهبان، $\mathbf{C}$ را برمی‌گرداند، حالتِ خودِ دفتر را. این همان یک عمل است که پاسخش باید از دفتر بیاید، و چون تنها خواندنی است چیزی به حریف نمی‌دهد که پیش‌تر نتوانسته باشد استنتاج کند. هستهٔ **Status** پیش‌نویس‌های قدیمی‌تر هم با آن می‌رود، چون پوشانه‌ای نمانده که از آن برکنار بماند.

فساد کار بیرون است. سطر 3 $\text{id}.F \in \{\mathbf{A}, \mathbf{Z}\}$ را می‌پذیرد، پس حریف و محیط هر یک می‌توانند طرفی را در $\mathbf{C}$ بگذارند و هیچ کارکردِ استانداردِ اصلاً نمی‌تواند چیزی آنجا بگذارد. نگهبان بالای آن، واسطهٔ فاسدکننده را به همان طرفی که فاسدش می‌کند گره می‌زند و می‌پرسد $\text{id}.P = \text{id}.P$: برای فاسدکردن P ، حریف باید در $(\mathbf{A}, P)$ بدود، و محیط باید $(\mathbf{Z}, P)$ را ادعا کند، که سطر 3 می‌گذارد ریشه‌اش این کار را برای هر طرفی بکند. هر دو می‌توانند بخوانند هم: $\mathbf{A} \cup \mathbf{Z} \subseteq \mathcal{C}.\mathbf{N}$ ، و **FullStatus** کل $\mathbf{C}$ را برمی‌گرداند، پس هر یک می‌تواند در هر نقطه‌ای بپرسد و همیشه کل مجموعه را می‌آموزد. و تنها طرف‌های اصلی می‌توانند فاسد شوند: **FullCorrupt** می‌پرسد $\text{id}.P \in \{0, 1\}^*$ ، پس

نام‌های رزرو بنا بر ساخت درستکار می‌مانند.

هم‌اندازگی درست درمی‌آید، و آنچه تأمینش می‌کند خواندن است و نه هیچ تقسیم اینکه چه کسی می‌تواند بنویسد. ماشین نشسته در (A, P) در یک اجرای آزمایشی تقلید A است و در اجرای دیگر S ، و هیچ چیز آن دو را به فاسدکردن همان طرف‌ها وادار نمی‌کند. اما S در هر دو سو همان یک ماشین است و در هر دو C را می‌خواند، پس اشغال‌گر جایگاه اگر مجموعه دیگری را فاسد کند، محیطی که تنها بپرسد او را می‌گیرد. پس هم‌اندازگی نتیجه است و نه فرض: هیچ محیط مجازی آن دو مجموعه فساد را با مزیتی بیش از آنچه تعریف 4.11 به شبیه‌ساز می‌دهد از هم تشخیص نمی‌دهد. و اینکه محیط می‌تواند برای خودش فاسد کند این را تنها تیزتر می‌کند، چون فسادهایی که او می‌کند در هر دو سو از خود اوست؛ آنچه برای گرفتن می‌ماند ابتکار خود اشغال‌گر است.

فساد تنها با نام طرف اندیس‌گذاری می‌شود، پس P نمی‌تواند در یک شناسه فرایند فاسد باشد و در دیگری درستکار.¹

3.2 نشت

کارکرد $\mathcal{F}$ ، عمل نشت	
id , P , N , U , par	
$\text{id} \leftarrow \text{id.FullLeak}()$ از id'	
1: require $\text{id}.pid = \mathcal{F}.pid \wedge \text{id}.P \in \mathcal{F}.P$	دارد وجود واسط /
2: require $\text{id}'.F = A \wedge \text{id}'.P = \text{id}.P$	طرف این برای A ، /
3: $C \leftarrow (C, \text{id}.P).\text{FullStatus}()$ as id	
4: require $\text{id}.P \in C$	می‌کند نشت فاسد طرف تنها /
5: return $\text{Silence}\{\text{id}'' : \text{id}'' \neq \text{id}\}, \text{id}.Leak() \text{ from } \text{id}'\}$	

این سه خواسته همان‌هایی هستند که پوشانه عمومی بخش 3.5 تحمیل می‌کند، و از نو بیان شده‌اند چون نشت به شیوه معمول از هسته به‌دست نمی‌آید. سطر 1 نیمه وجود **Guard** است؛ نیمه فراخواننده عموماً ضعیف‌تر است، چون A باید بتواند از کارکردی نشت بگیرد که او را به‌عنوان فراخواننده نمی‌پذیرد، پس سطر 2 تنها می‌خواهد که حریف برای طرفی عمل کند که دارد می‌خواندش. سطر 4 دروازه فساد است. بی آن، حالت طرف درستکار به میل خواندن می‌شد، و همین است که سطر 3 از واسط کامل هر جای دیگری ردش می‌کند. خموش‌کننده همان خموش‌کننده سطر 5 است، پس هسته نشتی که فراخوان می‌گذارد، مثل هر هسته دیگری هویت خودش را ادعا می‌کند.

برای کارکرد ایده‌آل، **Leak** بخشی از مشخصه است؛ پیام‌هایی که تا اینجا دریافت شده و محتوای فهرست‌های درونی نمونه‌های معمولند. برای پروتکل‌های واقعی

¹ می‌شد به‌جایش یک هویت را فاسد کرد — C هویت‌ها را جمع می‌کرد و هر $\text{id}.P \in C$ به $\text{id} \in C$ بدل می‌شد — به بهای «پخش‌شدن» فسادها به زیرپروتکل‌ها. قرارداد طرف‌گستر پاکیزه‌تر است، چون یک خانواده کامل را یک‌جا آلوده می‌کند.

می‌توان آن را چنان نشان داد که هیچ نشت نکند، که یعنی مدلی از بی‌نشتی کامل؛ و چاره همان کارکردهای ایده‌آلی هستند که می‌گویند کدام بخش‌های حالت نشت می‌کنند. پروتکلی نشت‌خوش ساخت است اگر بدنه هر ماشین بین فراخوانی‌ها پاک باشد، هر تصادف نمونه‌برداری شده از راه $\mathcal{F}_{\text{Rand}}$ کشیده شود و هر حالتی که از فراخوانی‌ای به فراخوانی دیگر می‌پایید در $\mathcal{F}_{\text{Store}}$ نگه داشته شود و نه روی نوار ماشین — که هر یک آن را از راه Leak خودش رو می‌کند — و ورودی‌ها و خروجی‌ها تنها آنجا نشت کنند که پروتکل آن‌ها را از راه کارکرد ورودی-خروجی $\mathcal{F}_{\text{IO}}$ می‌گذارند، که پیش‌فرض نیست. آن‌گاه نوارهای کار هرگز نشت نمی‌کنند، چون پاک‌ی بین فراخوانی‌ها چیزی رویشان نمی‌گذارد. هیچ نتیجه زیر هیچ‌یک از این‌ها را مفروض نمی‌گیریم؛ این شرطی است که یک تحقق باید برآورده کند تا Leak آن معنایی داشته باشد، و ما ثبتش می‌کنیم و به کارش نمی‌بریم. $\mathcal{Z}$ نمی‌تواند این واسطه را فرا بخواند، پس برای تمایزناپذیری لازم نیست محقق شود. با این حال باید شبیه‌سازی‌شدنی باشد، چون حریف می‌تواند فرایش بخواند.

3.3 محیط

ما کارکرد ویژه‌ای به نام $\mathcal{Z}$ تعریف می‌کنیم، با یک واسطه یگانه به‌ازای هر id ، یعنی $\text{id}.\text{FullZ}$ ، که هسته $\text{id}.\mathcal{Z}$ را نگهداری و خاموش می‌کند. میدان $\mathcal{Z}.\mathbf{N} = \mathbf{A} \cup \mathbf{Z}$ سزاوار تأکید است: حریف و خود محیط را می‌پذیرد و هیچ چیز دیگر را، پس هیچ کارکرد استاندارد نمی‌تواند $\mathcal{Z}$ را فرا بخواند. هر چیز فراخ‌تری می‌گذاشت پروتکلی محیط را به‌عنوان یکی از زیرروال‌های خودش فرا بخواند — یک «پروتکل باز»، که زیرروال‌هایش نامعین می‌مانند. دو چیز متمایز این را ممنوع می‌کنند. تنگی $\mathcal{Z}.\mathbf{N}$ خود فراخوان را رد می‌کند: شناسه فرایند فراخواننده استاندارد در Std است، که هیچ کجا با $\mathbf{A} \cup \mathbf{Z}$ برخورد نمی‌کند، پس سطح 2 از Guard ، هر چه فراخواننده اعلام کند، ردش می‌کند. تعریف 1.3 نیمه دیگر بازبودن را رد می‌کند، یعنی عضوی که نام وابستگی‌ای را می‌برد که هیچ عضوی فراهمش نمی‌کند. اینجا اولی است که کار می‌کند: نام‌بردن از $\mathcal{Z}.\text{wf}$ را نمی‌لغزاند، چون $\mathcal{Z} \in \pi^+$ چنین درایه‌ای را برآورده‌شدنی می‌کند، پس اگر آن میدان فراخ‌تر بود، سیستمی می‌توانست خوش‌ساخت باشد و با این حال باز.

آن $\mathcal{Z}$ در $\mathcal{Z}.\mathbf{N}$ به دلیلی دیگر آنجاست: می‌گذارد $\mathcal{Z}$ خودش را فرا بخواند. واسطه‌های $\mathcal{Z}$ محیط در طرف‌های مختلف جدایند، و پذیرفتن $\mathcal{Z}$ همان است که می‌گذارد آن‌ها به یکدیگر برسند.

اینکه کدامشان می‌تواند به کدام برسد را مجموعه خاموش‌شده حل می‌کند، و آنجا کاری بیش از دور نگه داشتن $\mathcal{Z}$ از سیستم‌ها می‌کند. هر ماشین دیگری به جایی که می‌دود گره خورده است: کارکرد استاندارد تنها هویت خودش را می‌تواند ادعا کند، و $\mathcal{A}$ روی $\{\text{id} \neq \text{id}'\}$ خاموش است. هیچ چیز $\mathcal{Z}$ را به‌تنهایی گره نمی‌زد، چون Silence تنها id' ادعایی را آزمون می‌کند و Guard تنها هدف را، پس $(\mathcal{Z}, P)$ و $(\mathcal{Z}, P')$ جایگزین‌پذیر می‌شدند و مؤلفه طرف یک هویت $\mathcal{Z}$ هیچ خبری حمل نمی‌کرد. سطح 3 این گره را می‌دهد، و تا آنجا که کاربردها می‌گذارند شل: واسطی در id می‌تواند id' را ادعا کند هرگاه $\text{id}.P = \text{id}'.P$ یا $\text{id}.P = \mathcal{Z}$ ، یا $\text{id}.P = \mathcal{Z}$. به زبان ساده، برای طرف خودت عمل کن، یا برای ریشه؛ و ریشه می‌تواند برای هر کسی عمل کند.

هر دو راه فرار لازمند. بی $\text{id}.P = Z$ ریشه در تنگنا می‌ماند، و ریشه همان جایی است که **Exec** از آن آغاز می‌کند و تنها واسطی است که می‌تواند به دیگران برسد. بی $\text{id}.P = Z$ برگ‌ها در تنگنا می‌مانند: تنها راه بازگشت حریف به محیط $\rightarrow (A, P)$ است، و **Mediate** A را در $(A, \text{id}.P)$ بیدار می‌کند، برای طرفی که پاسخش را می‌دهد؛ پس محیطی که نیاز داشته باشد آنچه حریف در P گفته را با آنچه در P' گفته بسنجد، هرگز نمی‌توانست آن دو را کنار هم بیاورد – این واسط‌ها جدایند، و نمی‌توانند از راه حالت مشترک هم آن را یک‌کاسه کنند – و رده تعریف 4.11 واقعاً کوچک می‌شد. با هر دو راه فرار، همبندی ستاره‌ای دوسویه است که در (Z, Z) ریشه دارد، و ما چیزی نمی‌شناسیم که محیطی نامقید بتواند بکند و این ستاره نتواند. سطر 2 در همین روح مقید می‌کند: جلوی ادعای (A, P) به دست Z را می‌گیرد، و از این راه جلوی گذشتنش از دروازه فساد سطر 3 و راندن واسط طرف درستکار را – توانی که حریفی که او نقشش را بازی می‌کرد هم ندارد، چنان که نکته 4.8 در باب شبیه‌سازی که روی کمتری خموش شده ثبت می‌کند. آنچه آن سطر دیگر لازم نیست بکند، دور نگه داشتن Z از **FullCorrupt** است، که سطر 3 او را زیر نام خودش به آن می‌پذیرد.

$A \cup Z$ با هم همان است که کاربردها لازم دارند و نه بیشتر. پذیرفتن **A** ضروری است چون محیط هیچ راه دیگری به اجرا ندارد: وقتی Z ژتون را واگذارد، دوباره تنها زمانی می‌دود که چیزی فرایش بخواند، و تنها حریف می‌تواند؛ و همان کانال است که Z از آن هر چه حریف بخواهد بگوید می‌شنود. پذیرفتن **Z** ضروری است چون محیط یک واسط به‌ازای هر طرف است و نه یک ماشین، و بی آن ریشه به هیچ‌یک از آن‌ها نمی‌رسید. جز این دو هیچ ضروری نیست: کارکردی که ناچار بود Z را فرا بخواند با آن مثل زیروالی رفتار می‌کرد که خودش تعریفش نکرده، و $\mathcal{C}$ فسادها را به هیچ‌کس گزارش نمی‌دهد – حریف می‌داند چه فاسد کرده، چون خودش کرده – و همین است که چرا Z باید مجموعه فساد را بپرسد. پس محیط تنها از آن دو ماشینی در دسترس است که بیرون هر سیستمی‌اند.

محیط Z

$\text{par} := \mathcal{J} \quad \mathcal{U} := \{(A, \text{serves})\} \quad \mathcal{N} := A \cup Z \quad \mathcal{P} := \{0, 1\}^* \cup \{A, Z\} \quad \text{id} := Z$

id از $\text{id}.\text{FullZ}(\text{in})$

1: **require** $\text{Guard}_Z(\text{id}, \text{id}')$

2: $\bar{\mathcal{J}} \leftarrow \mathcal{J} \cup \{\text{id}'' : \text{id}''.\text{F} = A\}$

3: $\cup \{\text{id}'' : \text{id}.P \neq Z \wedge \text{id}''.\mathcal{P} \notin \{\text{id}.P, Z\}\}$

4: **out** $\leftarrow \text{Silence}(\bar{\mathcal{J}}, \text{id}.Z(\text{in}) \text{ from } \text{id}')$

5: **return** out

// دسترس از دور A
// ریشه یا خود، طرف

3.4 حریف

حریف A همان شکل را دارد: یک واسط یگانه به‌ازای هر id ، یعنی $\text{id}.\text{FullA}$ ، که هسته $\text{id}.A$ را نگهبانی و خموش می‌کند. آنچه او را از محیط جدا می‌کند این است که چه کسی می‌تواند به او برسد. آنجا که $\mathcal{N}$ تنها $A \cup Z$ است، میدان $A \cup Z \cup \text{Std}$ ، هر

شناسه فرایندی را در بر دارد که هرگز فراخوانی می‌گذارد. تنها شناسه $\mathcal{C}$ نیست، و $\mathcal{C}$ هیچ فراخوانی نمی‌گذارد. این فراخی یک خواسته است و نه یک راحتی: قلاب‌های واسطه‌گری فصل 2، و فراخوان‌هایی که کارکردهای ایده‌آل از راه کوتاه‌نویسی $A(\cdot)$ می‌گذارند، هر دو فراخوان‌هایی هستند که ماشینی در حال عمل برای طرفی، روی حریف می‌گذارد. این واسطه‌ها همان چیزی هستند که در پشتی در این نمادگذاری چنین می‌نماید. حریف ژتونی از خودش ندارد و نمی‌تواند فراخوانی را آغاز کند، پس هر حمله‌ای که ترتیب می‌دهد از آنجا شروع می‌شود که ماشین دیگری مهار را به او می‌سپارد: واسطی یک طرف از راه **Mediate**، یا مستقیماً محیط، که اصلاً همان راهی است که محیط به حریف دستور می‌دهد. آنچه او در پاسخ می‌تواند بگوید تنگ است: مجموعهٔ خموش‌شده $\{id'' : id'' \neq id\}$ است، پس واسطی از A تنها می‌تواند همان هویتی را ادعا کند که در آن می‌دود.

در درون واسطی با هویت id ، ما $A(in)$ را برای فراخوان $FullA(in)$ به‌عنوان id می‌نویسیم، و $Z(in)$ را به همین شکل. ادعا باید id باشد و نه هیچ چیز دیگر، چون سطر 5 هسته را روی هر چیزی جز هویت خودش خموش می‌کند؛ پس حریف فراخواننده را به‌عنوان شناسهٔ ادعایی می‌آموزد و میدان جداگانه‌ای برایش لازم ندارد. هستهٔ او به‌صورت $id.A(in)$ از id' وارد می‌شود، همان شکلی که **Silence** اعلام می‌کند و همان شکلی که هر هستهٔ دیگری در آن وارد می‌شود. مؤلفهٔ طرف هدف $id.P$ است، پس فراخوان این را هم ثبت می‌کند که به‌نیابت چه کسی گذاشته شده است.

حریف A

$par := \perp$, $U := \{\{Z, serves\}\}$, $N := Std \cup A \cup Z$, $P := \{0, 1\}^* \cup \{A, Z\}$, $pid := A$

id از $id.FullA(in)$

```

1: require GuardA(id, id')
2: out ← Silence({id'' : id'' ≠ id}, id.A(in) from id')
3: return out

```

3.5 واسطه کامل

هر چه پوشانه لازم دارد اکنون سر جایش است: **Guard** و **Mediate** و **Silence** از فصل 2، و دفتر $\mathcal{C}$ از بخش 3.1 که $\mathcal{C}$ را می‌دهد. واسطهٔ کامل یک کارکرد استاندارد ترکیب همین‌هاست.

سطرهای 2-4 و 6-7 فساد را می‌گردانند، و تنها جایی هستند که واسطهٔ کامل با دفتر مشورت می‌کند. دو بار خوانده می‌شود چون $\mathcal{C}$ با پیشرفت اجرا تغییر می‌کند و هر خواندن تنها یک عکس فوری است: آزمون‌های پیش از سطر 6 اولی را به کار می‌برند و آزمون سطر 7 دومی را. عضویت یکنواست، چون **Corrupt** هرگز جز افزودن نمی‌کند، پس طرفی که در خواندن اول درستکار است می‌تواند در خواندن دوم فاسد باشد، اما هرگز برعکس.

سطر 3 یک ترکیب را رد می‌کند: حریف در طرف درستکار، که آنجا کاری ندارد. هر چیز دیگری می‌گذرد، و **Mediate** باقی را حل می‌کند. پس طرف درستکار برای همه جز

کارکرد $\mathcal{F}$ par, $\mathcal{U}$, $\mathcal{N}$, $\mathcal{P}$, pidid' از id.FullOp(in)

```

1: require Guard $\mathcal{F}$ (id, id')
2:  $\mathcal{C} \leftarrow (\mathcal{C}, \text{id.P}).\text{FullStatus}()$  as id
3: require  $\neg(\text{id'.F} = A \wedge \text{id.P} \notin \mathcal{C})$  / فاسد طرف در تنها  $A$ 
4: Mediate( $\mathcal{C}$ , id.Op, in, id')
5: out  $\leftarrow \text{Silence}(\{\text{id''} : \text{id''} \neq \text{id}\}, \text{id.Op}(\text{in}) \text{ from id'}$ )
6:  $\mathcal{C} \leftarrow (\mathcal{C}, \text{id.P}).\text{FullStatus}()$  as id
7: Mediate( $\mathcal{C}$ , id.Op, out, id')
8: return out

```

A خطاب‌پذیر است و هسته خودش را می‌دواند؛ و طرف فاسد برای همه خطاب‌پذیر است، اما تنها A به هسته‌اش می‌رسد و هر فراخوانده دیگری را A به‌جای آن طرف پاسخ می‌دهد. محیط حالت ویژه خودش را لازم ندارد: چه هویت خودش را ادعا کند و چه هویت بیرونی پذیرفته‌ای را، طرف فاسد در هر حال از راه A پاسخ می‌دهد. می‌ماند طرف فاسدی که هر کسی جز A خطابش کند، که فراخوانش را نمی‌شود همین‌طور دواند، پس A به‌جای آن طرف پاسخ می‌دهد. سطرهای 4 و 7 همان یک پوشانه در دو لحظه‌اند. اولی طرفی را می‌گیرد که در لحظه ورود از پیش فاسد است و in را واگذار می‌کند، پس هسته هرگز نمی‌دود. دومی طرفی را می‌گیرد که در حین دویدن هسته‌اش در سطر 5 فاسد شد و out را واگذار می‌کند، که دیگر مال آن طرف نیست که برگرداندش. یکنوایی باعث می‌شود آن دو، حالت‌ها را دقیقاً یک بار بپوشانند: طرفی که در اولی فاسد است پیش‌تر برگشته، و هیچ طرفی در اولی فاسد و در دومی درستکار نیست. هر دو قلاب A را در $(A, \text{id.P})$ از راه واسط کاملش خطاب می‌کنند، پس حریفی که پاسخ می‌دهد مثل هر فراخوانده دیگری نگرانی و خموش شده است. سطر 8 مقدار خود هسته را برمی‌گرداند، که در دو حالت رخ می‌دهد: برای طرفی که سرتاسر درستکار است، و برای طرف فاسدی که خود A خطابش می‌کند، که دروازه می‌پذیردش و هیچ‌یک از دو قلاب نمی‌گیردش، چون هر دو $\text{id'.F} \neq A$ را آزمون می‌کنند.

نکته 3.2 (فساد میان فراخوان پیش‌دستانه نیست). هیچ‌چیز هسته‌ای را که پیش‌تر در حال دویدن است قطع نمی‌کند. فرض کنید هسته سطر 5 حریف را فرا بخواند – از راه $A(\cdot)$ ، یا از راه زیرروالی که واسطه‌گری می‌کند – و حریف پیش از برگرداندن مقداری، id.P را فاسد کند. دفتر این را بی‌درنگ ثبت می‌کند، چون $\mathcal{C}$ حالت خود اوست، اما هسته نه خبر می‌شود و نه متوقف: با هر چه حریف برگردانده از سر می‌گیرد و تا پایان می‌دود. تنها در لحظه بازگشت آن است که سطر 6 $\mathcal{C}$ را از نو می‌خواند، id.P را در آن می‌یابد، و out را در سطر 7 به **Mediate** می‌سپارد تا جایگزین شود.

پس حریف در طول چنین فراخوانی دو بار ژتون را در دست دارد، به این ترتیب: یک بار در درون هسته، برای فراخوانی که هسته گذاشت، و یک بار پس از بازگشت هسته،

برای واسطه‌گری خروجی آن. هیچ لحظه‌ای در میانه نمی‌گذارد آن فساد اثر کند، چون طرفی که در میان فراخوان فاسد می‌شود نمی‌تواند آنچه پیش‌تر کرده را نکرده کند. فراخوان از نو هم دروازه‌بانی نمی‌شود: سطر 3 در خواندن اول تصمیم گرفته شد، و فراخوانی که در درستکاری $id.P$ پذیرفته شده پذیرفته می‌ماند. پس آنچه پوشانه تضمین می‌کند از سقط‌کردن ضعیف‌تر است، و بس است: هیچ چیزی که طرف تازه‌فاسد حساب کرده باشد بی‌واسطه‌گری به فراخوانده او نمی‌رسد. فراخوان‌هایی که پس از فساد می‌رسند، به‌جایش در راه ورود گرفته می‌شوند، با سطر 4.

3.6 اجراها

در یک اجرا، دقیقاً یک ماشین ژتون فعال‌سازی را در دست دارد. فراخوان $out \leftarrow id.Op(in)$ کوتاه‌نویسی این است که فراخوانده با یک ادامه معلق شود و ژتون به واسط Op از $\mathcal{F}$ در هویت id سپرده شود. هر فراخوان میان ماشین‌ها یک واسط کامل را خطاب می‌کند، و $FullOp$ همان است که $Exec$ توزیعش می‌کند؛ هسته تنها به‌دست پوشانه‌ای که خدمتش می‌کند فراخوانده می‌شود، در درون ماشینی که حملش می‌کند، و هرگز از آن‌سوی یک سپردن ژتون. فراخوانی که به هویتی خطاب شود که هیچ ماشینی اشغالش نکرده با rej پاسخ می‌گیرد. یک نمونه می‌تواند چند ادامه معلق داشته باشد و می‌تواند در حال تعلیق دوباره وارد شود، به‌ویژه به‌دست A .

تعریف 3.3 (ماشین‌ها و اشغال). یک ماشین هر چیزی است که اجرا می‌دواندش: اعضای سیستم، و هر چه اجرا در کنار آن‌ها فراهم می‌کند. ماشین مجموعه متناهی‌ای از هویت‌ها را اشغال می‌کند – برای کارکرد، همان $IDS(\mathcal{F})$ از تعریف 1.1 – و $Exec$ درست همان فراخوان‌هایی را به او توزیع می‌کند که به هویت‌های اشغال‌شده‌اش خطاب شده‌اند. اشغال‌گران یک اجرا مجموعه‌های هویت دوبه‌دو مجزا حمل می‌کنند، و $Exec$ برای هیچ خانواده دیگری تعریف نشده است. اشغال‌گر جایگاه حریف ماشینی است که هویت‌های شناسه فرایند A را اشغال می‌کند و هیچ هویت دیگری را؛ حریف بخش 3.4 یکی از آن‌هاست، و هر شبیه‌ساز فصل 4 نیز. یک فعال‌سازی بازه ماکسیمالی است که در طول آن یک ماشین ژتون را در دست دارد.

هر فراخوان یک شناسه ادعایی id' حمل می‌کند. کارکردهای استاندارد تنها هویت همان واسطی را می‌توانند ادعا کنند که فراخوان را می‌گذارد؛ $\mathcal{Z}$ و A می‌توانند هویت‌های دیگری را ادعا کنند، مشروط به آزمون‌های فصل 2. آن آزمون‌ها همان نگهبان‌اند، که id' را در برابر هویت id واسط و در برابر پارامترهای کارکرد می‌سنجد، و ما آن‌ها را با گزاره‌های **require** مشخص می‌کنیم. نگهبانی که شکست بخورد مقدار ممتاز rej را به فرستنده برمی‌گرداند، و rej یک رد است و نه یک مقدار: هیچ ماشینی نمی‌تواند آن را به‌عنوان خروجی خودش برگرداند. این قاعده اعمال می‌شود و مفروض گرفته نمی‌شود – هسته‌ای که کدش rej برمی‌گرداند، و پاسخ واسطه‌گری‌شده rej از جایگاه

حریف، به جایش به صورت $\perp$ تحویل داده می‌شوند — پس فراخوانده‌ای که rej می‌گیرد می‌آموزد که نگهبانی فراخوانش را رد کرده، و هیچ چیز دیگری نمی‌تواند یکی بسازد. به ویژه $rej \neq \perp$.

تصادف به ازای هر ماشین است: هر ماشین نوار سکه خودش را حمل می‌کند، نامتناهی، یکنواخت، و مستقل از نوار هر ماشین دیگر، که تنها وقتی کدش نمونه‌برداری می‌کند خوانده می‌شود. احتمال‌ها روی یک اجرا، روی این نوارها به طور توأم اند. دسته‌کردن ماشین‌ها در یک اشغال‌گر — چنان که فصل 4 و بخش 4.4 می‌کنند — نوارها را یکی نمی‌کند: سکه‌های یک رونوشت میهمان همان‌اند که پیش از جذب بودند. جز سیستم π ، اجرا سه ماشین حمل می‌کند: محیط $\mathbb{Z}$ ، حریف A ، و دفتر فساد C ، که حالتش C را هر پوشانه‌ای می‌خواند. بنا بر فصل 1 هیچ‌یک عضو π نیست، و همین سطر 1 را یک معنای می‌کند — دفتر یک بار مقدار آغازین می‌گیرد، به عنوان آرگومان، و نه بار دوم به عنوان عضو — و π را همان شیئی نگه می‌دارد که طراح نوشت. رویه آن سه را مقدار آغازین می‌دهد، و سپس محیط را در (Z, Z) فعال می‌کند، چنان که گویی خودش را فرا خوانده است. آن نخستین فعال‌سازی مثل هر فعال‌سازی دیگری از واسط کامل $\mathbb{Z}$ می‌گذرد، پس محیط از همان آغاز خموش است، و $\mathbb{Z}$ به سادگی می‌گذرد، چون هویت ادعایی و هدف یکی‌اند. هر چه محیط برگرداند همان برآمد کار است.

اجرای Exec

سیستم π ، محیط $\mathbb{Z}$ ، حریف A ، دفتر فساد C $Exec(\pi, \mathbb{Z}, A, C)$:

```

1:  $C.Initialize()$ ;  $\mathcal{F}.Initialize()$  for every  $\mathcal{F} \in \pi$ 
2: handle  $(Z, Z).FullZ()$  from  $(Z, Z)$  with // دارد را ژتون و می‌شود فعال نخست  $\mathbb{Z}$ 
3: return out  $\mapsto$  out
4:  $id.FullOp(in)$  from  $id'$ ;  $k \mapsto k(id.FullOp(in)$  from  $id')$ 

```

اگر آن را به عنوان هندلر بخوانیم [12، 4]، اجرا دقیقاً یک عمل دارد: گذاشتن فراخوان. سطر 2 محیط را فعال می‌کند، و این تنها فعال‌سازی‌ای است که ماشینی آن را نگذاشته است. سطر 4 از آن پس هر فراخوانی را می‌گرداند، و ادامه k همان جایی است که ژتون به آن بازمی‌گردد: معلق‌کردن فراخوانده، سپردن ژتون به id ، و از سر گرفتن فراخوانده با پاسخ، درست همان معنایی است که فراخواندن k روی آن پاسخ دارد. دقیقاً یک ماشین در سرتاسر ژتون را در دست دارد. بند **return** وقتی رسیده می‌شود که برنامه محیط بایستد، و خروجی آن خروجی اجراست.

تقلید و ترکیب UC

4.1 جانشانی زیرسیستم

پروتکل را معمولاً در برابر زیرروالی می‌نویسند که بعداً پروتکل دیگری فراهمش می‌کند. جانشین کردن یکی به جای دیگری، عملی مجموعه‌ای روی سیستم‌هاست.

تعریف 4.1 (جانشانی زیرسیستم). بگذارید ρ سیستمی باشد و $\varphi \subseteq \rho$ زیرسیستمی از آن. برای سیستم π ، جانشانی φ با π در ρ این است:

$$\rho[\pi/\varphi] := (\rho \setminus \varphi) \cup \pi.$$

تا اینجا هیچ چیز جلوی π ورودی را نمی‌گیرد که روی شناسه فرایندی فرود بیاید که پروتکل پیرامون از پیش به کارش می‌برد، و این فراخوانی را با خطابی دوبه‌لو رها می‌کرد. ما این را ممنوع می‌کنیم و به تعمیرش نمی‌رویم.

تعریف 4.2 (جانشانی خوش‌ساخت). جانشانی φ با π در ρ خوش‌ساخت است اگر هیچ شناسه فرایندی هم در $\rho \setminus \varphi$ و هم در π رخ ندهد، یعنی اگر

$$\{\mathcal{F}.pid : \mathcal{F} \in \rho \setminus \varphi\} \cap \{\mathcal{F}.pid : \mathcal{F} \in \pi\} = \emptyset.$$

گزاره 4.3 (جانشانی). بگذارید ρ و π سیستم باشند، $\varphi \subseteq \rho$ ، و جانشانی φ با π در ρ خوش‌ساخت باشد. آنگاه $\rho[\pi/\varphi]$ یک سیستم است. و اگر علاوه بر آن $IDS(\rho) = IDS(\varphi)$ ، آنگاه $IDS(\rho[\pi/\varphi]) = IDS(\rho)$.

اثبات. شناسه‌های فرایند $\rho[\pi/\varphi]$ همان‌های $\rho \setminus \varphi$ به همراه همان‌های π هستند. هر سیستم درون خودش شناسه‌های متمایز حمل می‌کند و خوش‌ساختی آن دو گردابه را مجزا می‌کند، پس هیچ شناسه‌ای تکرار نمی‌شود؛ اعضا استاندارد و به‌شمار متناهی‌اند، که هر دو از ρ و π به ارث رسیده، پس $\rho[\pi/\varphi]$ یک سیستم است. آن ادعای اول تنها بر خوش‌ساختی جانشانی تکیه دارد و $IDS(\pi) = IDS(\varphi)$ هیچ کجای آن وارد نمی‌شود.

هویت‌ها فرضِ جداگانه‌ای لازم ندارند: هویت شناسهٔ فرایندش را در مؤلفهٔ اول حمل می‌کند، پس اعضای با شناسه‌های متمایز مجموعه‌های هویت مجزا دارند. و برای خود هویت‌ها،

$$\begin{aligned} \text{IDs}(\rho[\pi/\varphi]) &= \text{IDs}(\rho \setminus \varphi) \cup \text{IDs}(\pi) \\ &= \text{IDs}(\rho \setminus \varphi) \cup \text{IDs}(\varphi) = \text{IDs}(\rho), \end{aligned}$$

که در گام میانی $\text{IDs}(\pi) = \text{IDs}(\varphi)$ و در گام آخر $\varphi \subseteq \rho$ را به کار می‌برد. $\square$

دو نیمهٔ این گزاره در زیر به دو شکل متفاوت به کار می‌روند. اینکه $\rho[\pi/\varphi]$ یک سیستم است می‌گذارد فصل 4 اصلاً از آن سخن بگوید، و تنها بر خوش‌ساختی تکیه دارد. اینکه هویت‌هایش همان‌های ρ است $\text{IDs}(\pi) = \text{IDs}(\varphi)$ را لازم دارد، و هیچ نتیجه‌ای در فصل 4 آن را مفروض نمی‌گیرد: مجاز بودن در آنجا برای زوجی از سیستم‌ها و از راه اجتماع مجموعه‌های هویتشان بیان می‌شود، و گزارهٔ 4.16 و قضیهٔ 4.20 رو می‌گویند که هیچ نسبتی میان آن دو به کار نرفته است. هر جا آن تساوی خواسته باشد، مسدودسازی فراهمش می‌کند.

تعریف 4.2 شناسه‌های فرایند را حل می‌کند و گزارهٔ 4.3 هویت‌ها را. هیچیک به وابستگی نمی‌پردازد، و وابستگی همان جایی است که جانشانی می‌تواند بد پیش برود: نام‌هایی که $\rho \setminus \varphi$ به آن‌ها تکیه دارد پس از آنکه φ جایش را به π بدهد هنوز اشغال‌شده‌اند، اما هر خواسته در اشغال‌گر تازه از نو سنجیده می‌شود. دو شرط این شکاف را می‌بندند.

به زبان غیررسمی، قضیهٔ 4.4 زیر می‌گوید جانشانی خوش‌ساخت wf را نگه می‌دارد، هرگاه درایه‌های π ورودی در جهان تازه برآورده‌شدنی باشند، و هر جا درایه‌ای از $\rho \setminus \varphi$ شاهدش را در درون φ می‌یافت، عضوی از π به جای φ بایستد. فرض (ii) همان معنای این است که π جانشین آمادهٔ φ باشد، و تقلید آن را فراهم نمی‌کند.

قضیه 4.4 (جانشانی خوش‌ساختی را نگه می‌دارد). بگذارید ρ و π سیستم باشند، $\varphi \subseteq \rho$ ، جانشانی φ با π در ρ به معنای تعریف 4.2 خوش‌ساخت باشد، و $\text{wf}(\rho)$ فرض کنید

- (i) هر درایهٔ هر $\mathcal{F} \in \pi$ شاهدی در $(\rho[\pi/\varphi])^+$ دارد، به معنای تعریف 1.3؛ و
 - (ii) هرگاه $\mathcal{F} \in \rho \setminus \varphi$ درایه‌ای $(\mathcal{F}', R) \in \mathcal{F}.\mathbf{U}$ داشته باشد که برخی شاهدش به معنای تعریف 1.3 در φ باشد، آن‌گاه برخی $\mathcal{G} \in \pi$ در $\mathcal{G}.F = \mathcal{F}'.F$ و $\mathcal{G}.s = \mathcal{F}.s \vee \text{global}(\mathcal{G})$ و $R(\mathcal{F}, \mathcal{G})$ صادق باشند.
- آن‌گاه $\text{wf}(\rho[\pi/\varphi])$

اثبات. بنویسیم $\rho' := \rho[\pi/\varphi] = (\rho \setminus \varphi) \cup \pi$ ، که بنا بر ادعای اول گزارهٔ 4.3 یک سیستم است – ادعای دوم اینجا در دست نیست، چون $\text{IDs}(\pi) = \text{IDs}(\varphi)$ فرض این قضیه نیست. $\mathcal{F} \in \rho'^+$ و درایهٔ $(\mathcal{F}', R) \in \mathcal{F}.\mathbf{U}$ را بگیرید؛ باید شاهدی در ρ'^+ بنشانیم. بر حسب اینکه $\mathcal{F}$ از کجا می‌آید، سه حالت هست.

اگر $\mathcal{F} \in \pi$ فرض (i) درست همان ادعاست.

اگر $\mathcal{F}$ یکی از $\mathcal{Z}, \mathcal{A}, \mathcal{C}$ باشد، درایه‌هایش یک‌بار برای همیشه و مستقل از سیستم حل شده‌اند: $\mathcal{C.U}$ تهی است، و درایه‌های $\mathcal{Z}$ و $\mathcal{A}$ نام یکدیگر را می‌برند، با $\mathcal{A}, \mathcal{Z} \in \rho^+$ و serves که بینشان برقرار است، چنان که فصل 1 ثبت می‌کند. هر سه جهانی‌اند، پس عطف نشست بخشیده می‌شود.

اگر $\mathcal{F} \in \rho \setminus \varphi$ ، آن‌گاه $\mathcal{F} \in \rho^+$ ، پس $wf(\rho)$ شاهدی $\mathcal{G} \in \rho^+$ برای این درایه فراهم می‌کند. یا $\mathcal{G} \notin \varphi$ که در آن صورت $\mathcal{G}$ در $\rho^+ \cup \{\mathcal{Z}, \mathcal{A}, \mathcal{C}\} \subseteq \rho^+$ است و دوباره خدمت می‌کند: همان ماشین است با همان میدان‌ها، پس $\mathcal{G}.s, \mathcal{G}.F$ و هر چه R می‌خواند تغییر نکرده، و $\mathcal{F}$ هم تغییر نکرده است. یا $\mathcal{G} \in \varphi$ فرض (ii) عضوی از π فراهم می‌کند که همان سه شرط را به‌جای او برمی‌آورد. در هر دو حالت درایه شاهدی در ρ^+ دارد. $\square$

فرض (ii) همان معنای این است که π جانشین آماده φ باشد، و تقلید آن را فراهم نمی‌کند. تعریف 4.11 رفتار را در مرز مقایسه می‌کند؛ در باب میدان‌های اعلام‌شده ماشین‌های پشت آن مرز هیچ نمی‌گوید، و π ای که φ را بی‌نقص تقلید کند می‌تواند باز هم در پذیرفتن فراخوانده‌ای که φ می‌پذیرفت، یا خدمت‌دادن به طرفی که φ خدمت می‌داد، شکست بخورد. این آزمون ارزان است – روی درایه‌هایی از $\rho \setminus \varphi$ دامنه دارد که به درون φ اشاره می‌کردند و هیچ چیز دیگر – اما آزمون است، و جایش کنار خوش‌ساختی جانشانی است و نه در درون قضیه ترکیب.

4.2 تقلید و جذب

هر چه اجرا لازم دارد اکنون تعیین شده است، پس می‌توانیم بگوییم یک سیستم کی دست‌کم به اندازه دیگری امن است. آن دو زیر یک و همان محیط نشانده می‌شوند، پس نخست باید تعیین شود که او چه چیزی را می‌تواند ادعا کند؛ و این نقش $\mathcal{Z}.par$ در زیر است.

مقدار $Exec(\pi, \mathcal{Z}, \mathcal{A}, \mathcal{C})$ هر چیزی است که محیط برمی‌گرداند. ما محیط را چنان می‌گیریم که در ریشه یک بیت برمی‌گرداند – حدسش در باب اینکه در کدام سیستم نشانده شده – و اجرا را چنان می‌خوانیم که دقیقاً وقتی 1 برمی‌گرداند که محیط برگرداند. هیچ چیز دیگری در چارچوب آن مقدار را مقید نمی‌کند، پس این شرطی است بر محیط‌هایی که روی آن‌ها سور می‌زنیم، و نه نتیجه‌ای. آن‌گاه شکاف میان دو اجرا یک عدد است، و نه یک شباهت غیررسمی.

تعریف 4.5 (مزیت UC). بگذارید π و φ سیستم باشند. برای محیط $\mathcal{Z}$ ، حریف $\mathcal{A}$ و شبیه‌ساز $\mathcal{S}$ ،

$$\text{Adv}_{\pi, \varphi}^{\text{UC}}(\mathcal{Z}, \mathcal{A}, \mathcal{S}) := \left| \Pr[\text{Exec}(\pi, \mathcal{Z}, \mathcal{A}, \mathcal{C}) = 1] - \Pr[\text{Exec}(\varphi, \mathcal{Z}, \mathcal{S}, \mathcal{C}) = 1] \right|,$$

که احتمال‌ها روی سکه‌های هر ماشین آن دو اجرا هستند، و $\text{Exec} = 1$ کوتاه‌نویسی پیشامدی است که اجرا با برگرداندن 1 از سوی محیط می‌ایستد.

آن دو اجرا در سیستم و در ماشینی که هویت‌های حریف را اشغال می‌کند تفاوت دارند: $\mathcal{A}$ در یک سو، $\mathcal{S}$ در سوی دیگر. هر دو با همان یک جفت تعریف نوع‌بندی می‌شوند.

تعریف 4.6 (حریف‌ها). یک حریف ماشینی است که $\text{IDs}(\mathcal{A})$ را اشغال می‌کند و هیچ هویت دیگری را، و واسطه‌های حریف را حمل می‌کند – پوشانه بخش 3.4، نگیهان و خموش‌کننده، و هر واسط دیگری که به ماشین آن بخش داده شده (فصل 9 یکی می‌افزاید) – به دور هسته دلخواهی در هر یک از آن‌ها. ماشین بخش 3.4 یکی از آن‌هاست؛ بدل بخش 4.4 دیگری.

تعریف 4.7 (شبیه‌سازها). یک شبیه‌ساز حریفی است که دست‌کم $\text{Std} \cup \mathbf{Z}$ را می‌پذیرد – شناسه‌های فرایند هر کارکرد استاندارد به همراه شناسه محیط.

پس هر شبیه‌سازی حریف است، و همین است که می‌گذارد یکی از آن‌ها در رده حریف بایستد: گزاره 4.12 و قضیه 4.24 شبیه‌سازها را به‌عنوان حریف می‌خوانند، و شمول‌های بودجه‌ای بخش 4.5 آن دو رده را مستقیم می‌سنجند. اشغال و فراخوانده‌های پذیرفته همان معنای ایستادن در جایگاه حریف‌اند، پس هر چه بخش 3.4 در باب ماشین نشسته در (A, P) حل می‌کند بی‌تغییر برای شبیه‌ساز هم برقرار است: هیچ فراخوانی از خودش آغاز نمی‌کند، و خموش‌کننده‌اش تنها می‌گذارد هویتی را ادعا کند که در آن می‌دود. گزاره 4.19 تنها اشغال هر چه به آن داده شود را می‌خواهد؛ قضیه 4.29 پوشانه و فراخوانده‌های پذیرفته را هم به کار می‌برد، و نکته 4.8 توضیح می‌دهد چرا هیچ‌یک محدودیت نیست.

متر آن سه ماشین را به‌عنوان آرگومان می‌گیرد، پس تقلید را می‌شود با سوزدن روی رده‌هایی از آن‌ها و کران‌گذاشتن بر نتیجه بیان کرد. اینکه اصلاً چه محیط‌هایی به کار می‌آیند را همان زوجی که مقایسه می‌شود حل می‌کند.

نکته 4.8 (تقلید در باب جایگاه حریف چه چیزی را تعیین می‌کند). تعریف 4.10 زیر $\mathbb{Z}.par$ را مقید می‌کند و در باب ماشین‌های جایگاه حریف هیچ نمی‌گوید؛ تعریف‌های 4.6 و 4.7 هر چه لازم است می‌گویند، و آن کمتر از پنج میدان است. میدان‌های pid و $\mathbf{P}$ با اشغال مفت می‌آیند: $IDs(\mathcal{F}) = \{(\mathcal{F}.pid, P) : P \in \mathcal{F}.P\}$ پس اشغال $IDs(A)$ هر دو را سنجاق می‌کند، و $\mathcal{S}.pid = (A, 0, 0)$ در سطر 1 و $global(\mathcal{S})$ در سطر 3 را می‌دهد؛ و سنجاق کردن $\mathbf{P}$ همان است که خواسته است، چون اشغال‌گر جایگاهی که به طرف‌های کمتری خدمت دهد یک هویت A را تنها در یک سو اشغال نشده می‌گذارد، که در آن سو rej پاسخ می‌گیرد و در سوی دیگر پاسخی واقعی. میدان par هرگز خوانده نمی‌شود – مجموعهٔ خموش شده همان $\{id'' : id'' \neq id\}$ عینی سطر 2 است، که پوشانهٔ لازم آن را به‌عنوان کد حمل می‌کند – و این تشریفات نیست: شبیه‌سازی که روی کمتری خموش شده باشد می‌توانست هویت یک طرف را ادعا کند، از دروازهٔ فساد $A.F = id'$ در سطر 3 بگذرد، و واسط طرف درستکاری را براند، توانی که حریفی که او به‌جایش می‌ایستد ندارد. میدان $\mathbf{U}$ بی‌اثر است و تنها wf می‌خواندش، و آنجا π^+ نام A رزرو را می‌برد و نه اشغال‌گر جایگاه را.

می‌ماند $\mathbf{N}$ ، و همین است که چرا تعریف 4.7 آن را رو می‌خواهد. فراخ‌ترکردن ناممکن است، چون $A.N$ از پیش هر شناسهٔ فرایندی را در بر دارد که فراخوانی می‌گذارد؛ و تنگ‌ترکردن هیچ برای شبیه‌ساز نمی‌آورد و هر دو قضیهٔ زیر را می‌گیرد. حذف **Std** هر قلاب واسطه‌گری در طرف فاسد را به همان $\perp$ محسوب بخش 3.6 بدل می‌کند – و قضیهٔ 4.20 روی هر p سور می‌زند، پس شبیه‌سازی که تنها شناسه‌های فرایند φ را بپذیرد در نخستین جهان بیرونی که شناسه‌های دیگری داشته باشد با آن‌ها روبه‌رو می‌شود. حذف $\mathbf{Z}$ محیط را تنها در سوی ایده‌آل بیرون می‌گذارد، و سطر 2 در جایگاه، درست همان دستورهایی را رد می‌کند که حریف واقعی پاسخشان می‌دهد. و هیچ چیز در هیچ یک از این‌ها از زوجی که برایش شبیه‌سازی می‌شود نام نمی‌برد، و همین است که می‌گذارد قضیهٔ 4.20 شبیه‌ساز فرض را بی‌تغییر تحویل دهد.

تعریف 4.9 (محیط‌ها). یک محیط ماشینی از اجراست که $IDs(\mathbb{Z})$ را اشغال می‌کند به همراه هویت‌های تعداد متناهی کارکرد استاندارد که میهمان می‌کند – برای ماشین بخش 3.3 هیچ، و برای محیط‌های جذب‌شدهٔ تعریف 4.15 زیر، رونوشت‌هایی از پروتکل پیرامون. این ماشین واسط بخش 3.3 را، با خموش‌کننده و پارامتر par ، در هر هویت $\mathbf{Z}$ حمل می‌کند، و واسط کامل خود هر کارکرد میهمان را در هویت‌های آن کارکرد؛ **Exec** در هر هویتی که او اشغال می‌کند به او توزیع می‌کند، و خروجی‌اش همان است که بخش $\mathbb{Z}$ او برمی‌گرداند.

تعریف 4.10 (محیط‌های مجاز). برای سیستم π ، بنویسیم $\langle \pi \rangle := \{id : id.pid.F = \mathcal{F}.F \text{ some for } \mathcal{F} \in \pi\}$ برای بستار نامی آن: هویت‌های هر شناسه فرایندی که نامی را حمل می‌کند که π به کار می‌برد، اشغال‌شده یا نه، در هر نشست و هر نمونه. برای سیستم‌های π و φ ، محیط‌های مجاز برای این زوج این‌هايند:

$$\mathbb{Z}_{\pi, \varphi} := \{ \mathbb{Z} : \mathbb{Z}.par \supseteq \langle \pi \rangle \cup \langle \varphi \rangle \text{ و } \mathbb{Z} \in IDs(\pi) \cup IDs(\varphi) \text{ در هیچ هویتی از } \mathbb{Z} \}.$$

شرط دوم بهداشت اشغال است: اجرا تنها برای اشغال‌گرانی با مجموعه‌های هویت دوه‌دو مجزا تعریف شده است (تعریف 3.3)، پس محیطی که در هویتی میهمان کند که یکی از سیستم‌ها اشغالش کرده نمی‌توانست کنار آن نشاندۀ شود، و مجازبودن آن را برای هر دو جهان یک‌جا ممنوع می‌کند.

شمول در زیر توجیه می‌شود. آن‌گاه یک حکم تقلید سه رده را تعیین می‌کند: رده $\mathbb{A}$ از حریف‌ها (تعریف 4.6)، رده $\mathbb{S}$ از شبیه‌سازها (تعریف 4.7)، و رده $\mathbb{Z} \subseteq \mathbb{Z}_{\pi, \varphi}$ از محیط‌ها (تعریف 4.9). گرفتن قوی‌ترین حکم از این سه را می‌دهد، و خوانش پیش‌فرض است.

تعریف 4.11 (تقلید UC). بگذارید π و φ سیستم باشند، $\varepsilon \geq 0$ ، و $\mathbb{Z} \subseteq \mathbb{Z}_{\pi, \varphi}$ می‌گوییم π کارکرد φ را در ε و در برابر $(\mathbb{A}, \mathbb{S}, \mathbb{Z})$ تقلید-UC می‌کند اگر

$$\forall \mathcal{A} \in \mathbb{A} \exists \mathcal{S} \in \mathbb{S} \forall \mathbb{Z} \in \mathbb{Z} : \quad Adv_{\pi, \varphi}^{uc}(\mathbb{Z}, \mathcal{A}, \mathcal{S}) \leq \varepsilon.$$

ترتیب سورها جان این تعریف است: شبیه‌ساز می‌تواند به حریف وابسته باشد اما به محیط نه، پس یک $\mathcal{S}$ باید یک‌جا در برابر هر $\mathbb{Z} \in \mathbb{Z}$ کار کند. جز این، آن دو اجرا یک آزمایش‌اند که یک سیستم در آن عوض شده، و $\mathbb{Z}$ عیناً همان یک ماشین در هر دو سو است. ضعیف‌کردنش به $\forall \mathcal{A} \forall \mathbb{Z} \exists \mathcal{S} -$ شبیه‌سازی که برای محیطی که با آن روبه‌روست ساخته شده - مفهومی می‌دهد که تعریف 4.11 آن را نتیجه می‌دهد و عکسش را ما اثبات نمی‌کنیم، و آن دو در صورت عینی بخش 4.5 به صورت $\max_{\mathcal{A}} \min_{\mathcal{S}} \max_{\mathbb{Z}}$ و $\max_{\mathcal{A}} \max_{\mathbb{Z}} \min_{\mathcal{S}}$ از یک کمیت خوانده می‌شوند. نتایج ترکیب از این ضعیف‌سازی جان سالم می‌برند، چون هر اثبات زیر فرض را در محیطی می‌نشانند که نتیجه تعیینش کرده؛ پس آنچه این ترتیب می‌خرد معناست و نه ترکیب، چون تنها زیر آن است که $(\varphi, \mathcal{S})$ یک سیستم است که مثل $(\pi, \mathcal{A})$ رفتار می‌کند، هر که تماشا کند - جهانی ایده‌آل که می‌شود نامش برد و تحویلش داد.

تعریف 4.10 همان است که این مقایسه را معنادار می‌کند. چون $\mathbb{Z}.par$ خود $\mathbb{Z}$ را خموش می‌کند، او هیچ هویتی از درون آن را نمی‌تواند ادعا کند، و سه دسته باید دور از دسترس بمانند. هویتی در $IDs(\pi) \triangle IDs(\varphi)$ در یک جهان اشغال است و در دیگری آزاد، پس به محیطی که اجازه یابد به‌عنوان آن سخن بگوید پیش از آنکه چیزی

رخ دهد گفته می‌شود در کدام جهان است، و هیچ شبیه‌سازی نمی‌تواند این را تعمیر کند. هویتی که درونی هر دو است چیزی را جدا نمی‌کند، اما محیطی که آن را ادعا کند به‌عنوان جزئی از سیستم سخن می‌گوید و نه از بیرون، و محیط برای این نیست. و هویت اشغال‌نشده یک نام به‌کاررفته به‌اندازه اشغال‌شده کارساز است، چون **Guard** فراخواننده‌ها را با نام می‌پذیرد: فراخوانده محلی هر نمونه از نام والدش را می‌پذیرد و سطر 2 بیش از این نمی‌خواهد، پس محیطی که نمونه تازه‌ای از آن نام را ادعا کند زیرروال‌های درونی را درست همان‌گونه می‌راند که فراخواننده‌هایشان – و در برابر زوجی مسدودشده (فصل 6) رو جهان‌ها را از هم تشخیص می‌دهد، چون مسدودگر فراخوانی را رد می‌کند که اشغال‌گر اصیل می‌پذیرد. خموش کردن بستر هر سه را یک‌جا ممنوع می‌کند. آنچه ادعاکردنی می‌ماند نام‌های تازه است، و آن‌ها تنها به چیزی می‌رسند که کل **Std** را می‌پذیرد: کارکردهای جهانی، که در هر مقایسه‌ای که خودشان محل نزاع نباشند بین دو جهان مشترکند.

خموش‌سازی حکم می‌کند که $\approx$ چه چیزی را می‌تواند ادعا کند، و فراخواندن کانال دومی است که به آن دست نمی‌زند. هویتی از $IDs(\pi) \triangle IDs(\varphi)$ در هر دو جهان می‌تواند فرا خوانده شود: آنجا که هیچ چیز اشغالش نکرده پاسخ **rej** است (بخش 3.6)، و آنجا که چیزی اشغالش کرده پاسخ باز هم **rej** است – به شرطی که آن چیز **Z** را در سطر 2 رد کند. این شرط، شرطی است بر سیستم‌هایی که مقایسه می‌شوند و نه بر رده محیط، و زوجی که آن را بشکند به‌سادگی تقلیدشدنی نیست، چون هیچ شبیه‌سازی نمی‌تواند پاسخی از خود بسازد که جهان ایده‌آل ماشینی برای دادنش ندارد. همین است که چرا زیرروال‌های محلی فصل 20 **Z** را از **N** خود بیرون نگه می‌دارند.

چون مجموعه‌های کوچک‌تر کم‌محدودکننده‌ترند، کم‌محدودترین محیط‌های مجاز آن‌هایی هستند که **par**شان درست همان بستر است – و مسدودسازی فصل 6 آن‌ها را برای زوجی سازگار تحویل می‌دهد:

$$IDs(\bar{\pi}) = IDs(\bar{\varphi}) = IDs(\pi) \cup IDs(\varphi)$$

بنا بر ادعای دوم قضیه 6.3، و مسدودگر نامش را از عضوی که به‌جایش می‌ایستند رونویسی می‌کند، پس

$$\langle \bar{\pi} \rangle = \langle \bar{\varphi} \rangle = \langle \pi \rangle \cup \langle \varphi \rangle.$$

پس $\bar{\pi} := \langle \bar{\pi} \rangle$ تنها مقداری نیست که آن دو سیستم مسدودشده می‌توانند مشترک داشته باشند، بلکه ضعیف‌ترین محدودیتی است که آن زوج می‌پذیرد.¹

یک ویژگی این رابطه از پیش در دست است، و از گونه‌ای دیگر از هر چیز پس از آن است. نتایج ترکیب زیر – جانشانی، ترکیب موازی، تمامیت حریف بدل – نتیجه‌های یک لم جذب یگانه‌اند، که هر یک یک اجرا را بازگروه‌بندی می‌کند و همان یک مزیت را دو بار می‌خواند. تریایی هیچ بازگروه‌بندی نمی‌کند. اثباتش نامساوی مثلثی روی تعریف 4.5

¹سوزدن روی همه محیط‌ها بی‌اعتنا به **par** سازگار است اما برای به‌کارآمدن زیادی قوی است: شبیه‌ساز ناچار بود ترافیک درونی φ را فراخوان به‌فراخوان بازتولید کند. اگر می‌گذاشتیم شبیه‌ساز در هویت‌هایی که φ اشغال نمی‌کند کارکرد بزیاناند، این شرط شل می‌شد و تنها $IDs(\varphi) \setminus IDs(\pi)$ برای دریغ‌کردن می‌ماند – که برای جفت‌شدن معمول تهی است؛ و اینکه زایاندن در ترکیب چه هزینه‌ای دارد را به کار آینده وامی‌گذاریم.

است، به همراه این آزمون که ردهٔ نتیجه برای زوج بیرونی مجاز است، پس به متر تعلق دارد و نه به ماشین‌آلات، و به هیچ‌یک از دستگاه‌های باقی این بخش نیازی ندارد. به همین سبب همین‌جا بیان می‌شود.

گزاره 4.12 (تراپایی). بگذارید π, φ و ψ سیستم باشند و $Z \subseteq Z_{\pi, \varphi} \cap Z_{\varphi, \psi}$. اگر π کارکرد φ را در ε_1 و در برابر $(\mathbb{A}, \mathbb{S}, \mathbb{Z})$ تقلید-UC کند و φ کارکرد ψ را در ε_2 و در برابر $(\mathbb{S}, \mathbb{S}', \mathbb{Z})$ ، آنگاه π کارکرد ψ را در $\varepsilon_1 + \varepsilon_2$ و در برابر $(\mathbb{A}, \mathbb{S}', \mathbb{Z})$ تقلید-UC می‌کند.

اثبات. اول، نتیجه حکم مجازی از تقلید است. $\mathbb{Z}$ ای که در هر دو رده باشد $\mathbb{Z}.par \supseteq \langle \varphi \rangle \cup \langle \psi \rangle$ و $\langle \pi \rangle \cup \langle \varphi \rangle$ دارد، پس $\mathbb{Z}.par \supseteq \langle \pi \rangle \cup \langle \psi \rangle$ ؛ و در هیچ هویتی از $IDs(\varphi) \cup IDs(\psi)$ و نه از $IDs(\pi) \cup IDs(\varphi)$ میهمان نمی‌کند، پس در هیچ هویتی از $IDs(\pi) \cup IDs(\psi)$ هم نه – پس هر دو بند تعریف 4.10 برقرارند و $Z \subseteq Z_{\pi, \psi}$ و برای این، آن سه سیستم لازم نیست نسبتی با هم داشته باشند. اکنون بگذارید $\mathcal{A} \in \mathbb{A}$. تقلید اول $\mathbb{S} \in \mathbb{S}$ را فراهم می‌کند، و چون تقلید دوم حریف‌هایش را روی $\mathbb{S}$ سور می‌زند، خواندن $\mathbb{S}$ به‌عنوان حریفی در برابر φ به $\mathbb{S}' \in \mathbb{S}'$ می‌رسد. برای هر $\mathbb{Z} \in \mathbb{Z}$ ، نامساوی مثلثی می‌دهد

$$\begin{aligned} \text{Adv}_{\pi, \psi}^{\text{uc}}(\mathbb{Z}, \mathcal{A}, \mathbb{S}') &\leq \text{Adv}_{\pi, \varphi}^{\text{uc}}(\mathbb{Z}, \mathcal{A}, \mathbb{S}) + \text{Adv}_{\varphi, \psi}^{\text{uc}}(\mathbb{Z}, \mathbb{S}, \mathbb{S}') \\ &\leq \varepsilon_1 + \varepsilon_2, \end{aligned}$$

چون جملهٔ میانی $\Pr[\text{Exec}(\varphi, \mathbb{Z}, \mathbb{S}, \mathcal{C}) = 1] = 1$ حذف می‌شود. و چون $\mathbb{S}'$ تنها به $\mathcal{A}$ وابسته است، همان شبیه‌ساز لازم است. $\square$

سه ساخت زیر – محیط جذب‌شدهٔ اینجا، و آن دوی بخش 4.4 – چند ماشین را در یک اشغال‌گر اجرا تا می‌کنند. آن‌ها یک شکل مشترک دارند، که یک بار نامش می‌بریم.

تعریف 4.13 (دسته). یک دسته چند ماشین را با هم به‌عنوان یک اشغال‌گر یگانهٔ اجرا می‌دواند، و ژتون را بینشان همان‌گونه که **Exec** می‌کند دست‌به‌دست می‌کند و هر یک را همان‌گونه که **Exec** می‌کرد مقدار آغازین می‌دهد. دسته مجموعهٔ اعلام‌شده‌ای از هویت‌ها را اشغال می‌کند، که هویت‌های خود اعضا باید بپوشانندش، و خروجی و پارامترهایش را از عضوی معین می‌گیرد. فراخوان میان دو ماشین میهمان در درون خدمت می‌شود، با مسیریابی خود دسته و بی‌آنکه روی اجرا گذاشته شود؛ فراخوانی که از بیرون می‌رسد به‌دست عضوی که هویت هدف را حمل می‌کند خدمت می‌شود، از راه واسطهٔ کامل همان عضو، با نگرهبان و همه چیز؛ و فراخوانی که ماشین میهمان به بیرون می‌گذارد، هرگاه هدفش در مجموعهٔ اعلام‌شدهٔ بیرونی باشد روی اجرا گذاشته می‌شود و در غیر این صورت **rej** پاسخ می‌گیرد. یک نمونه می‌تواند علاوه بر این، فراخوان‌های خاصی را میان اعضایش به‌دست مسیریابی کند – یکی را پیش از دیگری بنشانند، حتی وقتی هدف

بیرون هویت‌های هر عضوی باشد – و چنین مسیریابی صریحی بر فراخوان‌هایی که نامشان می‌برد حکم می‌کند، و تنها باقی را به قاعده هدف بالا وامی‌گذارد. اینکه فراخوان بیرونی ماشین میهمان زیر چه ادعایی می‌رود بر اشغال می‌گردد، چون خموش‌کننده هسته یک واسط را می‌پوشاند و فراتر نمی‌رود (فصل 2). عضوی که دسته هویت‌هایش را اشغال می‌کند فراخوان‌هایش را زیر هویت خودش می‌فرستد و تنها با خموش‌کننده خودش روبه‌رو می‌شود؛ عضوی که دسته هویت‌هایش را اشغال نمی‌کند هیچ هویتی برای ادعا ندارد، پس دسته هر یک از فراخوان‌های بیرونی‌اش را بازنویسی می‌کند تا زیر هویتی برود که خودش داردش – و اینکه کدام هویت، بخشی از همان چیزی است که نمونه اعلامش می‌کند، در کنار مجموعه بیرونی‌اش – و ادعای بازنویسی‌شده با خموش‌کننده آن واسط روبه‌رو می‌شود. اینکه عضوی در کدام حالت است، همان است که دسته را به اجرایی که به جایش می‌ایستد وفادار نگه می‌دارد.

اینکه ترافیک درونی خدمت می‌شود و نه گذاشته، همان است که می‌گذارد دسته جایگاه حریف سیستمی را اشغال کند که هسته‌هایش او را پاسخگوینه فرا می‌خوانند، و ارزش گفتن دارد که چرا این تمایز موی دماغ نیست. پوشانه **Respond** (تعریف 9.1) هر فراخوانی را که پاسخ‌دهنده می‌گذارد رد می‌کند، پس دسته‌ای که ناچار بود فراخوان‌های درونی‌اش را بگذارد در حال پاسخ‌دادن به هیچ‌یک از اعضای خودش نمی‌رسید – و شبیه‌ساز فصل 20، که در درون هر یک از دو پاسخ پاسخگو سه دورگه میهمان را می‌راند، اصلاً نمی‌توانست بدود. خدمت‌کردن آن‌ها در درون، ژتون را همان جا که بود می‌گذارد، و این کل چیزی است که پاسخگویی می‌خواهد. آنچه دسته در درون یک پاسخ پاسخگو باز هم نمی‌تواند بکند، رسیدن به چیزی است که میهمانش نکرده: دفتر و کارکردهای مشترک بیرون اویند، پس فراخوان‌هایشان گذاشته می‌شود، و رد می‌شود.

تعریف 4.13 پیش‌تر می‌گویند ماشین میهمان دو گونه دارد، بر حسب اینکه دسته هویت‌هایش را اشغال می‌کند یا نه، و همین تفاوت کل آنچه در پی می‌آید است. ماشینی که دسته هویت‌هایش را نگه می‌دارد از توزیع اجرا محو می‌شود و در درون خدمت می‌شود؛ و ماشینی که دسته هویت‌هایش را واگذار می‌کند باید آن‌ها را به کسی بسپارد، و فراخوان‌هایش را باید همان کس به نیابت او بگذارد. هر دو حرکت پیش می‌آید – اولی پروتکلی را جذب می‌کند و دومی حریفی را – پس با هم تعریفشان می‌کنیم و یک بار درستی‌شان را اثبات می‌کنیم.

تعریف 4.14 (جذب). بگذارید $\mathcal{Z}$ محیطی باشد و H مجموعه متناهی‌ای از ماشین‌های یک اجرا، که به $H = H_{\text{keep}} \uplus H_{\text{cede}}$ شقه می‌شود و اعضای H_{cede} اشغال‌گران جایگاه حریفاند. جذب H در $\mathcal{Z}$ همان دسته (تعریف 4.13) $\mathcal{Z}^H$ را از $\mathcal{Z}$ با یک رونوشت از هر عضو H است؛ این دسته $\text{IDs}(\mathcal{Z}) \cup \text{IDs}(H_{\text{keep}})$ را اشغال می‌کند، خروجی و par را از $\mathcal{Z}$ می‌گیرد، و مجموعه بیرونی‌اش همان هویت‌هایی است که اجرای پیرامون هنوز خدمتشان می‌کند. عضوی از H_{keep} هویت‌هایش را

نگه می‌دارد، پس فراخوان‌هایش زیر ادعای خودش می‌روند و فراخوان‌هایی که به او خطاب شوند در درون دسته خدمت می‌شوند. عضوی از H_{cede} هیچ هویتی نگه نمی‌دارد، پس دسته هر فراخوان بیرونی $FullOp(x)$ را که او در حال دویدن در (A, P) می‌گذارد به این دستور بازنویسی می‌کند

$$(A, P).FullA(id''.Op, x) \text{ as } (Z, P),$$

و هویت‌هایی که او واگذار می‌کند در اجرای تازه به‌دست یک بازپخش برای او اشغال می‌شوند، که سرتاسر همان بدل $\mathcal{D}$ از بخش 4.4 است.

ما حالت واگذاری را تنها برای جایگاه حریف بیان می‌کنیم. تنها همین یکی در زیر به کار می‌رود، و تنها همین یکی است که واسطش شکل دستوری دارد که فراخوان‌ها را به آن بازنویسی کنیم – کارکرد استاندارد آنچه کد خودش نامش می‌برد را خدمت می‌کند و نمی‌توانست به‌جای فراخوان‌های ماشین دیگری بایستد.

تعریف 4.15 (محیط جذب‌شده). بگذارید $\rho \subseteq \varphi$ ، و π سیستمی باشد که به‌جای φ نشانده می‌شود، و $\mathcal{Z}$ محیطی. محیط جذب‌شده $\mathcal{Z}^{\rho \setminus \varphi}$ همان جذب (تعریف 4.14) $\rho \setminus \varphi$ در $\mathcal{Z}$ است با هر عضو نگه‌داشته و هیچ عضو واگذارشده، پس دسته $\mathcal{Z}$ است با یک رونوشت از هر $\mathcal{F} \in \rho \setminus \varphi$ ، که هر رونوشت هویت‌هایش را نگه می‌دارد؛ این دسته $IDs(\mathcal{Z}) \cup IDs(\rho \setminus \varphi)$ را اشغال می‌کند، خروجی و par را از $\mathcal{Z}$ می‌گیرد، و مجموعه بیرونی‌اش $IDs(\pi) \cup IDs(\varphi) \cup \{id : id.F \in \{A, C\}\}$ است. این یک محیط است (تعریف 4.9) و نه یک کارکرد، چون روی چند شناسه فرایند گسترده است. بالانویس تنها پوسته میهمان‌شده را ثبت می‌کند – مجموعه بیرونی π را هم می‌خواند – پس $\mathcal{Z}^{\rho \setminus \varphi}$ نسبت به جانشانی‌ای که بافت می‌دهد تعیین می‌شود، و دو جانشانی که $\rho \setminus \varphi$ مشترک دارند اما در π متفاوتند نام دو ماشین متفاوتند.

هر دو نیمه مجموعه بیرونی را همان چیزی وادار می‌کند که اجرای جذب‌شده باید بازتولیدش کند. اولی جایگاه است، که هویت‌هایش بر حسب اینکه محیط جذب‌شده در کدام جهان نشسته به π یا به φ می‌رسند. ما اجتماع را می‌گیریم و نه تنها $IDs(\varphi)$ را، تا همان یک ماشین در هر دو خدمت کند، و این حالا که آن دو لازم نیست همان هویت‌ها را اشغال کنند مهم است. دومی آن دو شناسه رزرو A و C است، که به‌عنوان شناسه نامیده شده‌اند و نه به‌عنوان ماشین، چون جایگاه حریف در سوی ایده‌آل $\mathcal{S}$ را در بر دارد و گزاره 4.19 در هر دو سو یک و همان ماشین جذب‌شده را لازم دارد. آن‌ها آنچنانچند چون ماشین‌های میهمان واسط‌های کامل‌اند و نه هسته‌های خالی: هر یک دفتر را در سطرهای 2 و 6 بخش 3.5 می‌خواند، و هر یک فراخوانی در طرف فاسد را از راه **Mediate** به حریف می‌سپارد، که به $(A, id.P)$ قلاب می‌شود. هیچ‌یک از این دو شناسه به هیچ سیستمی تعلق ندارد. بی این نیمه، فراخوان به کارکردی میهمان در نخستین سطرش رد می‌شد، و قلاب طرف فاسد به‌جای رسیدن به حریف رد می‌شد:

اجرای جذب شده درست همان جا که C به $\varphi \setminus \rho$ می‌رسید از بازگروه‌بندی اجرای بیرونی بودن دست می‌کشید. هر چیز دیگر با rej پاسخ می‌گیرد، چون در اجرای بیرونی هم چیزی خدمتش نمی‌کرد.

پارامتر دست‌نخورده منتقل می‌شود، و باید چنین باشد. آن مجموعهٔ خاموش‌شدهٔ واسط خود $\mathcal{Z}$ است، پس هر تغییری در آن تغییر می‌دهد که $\mathcal{Z}$ چه فراخوان‌هایی می‌تواند بگذارد - و $\mathcal{Z}$ همان یک ماشینی است که باید در هر دو سو یکسان رفتار کند. کسی ممکن است گمان کند کارکردهای میهمان نیاز دارند $IDs(\rho \setminus \varphi)$ از آن حذف شود تا بتوانند به‌عنوان خودشان سخن بگویند، اما ندارند: هر یک واسطی کامل است، و سطح 5 از پوشاندهٔ خودش همان است که ادعایش را مجاز می‌کند، و پوشاندهٔ $\mathcal{Z}$ فراتر از هستهٔ خود $\mathcal{Z}$ نمی‌رسد (تعریف 4.13). حذف آن هویت‌ها هیچ چیز تازه‌ای برایشان مجاز نمی‌کرد، و $\mathcal{Z}$ را درست روی همان هویت‌هایی خاموش می‌کرد که پیش از جذب رویشان خاموش بود.

پس جذب، محیطی برای زوج بیرونی را به محیطی برای زوج درونی بدل می‌کند و مجازبودن را نگه می‌دارد. دفتر رفتار ویژه‌ای لازم ندارد: چون عضو هیچ سیستمی نیست، نه در φ است و نه در $\rho \setminus \varphi$ ، پس محیط جذب‌شده میهمانش نمی‌کند و مسدودسازی هیچ مسدودگری برایش نمی‌سازد. مثل $\mathcal{Z}$ و A ، $Exec$ فراهمش می‌کند، و همان یک ماشین واحد در هر دو سوی هر مقایسه‌ای است. رونوشت‌های میهمان آن هویت‌هایشان را نگه می‌دارند، پس بنا بر تعریف 4.13 فراخوان‌هایشان را زیر ادعاهای خودشان می‌فرستند و با خاموش‌کننده‌های خودشان روبه‌رو می‌شوند - نه با خاموش‌کنندهٔ $\mathcal{Z}$ - و همین است که بازگروه‌بندی زیر لازم دارد؛ و ساخت‌های بخش 4.4 حالت مقابلند، چون حریفی را میهمان می‌کنند که دسته هویت‌هایش را اشغال نمی‌کند.

گزاره 4.16 (جذب مجازبودن را نگه می‌دارد). بگذارید ρ و π سیستم باشند، $\varphi \subseteq \rho$ ، و جانشانی φ با π در ρ خوش‌ساخت باشد. آنگاه

$$\mathcal{Z} \in \mathbb{Z}_{\rho[\pi/\varphi], \rho} \implies \mathcal{Z}^{\rho \setminus \varphi} \in \mathbb{Z}_{\pi, \varphi}.$$

اثبات. برای نیمهٔ par ، جذب این میدان را دست‌نخورده منتقل می‌کند، پس همین بس است که

$$\langle \rho[\pi/\varphi] \rangle \cup \langle \rho \rangle \supseteq \langle \pi \rangle \cup \langle \varphi \rangle.$$

هر نامی که φ به کار می‌برد ρ هم به کار می‌برد، چون $\varphi \subseteq \rho$ ، و هر نامی که π به کار می‌برد $\rho[\pi/\varphi]$ هم، چون $\pi \subseteq \rho[\pi/\varphi]$. برای نیمهٔ میهمانی، $\mathcal{Z}^{\rho \setminus \varphi}$ فراتر از هر چه $\mathcal{Z}$ میهمان کرده بود، در $IDs(\rho \setminus \varphi)$ میهمان می‌کند؛ و آن دومی از $IDs(\rho[\pi/\varphi]) \cup IDs(\rho)$ می‌پرهیزد، که $IDs(\pi) \cup IDs(\varphi)$ را در بر دارد؛ و $IDs(\rho \setminus \varphi)$ هیچ کجا با $IDs(\varphi)$ برخورد نمی‌کند، چون آن دو زیرسیستم‌های مجزای ρ اند، و هیچ کجا با $IDs(\pi)$ برخورد نمی‌کند، بنا بر خوش‌ساختی جانشانی - تنها جایی که این فرض به کار می‌رود، و تنها برای نوع‌بندی به کار می‌رود. هیچ نسبتی میان $IDs(\pi)$ و $IDs(\varphi)$ وارد نمی‌شود. یک چیز که این حکم نمی‌گویدش: ماشین جذب‌شده هویت‌هایی را در درون par خودش

ادعا می‌کند، و هر رونوشت میهمان هویت خودش را. این تناقضی نیست – بنا بر تعریف 4.13 ادعاهای رونوشت میهمان با خموش‌کننده خودش روبه‌رو می‌شوند و هرگز با خموش‌کننده محیط، و مجاز بودن شرطی است بر آن میدان، که هر جا آن میدانِ اعمال شود خوانده می‌شود. □

آنچه این قضیه را کارآمد می‌کند این است که جذب یک اجرا را بازگروه‌بندی می‌کند بی‌آنکه تغییرش دهد. و چون همان تناظر به کار بخش 4.4 هم می‌آید، پیش از لم آن را می‌نشانیم.

پیکربندی‌ها. مقصودمان از پیکربندی یک اجرا حالت آن بین فعال‌سازی‌هاست: حالت هر ماشین، بخش ناخوانده هر نوار سکه، اینکه کدام ماشین ژتون را در دست دارد و کجای کد خودش است، و فراخوانده‌هایی که معلق و چشم‌به‌راه پاسخند. فراخوان‌ها تودرتو می‌شوند، و بخش 3.6 می‌گذارد یک نمونه چند ادامه معلق داشته باشد، پس آخری از این‌ها یک زنجیره است و نه یک مجموعه: یک درایه به‌ازای هر فراخوانی که گذاشته شده و هنوز پاسخ نگرفته، به همان ترتیبی که گذاشته شده، که فراخوانده و فراخوانی که چشم‌به‌راهش است را ثبت می‌کند.

آن دو اجرا همان ماشین‌ها را نمی‌دانند – کار جذب همین است – اما وقتی $\mathcal{H}$ را به‌جای یکی، همان ماشین‌هایی بشماریم که دسته‌شان می‌کند، همان‌ها را می‌دانند، جز بازپخش‌هایی که هویت واگذار شده لازم دارد. تعریف 4.13 ژتون را بین آن‌ها همان‌گونه که Exec می‌کند دست‌به‌دست می‌کند، پس هر یک در هر دو سو به همان معنا آن را در دست دارد. با این شمارش، ماشین‌های آن دو اجرا جز بازپخش‌ها یک‌به‌یک متناظرند، و ما ι را برای این دوسویی می‌نویسیم: رونوشت میهمان هر عضو H به همان عضو، $\mathcal{Z}$ میهمان به $\mathcal{Z}$ ، و هر چه اجرا هنوز می‌داندش، $\mathcal{C}$ هم در آن، به خودش.

تعریف 4.17 (تناظر بازگروه‌بندی). پیکربندی $\mathcal{C}$ از اجرای چپ و پیکربندی $\hat{\mathcal{C}}$ از اجرای راست متناظرند، و می‌نویسیم $\hat{\mathcal{C}} \approx \mathcal{C}$ ، هرگاه

(C1) هر ماشین $\mathcal{C}$ و تصویرش زیر ι در همان حالت باشند – $\mathcal{C}$ هم در آن، پس آن دو بر $\mathcal{C}$ توافق دارند؛

(C2) هر ماشین و تصویرش همان نوار سکه ناخوانده را داشته باشند؛

(C3) همان ماشین در هر دو، زیر ι ، ژتون را در دست داشته باشد، در همان نقطه از همان عمل و با همان مقدارهای محلی؛ و

(C4) دو زنجیره فراخوانده معلق زیر ι توافق داشته باشند: طول برابر، و به‌ازای هر k ، درایه‌های k نام ماشین‌هایی را ببرند که ι یکی‌شان می‌کند و چشم‌به‌راه همان فراخوان باشند – همان هویت هدف، همان عمل، همان ورودی و همان هویت ادعایی.

هر جا خانواده راست بازپخی حمل کند، ι دوسویی‌ای است میان ماشین‌های چپ و ماشین‌های راست جز بازپخش، و (C4) با حذف قاب‌های خود بازپخش خوانده

می‌شود: فراخوان بازپخش شده یک قاب می‌افزاید که چپ ندارد، و دقیقاً یک بار با همان پاسخی که فراخوان چپ برگرداند از سر گرفته می‌شود – جز آنکه alrej که چپ می‌خواند می‌تواند در راست به صورت $\perp$ برسد، و این تنها جایی است که بی‌اعتنایی به رد خرج می‌شود.

تنها (C4) از این کمانک‌گذاری در خطر است. در چپ، هر تعلیقی را **Exec** ثبت می‌کند؛ در راست، تعلیق‌ها بین **Exec** و دسته تقسیم می‌شوند، و دسته آن‌هایی را ثبت می‌کند که به فراخوان‌های درونی خودش مربوطند. این بند می‌خواهد که آن دو زنجیره، وقتی زنجیره‌های خود دسته همان جا که گذاشته شده‌اند شمرده شوند، توافق داشته باشند. بازگروه‌بندی درست همین است: یک زنجیره تعلیق، که دو توزیع‌گر نگهش می‌دارند.

نخستین فعال‌سازی. آن دو اجرا در تناظر آغاز می‌کنند، و ارزش گفتن دارد که چرا، چون این همان یک فعال‌سازی است که تحلیل حالت لم به آن نمی‌رسد: هیچ ماشینی نگذاشته است. سطر 1^c و هر چه اجرا در هر دو سو هنوز می‌دواندش را مقدار آغازین می‌دهد، تعریف 4.14 هر رونوشت مبهمان را همان‌گونه که **Exec** می‌کرد مقدار آغازین می‌دهد، و بازپخش هیچ حالتی حمل نمی‌کند، پس هر ماشین و تصویرش در همان حالت آغاز می‌کنند، که (C1) است. هیچ نواری خوانده نشده، که (C2) را می‌دهد. عمل Initialize هیچ فراخوانی نمی‌گذارد، پس هیچ چیز معلق نیست و هر دو زنجیره تهی‌اند، که (C4) را می‌دهد – و با آن، دلیل اینکه ترتیبی که دسته رونوشت‌هایش را مقدار آغازین می‌دهد مشاهده‌شدنی نیست: آن ترتیب را از هیچ چیز پیکربندی نمی‌شود خواند. سپس سطر 2^{FullZ()} از (Z, Z) را فعال می‌کند. در چپ ژتون به $\mathbb{Z}$ می‌رود؛ در راست به $\mathbb{Z}^H$ ، که در آن (Z, Z) را $\mathbb{Z}$ مبهمان حمل می‌کند، همان که تعریف 4.14 دسته را وادار به اشغالش می‌کند. هر دو در همان هویت و روی همان ورودی به **FullZ** وارد می‌شوند، که (C3) است.

از آن پس هر فعال‌سازی یا فراخوانی است که گذاشته می‌شود یا مقداری که برگردانده می‌شود، و لم به این می‌رسد که نشان دهد تناظر از یکی از آن‌ها در هر دو سو جان سالم می‌برد، و پیکربندی‌های متناظر با هم و با همان خروجی می‌ایستند. بندهایش آن فعال‌سازی‌ها را حالت‌به‌حالت می‌گیرند.

لم 4.18 (جذب). بگذارید اجرایی اشغال‌گران $\mathbb{Z} \cup H \cup R$ را بدواند، با $\mathbb{C} \in R$ و $\mathbb{Z}$ محیط، و $H = H_{\text{keep}} \uplus H_{\text{cede}}$ ، و $\mathbb{Z}^H$ همان جذب تعریف 4.14 باشد. از هر عضو H_{cede} ، و از هیچ چیز H_{keep} ، بخواهید که

(آ) بی‌اعتنا به رد باشد (تعریف 4.28)؛

(ب) هیچ هسته R فراخوان پاسخگویی روی هویت‌هایی که او واگذار می‌کند نگذارد (تعریف 9.2)؛ و

(ج) $\mathbb{Z}.\text{par}$ هیچ هویت Z در بر نداشته باشد.

آن‌گاه، به شرط آنکه آن دو خانواده به معنای تعریف 3.3 خانواده‌های مجاز اشغال‌گر باشند،

$$\text{Exec}(\{Z\} \cup H \cup R) \quad \text{and}$$

$$\text{Exec}(\{Z^H\} \cup R \cup \{\text{identity ceded each at relay a}\})$$

همسان‌توزیعند – به‌عنوان توزیع برابرند، و نه تنها نزدیک.

اثبات. این استدلال یک استقرار روی فعال‌سازی‌هاست، و پنج بند زیر گامش را انجام می‌دهند. ماشین‌ها دو خانواده اشغال‌گر را جفت می‌کند و می‌سنجد که هر ماشین و تصویرش یکسان آغاز می‌کنند. مسیریابی می‌پرسد هر فراخوان کجا فرود می‌آید و چهار حالت می‌یابد – درونی گره، خروج از آن، ورود در هویتی نگه‌داشته، ورود در هویتی واگذارشده – که از آن‌ها تنها آخری میان دو خانواده تفاوت دارد. ادعاها بر حسب اینکه چه گونه‌ای از عضو فراخوان را گذاشته شده می‌شود: عضوی که هویت‌هایش را نگه می‌دارد در هر دو سو با همان خموش‌کننده‌ها روبه‌رو می‌شود، و عضو واگذارنده هیچ هویتی برای ادعا ندارد، پس فراخوان‌های بیرونی‌اش بازنویسی می‌شوند، و این بازنویسی باید از سه آزمون جان سالم ببرد – دسته از کجا می‌تواند سخن بگوید، آیا آن ادعا خموش شده، و آیا از نگهبان بازپخش می‌گذرد. ردها آن یک مقداری را جدا می‌کند که می‌تواند متفاوت باشد، یعنی rej ای که چپ می‌خواند و در راست به‌صورت $\perp$ می‌رسد، و فرض (a) را رویش خرج می‌کند؛ و فرض (b) مسئله آینه‌ای همان را سد می‌کند، یعنی فراخوان پاسخگو روی جایگاهی بازپخش شده. ژتون می‌سنجد که آن دو زنجیره تعلیق، وقتی زنجیره‌های خود دسته همان جا که گذاشته شده‌اند شمرده شوند، توافق دارند. فرض (c) در ادعاها خرج می‌شود، بر حق عضو واگذارنده برای ادعای (Z, P) . دو قرارداد از فصل‌های 1 و 3.6 در سرتاسر به کار می‌روند و ارزش بازگفتن دارند. ماشین‌های متمایز از سکه‌های مستقل می‌کشند، و دسته‌کردن ماشین‌ها در یکی نوارهایشان را یکی نمی‌کند، پس تصادف رونوشت میهمان همان است که پیش از جذب بود. و Initialize هیچ فراخوانی نمی‌گذارد، پس ترتیبی که دسته رونوشت‌های میهمانش را مقدار آغازین می‌دهد مشاهده‌ناپذیر است. اثبات استقرایی است روی فعال‌سازی‌ها: پیکربندی‌های آغازین متناظرند، چنان که پیش از لم نشان داده شد؛ پیکربندی‌های متناظر گام می‌زنند، و هر سو همان فعال‌سازی بعدی را تولید می‌کنند و به پیکربندی‌های متناظر گام می‌زنند، و هر ماشین و تصویرش همان سکه‌ها را به همان ترتیب می‌خوانند، که همان است که $(C2)$ تعریف 4.17 نگهش می‌دارد؛ و پیکربندی‌های ایستای متناظر یکسان خروجی می‌دهند، چون خروجی مال Z میهمان است، بنا بر تعریف 4.14. بندهای زیر گام استقرار را انجام می‌دهند: چه ماشین‌هایی هستند و یکسان آغاز می‌کنند؛ $(C1)$ فراخوان کجا می‌رود و چه ادعا می‌کند؛ $(C4)$ و ژتون چگونه حرکت می‌کند، $(C3)$ دو بند اول بین هر دو گونه عضو مشترکند؛ و آن دو بند پس از آن‌ها همان چیزی هستند که عضو واگذارنده هزینه می‌کند.

ماشین‌ها. چپ $\{Z\} \cup H \cup R$ را می‌دواند؛ راست $\{Z^H\} \cup R$ را به همراه یک بازپخش در هر هویت واگذارشده، و Z^H همان Z است با یک رونوشت از هر عضو H . اگر به‌جای یکی، همان ماشین‌هایی شمرده شود که دسته‌شان می‌کند، آن دو گردایه جز بازپخش‌ها

بر هم می‌افتند، پس t هر عضو H را با رونوشتش جفت می‌کند، هر عضو R و C را با خودش، و بازپخش‌ها را باقی می‌گذارد، که همان خوانشی است که تعریف 4.17 به آن‌ها می‌دهد. سطر 1 R را در هر دو سو مقدار آغازین می‌دهد، تعریف 4.14 رونوشت‌های میهمان را همان‌گونه که $Exec$ می‌کرد، و بازپخش بی‌حالت است، پس هر ماشین و تصویرش یکسان آغاز می‌کنند. اشغال بنا بر فرض در هر دو سو مجزاست، پس هر فراخوان هدفی یگانه دارد: در راست، دسته $IDS(\mathbb{Z}) \cup IDS(H_{keep})$ را در بر دارد و بازپخش‌ها هر چه H_{cede} رها کرده است.

مسیریابی. فراخوانی را بگیرد و بیرسید کجا فرود می‌آید. با هر دو سر آن در $\{Z\} \cup H$ ، در چپ $Exec$ توزیعش می‌کند و در راست در درون $\mathbb{Z}^H$ خدمت می‌شود، و در هر دو حال به‌دست همان ماشین. اگر از آن گروه بیرون رود، در چپ $Exec$ توزیعش می‌کند و در راست به بیرون گذاشته می‌شود، چون هدفش در مجموعه بیرونی است، که تعریف 4.14 آن را همان هویت‌هایی می‌کند که اجرا هنوز خدمتشان می‌کند. اگر در هویتی از H_{keep} وارد گروه شود، در چپ $Exec$ توزیعش می‌کند و در راست رونوشت میهمانی که آن هویت را حمل می‌کند خدمتش می‌کند، همان که دسته اشغالش کرده. اگر در هویتی از H_{cede} وارد شود، در چپ به عضو واگذارنده توزیع می‌شود و در راست به بازپخش، و این همان یک جایی است که دو خانواده تفاوت دارند و موضوع دو بند زیر است. اگر به هر جای دیگری خطاب شود، در هیچ‌یک از دو سو ماشینی ندارد: $Exec$ هدفی نمی‌یابد و $\mathbb{Z}^H$ پاسخ می‌دهد. نگهبان‌ها در فراخوانده و از شناسه ادعایی سنجیده می‌شوند، پس هرگاه ادعاها توافق کنند آن‌ها هم توافق می‌کنند.

ادعاها، برای عضوی که هویت‌هایش را نگه می‌دارد. هر چنین فراخوانی در هر دو سو همان شناسه ادعایی را حمل می‌کند، و آن دو خموش‌کننده‌ای که می‌توانستند یکی را تغییر دهند در هر دو سو همان دو خموش‌کننده‌اند. واسط میهمان بنا بر تعریف 4.13 با خموش‌کننده خودش حکم می‌شود و نه با خموش‌کننده محیط: هویت‌هایش را نگه می‌دارد، پس $Silence$ در $\mathbb{Z}$ به او نمی‌رسد. و چون واسطی کامل است، در هر دو سو درست هویت خودش را ادعا می‌کند، و سطر 5 هسته‌اش را روی $\{id \neq id\}$ خموش می‌کند؛ و فراخوان‌هایی که پوشانه‌اش می‌گذارد، روی C و از راه $Mediate$ ، در هر دو سو حتی بیرون همان خموش‌کننده می‌ایستند. واسط خود محیط با par حکم می‌شود، که جذب آن را بی‌تغییر منتقل می‌کند، پس $\bar{A}$ از همان مجموعه و در همان هویت حساب می‌شود و درست همان ادعاها را می‌پذیرد. هیچ‌یک از دو ماشین ادعایی به‌دست نمی‌آورد و از دست نمی‌دهد.

ادعاها، برای عضوی که آن‌ها را واگذار می‌کند. چنین عضوی هیچ هویتی از خودش در درون دسته برای ادعا ندارد، پس تعریف 4.14 فراخوان‌های بیرونی‌اش را بازنویسی می‌کند، و این بازنویسی باید از سه آزمون جان سالم ببرد. اول، دسته از کجا می‌تواند سخن بگوید. هرگاه عضو میهمان به‌نیابت طرف P بدود – یعنی آنجا که چپ او را در (A, P) در حال دویدن دارد – دسته در (Z, P) یا در (Z, Z) می‌دود، یا وگرنه $P = Z$: یا $\mathbb{Z}$ با فراخواندن یک هویت A واردش می‌کند، که در درون خدمت می‌شود و $Guard_A$ طرف ادعایی را وادار می‌کند طرف هدف باشد، و سطر 3 می‌گذارد واسطی در (Z, Q) طرف P را تنها وقتی ادعا کند که $Q \in \{P, Z\}$ یا $P = Z$ ؛ یا بازپخش واردش می‌کند، که فراخوان درونی‌اش درست (Z, P) را هدف می‌گیرد. دوم، آیا ادعای (Z, P) خموش شده

است: نه با گره طرف که همین حالا حل شد، نه با سطر 2 چون نامش Z است، و نه با par ، که فرض (c) آن را از هویت‌های Z تهی می‌کند. سوم، آیا از $Guard$ در بازپخش می‌گذرد: $P \in A.P$ در سطر 1، $Z \subseteq A.N$ با طرف‌های برابر در سطر 2، و $global$ در سطر 3. آنگاه بازپخش درست همان فراخوانی را می‌گذارد که چپ گذاشت، زیر درست همان ادعایی که چپ کرد، پس هر دو سو با همان عطف‌ها در همان فراخوانده روبه‌رو می‌شوند. فراخوانی که به هویت واگذار شده می‌رسد راه دیگر را با همان سه آزمون می‌پیماید: زیر نگهبانی که در چپ از آن گذشت به بازپخش می‌رسد، با فراخوانده‌اش در بار پیام به Z در (Z, P) سپرده می‌شود — که خموش‌کننده خود بازپخش و $Guard$ آن را می‌پذیرند، با $A \subseteq Z.N$ و $global(Z) -$ و از راه نگهبان همان واسط به واسط عضو میهمان در (A, P) می‌رسد، همان نگهبانی که در چپ از آن گذشت.

رد‌ها. یک مقدار می‌تواند متفاوت باشد. آنجا که فراخوانی که عضو واگذارنده می‌گذارد رد می‌شود، چپ rej را از فراخوان خودش می‌خواند، و در راست بازپخش همان rej را برمی‌گرداند و بخش 3.6 آن را به صورت $\perp$ به عضو میهمان تحویل می‌دهد. فرض (a) درست همان است که این را می‌بندد: ماشین بی‌اعتنا به رد به آن دو یکسان پاسخ می‌دهد، پس توزیعش تغییر نمی‌کند. فرض (b) همان است که جهت دیگر همین مسئله را می‌بندد، یعنی فراخوان پاسخگو روی هویتی واگذار شده: $Respond$ هر فراخوانی را که پاسخ‌دهنده می‌گذارد رد می‌کند، و بازپخش باید یکی بگذارد تا اصلاً پاسخ دهد، پس فراخوان پاسخگو روی جایگاهی بازپخش شده $\perp$ پاسخ می‌گرفت آنجا که چپ پاسخی با محتوا داد — و همین است که چرا هیچ هسته R نمی‌تواند یکی بگذارد. هیچ چیز دیگری از کار بازپخش دیدنی نیست: $Corrupt$ از این دور جان سالم می‌برد، چون دروازه‌اش $id'.F = A$ با ادعای بازپخش برآورده می‌شود، و آن همان یک فراخوانی است که بازنویسی باید زیر هویت A بگذاردش و دسته نمی‌تواند.

ژتون. $Exec$ یک عمل دارد، گذاشتن فراخوان، و ادامه آن همان جایی است که ژتون به آن بازمی‌گردد. در راست، فراخوان‌های درونی Z^H دیگر به آن هندلر نمی‌رسند، اما تعریف 4.13 ژتون را بین ماشین‌های میهمان همان‌گونه که $Exec$ می‌کند دست‌به‌دست می‌کند، پس هر فراخواننده درونی روی ادامه خودش معلق می‌شود و با همان پاسخ از سر گرفته می‌شود؛ و هر فراخوان بازپخش شده در راست یک قاب می‌افزاید که دقیقاً یک بار از سر گرفته می‌شود، و این همان حذفی است که تعریف 4.17 اجازه می‌دهد. دو ویژگی $Exec$ به کار می‌رود، و بخش 3.6 هر دو را به هر نمونه‌ای می‌دهد: ماشین می‌تواند چند ادامه معلق داشته باشد، و می‌تواند در حال تعلیق دوباره وارد شود. دومی همان است که فراخوانی که از R می‌رسد لازم دارد، در حالی که خود Z^H چشم‌به‌راه فراخوانی بیرونی است. در هر لحظه و در هر دو سو، دقیقاً یک ماشین ژتون را در دست دارد.

پس هر ماشین همان ورودی‌ها را به همان ترتیب می‌بیند و از همان نوار پاسخ می‌دهد، پس تناظر در هر فعال‌سازی نگه داشته می‌شود و آن دو اجرا به‌عنوان توزیع توافق دارند — به‌ویژه بر احتمال اینکه Z مقدار 1 برگرداند. $\square$

از دید نموداری، این لم یک تساوی است با دو حالتش پهلوی هم: عضو نگه‌داشته یک‌جا در دسته محو می‌شود، و عضو واگذار شده از راه بازپختی به بیرون پل می‌خورد.

$$\boxed{\mathcal{Z}} \cup \boxed{H_{\text{keep}}} \cup \boxed{H_{\text{cede}}} \cup \boxed{R} \stackrel{(a)(b)(c)}{=} \boxed{\mathcal{Z} \mid H_{\text{keep}} \mid H_{\text{cede}}} \longleftrightarrow \boxed{\mathcal{D}} \longleftrightarrow \boxed{R}$$

عضوی از H_{keep} – مثل کلی $\varphi \setminus \rho$ در شکل قضیه 4.20 بالا – یک‌جا جذب می‌شود: $\mathcal{Z}^H$ هویتش را اشغال می‌کند، پس R تنها به‌عنوان بخشی از دسته به او می‌رسد. عضوی از H_{cede} هم میهمان می‌شود و روی همان نوار می‌دود، اما دسته هویتش را اشغال نمی‌کند: بازپخش $\mathcal{D}$ (همان بدلِ بخش 4.4) به جای او می‌ایستد، فراخوان‌های R را با برچسب فراخوانده‌شان به $\mathcal{Z}$ می‌سپارد و فراخوان‌های بیرونی عضو میهمان را زیر ادعای اصلی‌اش از نو صادر می‌کند، پس هیچ‌یک از دو سو نمی‌تواند بفهمد که بازپخش میانشان نشسته است. فرض‌های (a)–(c) که تنها از H_{cede} خواسته می‌شوند، همان‌هایی هستند که این را تمایزناپذیر نگه می‌دارند: (a) درست همان بی‌اعتنایی به رد است، چون $\mathcal{D}$ هر rej را که بازپخش می‌کند به $\perp$ بدل می‌کند؛ و (b) و (c) فراخوان پاسخگو و برخورد ادعای خود $\mathcal{Z}$ را ممنوع می‌کنند، آن دو راه دیگری که پرش اضافی بازپخش می‌توانست از آن‌ها پیدا شود. شکل قضیه 4.20 همان حالت $H_{\text{cede}} = \emptyset$ است؛ و حالت آینه‌ای، یعنی یک اشغال‌گر واگذارشده که به جای کل جایگاه حریف می‌ایستد ($H_{\text{keep}} = \emptyset$)، همان است که ساخت حریف بدلِ بخش 4.4 رویش می‌دود.

دو نیمه این لم جدا از هم به کار می‌روند. جذب یک پروتکل هیچ واگذار نمی‌کند، پس هر سه فرض می‌افتند و آنچه می‌ماند یک کمانک‌گذاری محض است؛ و این همان حالتی است که قضیه 4.20 رویش می‌دود، و ارزش آن را دارد که جداگانه بیان شود، برای همین تأکید که اصلاً هیچ هزینه‌ای ندارد.

گزاره 4.19 (بازگروه‌بندی). بگذارید ρ و π سیستم باشند، $\varphi \subseteq \rho$ ، جانشانی φ با π در ρ خوش‌ساخت باشد، و $\mathcal{Z}$ محیطی باشد که در هیچ هویتی از $\text{IDs}(\rho) \cup \text{IDs}(\pi)$ میهمان نمی‌کند. آن‌گاه برای σ ی برابر با π یا φ ، و برای هر اشغال‌گر $\mathcal{X}$ از جایگاه حریف (تعریف 3.3)،

$$\text{Exec}(\rho[\sigma/\varphi], \mathcal{Z}, \mathcal{X}, \mathcal{C}) \quad \text{and} \quad \text{Exec}(\sigma, \mathcal{Z}^{\rho \setminus \varphi}, \mathcal{X}, \mathcal{C})$$

همسان‌توزیعند – به‌عنوان توزیع برابرند، و نه تنها نزدیک.

اثبات. لم 4.18 در $H = \rho \setminus \varphi$ با $H_{\text{cede}} = \emptyset$ ، به‌گونه‌ای که $\mathcal{Z}^H$ همان محیط جذب‌شده تعریف 4.15 باشد و هیچ فرضی از آن لم به کار نیاید: (a) و (b) و (c) از اعضای واگذارشده خواسته می‌شوند و هیچ عضو واگذارشده‌ای نیست. توجه کنید که $\rho[\varphi/\varphi] = \rho$ ، پس حالت $\sigma = \varphi$ همان سوی ایده‌آل است. آن دو خانواده اشغال‌گر مجازند: گردایه سمت چپ در هر دو حالت یک سیستم است، پس هیچ هویتی دو بار اشغال نمی‌شود – برای $\sigma = \pi$ چون خوش‌ساختی می‌گوید $\rho \setminus \varphi$ و π هیچ شناسه فرایند مشترکی ندارند، و برای $\sigma = \varphi$ چون آن گردایه خود ρ است – و در سمت راست، فرض میهمانی روی $\mathcal{Z}$ دسته را از $\text{IDs}(\sigma)$ برکنار نگه می‌دارد. $\square$

اکنون قضیه می‌گوید که جانشین کردن یک زیرسیستم به جای دیگری در درون یک پروتکل، هیچ مزیتی هزینه نمی‌کند.

به زبان غیررسمی، قضیه 4.20 می‌گوید: اگر π کارکرد φ را در ε تقلید-UC کند، آنگاه $\rho[\pi/\varphi]$ کارکرد ρ را در همان ε تقلید-UC می‌کند. رده‌های حریف و شبیه‌ساز بی‌تغییر منتقل می‌شوند؛ و محیط‌های نتیجه آن‌هایی هستند که برای زوج $(\rho[\pi/\varphi], \rho)$ مجازند و صورت جذب‌شده‌شان $\mathcal{Z}^{\rho \wedge \varphi}$ در رده فرض می‌افتد.

قضیه 4.20 (ترکیب). بگذارید ρ و π سیستم باشند، $\varphi \subseteq \rho$ ، و جانشانی φ با π در ρ به معنای تعریف 4.2 خوش‌ساخت باشد. فرض کنید

. تقلید-UC می‌کند (A, S, Z) و در برابر ε را در φ کارکرد π

آنگاه

، تقلید-UC می‌کند (A', S', Z') و در برابر ε را در ρ کارکرد $\rho[\pi/\varphi]$

که در آن

$$A' = A, \quad S' = S, \quad Z' = \{Z \in \mathcal{Z}_{\rho[\pi/\varphi], \rho} : Z^{\rho \wedge \varphi} \in Z\}.$$

و به‌صراحت،

$$\forall A \in A' \exists S \in S' \forall Z \in Z' :$$

$$\text{Adv}_{\rho[\pi/\varphi], \rho}^{\text{uc}}(Z, A, S) \leq \text{Adv}_{\pi, \varphi}^{\text{uc}}(Z^{\rho \wedge \varphi}, A, S) \leq \varepsilon.$$

اثبات. سه نسبت، از سه گونه. رده‌های حریف و شبیه‌ساز با رونویسی منتقل می‌شوند: حریف هر سیستمی را که حمله کند همان یک ماشین است، و هیچ چیز از آنچه شبیه‌ساز را نوع‌بندی می‌کند از زوجی که برایش شبیه‌سازی می‌شود نام نمی‌برد. رده محیط تنها چیزی است که حساب می‌شود و نه رونویسی، چون پیش‌نگاره رده فرض زیر جذب است. آنگاه کران دو گام است. گزاره 4.19، که یک بار در هر سو به کار می‌رود، مزیت بیرونی را دقیقاً به مزیت درونی بدل می‌کند – چون همان یک ماشین جذب‌شده در هر دو سو خدمت می‌کند، و همین است که مجموعه بیرونی‌اش برای مجازکردنش انتخاب شده بود – و فرض تقلید کرانی بر مزیت درونی می‌گذارد. پس نسبت اول آن نمایش یک تساوی است، که تنها برای یک‌دستی با نسبت دوم به‌صورت نامساوی نشان داده شده. آن سه نسبت از سه گونه‌اند. حریف‌ها بی‌تغییر منتقل می‌شوند چون آن فروکاست همان یک ماشین را تحویل می‌دهد: $A \in A'$ که به $\rho[\pi/\varphi]$ حمله می‌کند، به‌عنوان $A \in A$ که به π حمله می‌کند خوانده می‌شود. هر $A' \subseteq A$ به کار می‌آید، و تساوی بزرگ‌ترین چنین انتخابی است. شبیه‌سازها به دلیل آینه‌ای منتقل می‌شوند: $S \in S$ که فرض فراهمش می‌کند به‌عنوان شبیه‌ساز نتیجه پس داده می‌شود، پس هر $S' \supseteq S$ به کار می‌آید،

و تساوی کوچک‌ترین است. و اینکه اصلاً می‌شود پشش داد، همان نکته 4.8 است: هیچ چیز از آنچه شبیه‌ساز را نوع‌بندی می‌کند از زوجی که برایش شبیه‌سازی می‌شود نام نمی‌برد، پس شبیه‌سازی برای (π, φ) از پیش در جایگاه $(\rho[\pi/\varphi], \rho)$ می‌ایستد. محیط‌ها همان یک جایی هستند که چیزی حساب می‌شود و نه رونویسی. اینجا $\mathbb{Z}$ و $\mathbb{Z}'$ رده‌هایی برای دو زوج متفاوت از سیستم‌هایند، و $\mathbb{Z}'$ پیش‌نگاره $\mathbb{Z}$ زیر جذب است،

$$\mathbb{Z}' = ((\cdot)^{\rho \setminus \varphi})^{-1}(\mathbb{Z}) \cap \mathbb{Z}_{\rho[\pi/\varphi], \rho},$$

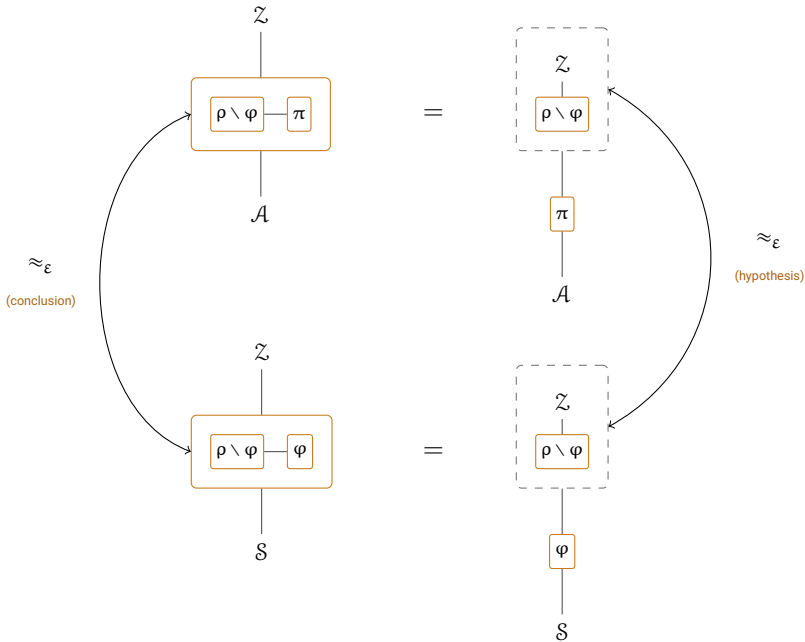
پس محیط بیرونی درست وقتی به شمار می‌آید که آنچه در جذب به آن بدل می‌شود یکی از آن‌هایی باشد که فرض می‌پوشاندش. این یک ردهٔ اصل است، چون بنا بر ساخت $\mathbb{Z}' \subseteq \mathbb{Z}_{\rho[\pi/\varphi], \rho}$ و وقتی $\mathbb{Z}$ بزرگ‌ترین ردهٔ مجاز یعنی $\mathbb{Z}_{\pi, \varphi}$ باشد، گزاره 4.16 شرط دوم را زائد می‌کند و پیش‌نگاره کلی $\mathbb{Z}_{\rho[\pi/\varphi], \rho}$ است: تقلید در برابر بزرگ‌ترین رده، بزرگ‌ترین رده را باز می‌دهد.

گزاره 4.16 مزیت سمت راست را هم تعریف‌شده می‌کند، بی‌آنکه فرضی از بستار بر محیط‌ها یا نسبتی میان $\text{IDs}(\varphi)$ و $\text{IDs}(\pi)$ لازم باشد. نامساوی اول همان گزاره 4.19 است که دو بار به کار رفته، با $\sigma = \pi$ و $\mathcal{X} = \mathcal{A}$ در سوی واقعی و $\sigma = \varphi$ و $\mathcal{X} = \mathcal{S}$ در سوی ایده‌آل. با تفریق،

$$\text{Adv}_{\rho[\pi/\varphi], \rho}^{\text{uc}}(\mathbb{Z}, \mathcal{A}, \mathcal{S}) = \text{Adv}_{\pi, \varphi}^{\text{uc}}(\mathbb{Z}^{\rho \setminus \varphi}, \mathcal{A}, \mathcal{S}),$$

پس در واقع یک تساوی است: آن دو آزمایش یک آزمایش‌اند که به‌گونهٔ دیگری کمانک‌گذاری شده، و همان یک $\mathcal{S}$ در هر دو سو خدمت می‌کند چون اجرای ایده‌آل با همان یک جذب بازگروه‌بندی می‌شود. نامساوی دوم همان فرض تقلید است که در $\mathbb{Z}^{\rho \setminus \varphi}$ به کار رفته، و آن در $\mathbb{Z}$ است چون $\mathbb{Z}'$ در $\mathbb{Z}$ است. قضیه نسبت اول را تنها برای یک‌دستی با نسبت دوم به‌صورت نامساوی نشان می‌دهد. $\square$

از دید نموداری، با تعیین یک $\mathcal{A} \in \mathcal{A}'$ ، یک $\mathcal{S} \in \mathcal{S}'$ که خدمت می‌کند و یک $\mathbb{Z} \in \mathbb{Z}'$ ، اثبات همان مربع جابه‌جایی زیر است، که به سبک نمودارهای ریسمانی [9] کشیده شده: جعبهٔ توپر یک سیستم است، سیم یک واسط، و جعبه‌ای که از پایین سیستم آویخته همان ماشینی است که در آن لحظه جایگاه حریفانهٔ آن را اشغال کرده است.



شکل 1: جانشرانی، به صورت یک مربع جابه جایی. خواندن هر ستون از بالا به پایین یک اجراست؛ جعبه نقطه چین زوجی را نشان می دهد که مبادله می شود. هر دو یال افقی جذبند، پس هر دو تساویند و نه کران، و یال راست همان فرض است. آن گاه نتیجه سمت چپ همان کمیت است، و نه تنها کمیتی با همان کران. (قضیه 4.20).

یال های بالا و پایین همان گزاره 4.19 اند، که به $(\pi, \mathcal{A})$ و به $(\varphi, \mathcal{S})$ به کار رفته — بازگروه بندی مفت است، پس هر دو تساویند و نه کرانی صرف، و همین است که چرا جعبه خط چین سمت راست (همان محیط جذب شده $\mathbb{Z}^{\rho \setminus \varphi}$ از گزاره 4.19) تنها چیزی است که میان یک ستون و همسایه اش تغییر می کند. یال راست همان فرض قضیه است که به آن محیط جذب شده به کار رفته. آن گاه یال چپ — نتیجه قضیه — وادار می شود: تعقیب مربع نشان می دهد آن دو $\approx_\varepsilon$ نه تنها هر دو با همان ε کران دار شده اند، بلکه بنا بر آن دو تساوی، عیناً همان یک کمیتند. و ترکیب کردن n مربع این چنین در یک زنجیره، یک دورگه در هر گام، درست همان قضیه 4.24 زیر است.

اکنون جانشرانی و ترائایی در صورتی ترکیب می شوند که ترکیب واقعاً در آن به کار می رود. گزاره 4.12 همراه متر در بالا بیان شد، چون اثباتش به هیچیک از این ماشین آلات نیاز نداشت؛ و اینجا همان جایی است که کارش را می کند.

نتیجه 4.21 (ترکیب با یک مشخصه). بگذارید π, ρ و φ سیستم باشند، $\varphi \subseteq \rho$ ، جانشرانی φ با π در ρ خوش ساخت باشد، و π کارکرد φ را در ε_1 و در برابر $(\mathcal{A}, \mathcal{S}, \mathbb{Z})$ تقلید-UC کند. $\mathbb{Z}'$ را برای رده قضیه 4.20 بنویسید. اگر ρ کارکرد φ را در ε_2 و در

برابر (S, S', Z') تقلید-UC کند، آن گاه $\rho[\pi/\varphi]$ کارکرد ψ را در $\varepsilon_1 + \varepsilon_2$ و در برابر (A, S, Z') تقلید-UC می‌کند.

اثبات. بنا بر قضیه 4.20، $\rho[\pi/\varphi]$ کارکرد ρ را در ε_1 و در برابر (A, S, Z') تقلید-UC می‌کند، و $Z' \subseteq Z_{\rho[\pi/\varphi], \rho}$. فرض دوم حکمی از تقلید در برابر همان رده است، پس $Z' \subseteq Z_{\rho, \psi}$ هم برقرار است. پس هر دو شمولی که گزاره 4.12 لازم دارد برقرارند، و آن گزاره به آن دو حکم ρ با در میانه اعمال می‌شود. $\square$

این همان صورتی است که ترکیب در آن به کار می‌رود. آدم ρ را با زیرروال ایده‌آل‌شده φ سر جایش تحلیل می‌کند و اثبات می‌کند که هر مشخصه ψ ای که بخواهد را تقلید می‌کند – در برابر رده‌هایی که نتیجه نامشان می‌برد و حریف‌هایشان شبیه‌سازهای گام اول‌اند؛ و آن‌گاه نتیجه، آن حکم را به سیستمی منتقل می‌کند که واقعاً می‌دود. یک فرض به‌سادگی از چشم می‌افتد: تقلید در برابر Z' ، به‌عنوان شرط جنبی تعریف 4.11، ادعا می‌کند که $Z' \subseteq Z_{\rho, \psi}$ – پس نام‌های ψ باید زیر par هر محیطی باشند که آن رده نگه می‌دارد، و هر جا نباشند آن نتیجه نمی‌گردد.

4.3 ترکیب موازی

قضیه 4.20 یک زیرسیستم را جانشین می‌کند. جانشین‌کردن چند زیرسیستم یک‌جا استدلال تازه‌ای نیست، بلکه n بار به‌کار بستن همان یکی در امتداد زنجیره‌ای از دورگه‌هاست؛ و آنچه دقت می‌خواهد، حساب‌داری آن سه رده در طول زنجیره است. فرض‌های مجزایبودن زیر تحت‌اللفظی‌اند، و تنها به‌سبب قرارداد فصل 1 برآورده‌شدنی‌اند: دفتر ماشینی از اجراست و عضو هیچ سیستمی نیست، و هرگز بیش از یکی از آن نیست. اگر عضو شمرده می‌شد، یک‌جا در هر φ_i و هر π_k می‌افتاد، پس هیچ دو زیرسیستمی از ρ مجزا نمی‌بودند و هیچ دو π_k شناسه‌های فرایند مجزا حمل نمی‌کردند؛ آن فرض‌ها برای $n \geq 2$ برآورده‌نشدن می‌شدند و باید به‌پیمانه $\mathcal{C}$ خوانده می‌شدند. بیرون نگه داشتن دفتر از سیستم‌ها همان است که می‌گذارد آن‌ها همان‌گونه که نوشته شده‌اند خوانده شوند.

تعریف 4.22 (جانشانی هم‌زمان). بگذارید ρ سیستمی باشد و $\varphi_1, \dots, \varphi_n \subseteq \rho$ زیرسیستم‌هایی دوه‌دو مجزا. برای سیستم‌های $\pi_1, \dots, \pi_n$

$$\rho[\pi_1/\varphi_1, \dots, \pi_n/\varphi_n] := \left(\rho \setminus \bigcup_{i=1}^n \varphi_i \right) \cup \bigcup_{i=1}^n \pi_i.$$

برای $0 \leq k \leq n$ بنویسیم $\rho_k := \rho[\pi_1/\varphi_1, \dots, \pi_k/\varphi_k]$ به گونه‌ای که $\rho_0 = \rho$ و $\rho_n = \rho'$. چون φ_i ها دوه‌دو مجزایند، φ_k از $k-1$ ک-جانشانی اول دست‌نخورده می‌ماند، پس $\varphi_k \subseteq \rho_{k-1}$ و

$$\rho_k = \rho_{k-1}[\pi_k/\varphi_k].$$

پس دوره‌های متوالی تنها در یک جانشانی تفاوت دارند، و همین می‌گذارد قضیه 4.20 در هر گام کار کند؛ و اثبات زیر زنجیره را به صورت n جهش بازی می‌پیماید، یکی به ازای هر جانشانی. جفت کردن گام‌ها به این هم نیاز دارد که رده‌ها تنظیم‌شدنی باشند، و هستند.

لم 4.23 (یکنوایی). فرض کنید π کارکرد φ را در ε و در برابر (A, S, Z) تقلید-UC می‌کند. آنگاه همین کار را در برابر (A^-, S^+, Z^-) هم می‌کند، برای هر $A^- \subseteq A$ هر رده شبیه‌ساز $S^+ \supseteq S$ و هر $Z^- \subseteq Z$.

اثبات. تعریف 4.11 حکمی از گونه $\forall \exists \forall$ روی آن سه رده است. تنگ کردن دامنه هر یک از دو سور عمومی و فراخ کردن دامنه سور وجودی، هر دو آن را نگه می‌دارند، و $\square$

به زبان غیررسمی، قضیه 4.24 می‌گوید جانشین کردن n زیرسیستم دوه‌دو مجزا یک‌جا، جمع خطاهای فردی را هزینه می‌کند: این همان قضیه 4.20 است که n بار در امتداد زنجیره‌ای از دوره‌ها به کار رفته، که حریف‌ها حریف‌های آخرین جانشانی‌اند، شبیه‌سازها شبیه‌سازهای اولین، و محیط‌ها آن‌هایی که یک‌جا برای هر گام مجازند.

قضیه 4.24 (ترکیب موازی). بگذارید ρ سیستمی باشد، $\varphi_1, \dots, \varphi_n \subseteq \rho$ دوه‌دو مجزا، و $\pi_1, \dots, \pi_n$ سیستم‌هایی چنان که π_k ها شناسه‌های فرایند دوه‌دو مجزا حمل کنند و، به ازای هر k ، π_k هیچ شناسه فرایندی با $\rho \setminus \varphi_k$ مشترک نداشته باشد. فرض کنید به ازای هر k

، تقلید-UC می‌کند (A_k, S_k, Z_k) و در برابر ε_k را در φ_k کارکرد π_k

و Z'_k را برای رده متناظری بنویسید که قضیه 4.20 به گام k ام می‌چسباند،

$$Z'_k = \{Z \in Z_{\rho_k, \rho_{k-1}} : Z^{\rho_{k-1} \setminus \varphi_k} \in Z_k\}.$$

و سرانجام فرض کنید

$$S_{k+1} \subseteq A_k \quad (1 \leq k < n), \quad Z^\star \subseteq \bigcap_{k=1}^n Z'_k.$$

آن‌گاه $\mathbb{Z}^\star \subseteq \mathbb{Z}_{\rho', \rho}$ ، و

$$\rho' := \rho[\pi_1/\varphi_1, \dots, \pi_n/\varphi_n] \text{ کاربرد}$$

. تقلید-UC می‌کند $(\mathbb{A}_n, \mathbb{S}_1, \mathbb{Z}^\star)$ و در برابر ε_k را در $\sum_{k=1}^n$

اثبات. یک استدلال دورگه روی n گام، و کار در این است که پیش از برداشتن هیچ‌یک از آن‌ها، بسنجیم هر گام نمونه‌ای مجاز از قضیه 4.20 است. سه مقدمه این کار را می‌کنند. اولی نشان می‌دهد هر جانشانی خوش‌ساخت است و هر ρ_k یک سیستم، با استقرا روی k ، تا قضیه ترکیب در هر گام اعمال شود. دومی نشان می‌دهد رده $\mathbb{Z}^\star$ یک‌جا برای هر زوج میانی مجاز است، و - با گرفتن اجتماع روی $k -$ برای زوج بیرونی هم، تا نتیجه خودش حکمی مجاز از تقلید باشد. سومی زنجیره شبه‌سازها را رو به پایین می‌سازد، با $\mathcal{A} := \mathbb{S}_{n+1}$ و هر $\mathbb{S}_k$ که گام k ام برای حریف $\mathbb{S}_{k+1}$ فراهم می‌کند، و همین است که $\mathbb{S}_1$ را تنها به $\mathcal{A}$ وابسته می‌کند. تنها پس از آن است که دورگه برداشته می‌شود: بازی‌های متوالی در یک زیرسیستم جانشین‌شده و یک حریف تفاوت دارند، آن تفاوت یک بار به کار بستن قضیه 4.20 است، و جمع n جهش تلسکوپی به $\sum_k \varepsilon_k$ می‌رسد. اول، هر گام خوش‌ساخت است. اعضای $\rho_{k-1} \setminus \varphi_k$ همان‌های $\rho \setminus \varphi_k$ هستند که $k - 1$ جانشانی اول سر جایشان می‌گذارند، به همراه $\pi_1, \dots, \pi_{k-1}$ ؛ و بنا بر فرض π_k با هیچ‌یک از این دو گروه شناسه فرایندی مشترک ندارد. پس جانشانی φ_k با π_k در ρ_{k-1} به معنای تعریف 4.2 خوش‌ساخت است. هر ρ_k یک سیستم است، با استقرا روی $k: \rho_0 = \rho$ یکی است، و $\rho_k = \rho_{k-1}[\pi_k/\varphi_k]$ بنا بر گزاره 4.3 از همان جانشانی خوش‌ساخت یکی است. پس قضیه 4.20 و گزاره 4.16، که ρ_{k-1} و π_k را سیستم می‌خواهند، به گام k ام اعمال می‌شوند.

بعد، مجازبودن. بگذارید $\mathbb{Z} \in \mathbb{Z}^\star$ و k را ثابت کنید. از $\mathbb{Z}_{\rho_k, \rho_{k-1}} \subseteq \mathbb{Z}'_k \subseteq \mathbb{Z}^\star$ به این می‌رسیم که

$$\mathbb{Z}.\text{par} \supseteq \langle \rho_k \rangle \cup \langle \rho_{k-1} \rangle,$$

پس $\mathbb{Z}$ برای زوج k ام دورگه‌ها مجاز است و مزیت $\text{Adv}_{\rho_k, \rho_{k-1}}^{\text{uc}}$ زیر تعریف‌شده است. گزاره 4.16، که به جانشانی φ_k با π_k در ρ_{k-1} اعمال می‌شود، آن‌گاه محیط جذب‌شده را در $\mathbb{Z}_{\pi_k, \varphi_k}$ می‌نشانند: پارامترش پارامتر $\mathbb{Z}$ است، که از پیش هویت‌های π_k و φ_k را در بر دارد، پس مزیت درونی $\text{Adv}_{\pi_k, \varphi_k}^{\text{uc}}$ هم تعریف‌شده است. با گرفتن اجتماع شمول‌های نمایش‌داده‌شده روی همه k ها،

$$\mathbb{Z}.\text{par} \supseteq \bigcup_{k=0}^n \langle \rho_k \rangle \supseteq \langle \rho' \rangle \cup \langle \rho \rangle,$$

و به همین شکل $\mathbb{Z}$ در هیچ هویتی از $\text{IDs}(\rho') \cup \text{IDs}(\rho) \supseteq \bigcup_k \text{IDs}(\rho_k)$ میهمان نمی‌کند - بنا بر بند میهمانی هر $\mathbb{Z}_{\rho_k, \rho_{k-1}}$ ، در $k = n$ و $k = 1 -$ پس هر دو بند تعریف 4.10 $\mathbb{Z}^\star \subseteq \mathbb{Z}_{\rho', \rho}$ را می‌دهند و نتیجه خودش حکمی مجاز از تقلید است.

$\mathcal{A} \in \mathbb{A}_n$ را ثابت کنید و $\mathcal{S}_{n+1} := \mathcal{A}$ بگذارید. برای $k = n, n-1, \dots, 1$ بگذارید $\mathcal{S}_k \in \mathbb{S}_k$ همان شبیه‌سازی باشد که گام k ام برای حریف $\mathcal{S}_{k+1}$ فراهم می‌کند. این در هر اندیسی مجاز است: در $k = n$ چون $\mathcal{S}_{n+1} = \mathcal{A} \in \mathbb{A}_n$ ، و در $k < n$ چون $\mathcal{S}_{k+1} \in \mathbb{S}_{k+1} \subseteq \mathbb{A}_k$. زنجیره شبیه‌سازها رو به پایین ساخته می‌شود، هر یک از یکی پیش از خودش، و $\mathcal{S}_1$ تنها به $\mathcal{A}$ وابسته است.

اکنون $\mathcal{Z} \in \mathbb{Z}^\star$ را ثابت کنید و این $n+1$ بازی را در نظر بگیرید

$$G_k := \text{Exec}(\rho_k, \mathcal{Z}, \mathcal{S}_{k+1}, \mathcal{C}), \quad p_k := \Pr[G_k = 1], \quad 0 \leq k \leq n.$$

اینجا G_0 اجرای ρ زیر $\mathcal{S}_1$ است و G_n اجرای ρ' زیر $\mathcal{A}$ ، پس این دنباله از ρ_0 آغاز می‌شود و به ρ_n می‌رسد. بازی‌های متوالی در یک زیرسیستم جانشین‌شده و در ماشینی که نقش حریف را بازی می‌کند تفاوت دارند، و آن تفاوت درست یک بار به‌کار بستن قضیه 4.20 است: برای $1 \leq k \leq n$

$$\begin{aligned} |p_k - p_{k-1}| &= \text{Adv}_{\rho_k, \rho_{k-1}}^{\text{uc}}(\mathcal{Z}, \mathcal{S}_{k+1}, \mathcal{S}_k) \\ &\leq \text{Adv}_{\pi_k, \varphi_k}^{\text{uc}}(\mathcal{Z}^{\rho_{k-1} \setminus \varphi_k}, \mathcal{S}_{k+1}, \mathcal{S}_k) \leq \varepsilon_k. \end{aligned}$$

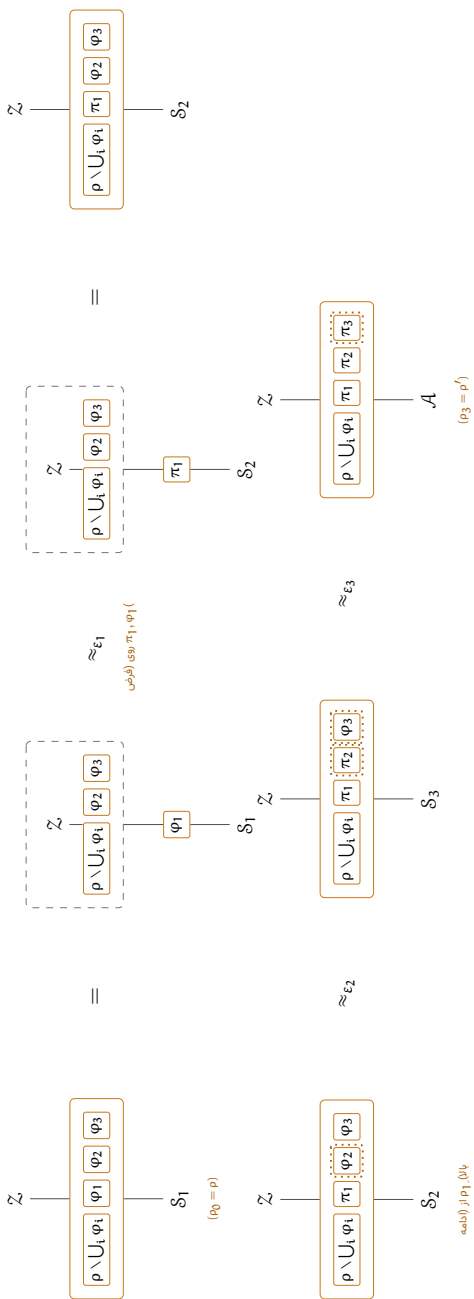
آن تساوی، تعریف 4.5 است که برعکس خوانده شده، چون G_k و G_{k-1} همان دو اجرایی هستند که مزیت $\text{Adv}_{\rho_k, \rho_{k-1}}^{\text{uc}}$ مقایسه‌شان می‌کند. نامساوی اول فروکاست قضیه 4.20 است، که در دست است چون بنا بر فرض $\mathcal{Z} \in \mathbb{Z}^\star \subseteq \mathbb{Z}'_k$ ، پس لم 4.23 گام k ام را حکمی در برابر $\mathbb{Z}^\star$ می‌کند. نامساوی دوم همان تقلید φ_k به‌دست π_k است، که به حریف $\mathcal{S}_{k+1}$ و محیط جذب‌شده اعمال شده.

با جمع n جهش و تلسکوپی‌شدن،

$$\text{Adv}_{\rho', \rho}^{\text{uc}}(\mathcal{Z}, \mathcal{A}, \mathcal{S}_1) = |p_n - p_0| \leq \sum_{k=1}^n |p_k - p_{k-1}| \leq \sum_{k=1}^n \varepsilon_k.$$

و چون $\mathcal{S}_1 \in \mathbb{S}_1$ تنها از $\mathcal{A}$ انتخاب شد و $\mathcal{Z} \in \mathbb{Z}^\star$ دلخواه بود، این همان حکم قضیه است. $\square$

از دید نموداری، جهش k همان مربع خود قضیه 4.20 است (همان شکل بالای اثباتش) با $\varphi_k \mapsto \varphi$ ، $\pi_k \mapsto \pi$ ، $\mathcal{S}_k \mapsto \mathcal{A}$ ، و پوسته $\varphi \setminus \rho$ که فراخ شده تا هر جایگاه دست‌نخورده را هم با خود ببرد. یک تصویر، که جهش 1 در آن کامل کشیده شده و جهش‌های 2 و 3 از همان جا که تمام می‌شود جمع‌وجور ادامه می‌یابند، از $\rho_0 = \rho$ تا $\rho_3 = \rho'$:



از چپ به راست بخوانید: ρ_0 (با همه φ ها) جذب می‌شود - باقی واقعی ρ و φ_2, φ_3 دست‌نخورده همراهش در $\mathbb{Z}$ تا می‌شوند و تنها φ_1 رو می‌ماند. فرض قضیه 4.20 آن را در ε_1 با π_1 عوض می‌کند؛ و پس‌دادن آن تایی خورده ρ_1 را از نو سوار می‌کند، و δ برای جورشدن از δ_1 به δ_2 پیش می‌رود. جهش‌های 2 و 3 همان مربع یکسانند، تنها فشرده به دو سر صافشان - کشیدن دوباره آن تکرار بود و نه افزودن - و جعبه‌های نقطه‌چین در هر جهش درست نشان می‌دهند کدام جایگاه φ_k است (که در آستانه برگشتن است) و کدام π_k (که همین حالا برگشته). جمع آن سه جهش و تلسکوپی‌شدن، چنان که در سطرهاي آخر اثبات بالا، سه ε_k جدا را - که هیچ‌یک در خود حکم قضیه نمی‌آیند - به آن یک کران $\rho' \approx \rho_{\varepsilon_1 + \varepsilon_2 + \varepsilon_3}$ بدل می‌کند.

فساد در طول زنجیره. فساد در طول زنجیره همان‌گونه رفتار می‌کند که در یک گام تنها. در درون یک بازی، دفتر یک ماشین است در زیر φ_k و π_k و $\rho \searrow \varphi_k$ ، به یک‌سان، و کلیدش طرف است، پس یک جهش نمی‌تواند طرفی را نیمه‌کاره فاسد کند؛ و بین بازی‌ها دفترها نمونه‌های جدایند و هیچ چیز از جهشی به جهش دیگر نمی‌برند. محیط در هر جهش همان طرف‌ها را فاسد می‌کند، چون همان ماشین است که همان فراخوان‌ها را زیر سطر 3 می‌گذارد؛ جایگاه ناچار نیست چنین کند، پس آن دو ماشینی که در هر جهش تغییر می‌کنند - زیرسیستم جانشین‌شده و اشغال‌گر جایگاه $\delta_k \rightarrow \delta_{k+1}$ - لازم نیست همان طرف‌ها را فاسد کنند؛ اما محیط **C** را بی‌واسطه‌گری می‌خواند (نکته 3.1)، پس δ_k ای که به‌گونه دیگری فاسد کند با همان ε_k گرفته می‌شود، و هم‌اندازی باز هم نتیجه است و نه فرض. طرف‌هایی که $\rho_{k-1} \searrow \varphi_k$ خدمتشان می‌دهد یکسان گردانده می‌شوند: قلاب‌های **Mediate** شان محیط جذب‌شده را به قصد جایگاه ترک می‌کنند، و همین است که چرا مجموعه بیرونی تعریف 4.15 شناسه‌های **A** و **C** را حمل می‌کند. پس قضیه تنها این را ادعا می‌کند که هیچ محیطی در $\mathbb{Z}^*$ الگوهای فساد را با بیش از $\sum_k \varepsilon_k$ از هم تشخیص نمی‌دهد.

نکته 4.25 (بزرگ‌ترین رده‌ها). اجتماع بستارهای دورگه صورتی بسته دارد. هر ρ_k از بخشی دست‌نخورده ρ به همراه $\pi_1, \dots, \pi_k$ تشکیل شده، پس

$$\bigcup_{k=0}^n \langle \rho_k \rangle = \langle \rho \rangle \cup \bigcup_{i=1}^n \langle \pi_i \rangle = \langle \rho' \rangle \cup \langle \rho \rangle.$$

اکنون فرض کنید هر $\mathbb{Z}_k$ بزرگ‌ترین رده مجاز یعنی $\mathbb{Z}_{\pi_k, \varphi_k}$ باشد. آنگاه گزاره 4.16 شرط دومی که $\mathbb{Z}'_k$ را تعریف می‌کند زائد می‌کند، پس $\mathbb{Z}'_k = \mathbb{Z}_{\rho_k, \rho_{k-1}}$ ، و بزرگ‌ترین انتخاب روایی $\mathbb{Z}^*$ این است

$$\bigcap_{k=1}^n \mathbb{Z}_{\rho_k, \rho_{k-1}} = \{ \mathbb{Z} : \mathbb{Z}.\text{par} \supseteq \bigcup_{k=0}^n \langle \rho_k \rangle \} = \mathbb{Z}_{\rho', \rho}.$$

پس ترکیب موازی در برابر بزرگ‌ترین رده‌ها، باز بزرگ‌ترین رده را تحویل می‌دهد، همان‌گونه که قضیه تک‌گامی می‌کند.

آن سه رده همان گونه رفتار می کنند که زنجیره نشان می دهد. حریف ها حریف های آخرین جانشانی اند، چون استدلال از حریفی در برابر $\rho_n = \rho'$ آغاز می کند؛ و شبیه سازها شبیه سازهای اولین، چون زنجیره با تولید شبیه سازی در برابر $\rho_0 = \rho$ تمام می شود. هر رده میانی دو بار خدمت می کند، به عنوان شبیه سازهای یک گام و حریف های گام بعد، و همین است که $S_{k+1} \subseteq A_k$ ترتیبش می دهد. محیط ها باید یک جا برای هر گام مجاز باشند، و اشتراک از همین جاست. و مزیت جمع شدنی است، چنان که استدلال دورگه روی n گام همیشه چنپنش می کند.

4.4 تمامیت حریف بدل

حریف بدل هر چه محیط بگویدش انجام می دهد و هر چه به او گفته شود پس گزارش می کند؛ هیچ راهبردی از خودش ندارد. هر چه حریف واقعی به دست می آورد، محیط می تواند با راندن بدل به دست آورد، و همین است که چرا می شود سورزدن روی حریف ها را با سورزدن روی محیط ها عوض کرد. این بخش او را تعریف می کند و اثبات می کند که، برای سیستم ها و رده هایی که قضیه دقیقشان می کند، هیچ از دست نمی رود – و نکته 9.4 آن یک رژیم را ثبت می کند که این ادعا به آن نمی رسد: هسته هایی که فراخوان پاسخگو می گذارند.

آنچه چارچوب از پیش تعیین کرده. بدل گونه تازه ای از ماشین نیست. او همان حریف بخش 3.4 است با هسته ای خاص، پس میدان های A را حمل می کند و فراخوان هایش از پوشانه A می گذرند. سه پیامد، شکل کد را پیش از نوشتن یک سطر آن تعیین می کنند. اول، او تنها یک هویت را می تواند ادعا کند. سطر 2 از جعبه A هسته را روی $\{id'' \neq id\}$ خموش می کند، پس هر فراخوانی که بدل می گذارد id را ادعا می کند، همان هویتی که در آن می دود. پس دستوری از شکل «فراخوان id'' .Op به عنوان id''' » را نمی شود برای id'' دلخواه انجام داد: بدل به عنوان $(A, id.P)$ سخن می گوید یا اصلاً سخن نمی گوید. آنچه محیط انتخاب می کند شناسه ادعایی نیست بلکه طرف است، با دستور دادن به بدل در (A, P) به جای (A, Q) ؛ و ریشه $\mathbb{Z}$ می تواند برای هر طرفی عمل کند و پس می تواند هر یک را برگزیند.

دوم، طرف هدف وادار شده است. محمول **Guard** از فراخوانده $id'.P = id.P$ را می پرسد، و ادعا $(A, id.P)$ است، پس بدل نشسته در (A, P) به واسطه های طرف P می رسد و به هیچ طرف دیگر. هدفی که A را نپذیرد، یا طرفی درستکار، فراخوان را با سطر 2 یا سطر 3 رد می کند؛ بدل آن رد را پس می دهد، و قاعده بخش 3.6 آن را به فراخواننده خود بدل به صورت $\perp$ تحویل می دهد.

سوم، راه او به محیط هم تعیین شده است: $(A, P) \rightarrow (Z, P)$ ، که $\text{Guard}_{\mathbb{Z}}$ می پذیردش، چون $A \subseteq \mathbb{Z}.N$ و طرف ها جورند.

کد. هسته بر حسب اینکه چه کسی فرایش خوانده توزیع می کند. فراخوان از سوی محیط دستوری است برای انجام دادن؛ و فراخوان از سوی هر چیز دیگر – یک قلاب واسطه گری، یا یک $A(\cdot)$ که در درون هسته ای گذاشته شده – پیامی است برای

تحويل دادن. در هر دو جهت، بار پیام همان شکلی را دارد که **Mediate** از پیش به کار می‌برد، یعنی یک عمل به همراه یک مقدار.

حریف بدل $\mathcal{D}$	
$\text{par} := \perp$	$\text{U} := \{\langle Z, \text{serve} \rangle\}$
$\text{N} := \text{Std} \cup \text{A} \cup \text{Z}$	$\text{P} := \{0, 1\}^* \cup \{\text{A}, \text{Z}\}$
$\text{pid} := \text{A}$	
$\text{id}' \text{ از } \text{id.A}(\text{in})$	
1: if $\text{id}'.F = \text{Z}$ then	
2: in تجزیه $(\text{id}''.\text{Op}, x)$ به صورت را	برگردان $\perp$ تجزیه‌ناپذیر: /
3: return $\text{id}''.\text{FullOp}(x)$ as id	بده انجام را دستور /
4: else	
5: return $(\text{Z}, \text{id.P}).\text{FullZ}(\text{id}', \text{in})$ as id	چیز همه و فراخوانده با بسپارش، Z به /

سطر 3 جهت بیرونی است: محیط نام واسط هدف و یک ورودی را می‌برد، و بدل درست همان فراخوان را می‌گذارد و درست همان پاسخ را برمی‌گرداند. چیزی بازرسی نمی‌شود و چیزی به یاد نمی‌ماند – بدل هیچ حالتی نگه نمی‌دارد، پس Initialize ندارد. سطر 5 جهت درونی است: هر چه به حریف می‌رسد، به همراه هویتی که از آن آمده، به محیط سپرده می‌شود، و هر چه محیط پاسخ دهد بی‌تغییر به آن فراخوانده برگردانده می‌شود. آن دو سطر همان یک بازپخش‌اند که در دو جهت مخالف خوانده شده، و همین است که بدل را شفاف می‌کند.

دستوری که سطر 2 نتواند تجزیه‌اش کند با $\perp$ پاسخ می‌گیرد: نگهبان‌ها هر چه را بدل نتواند بگذارد با rej پاسخ می‌دهند، اما بار پیامی که نام هیچ فراخوانی را نبرد به مقداری از خودش نیاز دارد، و $\perp$ همان است که ماشینی با هیچ حرفی برمی‌گرداند. یک جزء ارزش درنگ دارد. فراخوانده id' در سطر 5 به Z سپرده می‌شود و نه تنها بار پیام: بی آن، محیط نمی‌توانست بفهمد کدام واسط دارد می‌پرسد، و بازپخی که این را از دست بدهد نمی‌تواند شفاف باشد.

نشاندن حریف در میانه. تمامیت حکمی است در باب بردن حریف از اجرا به درون ماشین‌های پیرامونش، و برای هر حرکت نامی لازم دارد. هر دو، یک عمل دیده‌شده از دو سویند: $\mathcal{A}$ میان ماشینی و واسط بدل نشانده می‌شود، پس آنچه پیش‌تر به $\mathcal{A}$ می‌رسید حالا در درون به او می‌رسد، و آنچه $\mathcal{A}$ پیش‌تر می‌گذاشت حالا به صورت دستوری به $\mathcal{D}$ بیرون می‌رود.

تعریف 4.26 (حریف جذب‌شده در محیط). بگذارید $\mathcal{Z}$ محیطی باشد و $\mathcal{A}$ حریفی. محیط $\mathcal{Z}^{\mathcal{A}}$ همان دسته (تعریف 4.13) $\mathcal{Z}$ با یک رونوشت از $\mathcal{A}$ است، که آن رونوشت هویت‌هایش را نگه نمی‌دارد؛ این دسته تنها $\text{IDS}(\mathcal{Z})$ را اشغال می‌کند و خروجی و par را از $\mathcal{Z}$ می‌گیرد. فراخوانی که $\mathcal{Z}$ روی هویتی $\mathcal{A}$ می‌گذارد در درون به دست واسط $\mathcal{A}$ در آنجا خدمت می‌شود، و فراخوانی که $\mathcal{A}$ روی هویتی Z می‌گذارد به دست واسط $\mathcal{Z}$. هر فراخوان دیگری که $\mathcal{A}$ می‌همان می‌گذارد، روی $\text{id}''.\text{FullOp}(x)$ در حال دوییدن

در (A, P) ، به این دستور بازنویسی می‌شود

$$(A, P).FullA(id''.Op, x) \text{ as } (Z, P),$$

که کلیدش طرفی است که A در آن می‌دود و هرگز طرف هدفش نیست، پس فراخوانی که A جذب نشده تنها به عنوان (A, P) می‌توانست بگذارد در همان طرف بازپخش می‌شود و درست همان جا که بود پذیرفته یا رد می‌شود. فراخوانی که از هویتی A با بار پیام (id', in) می‌رسد، به واسطه A در (A, P) سپرده می‌شود، با نگرهبان و همه چیز، به عنوان فراخوانی از id' .

تعریف 4.27 (حریف نشانده پیش از شبیه‌ساز). بگذارید S شبیه‌سازی باشد و A حریفی. شبیه‌ساز $S[A]$ همان دسته (تعریف 4.13) S با یک رونوشت از A است؛ این دسته $IDS(A)$ را اشغال می‌کند و باقی میدان‌های S را حمل می‌کند – چون S است و نه A ، که او به جایش می‌ایستد، پس $S[A]$ هر جا که S شبیه‌ساز باشد شبیه‌ساز است. فراخوان از هویتی Z را A میهمان خدمت می‌کند؛ و فراخوان از هر شناسه فرابند دیگری، به ویژه یک قلاب واسطه‌گری، را S . فراخوان‌های A میهمان روی هویتی Z – راه بازگشتش به محیط – زیر (A, P) از دسته بیرون می‌روند به سوی Z بیرونی؛ فراخوان‌های بیرونی دیگرش به دستور تعریف 4.26 بدل می‌شوند، که S آن را در (A, P) به عنوان فراخوانی از (Z, P) خدمت می‌کند؛ و فراخوانی که S روی هویتی Z می‌گذارد، پس از گذر از $Guard_Z$ ، به A در (A, P) سپرده می‌شود.

هیچ‌یک از این دو ساخت گونه تازه‌ای از ماشین نیست. دسته $\mathcal{Z}^A$ آنچه محیط اشغال می‌کند را اشغال می‌کند و آنچه $\mathcal{Z}$ ادعا می‌کرد را ادعا می‌کند، پس درست وقتی مجاز است که $\mathcal{Z}$ مجاز باشد. او هیچ هویت A را اشغال نمی‌کند، چون آن‌ها در اجرای سمت راست مال بازپخش‌اند، پس بنا بر تعریف 4.13 A میهمان هیچ هویتی از خودش ندارد که فراخوانی را زیر آن بگذارد؛ و از همین جاست آن بازنویسی بالا، و از همین جاست آن تعهد – که در بند «جانشینی» در زیر انجام می‌شود – که ادعای بازنویسی‌شده را در برابر خموش‌کننده $\mathcal{Z}$ بسنجیم. جذب در فصل 4 آن حالت دیگر تعریف 4.13 است، و همین است که چرا کارکرد میهمان در آنجا به جایش ادعایش را نگه می‌دارد. و در همان حال، $S[A]$ را $IDS(A)$ اشغال می‌کند، که همان معنای ایستادن در جایگاه حریف است.

قضیه. تقلید تنها نسبت به بدل، همان تعریف 4.11 است با رده حریف که به همان یک ماشین $\mathcal{D}$ تنگ شده. ادعا این است که هیچ از دست نمی‌رود. یک شرط ملایم بر حریف لازم است، و آن بهای بازپخش‌بودن بدل است.

تعریف 4.28 (بی‌اعتنا به رد). حریفی بی‌اعتنا به رد است اگر رفتارش تغییر نکند وقتی هر rej که به عنوان پاسخ فراخوانی که خودش گذاشته دریافت می‌کند، با

⊥ عوض شود.

بدل چنین پاسخی را بازپخش می‌کند، و بنا بر بخش 3.6 return خودش از یک rej به صورت ⊥ به فراخوانده می‌رسد؛ پس آنجا که حریف جذب نشده با گذاشتن فراخوانی رد شده rej را مستقیم می‌خواند، همان حریف در حال راندن بدل ⊥ می‌خواند. حریف بی‌اعتنا به رد نمی‌تواند آن دو را از هم تشخیص دهد، و همین است که بازگروه‌بندی زیر لازم دارد. خود بدل بی‌اعتنا به رد است – پاسخش را بازپخش می‌کند بی‌آنکه رویش شاخه بزند – پس این فرض برای ردۀ بدل هیچ هزینه‌ای ندارد، و حریف واقعی‌ای که سیگنال رد درون چارچوبی را بخواند در هر حال یک صنعت مدل‌سازی است و نه یک حمله.

به زبان غیررسمی، قضیه 4.29 می‌گوید تقلید تنها نسبت به بدل هیچ از دست نمی‌دهد: هر چه حریف واقعی به دست می‌آورد، محیط می‌تواند با راندن بدل به دست آورد، و شبیه‌ساز $\mathcal{A}$ همان $\mathcal{S}_{\mathcal{D}}[\mathcal{A}]$ است – همان یک شبیه‌سازی که فرض فراهم می‌کند، با $\mathcal{A}$ که پیش از آن نشانده شده – که تنها به $\mathcal{A}$ وابسته است و نه به $\mathcal{Z}$.

قضیه 4.29 (تمامیت حریف بدل). بگذارید π و φ سیستم‌هایی باشند که هسته‌هایشان هیچ فراخوان پاسخگو نمی‌گذارند (تعریف 9.2)، $\varepsilon \geq 0$ ، $\mathbb{A}$ ردۀ حریف‌های بی‌اعتنا به رد باشد (تعریف 4.28)، $\mathbb{S}$ ردۀ شبیه‌سازها، و $\mathbb{Z} \subseteq \mathbb{Z}_{\pi, \varphi}$. فرض کنید

. تقلید-UC می‌کند $(\{\mathcal{D}\}, \mathbb{S}, \mathbb{Z})$ و در برابر ε را در φ کارکرد π

آنگاه

, تقلید-UC می‌کند $(\mathbb{A}, \mathbb{S}', \mathbb{Z}')$ و در برابر ε را در φ کارکرد π

که در آن

$$\begin{aligned}\mathbb{S}' &= \{ \mathcal{S}[\mathcal{A}] : \mathcal{S} \in \mathbb{S}, \mathcal{A} \in \mathbb{A} \}, \\ \mathbb{Z}' &= \{ \mathcal{Z} \in \mathbb{Z}_{\pi, \varphi} : \mathcal{Z}.\text{par} \cap \{ \text{id} : \text{id}.F = \mathcal{Z} \} = \emptyset \\ &\quad \wedge \mathcal{Z}^{\mathcal{A}} \in \mathbb{Z} \text{ به‌ازای هر } \mathcal{A} \in \mathbb{A} \}.\end{aligned}$$

و به‌صراحت، بگذارید $\mathcal{S}_{\mathcal{D}} \in \mathbb{S}$ همان شبیه‌سازی باشد که فرض برای $\mathcal{D}$ فراهم می‌کند. آنگاه

$$\forall \mathcal{A} \in \mathbb{A} \quad \forall \mathcal{Z} \in \mathbb{Z}' :$$

$$\text{Adv}_{\pi, \varphi}^{\text{uc}}(\mathcal{Z}, \mathcal{A}, \mathcal{S}_{\mathcal{D}}[\mathcal{A}]) \leq \text{Adv}_{\pi, \varphi}^{\text{uc}}(\mathcal{Z}^{\mathcal{A}}, \mathcal{D}, \mathcal{S}_{\mathcal{D}}) \leq \varepsilon,$$

پس شبیه‌سازی که تعریف 4.11 می‌خواهد همان $\mathcal{S}_{\mathcal{D}}[\mathcal{A}]$ است، که تنها به $\mathcal{A}$ وابسته است و نه به $\mathcal{Z}$.

اثبات. کل این استدلال دو بازگروه‌بندی است، یکی به‌ازای هر جهان، که در زیر (R1) و (R2) نامیده می‌شوند. با پذیرفتن آن‌ها، دو مزیت آن نمایش جمله‌به‌جمله توافق می‌کنند و فرض در $\mathbb{Z}^A$ کرانی بر دومی می‌گذارد، که همان قضیه است. هیچ‌یک کمانک‌گذاری محض نیست – اجراهای سمت راست ماشینی حمل می‌کنند که اجراهای سمت چپ ندارند، یعنی بازپختی که در هویت‌های واگذارشده ایستاده – پس هر دو نیمه واگذارنده لم 4.18 اند و نه نیمه گزاره 4.19. (R1) همان لم است که مستقیم به کار رفته، و سه فرضش همان سه فرض قضیه‌اند. (R2) همان استدلال است که دوباره با φ به‌جای π و $\mathcal{D}$ به‌جای $\mathcal{D}$ دوانده شده، و استدلالی از خود لازم دارد چون در آنجا حریف از دسته‌ای به دسته دیگر می‌رود و نه از اجرا به بیرون: آنچه اثبات آن لم از ماشین نشسته در جایگاه به کار برد بازگفته می‌شود و نشان داده می‌شود که برای $\mathcal{S}_{\mathcal{D}}[A]$ هم برقرار است. بند پایانی ترتیب سورها را می‌سند – $\mathcal{S}_{\mathcal{D}}[A]$ تنها به A وابسته است، و فرض هیچ از $\mathbb{Z}$ نمی‌داند. $A \in \mathbb{A}$ و $\mathbb{Z} \in \mathbb{Z}'$ را ثابت کنید، و بگذارید $\mathcal{D} \in \mathbb{S}$ همان شبیه‌سازی باشد که فرض برای $\mathcal{D}$ فراهم می‌کند. عضویت در $\mathbb{Z}'$ می‌دهد $\mathbb{Z}^A \in \mathbb{Z}$ ، پس فرض به آن اعمال می‌شود، و $\langle \pi \rangle \cup \langle \varphi \rangle$ و $\mathbb{Z}.par = \mathbb{Z}^A.par$ پس مزیت سمت راست آن نمایش تعریف‌شده است. قراردادهای گزاره 4.19 – سکه‌های مستقل، نوارهای یکی‌نشده، Initialize بی‌فراخوان – برقرارند. کار دو بازگروه‌بندی است، یکی به‌ازای هر جهان:

$$\text{Exec}(\pi, \mathbb{Z}, A, \mathbb{C}) \equiv \text{Exec}(\pi, \mathbb{Z}^A, \mathcal{D}, \mathbb{C}), \quad (\text{R1})$$

$$\text{Exec}(\varphi, \mathbb{Z}, \mathcal{S}_{\mathcal{D}}[A], \mathbb{C}) \equiv \text{Exec}(\varphi, \mathbb{Z}^A, \mathcal{S}_{\mathcal{D}}, \mathbb{C}), \quad (\text{R2})$$

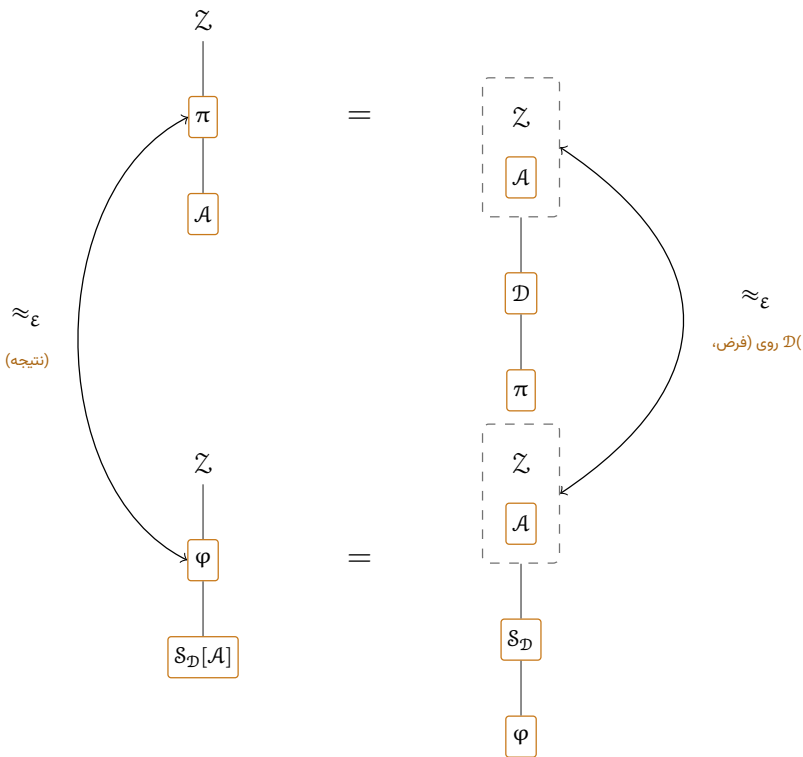
که هر یک تساوی توزیع‌هاست؛ و با پذیرفتن آن‌ها، دو مزیت جمله‌به‌جمله توافق می‌کنند و فرض در $\mathbb{Z}^A$ دومی را به ε کران می‌زند، که همان نمایش است. هیچ‌یک کمانک‌گذاری محض نیست – اجراهای سمت راست ماشینی حمل می‌کنند که اجراهای سمت چپ ندارند، یعنی بازپختی نشسته در جایگاه حریف – پس هر دو نیمه واگذارنده لم 4.18 اند و نه نیمه گزاره 4.19.

اثبات. (R1) لم 4.18 در $H = \{A\}$ با $H_{\text{keep}} = \emptyset$ به‌گونه‌ای که $\mathbb{Z}^H$ همان $\mathbb{Z}^A$ تعریف 4.26 باشد و بازپختی نشسته در $\text{IDs}(A)$ واگذارشده همان $\mathcal{D}$. سه فرض آن، سه فرض قضیه‌اند: A بی‌اعتنا به رد است چون $\mathbb{A}$ چنین است؛ هیچ هسته π فراخوان پاسخگو نمی‌گذارد، که همان فرض بر سیستم‌هاست؛ و $\mathbb{Z} \in \mathbb{Z}'$ هیچ هویت Z را خموش نمی‌کند. خانواده‌های اشغال‌گر مجازند، چون $\mathbb{Z}^A$ تنها $\text{IDs}(\mathbb{Z})$ را اشغال می‌کند و $\mathcal{D}$ همان هویت‌هایی را که A رها کرد. اثبات. (R2) اینجا A از دسته‌ای به دسته دیگر می‌رود و نه از اجرا به بیرون، پس لم 4.18 در ظاهر اعمال نمی‌شود و استدلال همان نیمه واگذارنده آن است که دوباره با φ به‌جای π و $\mathcal{D}$ به‌جای $\mathcal{D}$ دوانده شده. و اینکه این منتقل می‌شود، بسته به آن است که اثبات آن لم از ماشین نشسته در جایگاه چه به کار برد، و آن چیزی بیش از این نیست که نهبانش حکم‌های Guard_A را برمی‌گرداند، که بر حسب اینکه ادعا $\text{id}.F = Z$ دارد یا نه توزیع می‌کند، و – آنجا که A میهمان دستوری رده‌شده را می‌خواند، rej از $\mathcal{D}$ در درون دسته در سمت چپ در برابر $\perp$ محسوب از آن سوی جایگاه در سمت راست – که $\mathbb{A}$ بی‌اعتنا به رد است، درست

همان‌گونه که در (R1) آنچه تفاوت دارد این است که کدام ماشین $\mathcal{A}$ را در بر دارد – $\mathcal{Z}^{\mathcal{A}}$ در چپ، $\mathcal{S}_{\mathcal{D}}[\mathcal{A}]$ در راست – و تعریف‌های 4.27 و 4.26 آن دو را وادار می‌کنند همان تحویل‌ها را انجام دهند، که هر یک از راه واسطه نگهبانی‌شده ماشین گیرنده مسیریابی می‌شود، و آن دستور در یک سو در درون یک ماشین سفر می‌کند و در سوی دیگر از آن سوی اجرا. زنجیره $\varphi: \mathcal{S}_{\mathcal{D}} \rightarrow \mathcal{A} \rightarrow \mathcal{Z}$ در هر دو همان است، تنها در نقطه دیگری بریده شده: $\mathcal{S}_{\mathcal{D}}$ با همان دستورها به همان ترتیب و زیر همان ادعاها روبه‌رو می‌شود، بند «جانشینی» موبه‌مو برقرار است، و اشغال‌گر در هر دو حال با همان میدان‌ها داوری می‌شود – pid و P با اشغال، و N با تعریف 4.7، که $\mathcal{S}_{\mathcal{D}}$ برآورده‌اش می‌کند و $\mathcal{S}_{\mathcal{D}}[\mathcal{A}]$ به ارث می‌بردش، چون بنا بر تعریف 4.27 میدان‌های $\mathcal{S}_{\mathcal{D}}$ را حمل می‌کند؛ و هیچ نگهبانی باقی را نمی‌خواند. تعریف 4.27 تحویل سمت چپ را از راه همان واسطه نگهبانی‌شده‌ای مسیریابی می‌کند که Exec در سمت راست خدمتش می‌کند، و فراخوان‌های خود $\mathcal{A}$ میهمان به $\mathcal{Z}$ زیر (A, P) از دسته بیرون می‌روند، درست همان‌گونه که در سمت راست از Exec می‌گذرند.

نتیجه. بنا بر (R1) و (R2) دو مزیت آن نمایش برابرند، و فرض در $(\mathcal{D}, \mathcal{Z}^{\mathcal{A}})$ دومی را به ε کران می‌زند. شبیه‌ساز $\mathcal{S}_{\mathcal{D}}[\mathcal{A}]$ در $\mathcal{S}'$ است و تنها به $\mathcal{A}$ وابسته است، چون فرض هیچ از $\mathcal{Z}$ نمی‌داند، پس ترتیب سورهای تعریف 4.11 برآورده است؛ و $\mathcal{Z} \in \mathbb{Z}'$ دلخواه بود. $\square$

از دید نموداری، اثبات همان مربع جابه‌جایی قضیه 4.20 است (همان شکل بالای اثباتش)، که این بار از نیمه واگذارنده لم 4.18 ساخته شده و نه از نیمه نگه‌دارنده: آنچه به درون $\mathcal{Z}$ می‌رود بخشی از سیستم نیست بلکه خود حریف است، و بازپختی همان جا که او ایستاده بود می‌ایستد.



پایین‌چپ همان جایی است که قضیه از آن آغاز می‌کند: π و φ در سرتاسر دست‌نخورده، A اشغال‌گر جهان واقعی و $S_D[A]$ یعنی $-S_D[A]$ که پیش از آن نشانده شده - اشغال‌گر مشتق‌شده جهان ایده‌آل. یال‌های بالا و پایین همان (R1) و (R2) اثباتند: جذب A در Z ، از هر سویی که خوانده شود، باز هم تساوی توزیع‌هاست، درست همان‌گونه که در گزاره 4.19 بود - اما این بار نیمه واگذارنده لم 4.18 است که بهایش را می‌دهد، با فرض‌های (c)-(a) به جای هیچ، چون آنچه حرکت می‌کند خود حریف است و بازپختی (D)، و سپس S_D باید به جایش بایستد. همان یک Z^A جذب‌شده در هر دو سو خدمت می‌کند، و همین است که چرا یال راست اصلاً می‌تواند آن دو را بسنجد: آن یال همان فرض قضیه است، که تنها در برابر D بیان شده. با تعقیب مربع، درست همان‌گونه که در قضیه 4.20، یال چپ - نتیجه قضیه، که حالا برای هر A بی‌اعتنا به رد است و نه برای یک ماشین ثابت - وادار می‌شود: آن دو $\approx_\epsilon$ بنا بر آن دو تساوی، همان یک کمیتند.

آن سه رده همان‌گونه حرکت می‌کنند که در قضیه 4.20، و به همان دلایل. حریف‌ها آن‌هایی هستند که نتیجه رویشان سور می‌زند، و فرض تنها D را داشت؛ شبیه‌سازها حساب می‌شوند، یکی به‌ازای هر حریف، با نشانیدن آن حریف پیش از شبیه‌سازی که فرض

فراهم می‌کند؛ و محیط‌ها یک پیش‌نگاره‌اند، یعنی $\mathbb{Z}$ بیرونی درست وقتی به شمار می‌آید که آنچه در جذبِ هر A به آن بدل می‌شود یکی از آن‌هایی باشد که فرض می‌پوشاندش. گرفتنِ $\mathbb{Z}$ به‌عنوانِ بزرگ‌ترین ردهٔ مجاز، $\mathbb{Z}'$ را باز بزرگ‌ترین می‌کند – منهای تنها آن محیط‌هایی که هویت‌های $\mathbb{Z}$ خودشان را خموش می‌کنند، شرطی که هیچ مجازبودنی تحمیلش نمی‌کند و هیچ کاربردی نمی‌خواهدش، و کنار گذاشته می‌شود چون دستور تعریف 4.26 زیر ادعایی $\mathbb{Z}$ بیرون می‌رود و چنین محیطی آن را تنها در یک سو رد می‌کرد – چون جذبِ par را دست‌نخورده می‌گذارد.

اگر به معنای بخش 4.5 عینی خوانده شود، آن دو ساخت همان هزینه‌ای را دارند که حریف دارد: اگر A کرانداری $\mathbf{a}$ باشد، آن‌گاه $\mathbb{Z}^A$ به $\mathbf{a} \oplus \mathbf{z}$ نیاز دارد آنجا که $\mathbb{Z}$ به $\mathbf{z}$ نیاز داشت، و $[A]_{\mathcal{D}}$ به $\mathbf{a} \oplus \mathbf{s}$ آنجا که $\mathcal{D}$ به $\mathbf{s}$. شمارِ فراخوانی‌ها جمع می‌شود و زمان‌های هر فراخوانی ماکسیمم می‌گیرند، و هر دو مستقل از π و φ اند.

4.5 امنیت عینی

تعریف 4.11 در برابر رده‌های دلخواه بیان شده و در باب آنچه در آن‌هاست هیچ نمی‌گوید. پُرکردن‌شان با ماشین‌هایی با منابع کرانداری، هر حکمِ بخشِ پیشین را به حکمی عینی بدل می‌کند، و نسبت‌های میان رده‌ها را به حسابِ روی بودجه‌ها.

تعریف 4.30 (بودجه‌ها و رده‌های کرانداری). یک بودجه زوجی $\mathbf{a} = (t, r)$ از اعداد طبیعی است. ماشینی کرانداری $\mathbf{a}$ است اگر در هر فراخوانی دست‌بالا t گام بردارد و در کل دست‌بالا به r فراخوان پاسخ دهد؛ پس در هر اجرایی دست‌بالا $t+r$ گام می‌دود. ماشینی که در هر اجرایی هر دو کران را برآورد کرانداری $\mathbf{a}$ شمرده می‌شود، خواه قاعدهٔ ایستادن در خودش نوشته شده باشد خواه نه – و دسته‌هایی که در زیر از ماشین‌های کرانداری ساخته می‌شوند به همین معنا کرانداری دارند. بنویسیم $\mathcal{S}(\mathbf{a})$ و $\mathcal{A}(\mathbf{a})$ برای ردهٔ حریف‌های کرانداری $\mathbf{a}$ ، شبیه‌سازهای کرانداری $\mathbf{a}$ ، و محیط‌های کرانداری $\mathbf{a}$ که در $\mathbb{Z}_{\pi, \varphi}$ می‌افتند. بودجه‌ها مؤلفه‌به‌مؤلفه مرتب می‌شوند؛ و ماشین‌هایی که یک اجرا را شریکند – دسته‌ای از چند تا، یا شبیه‌سازی که حریفی را می‌دواند – با این عمل ترکیب می‌شوند

$$(t, r) \oplus (t', r') := (\max(t, t'), r + r'),$$

چون در هر فعال‌سازی یک ماشین می‌دود، پس زمانِ هر فراخوانی کرانِ بزرگ‌تر را می‌گیرد و شمارِ فراخوانی‌ها جمع می‌شود.

بودجه بخشی از ماشین است، و نه وعده‌ای در بابِ او. کران‌گذاشتن بر آن سه ماشین جایگاه هنوز برای کران‌گذاشتن بر یک اجرا بس نیست – اعضای سیستم هم ماشین‌اند، و هسته‌ای که روی خودش بازگشتی شود، یا دو عضو که تا ابد یکدیگر را فرا بخوانند، ژتون را از هر ماشینِ بودجه‌داری دور می‌برند و چیزی برای پس‌گرفتنش نمی‌بود. پس حکم‌های عینی روی سیستم‌های بودجه‌دار خوانده می‌شوند: هر عضو

هر سیستمی که مقایسه می‌شود بودجه‌ای از خودش حمل می‌کند، به همان معنای این تعریف که موبه‌مو اعمال شده. آن‌گاه اجرا تعداد متناهی ماشین می‌دواند، هر یک تعداد متناهی گام برمی‌دارد، و در هر لحظه دقیقاً یک ماشین گام می‌زند، پس اجرا در کل کران $\sum_i t_i$ ماشین‌هایش می‌ایستد و Exec رو تعریف شده است. ایستادن بر سر بودجه مقداری داده شده است: ماشینی که در میان فراخوان می‌ایستد هر فراخواننده معلق بر خودش را با $\perp$ از سر می‌گیرد، و محیطی که دور از بازگشت ریشه‌اش می‌ایستد چنان خوانده می‌شود که 0 برگردانده است. رده‌ها در بودجه یکنوایند، یعنی $\mathbb{A}(\mathbf{a}) \subseteq \mathbb{A}(\mathbf{a}')$ هرگاه $\mathbf{a} \leq \mathbf{a}'$ ، و همین برای $\mathbb{S}$ و $\mathbb{Z}_{\pi, \varphi}$. و همین لم 4.23 را حکمی در باب اعداد می‌کند: کوچک کردن بودجه حریف، بزرگ کردن بودجه شبیه‌ساز، یا کوچک کردن بودجه محیط، هر سه تقلید را نگه می‌دارند.

تعریف 4.31 (مزیت عینی UC). بگذارید π و φ سیستم‌های بودجه‌دار باشند، $\mathbf{a}$ و $\mathbf{s}$ بودجه‌هایی برای حریف و شبیه‌ساز، و $\mathbb{Z} \subseteq \mathbb{Z}_{\pi, \varphi}$ رده محیط‌ها. بگذارید

$$\text{Adv}_{\pi, \varphi}^{\text{UC}}[\mathbf{a}, \mathbf{s}, \mathbb{Z}] := \max_{\mathbf{A} \in \mathbb{A}(\mathbf{a})} \min_{\mathbf{S} \in \mathbb{S}(\mathbf{s})} \max_{\mathbb{Z} \in \mathbb{Z}} \text{Adv}_{\pi, \varphi}^{\text{UC}}(\mathbb{Z}, \mathbf{A}, \mathbf{S}),$$

و $\text{Adv}_{\pi, \varphi}^{\text{UC}}[\mathbf{a}, \mathbf{s}, \mathbb{Z}]$ را کوتاه‌نویسی حالت $\mathbb{Z} = \mathbb{Z}_{\pi, \varphi}(\mathbf{z})$ بگیرید، یعنی بزرگ‌ترین رده آن بودجه.

آرگومان سوم یک رده است و نه یک بودجه، چون بزرگ‌ترین رده به آن زوج سیستم‌ها وابسته است، و حکم‌های زیر مزیت‌هایی را می‌سنجند که روی زوج‌های متفاوت گرفته شده‌اند. هر جا رده مشترکی خواسته باشد، نمادگذاری باید بتواند بگویدش. آن سه عملگر، همان سه سور تعریف 4.11 اند که به عنوان یک مقدار خوانده شده‌اند، به همان ترتیب و به همان دلیل: شبیه‌ساز پس از حریف و پیش از محیط انتخاب می‌شود. اکسترمم‌ها حاصل می‌شوند، هرچند دلیلش لحظه‌ای وقت می‌خواهد، و آن دلیل متناهی بودن است و نه فشردگی. ماشین کراندار $\mathbf{a}$ در کل دست‌بالا $t_{\mathbf{r}}$ گام برمی‌دارد، پس دست‌بالا $t_{\mathbf{r}}$ نماد از هر ورودی و دست‌بالا $t_{\mathbf{r}}$ سکه می‌خواند؛ و تا حد رفتار، او نگاهی است از آنچه خوانده به آنچه کرده، و چنین نگاهی‌ها به شمار متناهی‌اند. سکه‌هایش آن‌ها را با احتمال‌های دوگانی مخرج $2^{t_{\mathbf{r}}}$ وزن می‌دهند، پس رده کراندار در کل تعداد متناهی توزیع رفتار القا می‌کند. و با بودجه‌دار بودن اعضای سیستم‌ها، $\Pr[\text{Exec} = 1]$ در هر انتخابی از رفتارها عددی خوش‌تعریف است، پس هر مزیت تعریف 4.5 یکی از تعداد متناهی مقدار را می‌گیرد، و ماکسیم یا مینیمم تعداد متناهی مقدار حاصل می‌شود. (روی رده دلخواه $\mathbb{Z}$ ، آن $\max$ بیرونی به عنوان سوپرمیم خوانده می‌شود.) پس

تقلید-UC می‌کند $(\mathbb{A}(\mathbf{a}), \mathbb{S}(\mathbf{s}), \mathbb{Z}_{\pi, \varphi}(\mathbf{z}))$ و در برابر ε را در φ کارکرد π

درست وقتی برقرار است که آن کمیت دست‌بالا ε باشد – و آن مینیمم حاصل‌شده همان است که جهت عکس را در مرز دقیق می‌کند. آن‌گاه یک ادعای امنیت عینی، یک

نامساوی تنه‌است که سه بودجه و یک احتمال را به هم مربوط می‌کند، بی هیچ پارامتر امنیتی و هیچ مجانبی در چشم.

ویژگی‌ها

تقلید دو سیستم را می‌سنجد. یک ویژگی از یکی چیزی می‌خواهد: که هیچ حریفی بازی بازی‌شده در برابر او را نبرد – که گزاره 5.9 آن را به نبرد بدل فرو می‌کاهد، با جایگاهی که در محیط جذب می‌شود. برای هیچ یک ماشین آلات تازه چندان نمی‌خواهیم. بازی کارکردی است مثل هر کارکرد دیگر، محیط همان حریفی است که بازی‌اش می‌کند، و لوله‌کشی بخش‌های پیشین از پیش حل کرده که چه کسی چه چیزی را می‌تواند فرا بخواند – پس یک ویژگی عبارت است از یک سیستم، یک رده محیط، و یک پیشامد، و میدان‌های تعریف 1.1 بیشتر کاری را می‌کنند که قواعد یک بازی روی کاغذ می‌کنند. آنچه باید افزوده شود کم است، و هر جا که بیاید نامش برده می‌شود: یک شرط بهداشتی بر سیستم بازی، و آن سه شرطی که بخش 4.4 از پیش از هر چه جابه‌جا می‌کند می‌خواهد.

تعریف 5.1 (بازی، سیستم بازی). یک بازی برای $\mathcal{F}$ کارکردی استاندارد به نام $\mathcal{G}_m$ است با

$$\begin{aligned}\mathcal{G}_m.\text{pid} &:= (\mathcal{G}_m.F, 0, 0), & \mathcal{G}_m.P &:= \mathcal{F}.P \cup \{Z\}, \\ \mathcal{G}_m.N &:= Z, & \mathcal{G}_m.U &:= \{(\mathcal{F}, \text{serve}_{\mathcal{G}_m})\},\end{aligned}$$

که در کنار واسطه‌های خودش یک یا چند نهایی‌سازی حمل می‌کند: واسطه‌هایی که نامشان با Fin آغاز می‌شود، پیشوندی که برای آن‌ها رزرو شده و هیچ واسط دیگری آن را حمل نمی‌کند، که هر یک در ریشه خدمت می‌شود، هر یک یک بیت برمی‌گرداند، و پس از آنکه یکی از آن‌ها خدمت شد هیچ واسط $\mathcal{G}_m$ – نهایی‌سازی یا غیر آن – خدمت نمی‌شود. اینجا

$$\text{serve}_{\mathcal{G}_m}(\mathcal{G}_m, \mathcal{F}) : \iff \mathcal{F}.P \supseteq \mathcal{G}_m.P \setminus \{Z\} \wedge \mathcal{G}_m.\text{pid} \in \mathcal{F}.N$$

همان serve است با ریشه که معاف شده. یک سیستم بازی عبارت است از $\mathcal{G}_m \cup \sigma := \{\mathcal{G}_m\}$ برای سیستم σ که $\mathcal{F}$ و هر چه $\mathcal{F}$ به کار می‌برد را در بر دارد و $\mathcal{G}_m$ را در بر ندارد، با $\text{wf}(\gamma)$ ، و با γ که به معنای نکته 6.5 در $\{\mathcal{G}_m\}$ تنگ است.

پس یک بازی می‌تواند چند ویژگی حمل کند: مثلاً FinCor و FinUF روی کارکرد امضا،

که اوراکل‌هایی را شریکند که ثبت می‌کنند چه چیزی امضا شده و تنها در حکمی که در پایان از آن‌ها خوانده می‌شود تفاوت دارند. محیط با انتخاب اینکه کدام نهایی‌سازی را فرا بخواند انتخاب می‌کند که بر کدام داوری شود، و آن انتخاب آخرین کاری است که می‌کند. پس بازی با چند نهایی‌سازی عطفی از ویژگی‌ها نیست، بلکه پوسته‌ای است که چند ویژگی را در بر دارد و هر یک مزیت خودش را در زیر دارد؛ محیطی که یکی را ماکسیمم کند لازم نیست دیگری را ماکسیمم کند، و هیچ چیز اینجا از او این را نمی‌خواهد.

بازی $\mathcal{G}_m$ برای $\mathcal{F}$: پوسته

$\text{par} := \perp$, $\mathcal{U} := \{(\mathcal{F}, \text{serve}_{\mathcal{G}_m})\}$, $\mathcal{N} := \mathbb{Z}$, $\mathcal{P} := \mathcal{F}.P \cup \{Z\}$, $\text{pid} := (\mathcal{G}_m.F, 0, 0)$

[:Initialize\(\)](#)

1: $\text{done} \leftarrow \text{false}$

2: بازی خود متغیرهای ...

[id' از id.Op\(in\)](#)

/ می‌شود بسته نهایی‌شدن با

3: **require** $\neg \text{done}$

4: بخواند $\text{id}.P$ طرف در را $\mathcal{F}$ می‌تواند که بازی، خود کد ...

5: **return** out

[id' از id.Leak\(\)](#)

/ نمی‌دارد نگه را طرفی هیچ راز بازی

6: **return** $\perp$

نهایی‌سازی‌های $\mathcal{G}_m$ ، یکی به‌ازای هر ویژگی؛ پوسته $\mathcal{F}$ مشترک است

هر یک به نام $\text{Fin} \dots$ ، هر یک خدمت‌شده در ریشه، هر یک بازی را می‌بندد

[id' از id.Fin\(\)](#) (شکل هر نهایی‌سازی)

/ نهایی‌سازی یک تنها

7: **require** $\neg \text{done}$

8: **require** $\text{id}.P = Z$

/ می‌ماند درست‌کار که ریشه، در

9: $\text{done} \leftarrow \text{true}$

10: $w \leftarrow$ پوسته حالت از خوانده‌شده ویژگی، این حکم

11: **return** w

[id' از ... id.FinUF\(\), id.FinCor\(\)](#)

12: ویژگی هر خود حکم سپس و سطر، سه همان ...

هر میدان وادار شده است، و ارزش گفتن دارد که با چه چیزی.

شناسه فرایند. در هر آزمایش یک بازی هست، پس نشست و نمونه هیچ خبری حمل نمی‌کنند و ما هر دو را در 0 تعیین می‌کنیم. نشست 0 قرارداد نیست بلکه یک خواسته است: محیط با ادعای هویت Z خودش به $\mathcal{G}_m$ می‌رسد، که سطر 2 می‌پذیردش چون $Z \subseteq \mathcal{G}_m.N$ ، و آن‌گاه سطر 3 می‌پرسد $\mathcal{G}_m.s = Z.s = 0$ چون $\mathcal{G}_m$ جهانی نیست. نکته 7.17 همان مشاهده است به‌طور کلی. بنا بر تعریف 1.3 یک $\mathcal{F}$ محلی نشست فراخواننده را شریک است، پس کل آزمایش در نشست 0 می‌نشیند، که همان جایی است که ماشین‌هایی که اجرا فراهم می‌کند هم در آن زندگی می‌کنند.

طرف‌های خدمت‌گیرنده. هسته تنها هویت خودش را ادعا می‌کند (سطر 5) و **Guard** می‌پرسد $id'.P = id.P$ ؛ پس G_m در طرف P به $\mathcal{F}$ در طرف P می‌رسد و به هیچ طرف دیگر؛ بازی $\mathcal{F}$ را هر بار در یک طرف تمرین می‌دهد، و محیط با انتخاب اینکه کدام هویت G_m را خطاب کند طرف را انتخاب می‌کند، و ریشه‌اش بنا بر سطر 3 آزاد است برای هر طرفی عمل کند. و $\mathcal{F}$ و P از همین‌جاست. ریشه برای نهایی‌سازی‌ها افزوده شده، و بند بعد می‌گوید چرا.

فراخواننده‌های پذیرفته. Z و هیچ چیز دیگر. نه **Std**، که G_m را جهانی می‌کرد و می‌گذاشت هر کارکرد استاندارد بازی را نهایی کند؛ و نه **A**، که آن را به حریف می‌سپرد. با Z تنها، سطر 2 هر فراخواننده‌ای جز محیط را رد می‌کند.

نام. دلخواه است، و نتیجه 7.16 همین را می‌گوید: تغییرنام بازی و $\mathcal{F}.N$ با هم هیچ احتمالی را تغییر نمی‌دهد، پس ویژگی‌ای که در برابر بازی‌ای با یک نام اثبات شود، در برابر همان بازی زیر هر نام دیگری هم برقرار است. و این مهم است چون سطر 5 می‌گذارد بازی تنها هویت خودش را ادعا کند، پس $\mathcal{F}.N$ باید نام بازی را ببرد، در حالی که آن $\mathcal{F}$ ی که در درون یک پروتکل عرضه می‌شود نام آن پروتکل را می‌برد؛ و آن نتیجه همان است که این دو را یک ویژگی می‌کند. آنچه دلخواه نیست، کد فراخواننده است، که هیچ تغییرنامی به آن دست نمی‌زند.

وابستگی. $serves_{G_m}$ ریشه را معاف می‌کند چون ریشه هویت حساب‌داری خود بازی است: $\mathcal{F}$ لازم نیست به طرفی خدمت دهد که بازی هرگز در آن فرایش نمی‌خواند. بازی‌ای که حکمش ناچار است از $\mathcal{F}$ پرس‌وجو کند – مثلاً برای تأیید جعلی که ادعا شده – دو گزینه دارد، و همان گزینه‌های معمولند: یا در طول فراخوان‌های اوراکل هر چه حکم لازم دارد را ثبت کند، یا از $\mathcal{F}$ بخواهد به ریشه هم خدمت دهد، که در آن صورت $serves_{G_m}$ به $serves$ بدل می‌شود.

تنگی. میدان‌های بالا به‌تنهایی بس نیستند، و آن شکاف شکاف نکته 6.5 است. محیط مجاز روی $\langle \gamma \rangle$ خموش است، پس هیچ هویتی از نامی که γ به کار می‌برد را نمی‌تواند ادعا کند – اما تعریف 4.10 تنها او را از میهمان‌کردن در هویت‌هایی که γ اشغال می‌کند منع می‌کند، و کارکرد میهمان واسطی کامل است که هویت خودش را از راه خموش‌کننده خودش ادعا می‌کند (تعریف 4.13). پس محیط می‌تواند ماشینی را در شناسه فرایند اشغال‌نشده‌ای از نامی به‌کاررفته میهمان کند: مثلاً در $(G_m.F, 0, 1)$ ، یعنی نمونه دوم ساختگی از نام خود بازی. عضوی از σ که فراخواننده‌اش را با نام می‌پذیرد – همان الگوی محلی فصل 1 – آن ماشین را می‌پذیرد، و فراخوان‌هایش از هر سطر **Guard** می‌گذرند: نام در N است، طرف‌ها جورند، نشست‌ها جورند، و **Mediate** در طرف درستکار شلیک نمی‌کند. آن‌گاه محیط اوراکلی روی $\mathcal{F}$ در دست داشت که بازی هیچ از آن ثبت نمی‌کند، و $Pr[Win_{Fin}] = 1$ برای هر بازی‌ای که حکمش این است که حریف چیزی تولید کرده که به او داده نشده بود.

تنگی در $\{G_m\}$ این را رد می‌کند، و کاملاً ردش می‌کند. هر عضوی جز بازی، کلی شناسه‌های فرایندی را می‌پذیرد که سیستم اشغالشان کرده، پس نمونه ساختگی در سطر 2 شکست می‌خورد؛ و شرط طرف نکته 6.5 هر هویتی را که فراخواننده سنجاق‌شده

می‌توانست از آن سخن بگوید اشغال شده می‌کند، همان جا که تعریف 4.10 میهمانی را ممنوع می‌کند. آن یک راهی که تنگی باز می‌گذارد – به عضوی از خود I ، که اینجا همان بازی است – با $\text{gm.N} = Z$ بسته می‌شود: کارکرد میهمان استاندارد است و شناسه فرایند استاندارد را ادعا می‌کند، که Z آن را کنار می‌گذارد، و هویت‌های Z خود محیط میهمان نیستند بلکه از خود اویند. پس بازی همان یک عضوی است که می‌تواند رده خالی‌ای از شناسه‌های فرایند را بپذیرد، و درست به این سبب در این کار ایمن است که Z هیچ نام استاندارد نیست.

نکته 5.2 (چرا نهایی‌سازی در ریشه خدمت می‌شود). شناسه‌های طرف در $\{0, 1\}^*$ می‌توانند فاسد شوند، و سطر 4 از واسط کامل فراخوان طرف فاسد را پیش از دوییدن هسته به حریف می‌سپارد. پس نهایی‌سازی‌ای که در چنین طرفی خدمت می‌شد، هرگاه آن طرف فاسد بود، حریف پاسخش می‌داد، و پاسخ «آیا بردم؟» هر چیزی می‌بود که حریف بگوید – درست همان مسئله نکته 3.1، و هیچ آزمونی در درون هسته نمی‌تواند تعمیرش کند، چون واسطه‌گری در پوشانه بالای آن رخ می‌دهد. ریشه همان چاره است، و چارچوب از پیش فراهمش می‌کند: FullCorrupt می‌پرسد $\text{id.P} \in \{0, 1\}^*$ ، پس Z هرگز به C وارد نمی‌شود، پس Mediate در $(\text{gm.pid}, Z)$ هرگز نمی‌تواند شلیک کند و بیتی که سطر 10 حساب می‌کند همیشه مال خود بازی است. شناسه‌های طرف رزرو برای درستکار نگه داشتن آن سه ماشین فراهم‌شده معرفی شده بودند؛ و بازی را بی هیچ هزینه اضافی درستکار نگه می‌دارند. آن‌گاه سطر 8 هر نهایی‌سازی را به همان هویت محدود می‌کند، پس هیچ جای دیگری جای بردن نیست.

نکته 5.3 (یک نهایی‌سازی، یک بار، و در آخر). اینکه دقیقاً یک نهایی‌سازی پاسخ می‌گیرد، و اینکه آن آخرین فراخوانی است که بازی پاسخ می‌دهد، اعمال می‌شود و مفروض گرفته نمی‌شود – و همان یک پرچم هر دو کار را می‌کند. سطر 9 پرچمی می‌نشانده که پوشانه نمی‌تواند دورش بزند، سطر 7 هر نهایی‌سازی پس از آن را رد می‌کند، چه به نام خودش چه به نام دیگری، و سطر 3 از آن پس هر واسط دیگری را رد می‌کند؛ و **require** رده‌شده بنا بر بخش 3.6 **rej** برمی‌گرداند، پس محیط می‌آموزد که بازی بسته است و هیچ چیز دیگر – دست‌کم در طرف درستکار. در طرف فاسد، سطر 4 پیش از ورود به هسته می‌رباید، پس فراخواننده به جای رد شدن پاسخ حریف را می‌گیرد؛ و این هیچ چیز را تغییر نمی‌دهد، چون جایگاه نمی‌تواند حالت بازی را بنویسد و پیشامد بُرد از پیش تعیین شده است. هیچ چیز هم جلوی محیط را از فراخواندن کارکردی جهانی پس از آن نمی‌گیرد، و این درست است – بازی تمام شده، و آنچه او آن‌گاه بکند نمی‌تواند تغییر دهد که چه بُرده است. ترتیب سطرها به همان دلیلی مهم است که $\mathcal{F}_{\text{Sig}}$ جدول‌هایش را از نو آزمون می‌کند: سطر 10 می‌تواند فراخوان بگذارد، و بخش 3.6 می‌گذارد نمونه معلق دوباره وارد شود، پس

پرچم پیش از حساب کردن حکم نشانده می‌شود و نه پس از آن. اگر برعکس بود، دو نهایی‌سازی تودرتو می‌توانستند هر دو بازی را باز ببابند. اینکه همان یک پرچم بر همه حکم می‌کند، همان است که آویختن چند نهایی‌سازی به یک پوسته را ایمن می‌کند. اگر هر یک پرچمی از خودش نگه می‌داشت، محیط می‌توانست دو بار نهایی کند و در یک اجرا بر دو ویژگی داوری شود – مثلاً حکم درستی را از حالتی بخواند که حکم جعل‌ناپذیری از پیش از آن حساب شده بود، با اوراکل‌هایی در میانه. شریک‌بودن *done* همان است که یک اجرا را به یک ویژگی محدود می‌کند، و همین است که چرا نهایی‌سازی‌ها در جعبه خودشان با پرچم پوسته می‌نشینند و نه هر یک با حساب‌داری خودش.

نکته 5.4 (فساد چه می‌خرد، و چه هزینه می‌کند). در طرف $P \in \{0, 1\}^*$ ، واسطه‌های خود بازی مثل واسطه‌های هر کارکرد دیگری پوشانده شده‌اند، پس به محض $P \in \mathbf{C}$ ، سطر 4 فراخوانی به $\mathcal{G}_m$ در P را می‌رباید: هسته نمی‌دود، بازی هیچ ثبت نمی‌کند، و فراخواننده پاسخ حریف را می‌گیرد. این هیچ برای محیط نمی‌آورد – *Mediate* پاسخی را جایگزین می‌کند و نه اینکه هسته‌ای را بدواند، پس حالت بازی هرگز به دست حریف نوشته نمی‌شود – و اوراکل آن طرف را از او می‌گیرد. و چون فساد تنها با طرف اندیس‌گذاری می‌شود (بخش 3.1)، فاسد کردن طرفی از $\mathcal{F}$ اوراکل بازی را در آن طرف هم می‌بندد. پس بازی‌ای که اوراکلی در طرف فاسد بخواهد باید آن را جایی خدمت کند که فساد به آن نمی‌رسد: در ریشه، یا در طرفی از خودش بیرون $\mathcal{F}.P$ ، که هر جا آن اوراکل هیچ فراخوانی روی $\mathcal{F}$ لازم نداشته باشد می‌تواند چنین کند.

آنچه حریف را از هسته‌های بازی بیرون نگه می‌دارد هیچ‌یک از آن دو سطری نیست که آدم به سویشان دست دراز می‌کند. سطر 3 $\mathcal{A}$ را در طرف فاسد می‌پذیرد و تنها در طرف‌های درست‌کار ردش می‌کند؛ و $\text{id}' . \mathcal{F} \neq \mathcal{A}$ *Mediate* را آزمون می‌کند، پس فراخوانی را که خود حریف می‌گذارد نمی‌رباید. پس فراخوانی از جایگاه در طرف فاسد هسته $\mathcal{G}_m$ را می‌دواند، درست همان‌گونه که بخش 3.5 در باب طرف فاسدی می‌گوید که $\mathcal{A}$ خطایش می‌کند. آنچه ردش می‌کند $\mathcal{G}_m.N = \mathbf{Z}$ در سطر 2 است، و هیچ چیز دیگر. تنگی آن میدان پاکیزه‌کاری نیست بلکه تنها چیزی است که میان حریف و کد خود بازی ایستاده است.

و $\mathcal{G}_m \text{ Leak} \perp$ مقدار $\perp$ برمی‌گرداند چون نشئت از *Guard* نمی‌گذرد (بخش 3.2): حریف می‌تواند آن را در طرف فاسد بخواند، هر چه $\mathcal{G}_m.N$ بگوید، پس بازی نباید هیچ چیزی آنجا نگه دارد که حاضر به واگذاشتنش نیست.

نکته 5.5 (چند نمونه از $\mathcal{F}$). بازی می‌تواند به هر چند نمونه از $\mathcal{F}$ که بخواهد خطاب کند، و آن نمونه‌ها به دو شکل می‌توانند متفاوت باشند. نمونه‌های داخل نشست خود بازی هیچ چیز اضافی لازم ندارند، چون سطر 3 نشست‌ها را می‌سجد و

شماره نمونه را نادیده می‌گیرد، پس $(\mathcal{F}, F, 0, i)$ به‌ازای هر i در دسترس است. نمونه‌های داخل نشست‌های دیگر $\text{global}(\mathcal{F})$ لازم دارند، که همان است که آن سطر را می‌بخشد. در هر دو حال، ادعا هویت خود $\mathcal{G}m$ است، چون سطر 5 هسته‌هایش را روی هر چیز دیگر خموش می‌کند، پس هر نمونه یک و همان فراخواننده را می‌بیند و کنترل دسترسی یک بار تعیین می‌شود. و آن نمونه‌ها واقعاً رونوشتند: بنا بر فصل 7 همان کد را می‌دوانند و میدان‌هایشان تنها در شناسه فرایند تفاوت دارد، در حالی که بخش 3.6 به هر یک حالت خودش و سکه‌های خودش را می‌دهد. پس ویژگی چندنمونه‌ای - n کلید، n نشست - با شمردنشان به‌دست بازی نوشته می‌شود، بی هیچ قراردادی اضافی در باب اینکه نمونه دوم چه معنایی دارد. آنچه شمردنشان نمی‌کند، ارزان‌ترکردن انتقال است. گزاره 5.10 یک زیرسیستم را جانشین می‌کند، پس بازی‌ای روی n نمونه که همه‌شان باید جانشین شوند n بار به‌کار بستن و $n\epsilon$ هزینه می‌کند، بنا بر قضیه 4.24 در امتداد زنجیره‌ای از دورگه‌ها - درست همان‌گونه که نکته 7.18 در سمت جابه‌جایی ثبت می‌کند. یک بار به‌کار بستن، ویژگی را برای نمونه‌هایی که φ در بر دارد منتقل می‌کند و در باب باقی هیچ نمی‌گوید.

نکته 5.6 (بازی نشست‌یکنوا نیست، و لازم نیست باشد). $\mathcal{G}m.N = Z$ با Z برخورد می‌کند در حالی که $\mathcal{G}m$ جهانی نیست، پس تعریف 7.2 در باب $\mathcal{G}m$ شکست می‌خورد و نتایج فصل 7 به سیستم بازی اعمال نمی‌شوند. و این همان‌گونه است که باید باشد. بازی با بند بالا به نشست 0 سنجاق شده، در هر آزمایش یکی از آن هست، و جابه‌جاکردنش چیزی نیست که کسی بخواهد؛ آنچه آن فصل به آن اعمال می‌شود همان $\mathcal{F}$ زیر آزمون است، که نکته 5.5 نمونه‌هایش را می‌شمارد.

اینکه چه محیط‌هایی می‌توانند بازی کنند، با همان رده‌ای حل می‌شود که فصل 4 از پیش تعریف می‌کند، که اینجا در یک سیستم تنها گرفته می‌شود.

تعریف 5.7 (محیط‌های ویژگی). محیط‌های مجاز برای سیستم بازی γ همان $\mathbb{Z}_{\gamma,\gamma}$ تعریف 4.10 اند: آن‌هایی که $\langle \gamma \rangle \geq \mathbb{Z}.\text{par}$ دارند و در هیچ هویتی از $\text{IDs}(\gamma)$ میهمان نمی‌کنند. $\mathbb{Z}_\gamma$ را برای آن‌هایی از میانشان بنویسید که هیچ هویت Z را خموش نمی‌کنند، همان‌هایی که جذب گزاره 5.9 می‌تواند جابه‌جایشان کند.

آن دو بند همان را می‌گویند که بازی از آن‌ها می‌خواهد. محیطی که روی بستار نامی خموش است، هیچ هویتی از نامی که γ به کار می‌برد را نمی‌تواند ادعا کند، پس نمی‌تواند به‌عنوان $\mathcal{F}$ ، یا یکی از زیروال‌های $\mathcal{F}$ ، یا نمونه دیگری از هر یک از آن‌ها سخن بگوید؛ آنچه می‌تواند ادعا کند هویت‌های Z خودش است، که به $\mathcal{G}m$ می‌رسند، و نام‌های تازه، که تنها به چیزی می‌رسند که کل **Std** را می‌پذیرد. پس محیط بازی را از راه واسطه‌های $\mathcal{G}m$ بازی می‌کند و تنها وقتی مستقیم به $\mathcal{F}$ دست می‌زند که $\mathcal{F}$ جهانی باشد - و آن‌گاه همان‌گونه

به آن دست می‌زند که هر کسی می‌تواند، که همان معنای جهانی بودن است و همان که تعریف 4.10 از پیش در زمینه تقلید می‌دهد. پس ویژگی یک $\mathcal{F}$ جهانی، ویژگی‌ای است در برابر حریفی که در آن شریک است، و بازی را باید با دانستن این نوشت. بند میهمانی همان بهداشتِ اشغالِ تعریف 3.3 است، و جلوی محیط را هم از نصبِ بازی دوم می‌گیرد.

تعریف 5.8 (بُرد، مزیت و ویژگی). بگذارید $\gamma = \{g_m\} \cup \sigma$ سیستم بازی باشد، Fin یکی از نهایی‌سازی‌های g_m ، $\mathbb{Z}$ مجاز برای γ (تعریف 5.7)، و $\mathcal{X}$ اشغال‌گر جایگاه حریف. Win_{Fin} را برای این پیشامد بنویسید که واسطه FullFin . $(g_m.\text{pid}, Z)$ در $\text{Exec}(\gamma, \mathbb{Z}, \mathcal{X}, \mathcal{C})$ مقدار 1 برمی‌گرداند. مزیت $\mathbb{Z}$ در ویژگی Fin از σ یکی از این دوست

$$\text{Adv}_{g_m, \sigma, \mathbb{Z}, \mathcal{X}}^{\text{Fin}} := 2 \Pr[\text{Win}_{\text{Fin}}] - 1,$$

$$\text{Adv}_{g_m, \sigma, \mathbb{Z}, \mathcal{X}}^{\text{Fin}} := \Pr[\text{Win}_{\text{Fin}}],$$

یعنی خوانش تصمیم و خوانش جست‌وجو، که احتمال روی سکه‌های هر ماشین آن اجراست. اینکه کدامیک مقصود است، به‌ازای هر نهایی‌سازی همان جا که بازی داده می‌شود تعیین می‌شود، و نه با این تعریف؛ و بند کالیبراسیون در زیر می‌گوید چگونه انتخاب کنیم. ما $\mathcal{X}$ را از زیرنویس می‌اندازیم هرگاه $\mathcal{D}$ باشد، و $\mathcal{F}$ را به‌جای σ می‌نویسیم هرگاه $\sigma = \{\mathcal{F}\}$. برای $\varepsilon \geq 0$ ، رده محیط $\mathbb{Z} \subseteq \mathbb{Z}_{\gamma, \gamma}$ و رده اشغال‌گران جایگاه $\mathcal{X}$ ، می‌گوییم σ ویژگی Fin را در ε و در برابر $(\mathbb{Z}, \mathcal{X})$ دارد اگر $\text{Adv}_{g_m, \sigma, \mathbb{Z}, \mathcal{X}}^{\text{Fin}} \leq \varepsilon$. و در ε و در برابر $\mathbb{Z}$ هرگاه $\mathcal{X} = \{\mathcal{D}\}$.

چون همان یک پرچم بازی را می‌بندد (نکته 5.3)، در هر اجرا دست‌بالا یک Win_{Fin} رخ می‌دهد: پیشامدهای نهایی‌سازی‌های متمایز مجزایند، و بازی‌ای با چند نهایی‌سازی چند ویژگی است که در چند اجرا سنجیده می‌شوند و نه چند عدد که از یک اجرا خوانده شوند.

زیر خوانش تصمیم، مزیت علامت‌دار است، آنجا که مزیت تعریف 4.5 چنین نیست، و همین ناهم‌سانی نکته اصلی است: محیط نمی‌تواند Win_{Fin} را برگرداند چنان که می‌تواند حدسی را برگرداند، چون آن پیشامد حکم بازی است و نه حکم خودش، پس بازی‌ای که نمی‌تواند بُرد امتیاز 1- می‌گیرد و چیزی نیست که از آن قدر مطلق بگیریم. Win_{Fin} پیشامدی از اجراست و نه خروجی آن، و این عامدانه است: خروجی اجرا هر چیزی است که محیط برمی‌گرداند (فصل 4)، و ویژگی حدسی نیست که گزارش شود بلکه چیزی است که رخ داده. با این تمایز هیچ از دست نمی‌رود، چون نکته 5.2 آن دو را همان جا که مهم است جور می‌کند. بیتی که سطر 10 حساب می‌کند همان بیتی است که نهایی‌سازی بی‌واسطه‌گری به محیط می‌سپارد، پس می‌توانیم نام محیطی را ببریم که گزارشش می‌کند: برای Fin ثابت، $\mathbb{Z}^*$ را برای $\mathbb{Z}$ بنویسید که بازگشتش در ریشه با این عوض شده: «اگر هر فراخوانی که من روی FullFin . $(g_m.\text{pid}, Z)$ گذاشتم با 1 پاسخ گرفت، 1 برگردان، و در غیر این صورت 0.» هر بخش آن جای خود را به‌دست می‌آورد.

نام هویت برده می‌شود چون نهایی‌سازی‌ای که در طرف دیگری خطاب شود، هر جا آن طرف درستکار باشد در سطر 8 شکست می‌خورد و هر جا فاسد باشد حریف پاسخ می‌دهد، پس مقداری که بازمی‌گردد لازم نیست مال بازی باشد؛ و در ریشه هیچ‌یک از این دو رخ نمی‌دهد، بنا بر نکته 5.2. نام عمل برده می‌شود چون $Z \in \mathcal{G}m.P$ ، پس سطر 1 هر واسط $\mathcal{G}m$ را در ریشه خدمت می‌کند، اوراکل‌های خود بازی هم در آن - و چون با چند نهایی‌سازی، صورت گزارش‌دهنده باید بگوید بر کدام ویژگی گزارش می‌دهد، یک Z^* به‌ازای هر Fin . آن آزمون در برابر 1 است و نه خواندن پاسخ به‌عنوان یک بیت، چون نهایی‌سازی بعدی بنا بر نکته 5.3 با rej بازمی‌گردد، و rej بیت نیست (بخش 3.6). و هر فراخوان است و نه اولی، چون اولی که گذاشته می‌شود می‌تواند رد شود - محیطی که (Z, P) را برای $P \neq Z$ ادعا کند در آزمون طرف سطر 2 شکست می‌خورد و $done$ ننشسته می‌ماند - در حالی که یکی بعدی خدمت می‌شود؛ و با سورزدن هیچ ضعیف نمی‌شود، چون دست‌بالا یک فراخوان هرگز با 1 پاسخ می‌گیرد، زیرا بازی یک نهایی‌سازی را خدمت می‌کند و باقی را رد. پس $Exec$ درست وقتی 1 برمی‌گرداند که Win_{Fin} رخ دهد، و Z^* می‌تواند حسابش کند، چون هر فراخوان روی آن واسط، فراخوانی است که خودش گذاشته: $\mathcal{G}m.N = Z$ هیچ فراخواننده دیگری را نمی‌پذیرد، و کارکردی که محیط میهمانش می‌کند شناسه استاندارد را ادعا می‌کند، که Z کنارش می‌گذارد. آن‌گاه

$$\Pr[Exec(\gamma, Z^*, \mathcal{X}, \mathcal{C}) = 1] = \Pr[Win_{Fin} \text{ in } Exec(\gamma, Z, \mathcal{X}, \mathcal{C})],$$

چون آن دو ماشین همان فراخوان‌ها را می‌گذارند و تنها در مقداری که با آن می‌ایستند تفاوت دارند. و همین Z^* را در هر رده‌ای می‌نشانند که Z در آن است و عضویتش تنها بر par و میهمانی می‌گردد - $Z_{\gamma, \gamma}$ یکی از آن‌ها، و رده‌های بودجه‌دار تعریف 4.30 تا همان یک گام اضافی خواندن یک بیت. گزاره 5.10 همان جایی است که همه این‌ها به کار می‌رود.

چرا بدل. اشغال‌گر پیش‌فرض جایگاه همان بدل بخش 4.4 است، و دلیلش این است که بازی یک حریف دارد و آن حریف محیط است. بدل در هر دو جهت بازپخش می‌کند و هیچ راهبردی نگه نمی‌دارد، پس محیطی که او را می‌راند هر چه جایگاه می‌تواند بکند در دست دارد؛ و آنجا که هسته‌های خود $\mathcal{F}$ به حریف می‌رسند - همان $A(0)$ بخش 3.4، که راهی است که کارکرد ایده‌آل مقداری را نامعین می‌گذارد - آن‌گاه محیط است که پاسخ می‌دهد، و این درست همان آزادی‌ای است که بازی می‌خواهد به او بدهد. پس انداختن $\mathcal{X}$ از زیرنویس در حالت بدل هیچ عمومیتی را از دست نمی‌دهد بلکه انتخابی است در باب اینکه چه کسی بازی می‌کند؛ و صورت بلندتر نگه داشته می‌شود چون گزاره 5.10 شبیه‌سازی را در آن جایگاه لازم دارد. و اینکه این کوتاه‌نویسی هیچ هزینه‌ای ندارد مسئله ذوق نیست، و دلیلش ساخت بخش 4.4 است: جایگاه در محیط جذب می‌شود.

گزاره 5.9 (جایگاه در محیط جذب می‌شود). بگذارید $\gamma = \{\mathcal{G}m\} \cup \sigma$ سیستم بازی‌ای باشد که هسته‌هایش هیچ فراخوان پاسخگو نمی‌گذارند (تعریف 9.2)، $\mathcal{X}$ اشغال‌گری بی‌اعتنا به رد از جایگاه حریف (تعریف 4.28)، و $Z \in Z_\gamma^\circ$ آن‌گاه Z^x

تعریف 4.26 باز هم در Z°_γ می‌افتد، و به‌ازای هر نهایی‌سازی Fin از gm

$$\text{Adv}^{\text{Fin}}_{\text{gm}, \sigma, Z, X} = \text{Adv}^{\text{Fin}}_{\text{gm}, \sigma, Z^X}.$$

پس اگر σ ویژگی Fin را در ε' و در برابر Z°_γ داشته باشد، آن را در ε' و در برابر (Z°_γ, X) هم دارد، به‌ازای هر رده X از اشغال‌گران بی‌اعتنا به رد. و اگر عینی خوانده شود، محیط کراندار Z و اشغال‌گر کراندار X محیطی جذب‌شده می‌دهند که کراندار $Z \oplus X$ است.

اثبات. لم 4.18 در $H = \{X\}$ با $H_{\text{keep}} = \emptyset$ می‌دهد

$$\text{Exec}(\gamma, Z, X, \mathcal{C}) \equiv \text{Exec}(\gamma, Z^X, \mathcal{D}, \mathcal{C}),$$

و سه فرضش همان سه فرضی است که اینجا مفروض گرفته شده، یکی به یکی. اکنون WinFin پیشامد این است که واسط FullFin در $(\text{gm.pid}, Z)$ مقدار 1 برمی‌گرداند، و جذب تنها به جایگاه حریف دست می‌زند - gm آن هویت را در هر دو سو اشغال می‌کند - پس آن پیشامد به‌ازای هر Fin یکسان یک احتمال دارد، و تعریف 5.8 زیر هر دو خوانش یک عدد می‌دهد. مجازبودن همان مشاهده بخش 4.4 است که Z^X آنچه محیط اشغال می‌کند را اشغال می‌کند و $Z \text{ par}$ را حمل می‌کند، پس درست وقتی مجاز است که Z مجاز باشد؛ و بودجه همان $Z \oplus a$ فصل 8 است. جمله آخر همان نمایش است که به‌ازای هر X به‌ترتیب خوانده می‌شود. $\square$

فرض‌ها همان فرض‌های قضیه 4.29 اند و نه بیشتر: سیستم بازی‌ای که فراخوان پاسخگو می‌گذارد بیرون این است، بنا بر نکته 9.4، و همچنین محیطی که هویت‌های Z خودش را خموش می‌کند، چون دستور تعریف 4.26 زیر ادعایی Z بیرون می‌رود. هیچ یک شرطی بر آنچه اشغال‌گر می‌کند نیست، و نکته اصلی همین است - بازی لازم نیست به این اهمیت دهد که ماشین نشسته در جایگاه منصفانه بازی می‌کند یا نه، تنها به این که می‌شود جابه‌جایش کرد.

یک شرط دامنه با همه این‌ها می‌آید، و در سرتاسر همان یک شرط است. گزاره بالا بر بازگروه‌بندی قضیه 4.29 تکیه دارد، که تنها برای سیستم‌هایی برقرار است که هسته‌هایشان هیچ فراخوان پاسخگو نمی‌گذارند، و نکته 9.4 نشان می‌دهد بازپخش در غیر این صورت کجا شکست می‌خورد؛ و همان شرط بر خود قضیه 4.29 حکم می‌کند، که همان است که فرض تقلید را از رده فراختر حریف برای گزاره 5.10 فراهم می‌کند، و نتیجه 5.11 آن را یک‌جا در هر دو سو به ارث می‌برد. پس سیستم بازی‌ای که $\mathcal{F}$ آن از راه A^1 به جایگاه می‌رسد بیرون هر سه می‌نشیند و باید فرضی تنها-بدل خودش را مستقیم برقرار کند. و اینکه نظریه پاسخگوی بازی‌ها چه شکلی می‌بود، همان جا می‌گذاریم که نکته 9.4 می‌گذاردش.

کالبراسیون. اینکه نهایی‌سازی کدامیک از دو خوانش تعریف 5.8 را می‌گیرد مسئله ذوق نیست، و قاعده این است. ضریب $1 - 2\text{Pr}[\text{WinFin}]$ ویژگی را به‌عنوان تصمیم

می‌خواند: محیطی که حدس می‌زند با احتمال $\frac{1}{2}$ می‌برد و امتیاز 0 می‌گیرد، و محیطی که همیشه می‌برد امتیاز 1. و این آنجا درست است که حدس یک راهبرد باشد – آنجا که حکم بیتی است که محیط می‌توانست برایش سکه بیندازد. و برای ویژگی جست‌وجو نادرست است – مثلاً جعل ناپذیری، که در آن بردن قرار است رو سخت باشد – چون آنجا همان کمیت $\Pr[\text{Win}_{\text{Fin}}] = (1 + \text{Adv}_{\mathcal{G}_{m,\sigma,\mathcal{Z}}}^{\text{Fin}})/2$ خوانده می‌شود، پس کران ε بر مزیت، کران $(1 + \varepsilon)/2$ بر احتمال بُرد است، که برای ε کوچک مقیاس نادرستی است: ε برابر 0 به جعل‌کننده‌ای اجازه می‌داد که نیمی از اوقات موفق شود. پس ویژگی‌های جست‌وجو خود $\Pr[\text{Win}_{\text{Fin}}]$ را می‌گیرند. و اعلام خوانش به‌ازای هر نهایی‌سازی و نه تعیین یکی به‌طور سراسری، همان است که می‌گذارد یک پوسته هر دو گونه را یک‌جا حمل کند. هر ویژگی این کتاب از گونه دوم است – بردن هر یک از بازی‌های زیر قرار است رو ناممکن باشد و نه تنها بهتر از یک حدس نباشد – پس همه‌شان خوانش جست‌وجو را می‌گیرند؛ و خوانش تصمیم برای ویژگی‌هایی هست که در آن‌ها حدس همان راهبرد است، و تمایز ناپذیری‌ای که به‌صورت بازی نوشته شود یکی از آن‌هاست. این انتخاب مفت نیست، و گزاره 5.10 همان جایی است که بهایش داده می‌شود. دو نمایش آن همان یک کران در دو مقیاسند – 2ε برای خوانش تصمیم، ε برای خوانش جست‌وجو – پس هر نهایی‌سازی انتقال را در مقیاسی به ارث می‌برد که خوانش خودش تعیین می‌کند، و بازی‌ای که آن دو را می‌آمیزد در هر دو به ارث می‌برد. و سرانجام، دلیل تعریف‌کردن ویژگی‌ها به این شکل و نه در کنار چارچوب: تقلید آن‌ها را حمل می‌کند.

گزاره 5.10 (ویژگی‌ها در امتداد تقلید منتقل می‌شوند). بگذارید $\gamma = \{\mathcal{G}_m\} \cup \sigma$ بگذارید سیستم بازی باشد، $\varphi \subseteq \sigma$ ، جانشانی φ با π در γ به معنای تعریف 4.2 خوش‌ساخت باشد، و $\gamma[\pi/\varphi]$ هم سیستم بازی باشد. فرض کنید

. تقلید-UC می‌کند $(\{\mathcal{D}\}, \mathbb{S}, \mathbb{Z})$ و در برابر ε را در φ کارکرد π

آن‌گاه $\mathcal{S} \in \mathbb{S}$ هست چنان که به‌ازای هر نهایی‌سازی Fin از $\mathcal{G}_m$ و هر $\mathcal{Z} \in \mathbb{Z}$ که صورت گزارش‌دهنده‌اش $\mathcal{Z}^{\gamma \setminus \varphi} \in \mathbb{Z}$ دارد – شرطی که گزاره 4.16 هرگاه $\mathbb{Z}$ بزرگ‌ترین رده مجاز باشد تُهی‌اش می‌کند، هرچند نه وقتی بزرگ‌ترین رده یک بودجه باشد، چون جذب $\mathcal{C}_{\gamma \setminus \varphi}$ هزینه می‌کند و صورت گزارش‌دهنده یک گام بیشتر –

$$\text{Adv}_{\mathcal{G}_m, \sigma[\pi/\varphi], \mathcal{Z}}^{\text{Fin}} \leq \text{Adv}_{\mathcal{G}_m, \sigma, \mathcal{Z}, \mathcal{S}}^{\text{Fin}} + 2\varepsilon, \quad (\text{خوانش تصمیم})$$

و، با نوشتن Win_{Fin} و Win'_{Fin} برای پیشامدهای بُرد آن دو اجرایی که نمایش بالا مقایسه‌شان می‌کند – یعنی واقعی و ایده‌آل –

$$\Pr[\text{Win}_{\text{Fin}}] \leq \Pr[\text{Win}'_{\text{Fin}}] + \varepsilon. \quad (\text{خوانش جست‌وجو}).$$

اثبات. سه حرکت و یک کران. اول، سنجیده می‌شود که این حکم در باب همان سیستم‌هایی است که نامشان می‌برد، و همین جایی است که فرض روی $\gamma[\pi/\varphi]$ خرج می‌شود: تقلید نه مجموعه طرف بازی را در اشغال‌گر تازه فراهم می‌کند و نه شناسه فرایند آن را در N آن اشغال‌گر، و تنگی هم علاوه بر آن بخشی از تعریف 5.1 است. بعد، صورت گزارش‌دهنده تعریف 5.8 پیشامد بُرد را به بیت خروجی بدل می‌کند، با بازنویسی آنچه $\mathcal{Z}$ در ریشه برمی‌گرداند و بی تغییر هیچ اجرایی. بعد، جذب خود بازی را به درون محیط می‌برد، تا محیطی که بازی را بازی می‌کند به محیطی بدل شود که آن دو سیستم را تشخیص می‌دهد. آنچه می‌ماند قضیه 4.20 است که در همان محیط جذب‌شده به کار رفته. خوانش جست‌وجو همان کرانی است که این می‌دهد؛ و خوانش تصمیم همان کران است که دو برابر شده، و هیچ چیز این استدلال Fin را نمی‌خواند، پس به‌ازای هر نهایی‌سازی جداگانه برقرار است. اول، آن دو مزیت تعریف‌شده‌اند و در باب همان سیستم‌هایی هستند که حکم نامشان می‌برد. چون $\sigma \notin \mathcal{G}m$ ، داریم $\varphi \notin \mathcal{G}m$ ، پس $\gamma[\pi/\varphi] = \{\mathcal{G}m\} \cup \sigma[\pi/\varphi]$ و $\mathcal{Z}_{\gamma[\pi/\varphi], \gamma}$ در هر دوی $\mathcal{Z}_{\gamma, \gamma}$ و $\mathcal{Z}_{\gamma[\pi/\varphi], \gamma[\pi/\varphi]}$ می‌افتد، چون هر بند تعریف 4.10 وقتی اجتماع دو بستر به یکی کوچک شود ضعیف می‌شود. و اینکه سمت چپ اصلاً یک مزیت ویژگی است، همان جایی است که فرض روی $\gamma[\pi/\varphi]$ خرج می‌شود، و مفت نیست: تقلید نه $\mathcal{G}m.P = F'.P \cup \{Z\}$ را در اشغال‌گر تازه F' فراهم می‌کند و نه $\mathcal{G}m.pid \in F'.N$ را، که دومی همان فرض (ii) قضیه 4.4 است و هیچ بخشی از قضیه 4.20 نیست. و این همه آنچه می‌خرد نیست: تنگی بخشی از تعریف 5.1 است، پس آن فرض علاوه بر این می‌خواهد که اعضای π فراخوانده‌هایشان را با شناسه فرایند سنجاق کنند، آنجا که اعضای φ چنین می‌کردند. و تقلید این را از همه کمتر فراهم می‌کند، چون رفتار را در مرز می‌سنجد و در باب میدان‌های اعلام‌شده پشت آن هیچ نمی‌گوید.

بگذارید $\mathcal{S} \in \mathcal{S}$ همان شبیه‌سازی باشد که فرض برای $\mathcal{D}$ فراهم می‌کند؛ او تنها به $\mathcal{D}$ وابسته است، پس یک $\mathcal{S}$ به هر $\mathcal{Z}$ خدمت می‌کند. چنین $\mathcal{Z}$ را ثابت کنید، Fin را ثابت کنید، و صورت گزارش‌دهنده $\mathcal{Z}^*$ را برای آن Fin از بند پس از تعریف 5.8 بگیرید، که می‌دهد

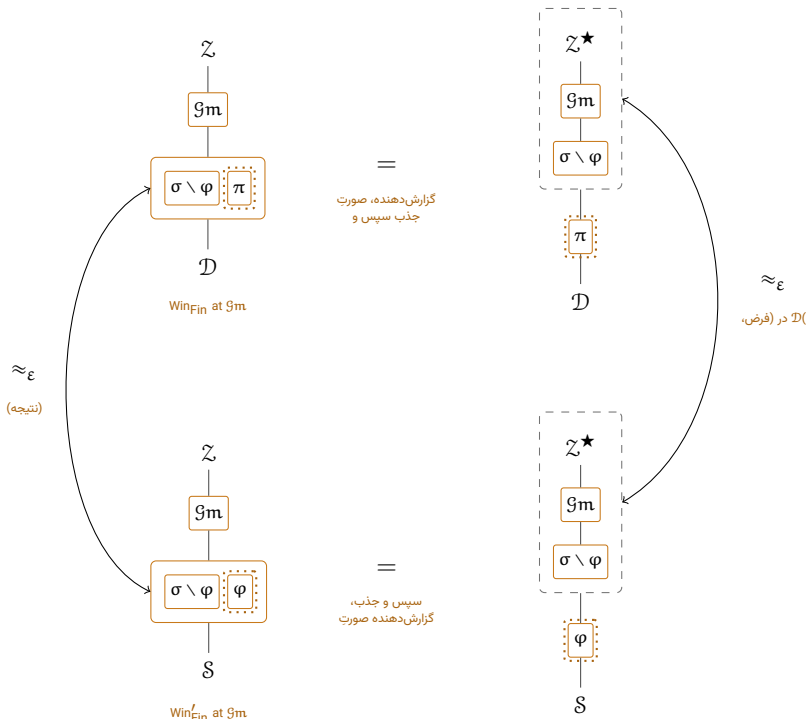
$$\Pr[\text{Exec}(\delta, \mathcal{Z}^*, \mathcal{X}, \mathcal{C}) = 1] = \Pr[\text{Win}_{Fin} \text{ in } \text{Exec}(\delta, \mathcal{Z}, \mathcal{X}, \mathcal{C})]$$

به‌ازای هر یک از دو سیستم بازی δ و هر یک از دو اشغال‌گر $\mathcal{X}$ ؛ و آن فرض $\mathcal{Z}^*$ را در رده $\mathcal{Z}'$ می‌نشانند که قضیه 4.20 به این جانشانی می‌چسباند، و $\mathcal{Z}^* \in \mathcal{Z}_{\gamma[\pi/\varphi], \gamma}$ چون $\mathcal{Z}$ چنین است و آن دو بر par و بر آنچه میهمان می‌کنند توافق دارند. آن‌گاه قضیه 4.20، که به جانشانی φ با π در γ و در حریف $\mathcal{D}$ و محیط $\mathcal{Z}^*$ به کار می‌رود، اختلاف آن دو احتمال سمت چپ را - با $\gamma[\pi/\varphi] = \delta$ و $\mathcal{X} = \mathcal{D}$ در برابر $\gamma = \delta$ و $\mathcal{S} = \mathcal{X} -$ به ε کران می‌زند. پس $\Pr[\text{Win}_{Fin}]$ میان آن دو دست‌بالا ε تفاوت دارد، که همان نمایش دوم است؛ و مزیت زیر خوانش تصمیم، که $1 - 2\Pr[\text{Win}_{Fin}]$ است، آن‌گاه دست‌بالا 2ε تفاوت دارد، که همان نمایش اول است. ترتیب برای کاربرد مهم است و نه برای اثبات: کران احتمال همان است که استدلال می‌دهد، و کران خوانش تصمیم همان است که دو برابر شده. زیر خوانش جست‌وجو، مزیت همان احتمال است، پس نمایش دوم

از پیش همان حکم انتقال است و آن ضریب دو هرگز پیش نمی‌آید. و هیچ چیز این استدلال را نمی‌خواند، پس به‌ازای هر نهایی‌سازی جداگانه برقرار است. $\square$

دو چیز در باب آن اثبات. آن فروکاست همان فروکاست کتاب درسی است، و جذب همان است که چنیش می‌کند: محیط جذب‌شده $\mathcal{Z}^{\gamma \setminus \varphi \star}$ از تعریف 4.15 خود بازی را به همراه باقی $\gamma \setminus \varphi$ می‌همان می‌کند، پس «محیطی که بازی را در برابر π بازی می‌کند» با بردن بازی به درون او به «محیطی که π را از φ تشخیص می‌دهد» بدل می‌شود، و همان است که قضیه 4.20 در بابش اثبات شده بود. و ویژگی سمت راست در برابر $\mathcal{S}$ در جایگاه بیان شده و نه در برابر بدل، و همین است که چرا تعریف 5.8 اشغال‌گر را به‌عنوان آرگومان نگه می‌دارد – اما این تعهدی نیست که کسی ناچار باشد در آن صورت برآورده‌اش کند، چون جایگاه هم در محیط جذب می‌شود.

این فروکاست ارزش کشیدن دارد، چون یک حرکت است و آن حرکت به‌سادگی از چشم می‌افتد. همان مربع جابه‌جایی شکل 1 است، با همان قراردادها و با جعبه نقطه‌چین که باز زوج مبادله‌شده را نشان می‌دهد – اما این بار بازی است که حرکت می‌کند:



شکل 2: انتقال ویژگی، به صورت همان مربع که این بار بازی جابه‌جا می‌شود و نه پروتکل. در ستون چپ، $\mathcal{G}_m$ در درون سیستم نشسته است، همان‌جا که جای بازی است؛ و در ستون راست، در درون جعبه خط‌چین می‌نشیند، به عنوان بخشی از ماشینی که تشخیص می‌دهد. گذشتن از آن مرز، کل فروکاست است. (گزاره 5.10).

با تماشای $\mathcal{G}_m$ بخوانیدش. در ستون چپ او در درون سیستم نشسته است، همان‌جا که جای بازی است: محیط بازی‌اش می‌کند، و Win_{Fin} حکمی است که او برمی‌گرداند. در ستون راست در درون جعبه خط‌چین می‌نشیند، به عنوان بخشی از ماشینی که تشخیص می‌دهد. گذشتن از آن مرز کل فروکاست است – محیطی که بازی را بازی می‌کند در برابر π به محیطی بدل می‌شود که π را از φ تشخیص می‌دهد – و به محض آنکه بازی آن‌سو رفت، آنچه می‌ماند پرسشی خالی از تقلید است، و همان است که قضیه 4.20 در بابش اثبات شده بود.

آن دو یال افقی مفتند. صورت گزارش‌دهنده تنها آنچه $\mathcal{Z}$ در ریشه برمی‌گرداند را بازنویسی می‌کند و هر فراخوانی که می‌گذارد را دست‌نخورده می‌گذارد، پس هیچ اجرایی را تغییر نمی‌دهد؛ و جذب همان لم 4.18 است، که این‌همانی اجراهاست و نه یک کران. پس یال راست تنها چیزی است که بهایش داده می‌شود، و همان فرض است که در $\mathcal{D}$ به کار رفته – و همین است که چرا سطر پایین $\mathcal{S}$ را در جایگاه دارد و نه بدل را، و پس چرا اصلاً تعریف 5.8 اشغال‌گر را به عنوان آرگومان نگه می‌دارد. آن‌گاه یال چپ وادار می‌شود، درست همان‌گونه که در شکل 1: تعقیب مربع آن دو ε را همان یک کمیت می‌کند و نه دو کران که تصادفاً یک ε را شریکند.

یک خوانش آن کمیت خوانش همین گزاره است، و آن دیگری دو برابر آن. زیر خوانش جست‌وجو، مزیت ویژگی همان $\Pr[\text{Win}_{\text{Fin}}]$ است، پس یال چپ از پیش همان حکم انتقال است و هیچ ضریب دویی پیدا نمی‌شود؛ و زیر خوانش تصمیم، مزیت $1 - 2\Pr[\text{Win}_{\text{Fin}}]$ است، که دو برابرش می‌کند. هر اثبات ویژگی هر کارکردی در این کتاب نمونه‌ای از همین یک مربع است، و همین است که چرا این مربع سزاوار نمودار است و نمونه‌هایش نه.

نتیجه 5.11 (انتقال، در برابر هر حریفی). در زمینه گزاره 5.10، علاوه بر آن فرض کنید که هسته‌های γ و $\gamma[\pi/\varphi]$ هیچ فراخوان پاسخگو نمی‌گذارند (تعریف 9.2)، که شبیه‌ساز $\mathcal{S}$ ی که آن گزاره فراهم می‌کند و اشغال‌گر $\mathcal{A}$ زیر بی‌اعتنا به ردند (تعریف 4.28)، و که محیط‌هایی که نامشان برده می‌شود در $\mathbb{Z}_\gamma^\circ$ می‌افتند. نهایی‌سازی Fin از $\mathcal{G}_m$ را ثابت کنید. اگر σ ویژگی Fin را در ε' و در برابر $\mathbb{Z}_{\gamma,\gamma}$ به معنای تعریف 5.8 داشته باشد، آن‌گاه

$$\text{Adv}_{\mathcal{G}_m, \sigma[\pi/\varphi], \mathbb{Z}_\gamma, \mathcal{A}}^{\text{Fin}} \leq \varepsilon' + 2\varepsilon$$

به‌ازای هر اشغال‌گر $\mathcal{A}$ از جایگاه حریف و هر $\mathcal{Z}$ ی که آن گزاره می‌پذیرد و $\mathcal{Z}^{\mathcal{A}}$ در میانشان است. و اگر به‌جای آن فرض روی مزیت‌ها، سوی ایده‌آل زیر همان سورزدن $\Pr[\text{Win}'_{\text{Fin}}] \leq p$ را برآورد، آن‌گاه $p + \varepsilon$ ، $\Pr[\text{Win}_{\text{Fin}}] \leq p$ ، با Win'_{Fin} و Win_{Fin} که پیشامدهای واقعی و ایده‌آل را مثل گزاره 5.10 نام می‌برند – و این همان صورتی

است که ویژگی جست‌وجو در آن به ارث می‌برد، و نمایش اول مال خوانش تصمیم است.

اثبات. سه گام، که دو گام بیرونی‌اش جذبند. بنا بر گزاره 5.9 در سیستم بازی واقعی، گزاره 5.10 $\text{Adv}_{\text{gm}, \sigma[\pi/\varphi], \mathcal{Z}, \mathcal{A}}^{\text{Fin}} = \text{Adv}_{\text{gm}, \sigma[\pi/\varphi], \mathcal{Z}, \mathcal{A}}^{\text{Fin}}$ که سمت راستش حالت بدل است. گزاره 5.10 در $\mathcal{Z}^{\mathcal{A}}$ آن را به $\text{Adv}_{\text{gm}, \sigma, \mathcal{Z}^{\mathcal{A}}, \mathcal{S}}^{\text{Fin}} + 2\varepsilon$ کران می‌زند. و گزاره 5.9 دوباره، این بار در سیستم بازی ایده‌آل، جمله اول را به $\text{Adv}_{\text{gm}, \sigma, (\mathcal{Z}^{\mathcal{A}})^{\mathcal{S}}}^{\text{Fin}}$ بازنویسی می‌کند، که فرضی روی σ آن را به ε' کران می‌زند، و آن ماشین دوبار جذب‌شده بنا بر دو بار به‌کار بستن همان گزاره محیطی مجاز است. صورت احتمالی همان سه گام است که روی نمایش دوم گزاره 5.10 خوانده شده و نه روی اولی، چون جذب اصلاً هیچ احتمالی را تغییر نمی‌دهد. $\square$

دو چیز که این می‌خرد. فرضی سوی ایده‌آل اکنون همان ویژگی در برابر بدل تنهاست — یک حکم در باب σ ، که روی محیط‌ها سور خورده و روی هیچ چیز دیگر — آنجا که گزاره 5.10 به‌تنهایی ویژگی را در برابر همان شبیه‌ساز خاصی می‌خواست که تقلید فراهم کرده بود. هیچ چیز لازم نیست در باب اینکه آن شبیه‌ساز چگونه فاسد می‌کند یا نشد می‌دهد مفروض شود — تنها اینکه روی رد درون چارچوبی شاخه نمی‌زند، که بخش 4.4 از هر ماشینی که جابه‌جا می‌کند می‌خواهد: هر کار دیگری هم که بکند، جذبش در محیط او را به بخشی از ماشینی بدل می‌کند که فرضی سوی ایده‌آل از پیش می‌پوشاندش. و نتیجه، ویژگی سیستم واقعی در برابر هر حریفی است و نه تنها در برابر بدل، با همان حرکت که در سوی دیگر به کار رفته. و اگر عینی خوانده شود، آن دو جذب کل بها هستند: برای $\mathcal{A}$ کراندار $\mathbf{a}$ و $\mathcal{S}$ کراندار $\mathbf{s}$ ، به درونی‌ترین حکم در بودجه محیط $\mathbf{z} \oplus \mathbf{a} \oplus \mathbf{s}$ می‌رسیم، یک هزینه میهمانی به‌ازای هر ماشین جابه‌جاشده، و فصل 8 هیچ چیز دیگری حساب نمی‌کند.

نکته 5.12 (اشغال‌گر جایگاه در سوی ایده‌آل چه می‌کند). هیچ چیز اینجا برای اثبات بالا لازم نیست، چون آن اثبات هر کاری که جایگاه بکند را در ε که قضیه 4.20 فراهم می‌کند جذب می‌کند. آنچه در میان است، به‌جایش، هزینه فرض سمت راست است: آن دو اجرایی که این گزاره مقایسه‌شان می‌کند، هم در جایگاه تفاوت دارند و هم در سیستم. فساد به دو نیم می‌شود، و تنها نیمی از آن محل نزاع است. آن نیمی که نیست، همان است که محیط برای خودش می‌کند: سطر $\text{id}.F \in 3$ $\{A, Z\}$ را می‌پذیرد، پس محیط برای فسادهایی که می‌خواهد به همکاری هیچ‌کس نیاز ندارد، و با ادعای (Z, P) از ریشه‌اش آن‌ها را در هر دو سو یکسان می‌کند. آن نیمی که محل نزاع است، ابتکار خود اشغال‌گر است: $\mathcal{S}$ می‌تواند طرف‌هایی را فاسد کند که هیچ‌کس از او نخواست، یا از آنچه از راه جایگاه از او خواسته می‌شود سر باز زند، آنجا که $\mathcal{D}$ بازپخش می‌کرد. همین است که چرا سمت راست گزاره 5.10 همان ویژگی در برابر $\mathcal{S}$ است و نه در برابر بدل، و چرا تعریف 5.8 اشغال‌گر را به‌عنوان آرگومان نگه می‌دارد. آنچه σ ایده‌آل در آن صورت باید برآورده کند، همان بازی در

برابر هر شبیه‌سازی است که تقلید ممکن است تولید کند؛ و نتیجه 5.11 آن را باز به بدل فرو می‌کاهد، و این کار را با بردن شبیه‌ساز به درون محیط می‌کند و نه با خواستن چیزی از او.

هیچ چیز از ε نمی‌گریزد، و دفتر دلیلش است. محیط C را از راه **FullStatus** می‌خواند، که هیچ واسطه‌گری نمی‌کند و کل مجموعه فاسدها را برمی‌گرداند (نکته 3.1)، پس می‌تواند C را در هر نقطه‌ای بخواند و آن را در برابر طرف‌هایی که خودش فاسد کرده به همراه هر چه به جایگاه دستور داده بیفزاید بسنجد. بگذارید $\mathcal{Z}$ محیطی باشد که آن رده می‌پذیردش، و $\mathcal{Z}'$ همان $\mathcal{Z}$ باشد با این سنجش که افزوده شده و بازگشت 0 آنجا که شکست بخورد. در سوی واقعی هرگز شکست نمی‌خورد، چون $\mathcal{D}$ همان را فاسد می‌کند که به او گفته شده و هیچ چیز دیگر؛ و در سوی ایده‌آل با هر احتمال δ ی که شبیه‌ساز کج می‌رود شکست می‌خورد، در هر دو جهت — سر باز زدن از آنچه خواسته شده، یا فاسدکردن ناخواسته. پس $\mathcal{Z}'$ با مزیت δ تشخیص می‌دهد، و فرض تقلید $\varepsilon \leq \delta$ می‌دهد. پس آدم می‌تواند چنان استدلال کند که گویی $\mathcal{S}$ فسادهایی را که به او دستور داده شده بازتاب می‌دهد، به بهایی که از پیش پرداخته شده — و این همان همان‌دازگی بخش 3.1 است که برای ویژگی‌ها خوانده شده. داشتنش ارزش دارد، اما این نیست که فرض سمت راست گزاره 5.10 را انجام می‌دهد؛ جذب جایگاه انجامش می‌دهد، و آن اصلاً هیچ از رفتار شبیه‌ساز نمی‌خواهد.

نکته 5.13 (نشت در سوی ایده‌آل). نشت هم مثل فساد بین دو سو تقسیم می‌شود، و یک سنجش از خود لازم دارد چون از **Guard** نمی‌گذرد. تنها جایگاه می‌تواند یک Leak را بخواند: سطر 2 می‌پرسد $\text{id}.F = A$ ، و محیط نمی‌تواند چنین هویتی را ادعا کند. پس gm.N آنجا از بازی محافظت نمی‌کند، و چیز دیگری باید بکند: دروازه سطر 4 به همراه طرف، چون نشت بازی تنها در طرف فاسد خواندنی است، ریشه هرگز فاسد نیست، و در هر طرف دیگری gm.Leak بنا بر نکته 5.4 مقدار $\perp$ برمی‌گرداند. پس حالت بازی از آن راه در هیچ‌یک از دو سو به حریف نمی‌رسد. آنچه میان دو سو تفاوت دارد، نشت φ در برابر نشت π است، که $\mathcal{S}$ باید تولیدش کند؛ و آن بخشی معمولی از تقلید است، که با همان ε کران‌دار شده، و ویژگی آن کران را با هیچ چیز دیگری برای سنجش به ارث می‌برد.

مسدودگرها

تعریف 4.10 از فصل 4 محیط را در هر دو جهان روی یک و همان بستار نامی خموش می‌کند، پس ادعاهایی که برای او باز است به این وابسته نیست که با کدام جهان روبه‌روست – و گزاره 4.16 و قضیه 4.20 اصلاً هیچ نسبتی میان آن دو مجموعه هویت به کار نمی‌برند. آنچه مسدودسازی می‌خرد تنگ‌تر است: آن بستار را به‌عنوان مجموعه خموش‌شده دقیق دست‌یافتنی می‌کند، و پس کم‌محدودترین رده مجاز را تحویل می‌دهد، چنان که بحث تعریف 4.10 در فصل 4 ثبتش می‌کند. یک مسدودگر هویت‌ها را اشغال می‌کند و هیچ چیز دیگر. هیچ فراخوانده‌ای را نمی‌پذیرد، $N := \emptyset$. پس عطف سطر 2 از **Guard** به‌ازای هر فراخوانی نادرست است و فراخوانده **rej** می‌گیرد. پس هسته **Block** او هرگز وارد نمی‌شود و بدنه‌اش تشریفات است. نشت از **Guard** نمی‌گذرد، پس هسته **Leak** طرف فاسد در دسترس است؛ و مقدار $\perp$ برمی‌گرداند، که همه آن چیزی است که ماشین بی‌حالت برای دادن دارد. هر دو هسته‌اند، که چنان پوشانده شده‌اند که تعریف 1.2 از هر کارکرد استاندارد می‌خواهد، و همان پوشانه است که اولی را دست‌نیافتنی می‌کند.

مسدودگر $B[\mathcal{F}]$	
$\text{par} := \perp, \quad \mathcal{U} := \emptyset, \quad \mathcal{N} := \emptyset, \quad \mathcal{P} := \mathcal{F}.\mathcal{P}, \quad \text{pid} := \mathcal{F}.\text{pid}$	
$\text{id}' \text{ از } \text{id.Block()}$	
1: return $\perp$	دست‌نیافتنی Guard به‌سبب /
$\text{id}' \text{ از } \text{id.Leak()}$	
2: return $\perp$	نیست حالتی اما دسترس، در /

مسدودگر $B[\mathcal{F}]$ با کارکردی که به‌جایش می‌ایستد اندیس‌گذاری می‌شود، و از همان کارکرد آن دو میدانی را رونویسی می‌کند که مجموعه هویت را تعیین می‌کنند و هیچ چیز دیگر. بنا بر ساخت $\text{IDs}(B[\mathcal{F}]) = \text{IDs}(\mathcal{F})$ ؛ او درست هویت‌های $\mathcal{F}$ را اشغال می‌کند، با نشست و نمونه، و هیچ چیز دیگر را. و چون دست‌نیافتنی است، به‌جای $\mathcal{F}$ به‌عنوان اشغال‌گر آن هویت‌ها می‌ایستد و از هیچ جهت دیگری.

بنویسیم $I_\pi := \{\mathcal{F}.\text{pid} : \mathcal{F} \in \pi\}$ برای شناسه‌های فرایندی که یک سیستم به کار می‌برد. بنا بر تعریف 1.2 هر $q \in I_\pi$ را دقیقاً یک عضو حمل می‌کند، که آن را π_q می‌نویسیم، پس $\pi = \{\pi_q : q \in I_\pi\}$ و $\text{IDs}(\pi) = \bigcup_{q \in I_\pi} \text{IDs}(\pi_q)$. مسدودگرها

با شناسه‌ها اندیس‌گذاری می‌شوند و نه با اعضا، و این تمایز مهم است: یک شناسه فرایند می‌تواند در هر دو جهان رخ دهد در حالی که کدِ متفاوتی حمل می‌کند، که در آن صورت φ_q عضو π نیست هرچند π آن q را به کار می‌برد، و مسدودگری برای چنین q ی دو ماشین را در یک شناسه می‌نشاند.

به جایش، اندیس‌گذاری با $I_\varphi \setminus I_\pi$ مسدودگر را از آنچه از پیش آنجاست برکنار نگه می‌دارد. هیچ عضوی از π چنین q ی را حمل نمی‌کند، پس هیچ هویتی از $IDS(\pi)$ آن q را در مؤلفهٔ اولش ندارد و $IDS(\varphi_q) \cap IDS(\pi) = \emptyset$. پس مسدودگری که از φ_q ساخته شود نه روی هویتی فرود می‌آید که π از پیش خدمتش می‌کند و نه روی شناسهٔ عضوی موجود. شناسه‌های متمایز مسدودگرهای دویه‌دو مجزا می‌دهند، و همهٔ آن‌ها برای π تازه‌اند، پس سیستم بزرگ‌شده باز در سرتاسر شناسه‌های متمایز دارد. آن زوج سیستمی که مقایسه می‌شود باید علاوه بر این، هر جا هم‌پوشانی دارند توافق داشته باشند.

تعریف 6.1 (سازگاری). سیستم‌های π و φ سازگارند اگر، هرگاه شناسهٔ فرایندی در هر دو رخ دهد، آن دو کارکردی که در آن نشسته‌اند بر طرف‌های خدمت‌گیرنده‌شان و بر اینکه چه کسی را می‌پذیرند توافق داشته باشند؛ و باقی میدان‌ها – آنچه به کار می‌برند، پارامترهایشان، و کدی که می‌دوانند – می‌توانند متفاوت باشند.

سازگاری در دقیقاً یک نقطه به کار می‌رود: شناسه‌هایی که میان دو جهان مشترکند. برای چنین شناسه‌ای هیچ مسدودگری افزوده نمی‌شود، پس هر چه آن شناسه می‌آورد باید از پیش در دو سو جور باشد. بی آن، شناسهٔ مشترک می‌توانست در π به طرف‌هایی خدمت دهد و در φ به طرف‌هایی دیگر، و آن تفاوت پوشانده نمی‌ماند. و خواستن توافق هم بیش از آن است که استدلال لازم دارد، چون به‌محض تعیین شناسهٔ فرایند تنها P در مجموعهٔ هویت وارد می‌شود؛ اما هیچ هزینه‌ای ندارد، چون در هر حالتی که ما به کار می‌بریم شناسهٔ مشترک را همان یک کارکرد در هر دو سو حمل می‌کند.

تعریف 6.2 (سیستم مسدودگر، سیستم‌های مسدودشده). برای سیستم‌های π و φ ، سیستم مسدودگر $Blockers(\varphi \setminus \pi)$ این است

$$Blockers(\varphi \setminus \pi) := \{B[\varphi_q] : q \in I_\varphi \setminus I_\pi\},$$

و سیستم‌های مسدودشدهٔ آن زوج این‌هايند

$$\bar{\pi} := \pi \cup Blockers(\varphi \setminus \pi), \quad \bar{\varphi} := \varphi \cup Blockers(\pi \setminus \varphi).$$

آرگومان $Blockers$ نمادگذاری است و نه ورودی: این ساخت شناسه‌های $I_\varphi \setminus I_\pi$ و اشغال‌گران φ در آن‌ها را می‌خواند، و نه آن مجموعهٔ عضوی را که آن تفاضل نشان می‌دهد – در شناسهٔ مشترکی که کدِ متفاوتی حمل می‌کند، $\varphi_q \in \varphi \setminus \pi$ است و با این حال هیچ مسدودگری ساخته نمی‌شود. پس هر شناسه در $I_\varphi \setminus I_\pi$ با مسدودگر

کارکردی پوشانده می‌شود که در آن زندگی می‌کند، با مجموعه طرف همان کارکرد. هر جهان درست همان شناسه‌های فرایندی را به دست می‌آورد که دیگری دارد و او ندارد، و این همه آنچه مسدودسازی باید به آن برسد است. شناسه‌های مشترک دو جهان هیچ مسدودگری نمی‌گیرند، و همان جایی است که تعریف 6.1 کارش را می‌کند.

به زبان غیررسمی، قضیه 6.3 می‌گوید مسدودسازی دو جهان را برابر می‌کند: هر سیستم مسدودشده باز یک سیستم است، و برای زوجی سازگار، آن دو یک و همان مجموعه هویت را اشغال می‌کنند، یعنی $IDS(\pi) \cup IDS(\varphi)$ — چون هر جهان درست همان شناسه‌های فرایندی را به دست آورده که دیگری دارد و او ندارد.

قضیه 6.3 (برابرسازی). بگذارید π و φ سیستم باشند. آنگاه سیستم‌های مسدودشده $\bar{\pi}$ و $\bar{\varphi}$ از تعریف 6.2 سیستمند. و اگر π و φ علاوه بر آن سازگار باشند، آنگاه

$$IDS(\bar{\pi}) = IDS(\bar{\varphi}) = IDS(\pi) \cup IDS(\varphi).$$

اثبات. دو ادعا، که تنها دومی سازگاری را لازم دارد. اولی آن سه چیزی را که تعریف 1.2 از یک سیستم می‌خواهد — متناهی بودن، استاندارد بودن، شناسه‌های فرایند متمایز — از هر سیستم مسدودشده می‌خواهد و از اندیس‌گذاری به‌دستشان می‌آورد: مسدودگرها با شناسه‌هایی اندیس‌گذاری می‌شوند که جهان دیگر دارد و این یکی ندارد، پس روی هیچ عضو موجودی فرود نمی‌آیند. دومی آن دو مجموعه هویت را اندیس‌به‌اندیس روی $I_\pi \cup I_\varphi$ می‌سند. در شناسه‌ای که تنها یک جهان به کارش می‌برد، هر دو سو همان مجموعه را می‌آورند، چون مسدودگر آن دو میدانی را رونویسی می‌کند که مجموعه هویت را تعیین می‌کنند. در شناسه مشترک هیچ مسدودگری ساخته نمی‌شود، پس آن دو $IDS(\pi_q)$ و $IDS(\varphi_q)$ را همان‌گونه که هستند می‌آورند — و سازگاری همان یک چیزی را فراهم می‌کند که از اندیس‌گذاری در نمی‌آید، یعنی اینکه مجموعه‌های طرف خدمت‌گیرنده جورند. و این تنها جایی است که به کار می‌رود. برای ادعای اول، سیستم بودن سه چیز را از $\bar{\pi}$ می‌خواهد: که متناهی باشد، که اعضایش استاندارد باشند، و که شناسه‌های فرایند متمایز حمل کنند. متناهی بودن بی‌درنگ است، چون اعضای افزوده با $I_\varphi \setminus I_\pi \subseteq I_\varphi$ اندیس‌گذاری می‌شوند. استاندارد بودن به ارث می‌رسد: مسدودگر همان پوشانه‌هایی را حمل می‌کند که تعریف 1.2 از کارکرد استاندارد می‌خواهد، یعنی Block چنان که در بخش 3.5 و Leak چنان که در بخش 3.2، و شناسه فرایندش از عضوی از φ رونویسی شده، پس نامش هیچ‌یک از A, Z, C نیست. برای تمایز، اعضای $\varphi \setminus \pi$ Blockers همان $\mathcal{B}[\varphi_q]$ ها با $q \in I_\varphi \setminus I_\pi$ هستند، و $q \cdot pid = \varphi_q \cdot pid = q$. این شناسه‌ها دوبه‌دو متمایزند و هیچ‌یک در I_π نیست، پس اعضای $\bar{\pi}$ شناسه‌های فرایند متمایز حمل می‌کنند و $I_\pi = I_\pi \cup I_\varphi$. و همان استدلال به $\bar{\varphi}$ اعمال می‌شود.

برای ادعای دوم، $IDS(\mathcal{B}[\mathcal{F}]) = IDS(\mathcal{F})$ به‌ازای هر $\mathcal{F}$ برقرار است، چون مسدودگر آن دو میدانی را رونویسی می‌کند که مجموعهٔ هویت را معین می‌کنند. پس

$$IDS(\bar{\pi}) = \bigcup_{q \in I_{\pi}} IDS(\pi_q) \cup \bigcup_{q \in I_{\varphi} \setminus I_{\pi}} IDS(\varphi_q),$$

$$IDS(\bar{\varphi}) = \bigcup_{q \in I_{\varphi}} IDS(\varphi_q) \cup \bigcup_{q \in I_{\pi} \setminus I_{\varphi}} IDS(\pi_q),$$

و هر دو اجتماع با $I_{\pi} \cup I_{\varphi}$ اندیس‌گذاری شده‌اند. آن‌ها را در هر q از آن مجموعه بسنجید. اگر $q \in I_{\pi} \setminus I_{\varphi}$ آن‌گاه هر دو $IDS(\pi_q)$ را می‌آورند، و اگر $q \in I_{\varphi} \setminus I_{\pi}$ آن‌گاه هر دو $IDS(\varphi_q)$ را. و اگر q در هر دو باشد، اولی $IDS(\pi_q)$ را می‌آورد و دومی $IDS(\varphi_q)$ را. هر دو در اندیس q حمل می‌شوند، پس $\pi_q.pid = \varphi_q.pid = q$ رو برقرار است، و سازگاری همان یک چیزی را فراهم می‌کند که از اندیس‌گذاری در نمی‌آید، یعنی $\pi_q.P = \varphi_q.P$. و مجموعهٔ هویت با همان دو میدان تعیین می‌شود، پس $IDS(\pi_q) = IDS(\varphi_q)$. پس آن دو اجتماع جمله‌به‌جمله توافق می‌کنند، و هر یک برابر این است

$$\bigcup_{q \in I_{\pi}} IDS(\pi_q) \cup \bigcup_{q \in I_{\varphi}} IDS(\varphi_q) = IDS(\pi) \cup IDS(\varphi).$$

□

ادعای دوم یک جدول است. شناسهٔ فرایند در یکی از سه جا می‌افتد، و قضیه همان است که در هر یک می‌نشیند:

$q \in I_{\pi} \cup I_{\varphi}$ فرایند شناسهٔ هر به‌ازای ستون یک

	$I_{\pi} \setminus I_{\varphi}$	$I_{\pi} \cap I_{\varphi}$	$I_{\varphi} \setminus I_{\pi}$
$\bar{\pi}$	π_q	π_q	$\mathcal{B}[\varphi_q]$
$\bar{\varphi}$	$\mathcal{B}[\pi_q]$	φ_q	φ_q

$IDS(\mathcal{B}[\pi_q]) = IDS(\pi_q)$
می‌کند رونویسی را P و pid مسدودگر

$\pi_q.P = \varphi_q.P$ سازگاری
می‌رود کار به که جایی تنها

$IDS(\mathcal{B}[\varphi_q]) = IDS(\varphi_q)$
می‌کند رونویسی را P و pid مسدودگر

می‌کنند اشغال را $IDS(\pi) \cup IDS(\varphi)$ سطر دو هر پس

— مسدودگر یک سایه‌دار:
می‌کند، اشغال را هویت‌ها
نمی‌پذیرد را فراخوانده‌ای هیچ

شکل 3: شناسهٔ فرایند کجا می‌تواند بیفتد، و مسدودسازی در هر جا چه می‌کند. آن سه ستون، سه حالت ادعای دوم قضیهٔ 6.3 اند.

در ستون‌های بیرونی، جهانی که آن شناسه را ندارد مسدودگری به‌دست می‌آورد، و مسدودگر درست هویت‌های کارکردی را اشغال می‌کند که به‌جایش می‌ایستد، چون تنها آن دو میدانی را رونویسی کرده که مجموعهٔ هویت را تعیین می‌کنند. پس آن دو سطر همان هویت‌ها را در آنجا می‌آورند — هرچند یکی کارکردی کارآمد می‌آورد و دیگری

ماشینی که هیچ پاسخ نمی‌دهد، و همین است نکتهٔ سایه‌زدن متفاوتشان. برابرکردن مجموعه‌های هویت همهٔ آن چیزی است که مسدودسازی باید به آن برسد، و بهتر است همهٔ آن چیزی هم باشد که به آن می‌رسد: مسدودگری که چیزی پاسخ می‌داد تغییر می‌داد که محیط چه می‌تواند بیاموزد، و مقایسه‌ای که برای مجازکردنش ساخته شده بود بی‌ارزش می‌شد.

ستون میانی همان جایی است که آن فرض در آن زندگی می‌کند. برای شناسهٔ مشترک هیچ مسدودگری ساخته نمی‌شود، پس آن دو سطر $IDS(\pi_q)$ و $IDS(\varphi_q)$ را همان‌گونه که هستند می‌آورند، و هیچ چیز اندیس‌گذاری آن‌ها را برابر نمی‌کند – هر دو در q حمل می‌شوند، پس شناسه‌های فرایند رو جورند، اما مجموعه‌های طرف خدمت‌گیرنده لازم نیست جور باشند. سازگاری درست همان فرض است که جورند، و این ستون تنها جایی است که به کار می‌رود. و اگر برعکس خوانده شود، همان است که آن فرض هزینه می‌کند: شناسهٔ مشترکی که در دو جهان به طرف‌های متفاوتی خدمت دهد تفاوتی است که مسدودسازی نمی‌تواند بپوشاندش، چون آن یک ابزاری که دارد – افزودن یک اشغال‌گر – همان جا که اشغال‌گری از پیش حاضر است در دست نیست.

تنها ادعای دوم هیچ‌یک از این‌ها را لازم دارد. اینکه π و φ اصلاً سیستمند هیچ از سازگاری نمی‌خواهد، و همین است که چرا قضیه آن دو را جداگانه بیان می‌کند و چرا گزارهٔ 6.4 زیر می‌تواند تنها بر اولی تکیه کند.

مسدودسازی سیستم را بزرگ می‌کند، و بزرگ‌کردن سیستم یکی از دو راهی است که تعریف 1.3 می‌توانست از هم بپاشد.

گزاره 6.4 (مسدودسازی خوش‌ساختی را نگه می‌دارد). بگذارید π و φ سیستم‌هایی با $Wf(\pi)$ و $Wf(\varphi)$ باشند. آن‌گاه $Wf(\pi)$ و $Wf(\varphi)$.

اثبات. π بنا بر ادعای اول قضیهٔ 6.3 یک سیستم است، که هیچ سازگاری‌ای نمی‌خواهد، پس $Wf(\pi)$ پرسشی مجاز است. ما برای π استدلال می‌کنیم؛ و استدلال φ همان است با $Wf(\varphi)$ به جای $Wf(\pi)$. $\mathcal{F} \in \pi^+$ و $\mathcal{F} \in \mathcal{F}, R) \in \mathcal{F}'$ را بگیرید.

اگر $\mathcal{F}$ یکی از مسدودگرهای افزوده باشد چیزی برای سنجیدن نیست: مسدودگر بنا بر ساخت $U = \emptyset$ دارد، پس هیچ درایه‌ای نمی‌آورد. در غیر این صورت $\mathcal{F}$ در π^+ می‌افتد، و $Wf(\pi)$ شاهدی $\mathcal{G} \in \pi^+$ برای آن درایه فراهم می‌کند. آن شاهد هنوز حاضر است، چون $\pi^+ \subseteq \pi^+$ ، و مسدودسازی میدان هیچ عضوی را تغییر نمی‌دهد، پس $\mathcal{G}, \mathcal{F}$ ، $\mathcal{G}, s$ و هر چه R می‌خواند همان‌اند که بودند. پس آن سه شرط باز هم در باب همان $\mathcal{G}$ برقرارند.

این استدلال هیچ‌چیز در باب مسدودگرها به کار نمی‌برد جز U تهی‌شان: بزرگ‌کردن سیستمی به سیستمی دیگر خوش‌ساختی را نگه می‌دارد هرگاه اعضای افزوده هیچ درایه‌ای از خودشان حمل نکنند، چون شاهدی که یک بار حاضر باشد هرگز برداشته نمی‌شود و میدان‌هایی که با آن‌ها دآوری می‌شود هرگز تغییر نمی‌کنند. نه سازگاری به کار می‌رود و نه ادعای دوم قضیهٔ 6.3. $\square$

نکته 6.5 (تحقق‌های تنگ). بستار ادعاهای خود محیط را از نام‌های به‌کاررفته دور نگه می‌دارد؛ و میهمانی آن یک راه دورزدن آن است. محیط مجاز می‌تواند ماشینی را در هویت اشغال‌نشده نامی به‌کاررفته میهمان کند – مثلاً نمونه دوم ساختگی از یک فراخواننده – و ماشین میهمان واسطی کامل است که هویت خودش را از راه خموش‌کننده خودش ادعا می‌کند و نه از راه *par* (تعریف 4.13). در برابر زوجی که اعضایش آن نام را می‌پذیرند، آن‌گاه دو جهان می‌توانند در هویتی که تنها یکی از آن‌ها اشغالش کرده متفاوت پاسخ دهند، و هیچ شرطی بر آن رده نمی‌تواند آن مزاحم را رد کند بی‌آنکه جذب را رد کند: محیط جذب‌شده تعریف 4.15 ماشین‌هایی درست به همان شکل میهمان می‌کند – اشغال‌گران بیرونی در نام‌های پذیرفته – و از آن مزاحم تنها در کدی که می‌دواند تفاوت دارد، که مجازبودن به درستی نمی‌خواندش. بخشی از تعمیر به تحقق تعلق دارد. سیستمی را در زیرمجموعه *I* از اعضایش تنگ بنامید اگر هر عضو بیرون *I* کل شناسه‌های فرایندی را بپذیرد که سیستم اشغالشان کرده و نه نام‌ها را، و تنها به طرف‌هایی خدمت دهد که هر یک از فراخواننده‌های پذیرفته‌اش خدمتشان می‌دهد – که با *serves* این مجموعه طرف عضو خصوصی را درست همان مجموعه طرف فراخواننده‌هایش می‌کند، و همان است که زیرروال خصوصی است. تنگی راه میهمانی را به اعضای بیرون *I* می‌بندد: شناسه فرایند اشغال‌نشده بیرون هر *N* سنجاق‌شده است و در سطح 2 شکست می‌خورد، و شرط طرف هر هویتی را که فراخواننده سنجاق‌شده می‌توانست از آن سخن بگوید اشغال‌شده می‌کند، همان‌جا که تعریف 4.10 میهمانی را ممنوع می‌کند؛ پس فراخوانی که مزاحم به عضوی بیرون *I* خطاب کند رد می‌شود، در سوی ایده‌آل هم، با *N* تهی مسدودگر یا با *Exec* که هیچ اشغال‌گری نمی‌یابد. آنچه تنگی نمی‌بندد، راه به عضوی از خود *I* است: عضو واسطی که نامی به‌کاررفته و خالی را می‌پذیرد – همان الگوی محلی فصل 1 – درست همان‌گونه که عضو خصوصی، به‌دست نمونه جعلی میهمان در دسترس است، و تنگی هیچ عضوی از *I* را مقید نمی‌کند. سنجاق‌کردن آن‌ها هم، یا پذیرفتن تنها $Z \cup A$ آنجا که محیط مستقیم به آن‌ها می‌رسد، آن را می‌بست؛ و ما اینجا به دنبال شرط کافی عمومی نمی‌رویم. تنگی شرطی بهداشتی است در روح پروتکل زیرروال محترم، که برای نشت‌نکردن اعضای خصوصی ضروری است؛ و توصیف کامل اینکه مقایسه در برابر زوج‌های مسدودشده کی وفادار است را باز می‌گذاریم.

جابه‌جایی و تغییرنام

فصل 1 ثبت می‌کند که کد یک کارکرد به‌سختی نشست خودش را می‌خواند و هرگز شماره نمونه‌اش را: s در سطر 3 از **Guard**، در پوشانه‌هایی که فرایش می‌خواند، و در تعریف 1.3 رخ می‌دهد، و i هیچ کجا. این فصل آن مشاهده را به یک حکم بدل می‌کند. تغییرنام نشست‌ها و نمونه‌ها یکرختی اجراهاست، پس تقلیدی که در یک شناسه فرایند اثبات شود در هر شناسه دیگری برقرار است، در همان ε و در همان بودجه‌ها. یک شرط بهداشتی لازم است و نه بیشتر – که کارکردی که از بیرون هر نشستی در دسترس است از هر نشستی در دسترس باشد، که هر ماشینی که این کتاب مشخص می‌کند برآورده‌اش می‌کند – و جداکردن آن نیمی از کار است. و همین است که می‌گذارد تحقق برای یک نشست تنها نوشته شود، چنان که $\mathcal{F}_{\text{Sig}}$ در فصل 15 نوشته شده، و چنان خوانده شود که باقی را می‌پوشاند.

تعریف 7.1 (جابه‌جایی). یک جابه‌جایی جایگشت θ از شناسه‌های فرایند است چنان که

$$(i) \quad \theta(q).F = q.F \text{ به‌ازای هر } q;$$

$$(ii) \quad \theta(q).s = \theta(q').s \iff q.s = q'.s \text{ به‌ازای هر } q, q' \text{ با } q, q' \in \{0, 1\}^*$$

$$(iii) \quad \theta(q) = q \text{ به‌ازای هر یک از آن سه شناسه } q \in \{(A, 0, 0), (Z, 0, 0), (C, 0, 0)\} \text{ که اجرا فراهم می‌کند.}$$

شرط‌های (i) و (ii) درست همین را می‌گویند که روی نام‌های استاندارد

$$\theta(F, s, i) = (F, \hat{\theta}(s), \theta_{F,s}(i))$$

برای جایگشت $\hat{\theta}$ از شناسه‌های نشست و، به‌ازای هر نام و نشست، جایگشت $\theta_{F,s}$ از شماره‌های نمونه – و ما دو مؤلفه جابه‌جایی را به این شکل می‌نویسیم و نه با حرف‌هایی از خودشان، چون σ و ι در گزاره 4.19 و تعریف 4.17 گرفته شده‌اند، و اثبات‌های زیر نمادگذاری آن‌ها را وارد می‌کنند. نام‌ها نگه داشته می‌شوند، چون **Guard** و wf با نام جور می‌کنند و هیچ چیز نمی‌تواند به کارکرد دیگری بدل شود؛ نشست‌ها به‌صورت بلوک جایگشت می‌یابند، چون سطر 3 می‌پرسد آیا دو شناسه یک نشست را

شریکند و نه اینکه کدام نشست؛ و نمونه‌ها آزادانه در درون یک بلوک جایگشت می‌یابند، چون آن مؤلفه هیچ کجا خوانده نمی‌شود.

شرط (iii) آن سه شناسهٔ فرایندی را سنجاق می‌کند که به‌صورت عینی در کد رخ می‌دهند: A ، در قلاب $(A, id.P)$ از **Mediate**؛ C ، در خواندن دفتر $(C, id.P)$ ؛ در سطرهای 2 و 6؛ و Z ، در سطر 2 و در بازپخش سطر 5. هر هویتی که در این کتاب رو نامش برده شده، یکی از این سه است که با طرفی در دامنه جفت شده، پس سنجاق کردن آن سه همه‌شان را سنجاق می‌کند – و همان سه را سنجاق می‌کند و نه بیشتر. و نباید چنان خوانده شود که هر شناسه‌ای از نام رزرو را سنجاق می‌کند، و آن تفاوت آرایشی نیست: بنا بر فصل 1 مجموعه A به‌ازای هر s و i یک سه‌گانهٔ (A, s, i) دارد، که تنها $(A, 0, 0)$ هرگز رخ می‌دهد، پس شرطی که همه‌شان را ثابت کند و همراه (ii) نامقید خوانده شود، $\hat{\theta}(s) = s$ را در هر s می‌داد و هیچ جابه‌جایی‌ای باقی نمی‌گذاشت که اصلاً نشستنی را حرکت دهد. و محدودکردن (ii) به نام‌های استاندارد نیمهٔ دیگر همان دقت است: $\hat{\theta}$ در آنجا تعیین می‌شود، و شناسه‌های نام رزرو که هرگز رخ نمی‌دهند رها می‌شوند تا بی‌آزار در درون ردهٔ خودشان حرکت کنند، که گزارهٔ 7.5 نشان می‌دهد همان جایی است که می‌مانند.

هیچ چیز اینجا نشست 0 را سنجاق نمی‌کند، و $\hat{\theta}$ آزاد است حرکتش دهد. آن آزادی بهایی دارد، و ارزش دیدن دارد که دقیقاً کجا، چون این تنها نقطه‌ای است که چارچوب در آن یک نشست را از دیگری تشخیص می‌دهد. سطر 3 نشست فراخوانده را می‌سنجد در حالی که $global$ در باپ فراخوانده آزموده می‌شود، و آن سه ماشینی که اجرا فراهم می‌کند در نشست 0 می‌نشینند؛ پس فراخوانی از A یا Z به فراخوانده‌ای که جهانی نیست، درست وقتی از آن سطر می‌گذرد که نشست خود فراخوانده 0 باشد. فراخوانده‌ای با این شکل، جابه‌جایی‌ای که نشست 0 را حرکت می‌دهد را از جابه‌جایی‌ای که آن را رها می‌کند تشخیص می‌داد. هیچ چیز دیگر نمی‌تواند: برای هر فراخواندهٔ دیگری، ماشین‌های فراهم‌شده یا با $global$ بخشیده می‌شوند یا پیش از آنکه به نشست نگاه شود در سطر 2 رد می‌شوند. پس یک شرط بر فراخوانده آن شکاف را می‌بندد، و آن شرطی بهداشتی است و نه محدودیتی بر θ .

تعریف 7.2 (نشست‌یکنوا). کارکرد $\mathcal{F}$ نشست‌یکنوا است اگر

$$\mathcal{F}.N \cap (A \cup Z) \neq \emptyset \implies global(\mathcal{F}),$$

یعنی فراخوانده‌ای از بیرون هر نشستنی را تنها در صورتی می‌پذیرد که از هر نشستنی در دسترس باشد. سیستمی نشست‌یکنواست اگر هر یک از اعضایش چنین باشد، و محیطی نشست‌یکنواست اگر هر کارکردی که میهمان می‌کند چنین باشد.

نشست‌هایی که کارکرد نشست‌یکنوا می‌تواند از آن‌ها در دسترس باشد به نشستنی که خودش در آن نشسته وابسته نیست، و این همان است که جابه‌جایی از او می‌خواهد. آن سه ماشین فراهم‌شده این شرط را رو برآورده می‌کنند، چون هر سه جهانی‌اند – A و C با عطف اول $global$ ، و Z با عطف دوم – و هر اشغال‌گر جایگاه

حریف هم، که با همان عطف دوم جهانی است، و مسدودگر هم، که N او تهی است و پس سطر 2 او را هر نشستنی هم که باشد رد می‌کند. این‌ها هر N ی است که این کتاب اعلام می‌کند؛ و $\mathcal{F}_{\text{sig}}$ آن خودش را به‌عنوان پارامتر می‌گیرد، که در آن، سنجیدن این شرط یک سطر است. آنچه این شرط کنار می‌گذارد همان شکلی است که فصل 1 با گفتن اینکه $\mathcal{F}$ محلی می‌تواند خودش بگوید آیا A و Z می‌توانند فرایش بخوانند باز می‌گذارد: کارکرد محلی‌ای که حریف را می‌پذیرد، از جایگاه در نشست 0 در دسترس است و در هیچ نشست دیگری، و این همان یک چیزی است که جابه‌جایی نمی‌تواند حملش کند. نکته 7.17 آن جایگزین را ثبت می‌کند، یعنی فراخ‌کردن یک‌سطری سطر 3 که این شرط را یک‌جا بی‌نیاز می‌کرد.

تعریف 7.3 (کنش یک جابه‌جایی). جابه‌جایی θ روی هویت‌ها با $:= \theta(\text{id}, P)$ $(\theta(\text{id}), P)$ کنش می‌کند، و مؤلفه طرف را رها می‌کند، و روی مجموعه‌های هویت نقطه‌به‌نقطه. کارکرد $\mathcal{F}$ را به $\theta\mathcal{F}$ می‌فرستد با

$$\theta\mathcal{F}.\text{id} := \theta(\mathcal{F}.\text{id}), \quad \theta\mathcal{F}.\mathbf{P} := \mathcal{F}.\mathbf{P}, \quad \theta\mathcal{F}.\mathbf{N} := \theta(\mathcal{F}.\mathbf{N}), \\ \theta\mathcal{F}.\mathbf{U} := \theta(\mathcal{F}.\mathbf{U}), \quad \theta\mathcal{F}.\text{par} := \theta(\mathcal{F}.\text{par}),$$

که در آن θ روی درایه $\mathbf{U}$ با $(\theta\mathcal{F}', R) := (\theta\mathcal{F}', R)$ کنش می‌کند و روی par از راه هر هویت و شناسه فرایندی که در بر دارد؛ و $\theta\mathcal{F}$ همان $\mathcal{F}$ است با هر شناسه فرایندی که در آن رخ می‌دهد – در هدف یک فراخوان، در هویتی ادعایی، در آزمونی، یا در مقداری ذخیره‌شده – که با تصویرش عوض شده؛ و هر مجموعه هویتی که ماشین اعلام می‌کند و حسابش نمی‌کند هم با تصویرش عوض می‌شود، که برای دسته (تعریف 4.13) یعنی مجموعه اشغال‌شده‌اش، مجموعه بیرونی‌اش، و هر مسیریابی‌ای که به‌دست تعیین می‌کند. روی سیستم عضو به عضو کنش می‌کند، و روی هر ماشین دیگر اجرا به همان شکل، اعضای میهمان یک دسته هم در آن، به‌گونه‌ای که $\theta\mathcal{Z}$ تصویرهای آن چیزی را میهمان می‌کند که $\mathcal{Z}$ میهمان می‌کند. ما می‌خواهیم محمول‌های خواسته هم‌وز باشند، یعنی $R(\theta\mathcal{F}, \theta\mathcal{G}) \iff R(\mathcal{F}, \mathcal{G})$ ، چنان که serve هست، چون تنها $\mathbf{P}$ و عضویتی از $\mathbf{N}$ را می‌خواند. هر بند بالا نقطه‌به‌نقطه برچسب عوض می‌کند، پس این کنش یک کنش است: id به‌عنوان همانی کنش می‌کند و $\mathcal{F}'(\theta\mathcal{F}) = \theta'(\theta\mathcal{F})$. به‌ویژه θ^{-1} روی ماشین‌ها همان‌گونه کار θ را باز می‌گرداند که روی شناسه‌های فرایند، و همین است که می‌گذارد نتایج زیر در هر دو جهت استدلال کنند.

عوض‌کردن برچسب شناسه‌های فرایندی که برنامه نامشان می‌برد به‌خودی‌خود عوض‌کردن برچسب آنچه می‌کند نیست، و شکاف میان آن دو شرطی است بر مدل ماشین. ما بیانش می‌کنیم، چون لم 7.9 بی آن نادرست است.

تعریف 7.4 (کد هویت‌اتمی). کد هویت‌اتمی است اگر هویت‌ها و شناسه‌های فرایند را به‌عنوان ژتون‌های مات به کار برد. می‌تواند یکی را به مؤلفه نام یا طرفش تصویر کند؛ دو تای آن‌ها را، یا دو تا از مؤلفه‌های نام، طرف، نشست یا نمونه‌شان را، برای تساوی بسنجد؛ یکی را برای عضویت در مجموعه‌ای از آن‌ها که به کد داده شده بسنجد، چنان که سطر $\mathcal{F}.N\ 2$ را می‌سنجد و Silence را؛ با همان آزمون‌ها مجموعه‌ای از آن‌ها بسازد، چنان که سطرهای 2 و 3 را می‌سازند و سطر 5 $\{id \neq id\}$ را؛ نام یکی را رو ببرد؛ و مقداری را که یکی را حمل می‌کند بشکافد. اما نمی‌تواند با نشست یا شماره نمونه حساب کند: نه حساب عددی، نه ترتیب، و نه هیچ نشست یا نمونه‌ای که به‌تنهایی نوشته شود، جز در درون هویت یا شناسه فرایندی که رو نامش برده شده. هر یک از این‌ها یک گام شمرده می‌شود، هر هویتی هم که در کار باشد، همان‌گونه که تعریف 4.30 فراخوانی را یک پاسخ می‌شمارد.

هر ماشین این کتاب هویت‌اتمی است. محمول Guard نشست‌ها را برای تساوی می‌سنجد و هیچ چیز دیگر، هیچ هسته‌ای اصلاً آن را نمی‌خواند، و تنها شناسه‌های فرایندی که رو نامشان برده شده آن سه شناسه شرط اند (iii) ما مدل را در سرتاسر همین می‌گیریم، و همین است که تعریف 7.3 را سالم می‌کند: روی کد هویت‌اتمی، آن جانشانی تنها عینیت‌ها را بازنویسی نمی‌کند بلکه رفتار برچسب‌عوض‌شده را حساب می‌کند، چون هر عمل مجاز هم‌وز است – تصویرها چون θ نام‌ها و طرف‌ها را رها می‌کند، تساوی و عضویت چون θ یک‌به‌یک است و مجموعه‌هایی که سنجیده و ساخته می‌شوند با کد برچسب عوض می‌کنند، و هویت نام‌برده‌شده چون جانشانی آن را با تصویرش عوض می‌کند. و همین است که ماشین و تصویرش را وادار می‌کند همان گام‌ها را بردارند: همان عمل‌ها را به همان ترتیب انجام می‌دهند، چون جریان مهار در هر دو همان است، و بند آخر آن تعریف به‌ازای هر یک یک گام حساب می‌کند، هر شکلی هم که شناسه‌های درونش تغییرنام یافته باشند. پس بودجه‌های تعریف 4.30 بی‌تغییر منتقل می‌شوند – که مدل هزینه‌ای که نماد به‌نماد حساب می‌کرد آن را نمی‌داد، چون جابه‌جایی آزاد است رمزگذاری یک نشست را بلندتر کند.

بی آن محدودیت، هر دو شکست می‌خورند. هسته‌ای که $id.i \bmod 2$ برمی‌گرداند نام هیچ شناسه فرایندی را نمی‌برد، پس جانشانی دست‌نخورده‌اش می‌گذارد؛ و اگر به شماره نمونه دیگری برده شود متفاوت پاسخ می‌دهد، و ماشین برچسب‌عوض‌شده همان اصل برچسب‌عوض‌شده نیست. هسته‌ای که $(F, id.s, id.i + 1)$ را فرا می‌خواند به همان شکل هدفش را گم می‌کند. هیچ‌یک ماشینی نیست که کسی بنویسدش – تعریف 4.6 هسته دلخواه را مجاز می‌کند، و همین است که چرا این کنارگذاری باید بیان شود و مفروض گرفته نشود.

گزاره 7.5 (ناوردایی‌های پایه). بگذارید θ جابه‌جایی باشد، $\mathcal{F}$ کارکردی و π سیستمی. آن‌گاه

(i) $(\text{IDs}(\theta\mathcal{F}) = \theta(\text{IDs}(\mathcal{F}))$ و اشغال‌گرانی که مجموعه‌های هویتِ دوبه‌دو مجزا حمل می‌کنند آن‌ها را نگه می‌دارند؛

(ii) θ هر ردهٔ نامی را بر خودش می‌برد، پس $\theta(\text{Std}) = \text{Std}$ ، $\theta(\mathbf{A}) = \mathbf{A}$ و $\theta(\mathbf{Z}) = \mathbf{Z}$ ، و هر $\mathbf{N}$ که با نام اعلام شده – یک $\{\text{pid} : \text{pid}.F = F^1\}$ محلی هم در آن – تصویر خودش است؛

(iii) $\text{global}(\theta\mathcal{F}) \iff \text{global}(\mathcal{F})$ ؛

(iv) $\langle \theta\pi \rangle = \theta(\langle \pi \rangle)$ ؛

(v) $\theta\pi$ یک سیستم است، و $\text{wf}(\theta\pi) \iff \text{wf}(\pi)$ ؛ و

(vi) θ^{-1} باز یک جابه‌جایی است.

اثبات. شش بند، و چیز به‌کارآمد این است که ببینیم هر یک کدام‌یک از سه شرطِ تعریف 7.1 را خرج می‌کند. (i) شرط (i) و یک‌به‌یکی است: مجموعهٔ هویت با دو میدان تعیین می‌شود، که یکی برچسب عوض می‌کند و دیگری دست‌نخورده می‌ماند. (ii) تنها شرط (i) است – رده‌های نامی به درون خودشان می‌روند، و چون این یک دوسویی روی یک بخش‌بندی است، بر آن‌ها هم می‌روند، و همین است که آن سه مجموعهٔ شناسهٔ رزرو را ناوردا می‌کند. آن‌گاه (iii) از (ii) در می‌آید، چون global تنها یک نام و زیرمجموعه‌ای از Std را می‌خواند. (iv) باز شرط (i) است، چون بستارهای نامی با نام‌ها مشخص می‌شوند. (v) بند اصلی است: خوش‌ساختی منتقل می‌شود چون نام‌ها جورند، عطفِ نشست یا با global بخشیده می‌شود یا میانِ دو شناسهٔ استاندارد خوانده می‌شود که شرط (ii) آن را نگه می‌دارد، و رابطهٔ R بنا بر تعریف 7.3 هم‌وزر است؛ و عکسِ آن همان استدلال است که روی θ^{-1} دوانده می‌شود. و (vi) دلیل مجازبودن آن است – شرط‌های (i) و (ii) به‌صورت این‌همانی مؤلفه‌ها و به‌صورت یک هم‌ارزی بیان شده‌اند، پس در باب وارون هم برقرارند، و (iii) برقرار است چون θ آن سه شناسه را ثابت می‌کند. (i) مجموعهٔ هویت با pid و $\mathbf{P}$ تعیین می‌شود، و جابه‌جایی برچسبِ اولی را عوض می‌کند و دومی را رها؛ و مجزایبودن جان سالم می‌برد چون θ روی هویت‌ها یک‌به‌یک است.

(ii) بنا بر (i) تعریف 7.1 هر ردهٔ نامی به درون خودش می‌رود. رده‌ها شناسه‌های فرایند را بخش‌بندی می‌کنند، پس برای x در ردهٔ F ، نقطهٔ $\theta^{-1}(x)$ در رده‌ای می‌افتد که به درون خودش می‌رود و پس همان ردهٔ F است؛ پس آن نگاشت بر آن رده هم می‌رود. آن سه مجموعهٔ نمایش‌داده‌شده اجتماعی از رده‌های نامی‌اند، و مجموعه‌ای از شناسه‌ها که با یک نام مشخص شود اجتماعی از آن‌هاست.

(iii) $\text{global}(\mathcal{F})$ یا $\text{Std} \subseteq \mathcal{F}.\mathbf{N}$ را می‌خواند یا $\mathcal{F}.F \in \{A, Z, C\}$ را. دومی با (i)

تعریف 7.1 دست‌نخورده است؛ و برای اولی،

$$\text{Std} \subseteq \theta(\mathcal{F}.\mathbf{N}) \iff \theta^{-1}(\text{Std}) \subseteq \mathcal{F}.\mathbf{N},$$

و $\theta^{-1}(\text{Std}) = \text{Std}$ بنا بر بند (ii) بالا، چون θ یک دوسویی است.

(iv) بستارِ نامی با نام‌هایی که سیستمش به کار می‌برد مشخص می‌شود؛ و θ

آن‌ها را نگه می‌دارد، پس $\theta\pi$ همان نام‌هایی را به کار می‌برد که π ، و $\langle \theta\pi \rangle = \langle \pi \rangle$. و

(ii) $\langle \pi \rangle$ اجتماعی از رده‌های نامی است که در شناسه‌های طرف ضرب شده، پس بند $\langle \pi \rangle = \langle \theta(\pi) \rangle$ را هم می‌دهد.

(v) اعضای $\theta\pi$ شناسه‌های فرایند متمایز حمل می‌کنند، چون θ یک‌به‌یک است، و استانداردند، چون نام‌هایشان تغییر نکرده و پوشانه‌هایشان با عوض‌شدن برچسب دست‌نخورده مانده؛ و آن خانواده متناهی است. برای $\mathcal{F} \in \pi^+$ ، wf و درایه $(\mathcal{F}', R)$ با شاهد $\mathcal{G}$ را بگیرد. نام‌ها در هر دو سو جورند. برای عطف نشست $\mathcal{G}.s = \mathcal{F}.s$ یا $\mathcal{G}$ global برقرار است و بند (iii) بالا آن عطف را در هر دو سو می‌بخشد، یا استاندارد است – و آن‌گاه $\mathcal{F}$ هم چنین است، چون درایه‌های آن سه ماشین فراهم‌شده تنها نام یکدیگر را می‌برند و $\mathcal{U}$ تهی است – به‌گونه‌ای که هر دو شناسه استانداردند و (ii) تعریف 7.1 آن آزمون را نگه می‌دارد. و سرانجام $R(\mathcal{F}, \mathcal{G}) \iff R(\theta\mathcal{F}, \theta\mathcal{G})$ بنا بر همان هم‌ورزی‌ای که تعریف 7.3 می‌خواهد. درایه‌های آن سه ماشین فراهم‌شده در هر دو سو یکسان حل می‌شوند، چون هر میدانی که serves از آن‌ها می‌خواند با بند (ii) بالا و (iii) تعریف 7.1 ثابت شده است. و برای عکس، همان استدلال را روی θ^{-1} بدوانید، که بنا بر بند (vi) جابه‌جایی است، و از $\theta(\pi) = \pi^{-1}(\theta\pi)$ که تعریف 7.3 می‌دهد استفاده کنید.

(vi) شرط‌های (i) و (ii) تعریف 7.1 به‌صورت این‌همانی مؤلفه‌ها و به‌صورت یک هم‌ارزی بیان شده‌اند، پس در باب وارون هم برقرارند؛ و (iii) برقرار است چون θ آن سه شناسه را ثابت می‌کند. $\square$

دو تا از این‌ها سزاوار یک کلمه‌اند. بنا بر (ii)، تنها آن N های سنجاق‌شده نکته 6.5 اصلاً حرکت می‌کنند، و همراه اعضایی حرکت می‌کنند که سنجاقشان می‌کنند. و بنا بر (iv) بستارهای نامی ناوردایند، و همین است که باعث می‌شود گزاره 7.10 در برابر رده محیطی در بیاید که درست مثل پیش مقید شده و نه رده‌ای که تنها متناظر آن است.

تعریف 7.6 (هسته شناسه‌کور). هسته‌ای شناسه‌کور است اگر $\theta Op = Op$ به‌ازای هر جابه‌جایی θ : یعنی نام هیچ شناسه فرایندی را نمی‌برد که جابه‌جایی حرکتش دهد. می‌تواند نشست‌های دو هویت در دامنه را بسنجد، چنان که **Guard** می‌کند، و می‌تواند نام یکی از آن سه شناسه فرایندی را ببرد که (iii) تعریف 7.1 ثابت‌شان می‌کند – اما نه شناسه دیگری از نامی رزرو را، که (iii) آزاد می‌گذاردش حرکت کند.

هر هسته این کتاب شناسه‌کور است. پوشانه‌های فصل 2 تنها نام‌ها و طرف‌ها را می‌خوانند؛ **Mediate** به $(A, id.P)$ قلاب می‌شود و سطرهای 2 و 6 به $(C, id.P)$ ، که هر دو رزروند؛ جعبه $\mathcal{Z}$ را از par با آزمون‌هایی روی نام‌ها و طرف‌ها حساب می‌کند؛ همان فراخوانی را می‌گذارد که بار پیامش نامش می‌برد؛ و $id.P$ $\mathcal{F}_{Sig}$ و P خودش را می‌خواند و هیچ چیز دیگری از یک هویت.

این ویژگی از آنچه لم 7.9 لازم دارد قوی‌تر است، و ویژگی دیگری است. آن لم تعریف 7.4 را لازم دارد، که هر ماشین یک اجرا باید برآورده‌اش کند، محیط و اشغال‌گر جایگاه حریف هم در آن – و هیچ‌یک از آن دو نمی‌تواند شناسه‌کور باشد، چون هر یک ناچار است نام شناسه‌های فرایندی را ببرد که می‌رانندشان. آنچه شناسه‌کوری می‌خرد،

خوانش نتیجه است: برای سیستمی از هسته‌های شناسه‌کور، $\theta\pi$ همان π است با پنج میدان که برچسب عوض کرده‌اند و بی یک سطر کد بازنویسی‌شده، پس پروتکل جابه‌جاشده همان پروتکل است که جای دیگری می‌دود.

پیش از آن لم، نیمهٔ دیگر پرسشی که آن تعریف پاسخش می‌دهد: اجرا اصلاً کدام میدان‌ها را می‌خواند.

گزاره 7.7 (میدان‌های بی‌اثر). بگذارید $\mathcal{F}$ و $\mathcal{F}'$ کارکردهای استاندارد باشند که بر par , P , pid و کد توافق دارند.

(i) اگر بر N هم توافق داشته باشند، به‌گونه‌ای که تنها در U تفاوت داشته باشند، آن‌گاه هیچ اجرایی آن دو را تشخیص نمی‌دهد: در آن دو سیستمی که با نشانند یکی یا دیگری در آن شناسهٔ فرایند به‌دست می‌آیند، هر فعال‌سازی همان پیکربندی را تولید می‌کند.

(ii) اگر تنها در N تفاوت داشته باشند، آن‌گاه از همان حالت و روی همان فراخوان $\text{id}.$ **FullOp**(in) از id' یا هر دو پوشانه در سطر 1 با rej رد می‌کنند، یا هر دو می‌پذیرند و در همان نقطه و در همان حالت به همان هسته وارد می‌شوند.

اثبات. با بازرسی اینکه آن دو میدان در طول یک اجرا کجا خوانده می‌شوند. میدان U تنها به‌دستِ آزمون ایستای تعریف 1.3 و نتایج مربوط به آن خوانده می‌شود، و هرگز به‌دستِ **Exec** یا هیچ واسطی، و همین (i) را می‌دهد. خوانندگان N در درون یک اجرا دو تاینند: سطر 2 از **Guard**، و محمول **global** و پس سطر 3. هر دو در درون $\text{Guard}_{\mathcal{F}}$ می‌نشینند، که سطر 1 از واسط کامل پیش از هر چیز دیگری آن را **require** می‌کند. (این میدان بیرون اجرا هم خوانده می‌شود – با تعریف 1.3 از راه آن دو، با تعریف 6.1، و با شرط‌های نوع‌بندی تعریف 4.7 و نکتهٔ 6.5 – اما آن‌ها شرط‌هایی بر یک سیستمند و نه گام‌هایی از آن.) پس این میدان همان حکم را تعیین می‌کند و هیچ جای دیگری در پوشانه خوانده نمی‌شود: سطرهای میان سطر 1 و هسته – خواندن دفتر، دروازهٔ فساد، قلاب اول **Mediate**، مجموعهٔ خموش‌شدهٔ $\{\text{id}'' \neq \text{id}\}$ – تابع‌هایی از P , pid , par ، کد و آن فراخوانند، پس دو پوشانه‌ای که هر دو می‌پذیرند با هم به سطر 5 می‌رسند و در همان نقطه و در همان حالت به همان هسته وارد می‌شوند. و این (ii) است. $\square$

پس N کنترل دسترسی است و هیچ چیز دیگر، و U در هر اجرایی بی‌اثر است، چنان که نکتهٔ 4.8 از پیش برای جایگاه حریف ثبت می‌کند. بی‌اثری پس از نگهبان حکمی در باب یک گام است، و نمی‌شود آن را به حکمی در باب مقدار برگردانده‌شده قوی کرد. دلیل آشکار این است که فراخوانی که یکی می‌پذیرد و دیگری رد می‌کند، آن دو حالت را از هم دور می‌کند. و دلیل کم‌آشکارتر این است که حتی فراخوانی که هر دو بپذیرند می‌تواند متفاوت تمام شود، چون هسته‌ای که در سطر 5 وارد می‌شود می‌تواند به همین پوشانه دوباره وارد شود – بخش 3.6 مجاز می‌کند نمونه‌ای دوباره وارد شود، و سطر 5 می‌گذارد هسته هویت خودش را ادعا کند – و آن فراخوان درونی باز با $\text{Guard}_{\mathcal{F}}$ روبه‌رو می‌شود، همان جا که آن میدان تفاوت دارد. هسته‌ای که $\text{id}.$ **FullOp**(') از id به‌عنوان id می‌گذارد و تنها اگر پاسخ rej نبود 1 برمی‌گرداند، $\mathcal{F}$ که شناسهٔ فرایند خودش را می‌پذیرد را از

$\mathcal{F}'$ ی که نمی‌پذیرد جدا می‌کند، روی فراخوانی که هر دو پذیرفته بودند. آنچه جان سالم می‌برد همان ادعای دقیق است: آن دو تنها در حکم‌های نگهبان تفاوت دارند، و یک حکم متفاوت، بیرونی یا درونی، برای جداکردنشان بس است.

نکته 7.8 (آن دو میدانی که خوانده می‌شوند). نه $\mathbf{P}$ حکمی از این دست را می‌پذیرد و نه par ، و به دو دلیل متفاوت. میدان $\mathbf{P} \text{ IDs}(\mathcal{F})$ را تعیین می‌کند، پس تعیین می‌کند Exec اصلاً چه فراخوان‌هایی را به $\mathcal{F}$ توزیع می‌کند و عطف دوم سطر 1 را؛ و هسته‌ها مستقیم می‌خوانندش — $\mathcal{F}_{\text{Sig}} \text{ Initialize} - \mathcal{K} \cup \{\square\}$ نوع $\text{VK} : \mathcal{F}_{\text{Sig}}.\mathbf{P} \rightarrow$ را می‌دهد، و آزمون $\exists P' \notin \mathbf{C}$ در Verify روی همان دامنه سور می‌زند. پس بزرگ کردن یا کوچک کردن مجموعه طرف تقارن نیست. آنچه هست تغییرنام است: جایگزینی β از $\{0, 1\}^*$ که برای ثابت کردن A و Z توسیع یافته، و روی هویت‌ها، روی $\mathbf{P}$ ، روی $\mathbf{C}$ و روی حالت اندیس‌گذاری شده با طرف کنش می‌کند، بنا بر استدلال لم 7.9 یکریختی اجراها می‌دهد، به شرطی که هسته‌ها شناسه‌های طرف را تنها از راه تساوی و اندیس‌گذاری بخوانند — چنان که $\mathcal{F}_{\text{Sig}}$ می‌کند، چون $\exists P' \notin \mathbf{C} : \text{vk} = \text{vk}[P']$ آن هم‌وزن است. آن دو جایی هم که شناسه طرف جز این وارد می‌شود از آن جان سالم می‌برند: سطر 3 نام طرف Z را می‌برد، که β ثابتش می‌کند، و FullCorrupt $\text{id}.\mathbf{P} \in \{0, 1\}^*$ را می‌سنجد، که مجموعه‌ای است که β جایگزینش می‌دهد. میدان par بنا بر تعریف خود میدان به‌دست می‌شود، و $\mathcal{Z}.\text{par}$ علاوه بر آن به‌دست تعریف 4.10؛ و چیزی نیست که ناوردا باشد. جابه‌جایی حرکتش می‌دهد، و هر جا با نام اعلام شده باشد به خودش حرکتش می‌دهد، چنان که $\langle \pi \rangle := \mathcal{Z}.\text{par}$ چنین است.

به‌زبان غیررسمی، لم 7.9 می‌گوید که عوض کردن برچسب نشست‌ها و نمونه‌ها در سرتاسر یک اجرا هیچ چیز مشاهده‌شدنی را تغییر نمی‌دهد: این همان استقرای گزاره 4.19 است با θ به‌جای آن دوسویی جذب، و هر سنجش، هم‌وزنی یک سطر از یک پوشانه است.

لم 7.9 (جابه‌جایی). بگذارید θ جابه‌جایی باشد، π سیستمی نشست‌یکنوا، $\mathcal{X}$ اشغال‌گری از جایگاه حریف (تعریف 3.3)، و $\mathcal{Z}$ محیطی نشست‌یکنوا که در هیچ هویتی از $\text{IDs}(\pi)$ میهمان نمی‌کند، به‌گونه‌ای که Exec در آن چهار تعریف شده باشد. آن‌گاه

$$\text{Exec}(\theta\pi, \theta\mathcal{Z}, \theta\mathcal{X}, \mathcal{C}) \quad \text{and} \quad \text{Exec}(\pi, \mathcal{Z}, \mathcal{X}, \mathcal{C})$$

همسان‌توزیعند.

اثبات. تناظر تعریف 4.17 در $\theta := \iota$ ، که θ تا خوانده می‌شود: دو حالت، دو نقطه برنامه یا دو زنجیره فراخوانده متناظرند هرگاه یکی تصویر دیگری باشد. و چون θ یک دوسویی است این رابطه متقارن است، و اثبات باز استقرایی است روی فعال‌سازی‌ها،

که چهار بند زیر گامش را انجام می‌دهند. ماشین‌ها می‌سنجد که برچسب اشغال دوسویانه عوض شده، پس هر دو اجرا تعریف شده‌اند و در تناظر آغاز می‌کنند. نگهبان‌ها سه عطف $Guard$ را به ترتیب می‌گیرد؛ دو تای اول یک‌به‌یکی‌اند، و سومی آن یک بندی است که کار واقعی می‌کند، چون سطر 3 تنها جایی است که چارچوب یک نشست را از دیگری تشخیص می‌دهد – و همین جاست که نشست یکنوایی خرج می‌شود، و بی آن جابه‌جایی‌ای که نشست 0 را حرکت می‌دهد آن حکم را تغییر می‌داد. خموش‌کننده‌ها، واسطه‌گری، دفتر پوشانه‌ها را می‌سنجد، و می‌بیند که دفتر اصلاً تصویری لازم ندارد، چون فساد تنها با طرف اندیس‌گذاری می‌شود. مسیریابی، ادعاها، ژتون گام را می‌بندد، چون کد هویت‌امی در هر دو سو همان شاخه‌ها را می‌گیرد. آن تناظر همان تناظر تعریف 4.17 است با $\theta := \iota$ – هر ماشین با تصویرش جفت شده، و $\mathcal{C}$ با خودش – و با (C1)، (C3) و (C4) که تا θ خوانده می‌شوند: دو حالت، دو نقطه برنامه، دو زنجیره فراخواننده معلق متناظرند هرگاه یکی تصویر θ از دیگری باشد. و چون θ روی هویت‌ها یک دوسویی است این رابطه متقارن است، و مثل پیش اثبات استقرایی است روی فعال‌سازی‌ها. بندهای زیر گام را انجام می‌دهند.

ماشین‌ها. π یک سیستم است و برچسب اشغال دوسویانه عوض شده، بنا بر (i) و (v) گزاره 7.5؛ و چون $\mathcal{Z}$ برکنار از $IDS(\pi)$ میهمان می‌کند و $\theta\mathcal{Z}$ در تصویرهای آنچه $\mathcal{Z}$ میهمان می‌کند میهمان می‌کند، اشغال‌گران در سمت راست درست همان جا دوبه‌دو مجزایند که در سمت چپ، پس هر دو اجرا تعریف شده‌اند. سطر 1 $\mathcal{C}$ و هر عضو را در هر دو سو مقدار آغازین می‌دهد، و حالت آغازین هر عضو همان حالت برچسب‌عوض‌شده تصویرش است، که (C1) را در آغاز می‌دهد.

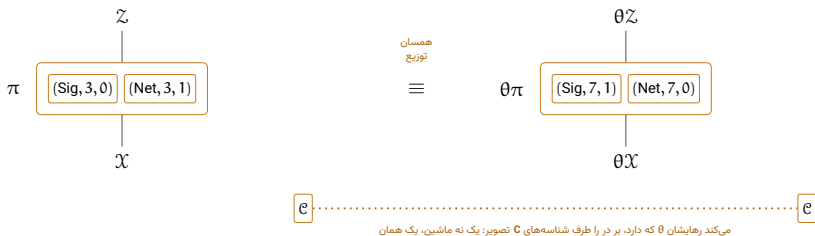
نگهبان‌ها. فراخوان $id.\text{FullOp}(in)$ از id' در $\mathcal{F}$ و تصویرش در $\theta\mathcal{F}$ را بگیرد. سطر 1: $id.\text{pid} = \mathcal{F}.\text{pid}$ اگر و تنها اگر $\theta(id.\text{pid}) = \theta(\mathcal{F}.\text{pid})$ ، بنا بر یک‌به‌یکی، و $id.P \in \mathbf{P}$ دست‌نخورده است. سطر 2: $id'.\text{pid} \in \mathcal{F}.\mathbf{N}$ اگر و تنها اگر $\theta(id'.\text{pid}) \in \theta(\mathcal{F}.\mathbf{N})$ ، باز بنا بر یک‌به‌یکی، و آزمون طرف مؤلفه‌هایی را می‌خواند که θ رهپاشان می‌کند. سطر 3 فصلی است که عطف دومش بنا بر (iii) گزاره 7.5 در دو سو جور است، پس فرض کنید $global(\mathcal{F})$ شکست بخورد – که در هر دو سو با هم شکست می‌خورد یا در هیچ‌یک – و اولی را در نظر بگیرید. هر ماشین آن اجرا نشست یکنواست: π و رونوشت‌های میهمان $\mathcal{Z}$ بنا بر فرض، و $\mathcal{X}$ و $\mathcal{C}$ و واسطه‌های خود $\mathcal{Z}$ چون شناسه‌های فرایندشان A, C و Z است، که $global$ با عطف دومش می‌پذیردشان. پس $\mathcal{F}.\mathbf{N}$ نه با A برخورد می‌کند و نه با Z ، و ادعایی که شناسه A یا Z را حمل کند در هر دو سو در سطر 2 رد می‌شود – چون (iii) آن شناسه‌ها را ثابت می‌کند، و $\theta(\mathcal{F}.\mathbf{N})$ درست مثل $\mathcal{F}.\mathbf{N}$ از A و Z خالی است، چون آن دو مجموعه بنا بر (ii) گزاره 7.5 زیر θ ناوردایند. و شناسه سوم فراهم‌شده اصلاً نمی‌تواند برسد، چون $\mathcal{C}$ هیچ فراخوانی نمی‌گذارد. پس ادعاهایی که به سطر 3 می‌رسند شناسه‌های فرایند استاندارد حمل می‌کنند، و id هم چنین، چون به محض شکست $global(\mathcal{F})$ نام $\mathcal{F}$ در $\{0, 1\}^*$ می‌افتد؛ پس $id.s = id'.s$ درست وقتی در باپ تصویرها برقرار است که در باپ اصل‌ها برقرار باشد، بنا بر (ii) تعریف 7.1. این همان یک بندی است که نشست یکنوایی برای آن است؛ و بی آن، جابه‌جایی‌ای که نشست 0 را حرکت می‌دهد این حکم را تغییر می‌داد. پس آن حکم در هر دو سو، در هر فراخوانده‌ای، همان است، از جمله در واسطه نشب بخش 3.2، که سه خواسته‌اش

تنها همان چیزی را می‌خوانند که این سطرها می‌خوانند.

خמוש‌کننده‌ها، واسطه‌گری، دفتر. مجموعهٔ خموش‌شدهٔ سطر 5 همان $\{id'' \neq id\}$ است، که تصویرش $\{id'' \neq \theta id\}$ است، پس هسته در هر دو سو هویت خودش را ادعا می‌کند و هیچ چیز دیگر را. مجموعهٔ $\bar{Z}$ با آزمون‌هایی روی نام‌ها و طرف‌ها حساب می‌شود، پس تصویر همان مجموعه‌ای است که در سمت چپ حساب شد، و درست تصویر آن چیزی را می‌پذیرد که آن یکی می‌پذیرد. پوشانهٔ **C Mediate** و $id.P$ و $id'.F$ را می‌خواند و به $(A, id.P)$ قلاب می‌شود؛ و سطرهای 2 و 6 $(C, id.P)$ را خطاب می‌کنند. هر شناسه‌ای که اینجا رو نامش برده شده یکی از آن سه است که (iii) تعریف 7.1 ثابتشان می‌کند. و خود دفتر اصلاً تصویری لازم ندارد: **C** شناسه‌های طرف را جمع می‌کند، که جابه‌جایی به آن‌ها دست نمی‌زند، پس $\mathcal{C}$ عیناً همان یک ماشین است که در پیکربندی‌های متناظر همان مجموعه را در بر دارد. و اندیس‌گذاری فساد تنها با طرف (بخش 3.1) همان است که این را می‌خرد.

مسیریابی، ادعاها، ژتون. **Exec** فراخوان را به اشغال‌گر هویت هدفش توزیع می‌کند، و هدف‌ها زیر θ متناظرند در حالی که اشغال هم بنا بر بند اول چنین است؛ و فراخوانی که به هویتی اشغال‌نشده خطاب شود در سمت چپ درست وقتی rej پاسخ می‌گیرد که تصویرش در سمت راست چنین پاسخ بگیرد. ادعاها بنا بر بند پیشین متناظرند. و چون کد هویت‌امی است (تعریف 7.4)، ماشین و تصویرش همان شاخه‌ها را می‌گیرند و فراخوان‌هایی می‌گذارند که تصویرهای θ از یکدیگرند، پس به همان نقطهٔ برنامه با مقدارهایی می‌رسند که θ مربوطشان می‌کند؛ و هر یک همان سکه‌ها را به همان ترتیب می‌خواند، که $(C2)$ است. نظم ژتون تغییر نکرده، چون **Exec** در هر دو سو یک عمل دارد و همان زنجیرهٔ تعلیق پیش می‌آید، که $(C3)$ و $(C4)$ است. پیکربندی‌های ایستای متناظر یکسان خروجی می‌دهند، چون خروجی بازگشت $\bar{Z}$ و $\theta\bar{Z}$ در (Z, Z) است، هویتی که (iii) تعریف 7.1 ثابتش می‌کند. پس آن دو اجرا به‌عنوان توزیع توافق دارند، به‌ویژه بر احتمال برگرداندن 1. $\square$

دیدن این لم آسان‌تر از بیانش است. یک اجرا، و همان اجرا با هر شناسهٔ فرایندی که تغییرنام یافته، پهلوی هم:



آن: درون در آزاد نمونه جابه‌جاشده، نشست بلوک نگه‌داشته، نام $\theta : (F, s, i) \mapsto (F, \theta(s), \theta_F(i))$ و $\bar{Z}$ و $\theta\bar{Z}$ پس سنجاق‌شده، $(Z, 0, 0)$ و $(A, 0, 0)$ و $(C, 0, 0)$ می‌کنند حرکت کدشان درون فرایند شناسه‌های تنها و می‌دارند نگه را هویت‌هایشان $\mathcal{C}$ و $\bar{X}$ و $\theta\bar{X}$ و $\theta\bar{Z}$ پس سنجاق‌شده، $(Z, 0, 0)$ و $(A, 0, 0)$ و $(C, 0, 0)$

شکل 4: یک اجرا در کنار همان اجرا با هر شناسهٔ فرایندی که تغییرنام یافته. لم 7.9 می‌گوید آن دو به‌عنوان توزیع توافق دارند.

هر چه جابه‌جایی مجاز است بکند در آن دو سیستم خواندنی است. نام نگه داشته می‌شود، چون **Guard** و **wf** با نام جور می‌کنند و هیچ چیز نمی‌تواند بی‌صدا به کارکرد دیگری بدل شود. نشست به صورت بلوک حرکت می‌کند – هر دو عضو با هم از 3 به 7 می‌روند – چون سطر 3 می‌پرسد آیا فراخوانده و فراخوانده یک نشست را شریکند و هرگز نمی‌پرسد در کدام نشستند. و نمونه آزاد است در درون آن بلوک حرکت کند، که اینجا 0 و 1 جا عوض می‌کنند، چون هیچ سطری از هیچ کدی آن را نمی‌خواند. آن سه شناسه‌ای که به صورت عینی رخ می‌دهند سنجاق شده‌اند، پس $\mathbb{Z}$ و $\mathbb{X}$ و $\mathbb{C}$ در هر دو سو در همان هویت‌ها می‌نشینند و تنها شناسه‌های فرایند درون کدشان بازنویسی می‌شوند.

$\mathbb{C}$ حتی آن هم نیست: همان یک ماشین است که همان مجموعه را در بر دارد، چون **C** شناسه‌های طرف را جمع می‌کند و جابه‌جایی به طرف دست نمی‌زند. اندیس‌گذاری فساد تنها با طرف (بخش 3.1) همان است که این را می‌خرد، و ارزشی توجه دارد که چه قدر می‌خرد – دفتری که با شناسه فرایند اندیس‌گذاری شده بود ناچار بود مثل هر چیز دیگر با θ منتقل شود، و آن‌گاه الگوی فساد چیزی می‌شد که جابه‌جایی می‌توانست آشفته‌اش کند.

یک هشدار که آن تصویر نمی‌تواند بدهد. اگر این‌گونه کشیده شود، آن لم مفت به نظر می‌رسد، و تقریباً هست: هر عطف نگهبان، هر خموش‌کننده، هر قلاب واسطه‌گری تنها نام‌ها و طرف‌ها و آن سه شناسه سنجاق‌شده را می‌خواند، که هر یک یا از θ جان سالم می‌برد یا با آن ثابت شده است. استثنا همان یک آزمون نشست است، و همان دلیلی است که نشست‌یکنوایی فرض است و نه یک نکته – فراخوانده‌ای که از بیرون هر نشستی در دسترس است اما در یکی از آن‌ها می‌نشیند، پیش و پس از θ که نشست 0 را حرکت می‌دهد به ماشین‌های فراهم‌شده متفاوت پاسخ می‌داد، و آن دو اجرا در همان یک سطر از هم می‌پاشیدند. آن بند اثبات کل کار است؛ و باقی حسابداری‌ای است که آن تصویر از پیش نشان می‌دهد.

گزاره 7.10 (تقلید منتقل می‌شود). بگذارید θ جابه‌جایی باشد و π ، φ سیستم؛ و برای (ii) و (iii) بگذارید آن دو و هر محیط رده‌هایی که نامشان برده می‌شود نشست‌یکنوا باشند، که (i) آن را لازم ندارد. آن‌گاه

$$(i) \quad \theta(\mathbb{Z}_{\pi, \varphi}) = \mathbb{Z}_{\theta\pi, \theta\varphi} \text{ و } \theta \text{ حریف‌ها را به حریف‌ها و شبیه‌سازها را به}$$

شبیه‌سازها می‌برد و بدل بخش 4.4 را ثابت می‌کند؛

$$(ii) \quad \text{اگر } \pi \text{ کارکرد } \varphi \text{ را در } \varepsilon \text{ و در برابر } (\mathbb{A}, \mathbb{S}, \mathbb{Z}) \text{ تقلید-UC کند، آن‌گاه } \theta\pi \text{ کارکرد}$$

$$\varphi \text{ را در } \varepsilon \text{ و در برابر } (\theta\mathbb{A}, \theta\mathbb{S}, \theta\mathbb{Z}) \text{ تقلید-UC می‌کند؛ و}$$

$$(iii) \quad \text{برای سیستم‌های بودجه‌دار و به‌ازای هر } [\mathbf{a}, \mathbf{s}, \mathbf{z}] = \text{Adv}_{\theta\pi, \theta\varphi}^{\text{uc}}[\mathbf{a}, \mathbf{s}, \mathbf{z}] = \text{Adv}_{\pi, \varphi}^{\text{uc}}[\mathbf{a}, \mathbf{s}, \mathbf{z}].$$

اثبات. سه ادعا، و هر یک بر بخش متفاوتی از گزاره 7.5 تکیه دارد. (i) حسابداری در باپ رده‌هاست: هر دو بند تعریف 4.10 منتقل می‌شوند، بند par چون بستارهای نامی زیر θ نوردایند و بند میهمانی بنا بر یک‌به‌یکی، و آن شمول در هر دو جهت می‌رود چون

θ^{-1} هم جابه‌جایی است؛ و حریف‌ها به حریف‌ها و شبیه‌سازها به شبیه‌سازها می‌روند چون میدان‌هایی که آن تعریف‌ها می‌خوانند همان‌هایی هستند که θ ثابتشان می‌کند، و بدل رو ثابت است، چون هسته‌اش شناسه‌کور است. آن‌گاه (ii) بی‌درنگ از لم 7.9 در می‌آید که به هر یک از دو جهان اعمال شده، و شبیه‌ساز پس داده شده همان $\theta\mathcal{S}$ است، که تنها به $\theta\mathcal{A}$ وابسته است. و (iii) صورت عینی است: θ دوسویی‌ای است از هر یک از آن سه ردهٔ بودجه‌دار بر رده‌ای که نتیجه رویش سور می‌زند، و اندیس‌گذاری دوبارهٔ یک تابع در امتداد دوسویی‌ها نه سوپریمم را تغییر می‌دهد و نه اینفیمم را. و نشست‌یکنوایی در سرتاسر نگه داشته می‌شود، که یک بار سنجیده می‌شود و نه در هر بخش. برای (i)، تعریف 4.10 دو چیز از $\mathcal{Z}$ می‌خواهد. بند par رو زیر θ ناورداست: $\theta\mathcal{Z}.par = \theta(\mathcal{Z}.par)$ درست وقتی $\langle \theta\pi \rangle \cup \langle \theta\varphi \rangle = \langle \pi \rangle \cup \langle \varphi \rangle$ را در θ دارد که $\mathcal{Z}.par$ در θ داشته باشد، چون بستارهای نامی بنا بر (iv) گزارهٔ 7.5 ناوردایند. بند میهمانی بنا بر یک‌به‌یکی منتقل می‌شود، چون $\theta\mathcal{Z}$ در تصویرهای آنچه $\mathcal{Z}$ میهمان می‌کند میهمان می‌کند و $IDs(\theta\pi) \cup IDs(\theta\varphi)$ بنا بر (i) همان گزاره تصویر $IDs(\pi) \cup IDs(\varphi)$ است. و آن شمول در هر دو جهت می‌رود، چون θ^{-1} بنا بر (vi) جابه‌جایی است. و برای جایگاه، حریف $IDs(\mathcal{A})$ را اشغال می‌کند، که هویت‌هایش همان شناسهٔ فرایندی را حمل می‌کنند که (iii) تعریف 7.1 ثابتش می‌کند و مؤلفه‌های طرفی که θ رهایشان می‌کند، و پوشانهٔ $\mathcal{A}$ را حمل می‌کند، که شناسه‌کور است؛ پس θ تنها برچسب هسته را عوض می‌کند و $\theta\mathcal{A}$ حریفی از همان شکل است. تعریف 4.7 $\mathbf{N} \supseteq \mathbf{Std} \cup \mathbf{Z}$ را می‌خواهد، که مجموعه‌ای است که θ بنا بر (ii) ثابتش می‌کند، پس شبیه‌سازها به شبیه‌سازها می‌روند. و هستهٔ بدل شناسه‌کور است – سطر 3 همان فراخوانی را می‌گذارد که بار پیامش نامش می‌برد – پس $\theta\mathcal{D} = \mathcal{D}$. و هیچ چیز از این دست در باب $\mathcal{A}$ بخش 3.4 ادعا نمی‌شود، که تعریف 4.6 هسته‌اش را دلخواه می‌گذارد، و هیچ چیز زیر آن را لازم ندارد.

برای (ii)، $\mathcal{A} \in \mathbb{A}$ را ثابت کنید و بگذارید $\mathcal{S} \in \mathbb{S}$ همان شبیه‌سازی باشد که فرض برایش فراهم می‌کند. با داشتن $\theta\mathcal{A} \in \theta\mathbb{A}$ ، $\theta\mathcal{S} \in \theta\mathbb{S}$ را پس بدهید؛ او تنها به $\theta\mathcal{A}$ وابسته است، پس ترتیب سورهای تعریف 4.11 برآورده است. به‌ازای هر $\theta\mathcal{Z} \in \theta\mathbb{Z}$ ، لم 7.9 که به هر یک از دو جهان اعمال شده می‌دهد

$$\text{Adv}_{\theta\pi, \theta\varphi}^{\text{uc}}(\theta\mathcal{Z}, \theta\mathcal{A}, \theta\mathcal{S}) = \text{Adv}_{\pi, \varphi}^{\text{uc}}(\mathcal{Z}, \mathcal{A}, \mathcal{S}) \leq \varepsilon,$$

چون آن دو اجرای سمت چپ تصویرهای θ از آن دوی سمت راستند. و بنا بر (i) آن نتیجه حکمی مجاز از تقلید است، چون $\theta\mathbb{Z} \subseteq \mathbb{Z}_{\theta\pi, \theta\varphi}$. نشست‌یکنوایی در سرتاسر نگه داشته می‌شود: θ بنا بر (ii) گزارهٔ 7.5 A و $\mathbf{Z}$ را ثابت می‌کند و بنا بر (iii) آن global را، پس $\theta\mathcal{F}$ درست وقتی نشست‌یکنواست که $\mathcal{F}$ چنین باشد، و همین در باب یک سیستم و یک محیط عضو به عضو. پس هر ردهٔ زیر از این جهت هم زیر θ بسته است.

برای (iii)، ماشین و تصویرش همان گام‌ها را برمی‌دارند و به همان فراخوان‌ها پاسخ می‌دهند، چون کد هویت‌اتمی است و پس جریان مهارش در هر دو همان است (تعریف 7.4)؛ و عضو یک سیستم بودجه‌دار و تصویرش به همان دلیل همان بودجه را حمل می‌کنند. پس θ دوسویی‌ای است از هر یک از آن سه رده بر رده‌ای که نتیجه رویش سور می‌زند: $\mathbf{a} \in \mathbb{A}$ و $\mathbf{s} \in \mathbb{S}$ بر خودشان، و $\mathbb{Z}_{\pi, \varphi}(\mathbf{z})$ بر $\mathbb{Z}_{\theta\pi, \theta\varphi}(\mathbf{z})$ بنا بر (i)، که هر

یک وارونی دارد که θ^{-1} القا می‌کند. لم 7.9 مزیت تعریف 4.5 را در سه‌گانه‌های متناظر برابر می‌کند، پس آن دو اکستریم تودرتوی تعریف 4.31 همان سه عملگرند که به یک تابع اندیس‌گذاری شده در امتدادِ دوسویی‌ها اعمال شده‌اند، و اندیس‌گذاری دوباره نه سوپریم را تغییر می‌دهد و نه اینفیم را. اینجا هیچ متناهی‌بودنی لازم نیست؛ آنچه متناهی‌بودن تعریف 4.31 حل می‌کند این است که آن اکستریم‌ها حاصل می‌شوند، که مسئلهٔ جداگانه‌ای است و در هر دو سو یکسان برقرار است. □

پس تقلید ویژگی یک سیستم است تا حدِ جابه‌جایی، و اثبات می‌تواند نشست و نمونه را از آغاز تعیین کند. و برای گفتن این در بابِ خانواده‌ای از نمونه‌ها، آدم باید بدانند دو فهرست از شناسه‌های فرایند اصلاً کی با یک جابه‌جایی مربوطند، و پاسخ سه شرط است.

گزاره 7.11 (چه وقت یک چندگانه به دیگری جابه‌جا می‌شود). بگذارید $\vec{q} = (q_1, \dots, q_m)$ و $\vec{q}' = (q'_1, \dots, q'_m)$ چندگانه‌هایی از شناسه‌های فرایند نام استاندارد باشند. جابه‌جایی θ با $q'_j = \theta(q_j)$ به‌ازای هر j هست اگر و تنها اگر، به‌ازای هر j و k ,

$$q_j.F = q'_j.F, \quad (q_j.s = q_k.s \iff q'_j.s = q'_k.s), \\ (q_j = q_k \iff q'_j = q'_k).$$

به‌ویژه، هر دو شناسهٔ فرایند از یک نام با یک جابه‌جایی مربوطند.

اثبات. ضرورت همان سه شرط تعریف 7.1 است که به‌ترتیب خوانده می‌شود. کفایت یک ساخت است: θ را از انتساب نشست بسازید، که شرط دوم آن را دوسویی‌ای خوش‌تعریف میان نشست‌هایی می‌کند که در دو سو رخ می‌دهند، و آن را به یک جایگشت توسیع دهید، چون آنچه باقی می‌ماند در هر سو یک اندازه است؛ سپس، در هر نام و نشست، $\theta_{F,s}$ را از انتساب نمونه بسازید، که شرط سوم و چهارم آن را یک‌به‌یک و پس باز توسیع‌پذیر می‌کنند. و با گرفتن همانی در هر جای دیگر، جابه‌جایی‌ای سوار می‌شود که $\vec{q}$ را به $\vec{q}'$ می‌برد، و (iii) برقرار است چون به شناسه‌های رزرو هرگز دست زده نشد. ضرورت همان شرط‌های تعریف 7.1 است که به‌ترتیب خوانده می‌شود: نام‌ها با (i)، شریک‌بودن یا نبودن نشست با (ii) و تساوی با یک‌به‌یک‌بودن θ .

برای کفایت، بگذارید S و S' همان تعداد متناهی نشست‌ها باشند که در $\vec{q}$ و در $\vec{q}'$ رخ می‌دهند. شرط دوم $q_j.s \mapsto q'_j.s$ را دوسویی‌ای خوش‌تعریف $S' \rightarrow S$ می‌کند – که پوشاست، چون هر $q'_j.s$ یک تصویر است – پس آنچه در دو سو باقی می‌ماند یک اندازه است، متناهی یا نامتناهی یکسان، و به جایگشت $\hat{\theta}$ از شناسه‌های نشست توسیع می‌یابد. اکنون نام F و نشست $s \in S$ را ثابت کنید و اندیس‌های i را در نظر بگیرید که $q_j.F = F$ و $q_j.s = s$. انتساب نمونهٔ آن‌ها، یعنی $q'_j.i \mapsto q_j.i$ ، خوش‌تعریف و یک‌به‌یک است: اگر $q_j.i = q_k.i$ آن‌گاه $q_j = q_k$ ، پس بنا بر شرط چهارم $q'_j = q'_k$ و نمونه‌ها جورند؛ و اگر $q_j.i \neq q_k.i$ آن‌گاه $q_j \neq q_k$ ، پس $q'_j \neq q'_k$ در حالی که

نام‌ها و نشست‌هایشان جورند، و نمونه‌هایشان تفاوت دارند. هر یک باز دوسویی‌ای است میان شماره‌های نمونه‌ای که در دو سو رخ می‌دهند، پس مثل $\hat{\theta}$ به جایگشت $\theta_{F,s}$ توسیع می‌یابد؛ و در هر نام و نشست دیگری همانی را بگیرید، و بگذارید θ روی نام‌های استاندارد به صورت $(F, s, i) \mapsto (F, \hat{\theta}(s), \theta_{F,s}(i))$ و روی نام‌های رزرو به صورت همانی کنش کند. آن‌گاه θ تعریف 7.1 را برآورده می‌کند، و (iii) چون شناسه‌های رزرو رو ثابتند، و $\vec{q}$ را به $\vec{q}'$ می‌برد. و برای ادعای آخر، یک زوج تنهای q, q' از یک نام آن سه شرط را نهد و $\vec{q}$ را برآورده می‌کند.

نتیجه 7.12 (یک نمونه پس است). زوجی از نگاشت‌های $\vec{q} \mapsto \pi(\vec{q})$ و $\vec{q} \mapsto \varphi(\vec{q})$ را خانواده نمونه بنامید اگر $\pi(\theta\vec{q}) = \theta(\pi(\vec{q}))$ و $\varphi(\theta\vec{q}) = \theta(\varphi(\vec{q}))$ به‌ازای هر جابه‌جایی θ ، که در آن $\vec{q}$ فهرست شناسه‌های فرایندی است که $\pi(\vec{q})$ و $\varphi(\vec{q})$ با هم اشغالشان می‌کنند – یعنی یک پروتکل، زیرروال‌های خصوصی‌اش و مشخصه‌ای که در برابرش سنجیده می‌شود، که همه با یک چندگانه اندیس‌گذاری شده‌اند و با آن حرکت می‌کنند. فرض کنید $\pi(\vec{q})$ و $\varphi(\vec{q})$ نشست‌یکنوا باشند و $\pi(\vec{q})$ کارکرد $\varphi(\vec{q})$ را در ε و در برابر $(A(a), S(s), \mathbb{Z}_{\pi(\vec{q}), \varphi(\vec{q})}(z))$ که به محیط‌های نشست‌یکنوا تنگ شده تقلید-UC کند، برای یک $\vec{q}$ تنها. آن‌گاه در هر $\vec{q}'$ که سه شرط گزاره 7.11 را برآورده کند، $\pi(\vec{q}')$ کارکرد $\varphi(\vec{q}')$ را در همان ε و در برابر $(A(a), S(s))$ و همان رده تنگ‌شده برای آن زوج تقلید-UC می‌کند. به‌ویژه، برای خانواده‌ای که یک شناسه فرایند را اشغال می‌کند، تقلید در یک شناسه تقلید در هر شناسه آن نام را می‌دهد.

اثبات. گزاره 7.11 جابه‌جایی‌ای فراهم می‌کند که $\vec{q}$ را به $\vec{q}'$ می‌برد، هم‌وزنی آن را به جابه‌جایی‌ای بدل می‌کند که آن زوج سیستم را به زوج نشسته در $\vec{q}'$ می‌برد، و گزاره 7.10 آن حکم را منتقل می‌کند – و نشست‌یکنوایی زوج جابه‌جاشده از نشست‌یکنوایی زوج اصلی می‌آید، چنان که اثبات آن گزاره ثبتش می‌کند. ادعای آخر همان بند پایانی گزاره 7.11 است: چندگانه‌ای با یک شناسه، آن سه شرط را در برابر هر شناسه دیگری از نام خودش برآورده می‌کند.

اندیس‌گذاری با کل چندگانه همان است که این شرط را برآورده‌شدنی می‌کند، و این نکته ارزش ثبت دارد، چون اندیس‌گذاری تنها با شناسه فرایند اصلی – که نخستین چیزی است که آدم می‌نویسد – چنین نمی‌کند. جابه‌جایی می‌تواند q داده‌شده را ثابت کند و با این حال عضوی خصوصی را که در شناسه دیگری نشسته حرکت دهد، پس هم‌وزنی تنها در q ، هر عضو $\pi(q)$ را وادار می‌کرد شناسه‌ای داشته باشد که کل پایاساز q ثابتش می‌کند؛ و تنها چنین شناسه‌هایی خود q و آن سه شناسه رزروند، چون هر شناسه دیگری با جابه‌جایی‌ای که q را ثابت می‌کند حرکت‌پذیر است. پس هم‌وزنی تنها در q تنها سیستم‌هایی را می‌پذیرد که اصلاً هیچ عضو خصوصی ندارند، و این حالتی نیست که کسی بخواهد.

و اگر به‌جایش برای زوجی از سیستم‌ها نوشته شود و بپرسیم چه چیز دیگری باید حرکت کند، آن حکم دو شقه می‌شود. نمونه تازه‌ای از یک نام را می‌شود به‌تنهایی برداشت؛ نشست تازه را نمی‌شود، و دلیلش شرط (ii) تعریف 7.1 است: جابه‌جایی

نشست‌ها را به صورتِ بلوک جایگشت می‌دهد، پس هیچ چیز نمی‌تواند یک عضو را از نشست s بیرون ببرد و عضو دیگری را در آن جا بگذارد.

نتیجه 7.13 (جابه‌جا کردن یک نمونه، جابه‌جا کردن یک نشست). بگذارید φ و π سیستم‌های نشست یکنوایی باشند که هر یک عضوی در یک و همان شناسه فرایند q_0 حمل می‌کند، و q شناسه فرایندی با $q_0.F = q.F$ باشد که هیچ یک از دو سیستم اشغالش نکرده. فرض کنید π کارکرد φ را در ε و در برابر $(A(a), S(s), Z_{\pi, \varphi}(z))$ تقلید-UC کند. در هر حالت زیر، نتیجه این است که زوج جابه‌جا شده در همان ε و در برابر $(S(s), A(a))$ و بزرگ‌ترین رده مجاز آن زوج در z تقلید-UC می‌کند.

(i) اگر $q.s = q_0.s$ ، آن‌گاه ترانهشت $(q_0 \ q) := \theta$ از شماره‌های نمونه در نام $q_0.F$ در همان نشست یک جابه‌جایی است، و شناسه فرایند هر عضو دیگری از هر یک از دو سیستم را ثابت می‌کند. پس $\theta\varphi$ و $\theta\pi$ همان φ و π اند با نمونه نشسته در q_0 که به q برده شده و هیچ چیز دیگری آشفته نشده – جز آنکه عضوی که نام نمونه جابه‌جا شده را در میدانی می‌برد، خواه با سنجاق‌کردنش در میان فراخوانده‌هایش به شیوه نکته 6.5 و خواه جز آن، آن ذکر را همراه خود می‌برد.

(ii) اگر $q.s \neq q_0.s$ ، هیچ جابه‌جایی‌ای q_0 را به q نمی‌برد در حالی که شناسه فرایندی از نام استاندارد را در نشست $q_0.s$ ثابت کند – پس هیچ عضوی از هیچ سیستمی که آنجا بیفتد ثابت نمی‌ماند، و آن بلوک یکجا حرکت می‌کند. جابه‌جایی‌ای که q_0 را به q ببرد هست؛ و اگر علاوه بر آن هیچ یک از دو سیستم عضوی در نشست $q.s$ نداشته باشد، می‌شود یکی را چنان برگزید که هر عضو φ و π را که در نشست $q_0.s$ می‌افتد به نشست $q.s$ ببرد و شناسه فرایند هر عضوی که جای دیگری می‌افتد را ثابت کند. بی آن فرض افزوده، اعضایی که از پیش در نشست $q.s$ هستند هم حرکت می‌کنند – بنا بر همان استدلال که در q به کار می‌رود، هیچ جابه‌جایی‌ای که q_0 را به q می‌برد نمی‌تواند یکی از آن‌ها را ثابت کند – پس آن‌گاه نتیجه در باب زوجی است که دو بلوکش حرکت کرده و نه یکی.

اثبات. هر دو بخش به محض نمایاندن یک جابه‌جایی همان گزاره 7.10 اند، پس کار، نمایاندن یکی است. برای (i)، ترانهشت دو شناسه از یک نشست مشترک این کار را می‌کند، و سنجش این است که هیچ چیز اشغال شده دیگری را حرکت نمی‌دهد. برای (ii)، نیمه منفی نخست می‌آید – جابه‌جایی‌ای که q_0 را به شناسه‌ای از نشست دیگری می‌برد، بنا بر شرط (ii) تعریف 7.1، هر شناسه استاندارد آن نشست را حرکت می‌دهد – و نیمه مثبت آن‌گاه جابه‌جایی‌ای می‌سازد که نشست $q_0.s$ را حرکت می‌دهد و هر نشستِ سومی را ثابت می‌کند. جمله آخر، نیمه منفی را به θ^{-1} اعمال می‌کند، که بنا بر (vi) گزاره 7.5 جابه‌جایی است. هر دو بخش به محض نمایاندن آن جابه‌جایی همان گزاره 7.10 اند، و بخش (iii) آن گزاره رده‌ها را فراهم می‌کند. برای (i)، آن ترانهشت $\hat{\theta} = id$ دارد، پس شرط‌های (i) و (ii) تعریف 7.1 برقرارند، و (iii) برقرار است چون $q_0.F$

نام عضوی از یک سیستم است و پس هیچ‌یک از A ، Z ، C نیست. این جابه‌جایی تنها آن دو شناسه q_0 و q را حرکت می‌دهد، و q را هیچ‌یک از دو سیستم اشغال نکرده، پس هر شناسه اشغال‌شده دیگری ثابت است؛ و آنچه ثابت نیست میدانی است که نام q_0 را می‌برد، که تعریف 7.3 برچسبش را عوض می‌کند.

برای، (ii) فرض کنید $\theta(q_0) = q$ و بگذارید p هر شناسه فرایندی از نام استاندارد با $p.s = q_0.s$ باشد؛ بنا بر شرط (ii) تعریف 7.1، $\theta(p).s = \theta(q_0).s = q.s \neq p.s$. پس $\theta(p) \neq p$. برای وجود، θ را ترانهشت آن دو نشست بگیرد و جایگشت‌های نمونه‌ای که $q_0.i$ را به $q.i$ در نام $q_0.F$ در نشست $q_0.s$ می‌برند، و همانی در جای دیگر. آن θ نشست $q_0.s$ را به نشست $q.s$ می‌برد و هر شناسه نشستِ سومی را ثابت می‌کند؛ و اگر هیچ عضوی در نشست $q.s$ نیفتد، اعضایی که حرکت می‌دهد درست همان‌های نشست $q_0.s$ اند. برای جمله آخر، بگذارید θ q_0 را به q ببرد و m عضوی باشد که در نشست $q.s$ می‌افتد. آن‌گاه θ^{-1} بنا بر (vi) گزاره 7.5 جابه‌جایی است و q را به q_0 می‌برد با $q.s \neq q_0.s$ ، پس ادعای منفی که به آن اعمال شود هر شناسه استاندارد نشست $q.s$ را حرکت می‌دهد، m هم در میانشان؛ و $\theta(m) = m$ می‌داد $\theta^{-1}(m) = m$. پس θ m را هم حرکت می‌دهد. $\square$

نشست و نمونه تنها مؤلفه‌هایی نیستند که یک جایگشت می‌تواند حرکت دهد، و سومی همان است که پرسشی را حل می‌کند که فصل 5 پیش می‌آورد: بازی $\mathcal{F}$ را زیر ادعایی تمرین می‌دهد که نام خود بازی را حمل می‌کند (سطر 5)، پس $\mathcal{F}.N$ باید نام بازی را ببرد، در حالی که آن $\mathcal{F}$ ی که در درون پروتکلی عرضه می‌شود به‌جایش آن پروتکل را می‌پذیرد. آن دو در یک میدان تفاوت دارند، و آن همان میدانی است که **Guard** می‌خواندش.

تعریف 7.14 (تغییرنام). یک تغییرنام جایگشت v از شناسه‌های فرایند به این صورت است

$$v(F, s, i) = (\bar{v}(F), s, i)$$

برای جایگشت $\bar{v}$ از نام‌ها که A ، Z و C را ثابت می‌کند. این تغییرنام روی هویت‌ها، میدان‌ها، کد و مجموعه‌های اعلام‌شده همان‌گونه کنش می‌کند که تعریف 7.3 تعیین می‌کند، و علاوه بر آن هر نامی را که رو در کد رخ می‌دهد با تصویرش عوض می‌کند.

آن بند افزوده لازم است و پیش‌تر نبود. تعریف 7.3 شناسه‌های فرایند را جانشین می‌کند، که برای جابه‌جایی بس است، چون نام‌ها همان‌اند که جابه‌جایی نگه می‌دارد؛ اما تغییرنام حرکتشان می‌دهد، پس آزمونی در برابر نام خالی $id'.F \neq A$ در **Mediate**، یا آن $id'.F = Z$ که بدل بخش 4.4 رویش توزیع می‌کند – هم باید سفر کند. و برای ماشین‌های این کتاب هیچ کجا سفر نمی‌کند: تنها نام‌هایی که رو ذکر می‌کنند همان سه نامی است که $\bar{v}$ ثابتشان می‌کند، پس تغییرنام اصلاً هیچ سطری از کد را تغییر نمی‌دهد، تنها میدان‌ها را.

گزاره 7.15 (تغییرنام). بگذارید ν تغییرنام باشد. آن‌گاه

- (i) $\nu(\mathbf{Z}) = \mathbf{Z}$ و $\nu(\mathbf{A}) = \mathbf{A}$, $\nu(\mathbf{Std}) = \mathbf{Std}$; $\text{IDs}(\nu\mathcal{F}) = \nu(\text{IDs}(\mathcal{F}))$ (ii) به‌ازای هر سیستم π , هر اشغال‌گر $\mathcal{X}$ از جایگاه حریف و هر محیط $\mathcal{Z}$ که در هیچ هویتی از $\text{IDs}(\pi)$ میهمان نمی‌کند – بی‌آنکه هیچ نشست‌یکنوایی‌ای از هیچ‌یک خواسته شود –

$$\text{Exec}(\nu\pi, \nu\mathcal{Z}, \nu\mathcal{X}, \mathcal{C}) \quad \text{and} \quad \text{Exec}(\pi, \mathcal{Z}, \mathcal{X}, \mathcal{C})$$

همسان‌توزیعند؛ و

- (iii) $\mathbb{Z}_{\nu\pi, \nu\varphi} = \nu(\mathbb{Z}_{\pi, \varphi})$ اگر π کارکرد φ را در ε و در برابر $(\mathbf{A}, \mathbf{S}, \mathbf{Z})$ تقلید-UC کند آن‌گاه $\nu\pi$ کارکرد $\nu\varphi$ را در ε و در برابر $(\nu\mathbf{A}, \nu\mathbf{S}, \nu\mathbf{Z})$ تقلید-UC می‌کند؛ و مزیت‌های عینی تعریف 4.31 در هر بودجه‌ای جوړند.

اثبات. هر سه بخش، همان اثبات‌های متناظر جابه‌جایی‌اند که دوباره دوانده شده‌اند، و این پیش‌نگاه همان چیزی است که تغییر می‌کند. نام‌ها اکنون جایگشت می‌یابند و نه اینکه بر خودشان برده شوند، و همین نکتهٔ اصلی آن تعریف است؛ رده‌های رزرو و $\mathbf{Std}$ ثابت می‌مانند، و با آن‌ها global و نشست‌یکنوایی، در حالی که بستار نامی برده می‌شود و نه ثابت می‌ماند. در (ii) آن یک بندِ سختِ لم 7.9 – عطفِ نشست – بدیهی می‌شود، چون تغییرنام هیچ نشستی را حرکت نمی‌دهد، و همین است که چرا اصلاً هیچ فرضِ نشست‌یکنوایی‌ای اینجا پیدا نمی‌شود. و در (iii) آنچه از دست می‌رود خوانش است و نه نتیجه: ردهٔ محیطِ نتیجه، متناظرِ ردهٔ فرض است و نه خودِ آن. (i) همان اثباتِ گزارهٔ 7.5 است با دو تغییرِ حساب‌داری. رده‌های نامی اکنون جایگشت می‌یابند و نه اینکه بر خودشان برده شوند، پس $\mathbf{N}$ ی که با نام اعلام شده به همان $\mathbf{N}$ ی بدل می‌شود که با تصویرها اعلام شده – که کل نکتهٔ آن تعریف است – در حالی که $\mathbf{Std}$ ، که اجتماعِ هر ردهٔ استاندارد است، و $\mathbf{A}$ و $\mathbf{Z}$ ، که رده‌های رزرو یگانه‌اند، ثابت می‌مانند، و با آن‌ها global و تعریف 7.2 و شناسه‌های رزرو (iii) تعریف 7.1. و بستار نامی برده می‌شود و نه ثابت، چون $\langle \rangle$ با نام‌هایی مشخص می‌شود که سیستم به کار می‌برد. و برای Wf ، عطفِ نشست دست‌نخورده است، چون ν هیچ نشستی را حرکت نمی‌دهد، و جوړبودن نام درست وقتی در یک سو برقرار است که در سوی دیگر.

(ii) همان اثباتِ لم 7.9 است موبه‌مو، جز آن یک بندِ سختش، که بدیهی می‌شود: عطفِ اول سطر 3 عیناً دست‌نخورده است، چون ν هر نشست را رها می‌کند، و عطفِ دومش با (i) نگه داشته می‌شود. هیچ چیز لازم نیست کنار گذاشته شود، و همین است که چرا هیچ نشست‌یکنوایی‌ای اینجا پیدا نمی‌شود. هر سنجشِ دیگری نامی را برای تساوی می‌خواند یا عضویتی را در مجموعه‌ای که با کد سفر می‌کند، و تعریف 7.4

به همراه بند افزوده تعریف 7.14 آن جانشانی را وادار می‌کند رفتار برچسب‌عوض‌شده را مثل پیش حساب کند.

(iii) همان اثبات گزاره 7.10 است، که از بستارها تنها این را به کار برد که $\cup (\pi) \cup \langle \nu\varphi \rangle = \langle \nu\pi \rangle \cup \langle \nu\varphi \rangle$ و هرگز این را که سر جایشان می‌مانند. آنچه از دست می‌رود همان خوانشی است که پس از گزاره 7.5 ثبت شد: رده محیط نتیجه، متناظر رده فرض است و نه رده‌ای که یکسان مقید شده باشد، چون par اکنون باید نام تصویرها را ببرد. $\square$

نتیجه 7.16 (نام فراخواننده بی‌اهمیت است). بگذارید ν تغییرنام باشد و $\gamma = \sigma \cup \{g_m\}$ سیستم بازی برای $\mathcal{F}$ (تعریف 5.1). آن‌گاه $\nu\gamma = \{\nu g_m\} \cup \nu\sigma$ سیستم بازی است، νg_m بازی‌ای برای $\nu\mathcal{F}$ است، و به‌ازای هر نهایی‌سازی g_m از Fin

$$\text{Adv}_{\nu g_m, \nu\sigma, \nu Z, \nu X}^{\text{Fin}} = \text{Adv}_{g_m, \sigma, Z, X}^{\text{Fin}}$$

به‌ازای هر $Z \in Z_{\gamma, \gamma}$ و هر اشغال‌گر X از جایگاه حریف. و چون $\nu\mathcal{D} = \mathcal{D}$ ، صورت کوتاه‌شده چنین می‌شود

$$\text{Adv}_{\nu g_m, \nu\sigma, \nu Z}^{\text{Fin}} = \text{Adv}_{g_m, \sigma, Z}^{\text{Fin}}$$

پس σ ویژگی Fin را در ε و در برابر رده‌ای از محیط‌ها درست وقتی دارد که $\nu\sigma$ آن را در ε و در برابر تصویر آن رده داشته باشد.

اثبات. اینکه $\nu\gamma$ سیستمی با $\text{wf}(\nu\gamma)$ همان (i) است. و برای بازی‌بودن νg_m چهار میدان تعریف 5.1 را بسنجید: شناسه فرایندش $(\bar{\nu}(g_m.F), 0, 0)$ است، چون ν نه نشست را حرکت می‌دهد و نه نمونه را؛ P آن دست‌نخورده است و $\nu\mathcal{F}.P = \mathcal{F}.P$ پس همان $\nu\mathcal{F}.P \cup \{Z\}$ لازم است؛ N آن بنا بر (i) همان $\nu(Z) = Z$ است؛ و درایه‌اش $(\nu\mathcal{F}, \text{serve}_{g_m})$ است، و serve_{g_m} هم‌وز است، چون P و عضویتی از N را می‌خواند. سطر 8 باز هر نهایی‌سازی را در ریشه خدمت می‌کند، چون شناسه‌های طرف هیچ بخشی از شناسه فرایند نیستند، و تغییرنام نام هیچ واسطی را حرکت نمی‌دهد، پس Fin در هر دو سو نام همان یکی است. و برای مزیت، Win_{Fin} پیشامد این است که واسط FullFin در $(g_m.\text{pid}, Z)$ مقدار 1 برمی‌گرداند، و ν آن هویت را به $(\nu g_m.\text{pid}, Z)$ می‌برد؛ و بنا بر (ii) آن دو اجرا زیر آن تناظر همسان‌توزیعند، پس آن پیشامد در هر دو سو همان احتمال را دارد و تعریف 5.8 همان عدد را می‌دهد. و اینکه νZ برای $\nu\gamma$ مجاز است همان (iii) است. و برای صورت کوتاه‌شده، $\nu\mathcal{D} = \mathcal{D}$ ؛ میدان‌های بدل $\text{pid} := A$ و مجموعه‌هایی از شناسه‌هاست که (i) ثابتشان می‌کند، و هسته‌اش بر $\text{id}' . F = Z$ توزیع می‌کند و همان فراخوانی را می‌گذارد که با پیامش نامش می‌برد، پس هیچ نامی که ذکر می‌کند حرکت نمی‌کند. $\square$

اگر در ترانهشت $g_m.F$ با نامی که γ به کارش نمی‌برد خوانده شود، و با $g_m.F$ که تنها g_m حملش می‌کند، این همان را می‌گوید که آدم از تعریف 5.1 می‌خواهد.

بازی را می‌شود هر نامی داد و $\mathcal{F}.N$ را برای جورشیدن تغییرنام داد؛ هیچ میدان دیگری حرکت نمی‌کند، و بنا بر تعریف 7.14 هیچ سطری از کد هم. پس ویژگی به این بند نیست که بازی‌اش را چه می‌نامند، و آن $\mathcal{F}$ ی که ویژگی‌اش اثبات می‌شود می‌تواند هر نام فراخواننده‌ای را بپذیرد که استقرار به کار خواهد برد.

آنچه هیچ تغییرنامی نمی‌دهد، برنامه فراخواننده متفاوت است. تغییرنام بازی را به بازی می‌برد و پروتکل را به پروتکل، و هرگز بازی را به پروتکلی که $\mathcal{F}$ را به‌جد به کار خواهد برد – و پروتکلی که پاسخ‌های $\mathcal{F}$ را به حریف می‌سپارد هیچ ویژگی‌ای از $\mathcal{F}$ را برآورده نمی‌کند، پس هیچ چیز نمی‌توانست. گزاره 7.7 آن شکاف را از سوی دیگر تنگ می‌کند، چون N هیچ کجا جز در درون $\text{Guard}_{\mathcal{F}}$ خوانده نمی‌شود: ویژگی حکمی است در باب کد $\mathcal{F}$ و کنترل دسترسی گرد آن، و در باب هیچ چیز دیگری که آن میدان می‌توانست حمل کند. گذشتن از باقی همان است که تقلید برایش هست، و گزاره 5.10 همان گذشتن است. جابه‌جایی‌ها و تغییرنام‌ها ترکیب می‌شوند، چون هر یک یکرختی اجراهاست، پس سیستم را می‌شود یک‌جا در نام و نشست و نمونه جابه‌جا کرد.

نکته 7.17 (نشست 0 ممتاز است). نشست یکنوایی صنعت اثبات‌ها نیست، و کارکردی که کنار می‌گذارد کارکرد واقعی‌ای است. سطر 3 نشست فراخواننده را می‌سجد، و A و Z در نشست 0 می‌نشینند، پس کارکردی که به معنای فصل 1 محلی است و با این حال A را می‌پذیرد، درست وقتی از جایگاه حریف خطاب‌پذیر است که نشست خودش 0 باشد: با $\mathcal{F}.s \neq 0$ عطف $\text{id}.s = \text{id}.s$ شکست می‌خورد و $\text{global}(\mathcal{F})$ نجاتش نمی‌دهد. آن تفاوت مشاهده‌شدنی است – همان راهی است که $\mathcal{D}$ دستوری از بخش 4.4 را در طرف فاسد انجام می‌دهد – پس برای چنین زوجی هیچ جابه‌جایی‌ای که نشست 0 را حرکت دهد یکرختی نیست، و تعریف 7.2 ضروری است و نه راحت. و کم‌ترین چیزی هم هست که می‌شود کنار گذاشت: برای هر فراخواننده دیگری، ماشین‌های فراهم‌شده با global بخشیده می‌شوند یا در سطر 2 رد می‌شوند، پس هیچ شرط کوچک‌تری کار نمی‌کرد و هیچ شرط بزرگ‌تری لازم نیست.

شکلی که کنار می‌گذارد غریب نیست، و یک جا که خود این کتاب به‌سویش دست دراز می‌کند باید رو گفته شود. نکته 6.5، به‌عنوان راهی برای بستن راه میهمانی به عضوی واسط، پیشنهاد می‌کند که چنین عضوی تنها $Z \cup A$ را بپذیرد – محیط و حریف و هیچ‌کس دیگر. عضوی با آن شکل جهانی نیست، چون Std بیرون N او می‌افتد، پس نشست یکنوا نیست. و این هیچ عیب تعریف 7.2 نیست بلکه همان صنعت است که در جایی پیدا می‌شود که بیش از همه هزینه دارد: سطر 3 می‌گذارد محیط به چنین عضوی در نشست 0 برسد و در هیچ نشست دیگری، پس پروتکلی که اعضای واسطش درست $Z \cup A$ را می‌پذیرند تنها در نشست 0 راندنی است، هر چه جابه‌جایی هم در بابش بگوید. آن دو راهی که محیط مجاز به یک پروتکل دارد به همین نسبت تقسیم می‌شوند. می‌تواند نامی تازه ادعا کند، که سطر 2 تنها در عضوی می‌پذیردش که کل Std را می‌پذیرد – یعنی جهانی، پس مشترک بین نشست‌ها، پس نه یک اجرا از یک پروتکل. یا می‌تواند هویت Z خودش را ادعا کند،

که سطر 2 در عضوی می‌پذیردش که Z را می‌پذیرد - و آن‌گاه سطر 3 آن عضو را به نشست 0 سنجاق می‌کند. هیچ‌یک از دو راه به واسطه پروتکلی که محلی نشست خودش است در نشستی به انتخاب خودش نمی‌رسد. و اینکه محیط اصلاً می‌تواند هویت Z خودش را ادعا کند ارزش یادآوری دارد، چون همان است که راه دوم را در دسترس می‌کند: نام Z را هیچ سیستمی به کار نمی‌برد، پس هویت‌های Z بیرون $\langle \varphi \rangle \cup \langle \pi \rangle$ می‌افتند و تعریف 4.10 ادعای درونی‌شان می‌گذارد. عضوی که N او تنها رده نامی والدش است با هیچ‌یک از دو راه در دسترس نیست، و آنجا آن صنعت تنها از راه جایگاه حریف پیدا می‌شود. خواندن سطر 3 به صورت

$$(id'.s = id.s \vee global(\mathcal{F}) \vee id'.F \in \{A, Z\})$$

این شرط را بی‌نیاز می‌کرد. آن‌گاه نشست 0 هیچ نمی‌بخشید، کارکرد محلی‌ای که حریف را می‌پذیرد در هر نشستی یکسان از جایگاه در دسترس می‌بود، و تعریف 7.2 را می‌شد از هر حکم بالا انداخت: نتایج در باب همه سیستم‌ها برقرار می‌بودند و نه تنها نشست‌یکنواها. و هیچ چیز دیگر این فصل تغییر نمی‌کرد - شرط (iii) می‌ماند، چون شناسه‌های رزرو هر قاعده نشست هم که باشد رو در کد نامشان برده می‌شود، و شرط‌های چندگانه گزاره 7.11 از پیش از نشست 0 آزادند. آن فراخ‌کردن کوچک است و در روح باقی: آن دو ماشینی که نامشان می‌برد، هر جا $global$ در بابشان آزموده شود با عطف دوم $global$ جهانی‌اند، دروازه فساد سطر 3 باز A را به طرف‌های فاسد محدود می‌کند، و Z با N و با par خودش بیرون می‌ماند. و آنچه می‌خريد بیش از جابه‌جایی است: همان است که می‌گذاشت پروتکلی هم‌زمان محلی نشست خودش باشد و محیط براندش، که همان جفت‌شدنی است که آن دو راه بالا رد می‌کنند و پیشنهاد نکته 6.5 بی‌صدا مفروض می‌گیرد. ما همان قاعده تنگ‌تر را نگه می‌داریم، که قاعده کانتی [7] است، و هم آن گزینه و هم بهایش را همین‌جا ثبت می‌کنیم.

نکته 7.18 (جابه‌جایی قضیه چند نمونه‌ای نیست). گزاره 7.10 یک رونوشت را جابه‌جا می‌کند و هیچ حساب نمی‌کند. از آن در نمی‌آید، و درست هم نیست، که n رونوشت در n اندیس ε هزینه کنند و نه $n\varepsilon$: اجراهای $\pi(\vec{q}_1), \dots, \pi(\vec{q}_n)$ و $\varphi(\vec{q}_1), \dots, \varphi(\vec{q}_n)$ برچسب عوض شده یکدیگر نیستند - هیچ جابه‌جایی‌ای سیستمی را به n رونوشت از خودش نمی‌برد - و تنها راه میانشان همان زنجیره دورگه قضیه 4.24 است، که جمع می‌کند. آنچه آن دو نتیجه با هم می‌دهند همان حکمی است که آدم از یک اثبات تنها می‌خواهد: اثبات کنید $\pi(\vec{q})$ کارکرد $\varphi(\vec{q})$ را در یک $\vec{q}$ تقلید می‌کند؛ نتیجه 7.12 آن را در هر $\vec{q}$ دیگری در ε می‌دهد؛ و قضیه 4.24، که فرض‌های مجزایندش به این می‌رسد که $\vec{q}_k$ ها دوبه‌دو مجزا و برکنار از پوسته پیرامون باشند، هر n تا را یک‌جا در $n\varepsilon$ جانشین می‌کند، در

بودجه‌هایی که فصل 8 حساب می‌کند. و جابه‌جایی در ساخت‌های بخش 4.1 و فصل 6 هم تابعی است، چون هر یک از آن‌ها تنها نام‌ها و شناسه‌ها و مجموعه‌های هویت را می‌خواند: $\theta(\rho[\pi/\varphi]) = (\theta\rho)[\theta\pi/\theta\varphi]$ چون کنش یک‌به‌یک با تفاضل مجموعه‌ای جابه‌جا می‌شود، $\theta\bar{\pi} = \overline{\theta\pi}$ که سمت راستش، چنان که تعریف 6.2 می‌خواهد، به‌عنوان سیستم مسدودشده زوج $(\theta\pi, \theta\varphi)$ خوانده می‌شود – چون مسدودگر آن دو میدانی را رونویسی می‌کند که θ برچسبشان را عوض می‌کند و $\theta(I_\varphi \setminus I_\pi) = I_{\theta\varphi} \setminus I_{\theta\pi}$ و $\theta(\mathbb{Z}^{\rho \setminus \varphi}) = (\theta\mathbb{Z})^{\theta\rho \setminus \theta\varphi}$ چون تعریف 7.3 برچسب مجموعه بیرونی اعلام‌شده یک دسته را عوض می‌کند، که نیمه رزرو آن، یعنی $\{\text{id} : \text{id}.F \in \{A, C\}\}$ بنا بر (ii) گزاره 7.5 ثابت است. پس اثبات جابه‌جاشده ترکیب، همان ترکیب اثبات جابه‌جاشده است.

هزینه‌های عینی

دو رشته هزینه عینی بیرون خط اصلی می‌نشینند و همین‌جا جمع می‌شوند: هزینه یک سیستم در خودش، که از اعضایش خوانده می‌شود، و هزینه‌ای که نتایج ترکیب بخش 4.5 حساب می‌کنند، که حسابی است روی بودجه‌های تعریف 4.31. سیستمی که عضویه عضو بودجه‌دار شده، به‌عنوان یک کل هم هزینه‌ای دارد، و آن همان هزینه یک کارکرد است که یک سطح بالاتر خوانده شده.

تعریف 8.1 (هزینه یک سیستم). یک فراخوانی از سیستم π فراخوانی است که از بیرون π به یکی از اعضایش می‌رسد؛ و π آن را با آبخاری از فعال‌سازی‌های درونی خدمت می‌کند پیش از آنکه مهار به بیرون بازگردد. هزینه π زوج (T_π, R_π) است: زمان هر فراخوانی T_π ، یعنی بزرگ‌ترین شمار کل گام‌ها در یکی از آن آبخارها – که روی فعال‌سازی‌هایی که برمی‌انگیزد جمع شده، با یک ماشین که در هر لحظه می‌دود – و شمار فراخوانی R_π ، یعنی شمار فراخوانی‌هایی که π پاسخشان می‌دهد. پس π دست‌بالا در $T_\pi R_\pi$ گام می‌دود.

گزاره 8.2 (هزینه سیستم از هزینه اعضا). بگذارید $\pi = \{F_1, \dots, F_n\}$ باشد که هر F_i هزینه (t_i, r_i) دارد، و k_i کرانی بر فعال‌سازی‌های F_i در درون یک فراخوانی از π باشد. آن‌گاه

$$T_\pi \leq \sum_i k_i t_i, \quad R_\pi \leq \sum_i r_i,$$

و π دست‌بالا در $\sum_i t_i r_i$ گام می‌دود. اگر هر فراخوانی هر عضو را دست‌بالا یک بار فعال کند – یعنی π زیرروال محترم باشد، $k_i \leq 1 - \sum_i t_i$ ؛ و اگر علاوه بر آن هر فراخوانی همه اعضا را براند، $R_\pi \leq \min_i r_i$.

اثبات. فراخوانی F_i را دست‌بالا k_i بار فعال می‌کند، که هر بار دست‌بالا t_i گام است، پس دست‌بالا $\sum_i k_i t_i$ خرج می‌کند. فعال‌سازی‌های F_i در کل اجرا دست‌بالا r_i تائید، و هر فراخوانی دست‌کم یک فعال‌سازی است، پس فراخوانی‌ها دست‌بالا $\sum_i r_i$ تائید؛ و کل گام‌ها $\sum_i r_i t_i \leq \sum_i (F_i)$ است. و وقتی هر فراخوانی همه

اعضا را می‌راند، R_π فراخوانی هر $\mathcal{F}_i$ را دقیقاً R_π بار فعال می‌کند، پس $R_\pi \leq r_i$ به ازای هر i . $\square$

نتیجه 8.3 (هزینه یک اجتماع). بگذارید π_1 و π_2 سیستم‌هایی با شناسه‌های فرایند مجزا و هزینه‌های (T_1, R_1) و (T_2, R_2) باشند. اگر هیچ عضوی از یکی، عضوی از دیگری را فرا نخواند، آنگاه

$$\begin{aligned}(T_{\pi_1 \cup \pi_2}, R_{\pi_1 \cup \pi_2}) &= (T_1, R_1) \oplus (T_2, R_2) \\ &= (\max(T_1, T_2), R_1 + R_2).\end{aligned}$$

و اگر به جایش هر فراخوانی از π_1 دست‌بالا k فراخوان به درون π_2 بگذارد و گرافی فراخوان‌های متقابل بی‌دور باشد، آنگاه $T_{\pi_1 \cup \pi_2} \leq T_1 + k T_2$ از ورودی‌ای در π_1 ، و متقارناً، با $R_{\pi_1 \cup \pi_2} \leq R_1 + R_2$ ؛ و با فراخوان‌های متقابل بی‌کران، هزینه‌های خلاصه دیگر هزینه اجتماع را تعیین نمی‌کنند.

اثبات. بی فراخوان متقابل، فراخوانی از بیرون $\pi_1 \cup \pi_2$ به π_1 یا π_2 وارد می‌شود و آبخارش همان جا می‌ماند، که دست‌بالا T_1 یا T_2 هزینه دارد و به شمار فراخوانی همان زیرسیستم تعلق می‌گیرد؛ پس زمان هر فراخوانی $\max(T_1, T_2)$ است و شمارها جمع می‌شوند. فراخوان متقابل از π_1 به درون π_2 درونی آن اجتماع است و آبخارش را با یک فراخوانی از π_2 توسیع می‌دهد، که دست‌بالا T_2 گام است؛ و دست‌بالا k تای آن‌ها، بی‌دور، $T_1 + k T_2$ می‌دهد، در حالی که فراخوانی که یک بار بیرونی π_2 بود اما از π_1 گذاشته شده دیگر شمرده نمی‌شود، پس فراخوانی‌ها باز دست‌بالا $R_1 + R_2$ تایند. $\square$

شرط زیرروال محترم بی‌کار نیست: بی آن، یک فراخوان را می‌شود بی‌پایان دور سیستم چرخاند، و T_π به‌سوی کل $\sum_i t_i r_i$ واگرا می‌شود – همان نظمی که فراخواننده‌های عضو خصوصی را در نکته 6.5 سنجاق نگه می‌دارد. این هزینه سیستم دوگان $\oplus$ تعریف 4.30 است: میهمان کردن یک سیستم به‌عنوان دسته، با یک عضو در هر فعال‌سازی، اعضایش را با $\oplus$ به $(\max_i t_i, \sum_i r_i)$ ترکیب می‌کند، همان هزینه‌ای که $c_{p \setminus \emptyset}$ در هزینه‌های ترکیب زیر می‌پردازد؛ و فراخواندنش به‌عنوان زیرسیستم، زمان‌ها را روی یک آبخارش جمع می‌کند و فراخوان‌های بیرونی را می‌شمارد، که در حالت همگام $(\sum_i t_i, \min_i r_i)$ است. هر دو کرانی بر همان کل $\sum_i t_i r_i$ می‌گذارند، و دومی تنگ‌تر، هرگاه هر فراخوانی یک گذر کامل باشد. و $\oplus$ در سطح خود سیستم هم تکرار می‌شود: بنا بر نتیجه 8.3 زیرسیستم‌هایی که با هم برخورد نمی‌کنند با آن ترکیب می‌شوند، یک عملگر در هر دو مقیاس.

وقتی اعضای π یکدیگر را فرا می‌خوانند، زمان هر فراخوانی بر شکل آن فراخوانی می‌گردد، و یک حالت پاکیزه است.

گزاره 8.4 (سیستم‌های DAG). فرض کنید گراف فراخوان $\pi = \{\mathcal{F}_1, \dots, \mathcal{F}_n\}$ بی‌دور باشد و هر فراخوانی هر عضو درونی $\mathcal{F}_i$ را دست‌بالا $\max_{j: \mathcal{F}_j \rightarrow \mathcal{F}_i} t_j$ بار فعال کند – یعنی بزرگ‌ترین زمان هر فراخوانی در میان اعضای که فراپیش می‌خوانند. آن‌گاه

$$T_\pi \leq \sum_i \left(\max_{j: \mathcal{F}_j \rightarrow \mathcal{F}_i} t_j \right) t_i \leq \left(\max_i t_i \right) \sum_i t_i,$$

یعنی زمان هر فراخوانی‌ای که مستقل از عمق گراف است.

اثبات. فراخوانی‌ای که به یک سرچشمه وارد می‌شود هر $\mathcal{F}_i$ را a_i بار فعال می‌کند و $\sum_i a_i t_i$ گام خرج می‌کند. سرچشمه یک بار می‌دود و هر $\mathcal{F}_i$ درونی بنا بر فرض دست‌بالا $\max_{j: \mathcal{F}_j \rightarrow \mathcal{F}_i} t_j$ بار، پس $a_i \leq \max_i t_i$ در سرتاسر، و از این‌رو $T_\pi \leq \sum_i (\max_{j: \mathcal{F}_j \rightarrow \mathcal{F}_i} t_j) t_i \leq (\max_i t_i) \sum_i t_i$. $\square$

آن فرض جای خود را به‌دست می‌آورد، چون بی‌دوری به‌تنهایی کرانی بر T_π نمی‌گذارد. اگر n_{ji} را برای فراخوان‌هایی بنویسیم که $\mathcal{F}_j$ در یک فعال‌سازی $\mathcal{F}_i$ می‌گذارد، شمارِ فعال‌سازی‌های هر فراخوانی از $a_i = \sum_{j: \mathcal{F}_j \rightarrow \mathcal{F}_i} n_{ji}$ a_i در می‌آید، پس a_i مسیرهای وزن‌دار سرچشمه به $\mathcal{F}_i$ را می‌شمارد، و گراف همگرا آن‌ها را ضرب می‌کند: زنجیره‌ای از N الماس $s \rightarrow \{A_k, B_k\} \rightarrow C_k \rightarrow \{A_{k+1}, B_{k+1}\} \rightarrow \dots$ – برابر می‌کند، پس a در پایین 2^N است و زمان هر فراخوانی در عمق نمایی است. و سقف‌گذاشتن بر هر گره با ماکسیمم روی فراخواننده‌هایش و نه با جمعشان، درست همان است که جلوی انباشتن آن مسیرها را می‌گیرد – باز همان مضمون $\oplus$ -بر-جمع، و همتای DAG حالت زیرروال محترم $k_i \leq 1$ از گزاره 8.2، با یک فعال‌سازی که به $\max_{j: \mathcal{F}_j \rightarrow \mathcal{F}_i} t_j$ شل شده. سقف‌های فراخوانی در هر حال $a_i \leq r_i$ را کران می‌زنند؛ و نکته اینجاست که کرانی است تنها در زمان‌ها، مستقل از آن سقف‌ها و از عمق. همان نکته آخر کل حکم عمومی است: خود آن سقف‌ها همین حالا رده هزینه را بسته می‌کنند، بی هیچ شرطی بر ساختار فراخوان.

نتیجه 8.5 (ترکیب هزینه). سیستم $\pi = \{\mathcal{F}_1, \dots, \mathcal{F}_n\}$ که اعضایش هزینه‌های (t_i, r_i) دارند، دست‌بالا در $\sum_i t_i r_i$ گام و روی دست‌بالا $\sum_i r_i$ فراخوانی می‌دود، پس $T_\pi \leq \sum_i t_i r_i$ و $R_\pi \leq \sum_i r_i$ ، هر ساختار فراخوانی هم که داشته باشد. پس اگر هر t_i و r_i در پارامتر λ چندجمله‌ای باشد و n چندجمله‌ای کران باشد، (T_π, R_π) چندجمله‌ای است: یعنی سیستم‌های چندجمله‌ای کران زیر تشکیل بسته‌اند.

اثبات. کران‌های گام و فراخوانی همان گزاره 8.2 اند، و آبشار یک فراخوانی بخشی از کل اجراست، پس $T_\pi \leq \sum_i t_i r_i$ هم برقرار است. و جمع چندجمله‌ای‌های به‌شمار چندجمله‌ای، چندجمله‌ای است. $\square$

اینجا هیچ شرطی بر گراف فراخوان لازم نیست، به‌خلاف کران زمانی بی‌عمق گزاره 8.4، چون آن سقف‌ها حملش می‌کنند. آبخار همگرایی که عضوی را 2^N بار فعال می‌کرد تنها در صورتی می‌تواند چنین کند که آن عضو به 2^N فراخوان پاسخ دهد، پس $2^N - r_i \geq 0$ یعنی عضوی که از پیش بیرون رده چندجمله‌ای است؛ و در درون آن رده، آن سقف زیادی را رد می‌کند و آبخار را در $\sum_i r_i$ فعال‌سازی می‌برد. این همتای کارایی قضیه 4.20 است: آنجا که ترکیب امنیت را نگه می‌دارد $-\rho[\pi/\varphi]$ کارکرد ρ را تقلید می‌کند به محض آنکه π کارکرد φ را $-\rho[\pi/\varphi]$ ترکیب هزینه کارایی را نگه می‌دارد، پس پروتکلی که از کارکردهای چندجمله‌ای زمان سوار شده خودش چندجمله‌ای زمان است. و آن دو با هم همان چیزی هستند که دستگاه عینی برایش هست: پروتکل‌های امن-UC چندجمله‌ای زمان به پروتکل‌های امن-UC چندجمله‌ای زمان ترکیب می‌شوند، و همین است که می‌گذارد خوانش مجانبی زیر یک نتیجه باشد و نه نظریه‌ای از خودش. همان بسته‌بودن از عملی که قضیه ترکیب می‌دواند هم جان سالم می‌برد.

گزاره 8.6 (جانشانی چندجمله‌ای کرانی را نگه می‌دارد). بگذارید ρ و π سیستم‌هایی باشند که اعضایشان چندجمله‌ای و به‌شمار چندجمله‌ای‌اند، و جانشانی φ با π در ρ خوش‌ساخت باشد. آن‌گاه $\rho[\pi/\varphi]$ چندجمله‌ای کران است.

اثبات. اعضای $\rho[\pi/\varphi] = (\rho \setminus \varphi) \cup \pi$ (تعریف 4.1) همان اعضای ρ در بیرون φ هستند، که هر یک به‌عنوان عضو ρ چندجمله‌ای است، به همراه اعضای π ، که باز چندجمله‌ای‌اند؛ و شمارشان دست‌بالا $|\pi| + |\rho|$ است، که به‌شمار چندجمله‌ای است. $\square$ پس نتیجه 8.5 اعمال می‌شود.

پس سیستمی که پس از یک جانشانی می‌دود، هر جا مشخصه ρ و تحقق π کارا باشند کاراست — یعنی قضیه 4.20 که برای هزینه خوانده شده و نه برای امنیت، و روی همان عمل.

ترکیب، به‌زبان عدد. در قضیه 4.20 حریف‌ها و شبیه‌سازها بی‌تغییر منتقل می‌شوند، پس بودجه‌هایشان هم. محیط‌ها همان یک جایی هستند که چیزی حساب می‌شود، و آنچه جذب هزینه می‌کند هزینه میهمانی است. سیستم‌ها عضویه عضو بودجه‌دار شده‌اند، پس آن هزینه را می‌شود نگاه کرد: بنویسیم $\mathbf{c}_{\rho \setminus \varphi}$ برای ترکیب $\oplus$ بودجه‌های اعضای $\rho \setminus \varphi$ — بزرگ‌ترین زمان‌های هر فراخوانی‌شان، جمع شمار فراخوانی‌هایشان، همان هزینه دسته میهمان‌شده که دوگان هزینه زیرسیستم فراخوانده‌شده گزاره 8.2 است — که در هر اجرایی کرانشان می‌زند، هر که هم که براندشان. محیط جذب‌شده تعریف 4.15 $\mathbb{Z}$ و همان کارکردها را می‌دواند و هیچ‌چیز دیگر را، پس

$$\mathbb{Z} \in \mathbb{Z}_{\rho[\pi/\varphi], \rho}(\mathbf{z}) \implies \mathbb{Z}^{\rho \setminus \varphi} \in \mathbb{Z}_{\pi, \varphi}(\mathbf{z} \oplus \mathbf{c}_{\rho \setminus \varphi}),$$

که شمولش در $\mathbb{Z}_{\pi, \varphi}$ همان گزاره 4.16 است. و با دادن این به آن قضیه، نتیجه این است

$$\text{Adv}_{\rho[\pi/\varphi], \rho}^{\text{uc}}[\mathbf{a}, \mathbf{s}, \mathbf{z}] \leq \text{Adv}_{\pi, \varphi}^{\text{uc}}[\mathbf{a}, \mathbf{s}, \mathbf{z} \oplus \mathbf{c}_{\rho \setminus \varphi}].$$

پس ترکیب در حریف و در شبیه‌ساز مفت است، و سیستم پیرامون را یک بار هزینه می‌کند، در سمت محیط.

ترایایی، به زبان عدد. گزاره 4.12 می‌خواهد که ردهٔ شبیه‌ساز گام اول همان ردهٔ حریف گام دوم باشد، که به نامساوی‌ای میان بودجه‌ها بدل می‌شود: $\mathbb{S}(s) \subseteq \mathbb{A}(a')$ به محض $s \leq a'$ برقرار است. شبیه‌سازی که آدم حاضر است بسازد باید حریفی باشد که گام بعد حاضر است با آن روبه‌رو شود. آن گزاره یک ردهٔ مجاز برای هر دو گام هم می‌خواهد، و آن فرض را باید همین‌جا حمل کرد: برای $\mathbb{Z} \subseteq \mathbb{Z}_{\pi,\varphi} \cap \mathbb{Z}_{\varphi,\psi}$ ، که بنا بر اثبات گزاره 4.12 در $\mathbb{Z}_{\pi,\psi}$ هم می‌افتد،

$$\text{Adv}_{\pi,\psi}^{\text{uc}}[a, s', \mathbb{Z}] \leq \text{Adv}_{\pi,\varphi}^{\text{uc}}[a, s, \mathbb{Z}] + \text{Adv}_{\varphi,\psi}^{\text{uc}}[s, s', \mathbb{Z}].$$

بودجه‌ها به تنهایی اینجا کار نمی‌کنند. اگر با $\mathbf{z}$ در هر سه جا نوشته می‌شد، آن کمانک‌ها روی $\mathbb{Z}_{\varphi,\psi}(\mathbf{z})$ ، $\mathbb{Z}_{\pi,\varphi}(\mathbf{z})$ ، $\mathbb{Z}_{\pi,\psi}(\mathbf{z})$ دامنه می‌داشتند، و اولی در هیچ‌یک از آن دو دیگر گنجدیده نیست – به محیطی که برای زوج بیرونی مجاز است هیچ در باب $\text{IDS}(\varphi)$ گفته نمی‌شود – پس هیچ فرضی آن احتمال میانی را نمی‌پوشاند که نامساوی مثلثی حذف می‌کند. و نمایش ترکیب بالا چنین دردی ندارد: جذب آن یک رده‌اش را به دیگری می‌نگارد. و نتیجه 4.21 همان دو نمایش است که کنار هم گذاشته شده‌اند. در امتداد زنجیره قضیه 4.24 همان دو اثر انبار می‌شوند: خطاها جمع می‌شوند، هر جهش بودجه محیط را به اندازهٔ هزینهٔ میهمانی آنچه در آن گام جذب می‌کند فراخ می‌کند، و نسبت‌های ردهٔ نکته 4.25 به نامساوی‌های $s_{k+1} \leq a_k$ بدل می‌شوند.

تودرتویی، و اینکه آیا شبیه‌ساز می‌ترسد. بگویید حکم تقلید سرباری f دارد اگر به‌ازای هر بودجهٔ حریف $\mathbf{a}$ در برابر $(\mathbb{A}(\mathbf{a}), \mathbb{S}(f(\mathbf{a})), \cdot)$ برقرار باشد: یعنی هر چه به حریف داده شود، شبیه‌سازی که آدم می‌سازد f آن را هزینه می‌کند. در بودجهٔ هر-فراخوانی، آن دو مؤلفه متفاوت رفتار می‌کنند،

$$f(t, r) = (\max(t, \tau), \gamma r + \delta),$$

که شمار پرس‌وجو وابستهٔ خطی است – γ فراخوان‌هایی که شبیه‌ساز به‌ازای هر فراخوان حریفی که می‌داند می‌گذارد، و δ فراخوان‌های خودش – و زمان هر فراخوانی یک ماکسیمم است، با τ برای کار هر-فعال‌سازی خود شبیه‌ساز، چون در هر فعال‌سازی یک ماشین می‌دود و شبیه‌ساز به هر یک تنها کار کراندار می‌افزاید.

قضیه 4.20 اصلاً به f دست نمی‌زند. رده‌های حریف و شبیه‌سازش همان‌های فرضند، پس یک جانشانی تنها عیناً همان ماشینی را پس می‌دهد که فرض فراهم کرده بود و کل بها در سمت محیط پرداخته می‌شود. سربارها تنها در یک جا ترکیب می‌شوند: آنجا که شبیه‌سازی به‌عنوان حریف از نو خوانده می‌شود، که همان گزاره 4.12 است. آنجا s' از $\mathcal{S}$ ساخته می‌شود، که خودش از $\mathcal{A}$ ساخته شده، پس آن ترکیب سربار $f_2 \circ f_1$ دارد. و با به‌کاربستن نتایج k بار، بودجهٔ پرس‌وجو از $\tau_j = \gamma \tau_{j-1} + \delta$ پیروی می‌کند، یعنی

$$r_k = \gamma^k r_0 + \delta \frac{\gamma^k - 1}{\gamma - 1} \quad (\gamma \neq 1), \quad r_k = r_0 + k\delta \quad (\gamma = 1),$$

در حالی که زمان هر فراخوانی اشباع می‌شود، $t_k = \max(t_0, \tau_1, \dots, \tau_k)$ ، که با کار خود بزرگ‌ترین سطح کران خورده و در عمق ثابت است.

پس آن ترکییدن تمام و کمال در شمار پرس‌وجو زندگی می‌کند، و دوگانگی‌ای است که تنها بر γ می‌گردد. شبیه‌سازی که ناچار است به‌ازای برخی فراخوان‌های حریفی که می‌دواند بیش از یک فراخوان بگذارد $\gamma > 1$ دارد، و k بار به‌کار بستن γ^k هزینه دارد؛ به‌راستی نمایی، و در $\gamma = 2$ عمق 20 همین حالا ضریب 2^{20} ، یعنی حدود یک میلیون، روی شمار پرس‌وجو هزینه دارد. و شبیه‌سازی که هر فراخوان را یک بار بازپخش می‌کند و ترافیک کراندار خودش را می‌افزاید $\gamma = 1$ دارد، و k بار به‌کار بستن $k\delta + r_0$ هزینه دارد: خطی، با δ که یک بار به‌ازای هر سطح پرداخته می‌شود. این همان ویژگی‌ای است که وقتی شبیه‌سازی ساخته می‌شود باید سنجیده شود؛ و هیچ چیز اثبات‌شده در اینجا آن را نتیجه نمی‌دهد. و همین است که چرا شبیه‌سازهای خط‌مستقیم بیش از دلایل معمول مهم‌اند، چون عقب‌گرد راه استاندارد به‌دست آوردن $\gamma > 1$ است: یک شبیه‌ساز عقب‌گردی در هر جای آن برج، ضریب γ را روی هر سطح زیر خودش می‌گذارد.

همان فرمول بر پهنای هم حکم می‌کند و نه تنها بر عمق. قضیه 4.24 $\delta_1, \dots, \delta_n$ را با سپردن هر شبیه‌ساز به گام بعد به‌عنوان حریفش می‌سازد، پس n نمونه موازی n سربار را درست همان‌گونه ترکیب می‌کنند که برچی به عمق n ، و نکته 4.25 که با بودجه خوانده شود در هر پیوند $a_k \geq f_{k+1}(a_{k+1})$ را می‌خواهد. ترکیب n نمونه از پروتکلی با $\gamma = 1$ پرس‌وجوی اضافی هزینه دارد؛ و ترکیب n نمونه از پروتکلی با $\gamma = 2$ حتی برای n متوسط هم ناامیدکننده است.

هیچ چیز قابل مقایسه‌ای در سمت محیط رخ نمی‌دهد، و نه در زمان هر فراخوانی. هر سطح هزینه میهمانی پوستانه‌ای را که جذب می‌کند می‌افزاید، و پوستانه‌های یک برج مجزایند، پس تودرتویی به عمق k به درونی‌ترین حکم در بودجه $\mathbf{z} \oplus \bigoplus_{i \leq k} \mathbf{c}_i$ می‌رسد — که شمار فراخوانی‌اش همان شمار یک بار دواندن سیستم محیطی است، و زمان هر فراخوانی‌اش همان زمان بزرگ‌ترین پوستانه، هر چه هم آن برج عمیق باشد. خطاها هم به همین شکل رفتار می‌کنند، چون k بار به‌کار بستن گزاره 4.12 $\sum_i \varepsilon_i$ می‌دهد. پس هر جا ادعای امنیتی زیر تودرتویی نمایی تحلیل برود، سربار پرس‌وجوی شبیه‌ساز با $\gamma > 1$ تنها سرچشمه ممکن است: در هر فعال‌سازی یک ماشین می‌دود، پس نه زمان هر فراخوانی می‌تواند ضرب شود و نه بودجه محیط.

باز یافتن خوانش مجانی. هیچ چیز بالا از پارامتر امنیتی نام نمی‌برد. برای گرفتن آن حکم آشنا، همه چیز را با λ اندیس‌گذاری کنید، بگذارید بودجه‌ها در λ چندجمله‌ای باشند، و بخواهید که مزیت در آن ناچیز باشد؛ آن‌گاه رده‌ها به ماشین‌های چندجمله‌ای کران بدل می‌شوند و تعریف 4.11 به تعریف معمول تقلید UC صورت عینی همان صورتی است که نتایج ترکیب در آن اثبات شده‌اند، و صورت مجانبی نتیجه‌ای از آن است و نه برعکس — همان ترتیبی که امنیت اثبات‌پذیر عمل‌گرا مدت‌هاست برایش استدلال می‌کند [2].

فراخوان‌های پاسخگو

فصل 15 به وعده‌ای نیاز دارد که چارچوب تا اینجا هیچ راهی برای نوشتنش نداشته. وقتی Sign از حریف امضایی می‌خواهد، باید یکی پس بگیرد، و همین حالا پس بگیرد: اگر حریف آزاد بود پیش از پاسخ‌دادن برود و باقی اجرا را براند، آن طرف در میانه امضا معلق می‌ماند و محلی بودن از دست می‌رفت – همان بی‌درنگی‌ای که UC همگام از محیط پاسخگو می‌خواهد [11]. متن آنجا به محدودکردن رده شبیه‌ساز بسنده می‌کند. این فصل به جایش همان چیز را در کد می‌گوید.

چه چیزی باید ممنوع شود. نه اینکه حریف به نفع ما پاسخ دهد – او می‌تواند هر چیزی برگرداند، و فصل 12 به پاسخ بد می‌پردازد. آنچه باید ممنوع شود، رفتن ژتون است: میان دریافت فراخوان و بازگشت، پاسخ‌دهنده اصلاً هیچ فراخوانی نباید بگذارد. و این محدودیتی است بر فراخوان‌هایی که ماشین می‌گذارد، و این کتاب از پیش پوشانه‌ای درست برای همان دارد.

تعریف 9.1 (پاسخ‌دهی پاسخگو). **Respond** همان **Silence** در بیشینه خود است: $\mathcal{I}_{all}$ به عنوان مجموعه همه هویت‌ها،

$$\text{Respond}(\text{id}.\text{Op}(\text{in}) \text{ from } \text{id}')$$

$$:= \text{Silence}(\mathcal{I}_{all}, \text{id}.\text{Op}(\text{in}) \text{ from } \text{id}').$$

هر ادعایی در $\mathcal{I}_{all}$ می‌افتد، پس بنا بر فصل 2 هر فراخوانی که برنامه پوشانده شده صادر می‌کند با rej رد می‌شود، بی‌آنکه هرگز گذاشته شود. آن برنامه می‌تواند هر چه قدر بخواهد حساب کند و باز هم باید مقداری برگرداند؛ و آنچه نمی‌تواند بکند، واگذاشتن ژتون است.

خواننده‌شده به عنوان هندلر. اگر نوشته شود، **Respond** همان **Silence** است با بند آخرش که حذف شده. پوشانه **Silence** فراخوانی را که ادعایش را مجاز می‌داند پیش می‌فرستد، پس بند عملش تا حدی برانگیختن دوباره است؛ و **Respond** هیچ برانگیختن دوباره‌ای ندارد، تنها یک ثابت. به واژگان فصل 2، او اثر فراخوان محیطی را تخلیه می‌کند و نه اینکه پیش بفرستدش: در درون دامنه‌اش آن اثر به‌طور محلی و کامل گردانده

پوشانه Respond

برنامه id.Op(in) از id'

:Respond(id.Op(in) from id')

```

:1 handle id.Op(in) from id' with
:2   return out           ↦ out
:3   id''.FullOp(in'') as id''; k ↦ k(rej)

```

می‌شود، پس برنامه پوشانده شده نسبت به آن پاک است. و این تعریف محلی بودن هم هست، همان ویژگی‌ای که فصل 15 می‌خواست: فراخوانی روی جایگاه در درون فعال‌سازی خود پاسخ‌دهنده خدمت می‌شود و ژتون هرگز ترکش نمی‌کند. این همان بیان برای پاسخگویی است — هسته پاسخگو هسته‌ای است که اثر فراخوانش به Exec نمی‌گریزد — و خوانش عملیاتی از پی آن می‌آید، چون Exec درست همان فراخوان‌هایی را توزیع می‌کند که می‌گریزند. فراخوان پاسخگو یک فعال‌سازی پاسخ‌دهنده است و نه بیشتر.

تعریف 9.2 (واسط پاسخگو، فراخوان پاسخگو). حریف، در کنار FullA، واسط دومی به نام FullA¹ حمل می‌کند با همان نگهبان و همان هسته اما با Respond به جای Silence:

id.FullA¹(in) from id' : **require** Guard_A(id, id');
 return Respond(id.A(in) from id').

فراخوان پاسخگو فراخوانی روی FullA¹ است. در درون واسطی با هویت id ما A¹(in) را برای FullA¹(in).A(id.P) به عنوان id می‌نویسیم، همان‌گونه که A(in) کوتاه‌نویسی فراخوان معمولی است.

سه چیز از این در می‌آید، و آن‌ها بهایش‌اند.

پاسخ‌دهنده نمی‌تواند دفتر را بخواند. مجموعه C با فراخواندن FullStatus دسترس است، و آن فراخوان رد می‌شود، پس پاسخ پاسخگو از آنچه پاسخ‌دهنده از پیش می‌داند حساب می‌شود. نمی‌تواند واگذار کند: نه به زیرروالی، نه به طرف دیگری، نه به محیطی. و نمی‌تواند فاسد کند، چون Corrupt فراخوانی است مثل هر فراخوان دیگر. آخری سزاوار بیان جداگانه است، چون بخش 3.5 دفتر را دو بار می‌خواند و شکاف میان آن دو خواندن درست همان جایی است که فساد می‌تواند به درونش بلغزد.

گزاره 9.3 (فراخوان پاسخگو نمی‌تواند فاسد کند). در هر اجرایی، حالت دفتر C در لحظه بازگشت فراخوان پاسخگو همان حالت آن در لحظه گذاشتن آن فراخوان است.

اثبات. بنا بر بخش 3.1، C تنها در سطر 5 از FullCorrupt تغییر می‌کند، و تنها وقتی

آن واسط با فراخوانی در دسترس باشد. میان گذاشتن و بازگشت فراخوان پاسخگو، پاسخ‌دهنده در سرتاسر ژتون را در دست دارد، و بنا بر تعریف 9.1 هر فراخوانی که صادر کند پیش از گذاشته‌شدن رد می‌شود، پس **FullCorrupt** در دسترس نیست. و هیچ ماشین دیگری در آن بازه ژتون را در دست ندارد، پس هیچ‌یک هم نمی‌تواند به آن برسد. □

پس اگر هسته‌ای تنها از راه A^1 به حریف برسد، طرفی که خدمتش می‌کند در سطر 7 درست وقتی فاسد است که در سطر 3 فاسد بوده، و قلاب دوم واسطه‌گری به سبب آن فراخوان نمی‌تواند شلیک کند. نکته 3.2 توصیف می‌کند که وقتی شلیک می‌کند چه می‌شود؛ و پاسخگویی همان شرطی است که زیر آن شلیک نمی‌کند.

این همان است که فصل 15 می‌خواست. نوشتن A^1 به جای A در **Sign** و **Gen** و **Verify**، محلی‌بودن را ویژگی کد می‌کند و نه شرطی بر رده شبیه‌ساز، و پوشانه اعمالش می‌کند، پس در باب هر اشغال‌گر جایگاه حریف برقرار است – شبیه‌ساز هم در آن، چون تعریف 4.7 به شبیه‌ساز واسطه‌های حریف را می‌دهد، و $FullA^1$ یکی از آن‌هاست. و آنچه نمی‌کند این است که پاسخ را خوب کند: حریف پاسخگو باز هم می‌تواند $\perp$ برگرداند، و **San** آن را تعمیر می‌کند. پاسخگویی ژتون را باز می‌خرد؛ و پاک‌سازی، مقدار را.

نکته 9.4 (پاسخگویی و بدل). آن دو فصل در برابر هم می‌کشند، و ارزش دارد که رو گفته شود کجا. بدل بخش 4.4 به فراخوانی از سوی یک کارکرد با بازپخش آن به محیط در سطر 5 پاسخ می‌دهد – و آن یک فراخوان است، که **Respond** ردش می‌کند. پس $\mathcal{D}$ نمی‌تواند فراخوان پاسخگو را خدمت کند: آن بازپخش رد می‌شود، و قاعده بخش 3.6 پاسخش را به صورت $\perp$ تحویل می‌دهد، آنجا که حریف واقعی پاسخی با محتوا می‌داد. پس بازگروه‌بندی ($R1$) برای سیستمی که هسته‌هایش A^1 را به کار می‌برند در نخستین فراخوان پاسخگو شکست می‌خورد – و همین است که چرا قضیه 4.29 سیستم‌هایی را فرض می‌گیرد که هسته‌هایشان هیچ‌یک نمی‌گذارند. و بازیافتن تمامیت برای سیستم‌های پاسخگو به مفهومی متناظر در سمت محیط نیاز دارد، تا آن پرسش بازپخش‌شده بی‌جابه‌جاشدن ژتون پاسخ بگیرد؛ و آن را باز می‌گذاریم.

زبان هسته

فصل‌های 13 تا 21 کد کارکردها و پروتکل‌ها و بازی‌های این کتاب را می‌دهند، و آن کد قرار است به‌عنوان برنامه خوانده شود و نه به‌عنوان تصویری از یک برنامه – وعده‌ای که باید با قراردادی بیان‌شده نگه داشته شود و نه با حسن نیت خواننده. [5] درست همین وعده را برای اثبات‌های بازی‌محور با زبانی کوچک، قویاً نوع‌دار و پایان‌پذیر نگه می‌دارند که $\mathcal{L}$ می‌نامندش، و آن را با دستور زبان مستقل از متن (شکل 15 آن‌ها) و فهرستی کوتاه و نام‌دار از شکرهایی که رویش مجاز می‌دانند می‌دهند. این فصل همان دو گام را اینجا برمی‌دارد: دستور زبانی بسته برای آنچه هر فصل از این پس هسته می‌نامندش – یعنی بدنه یک عمل، که از پیش واژه خود این کتاب برای آن است – و فهرست کوتاه خود این کتاب از عدول‌ها از آن دستور زبان، که هر یک یک بار نامش برده می‌شود تا هر هسته‌ای که هنوز می‌آید در برابرش سنجیده شود و نه با اعتماد پذیرفته.

هر چه پیش از این فصل است معاف است، و عامدانه. **Silence** و **Guard** و **Mediate** (فصل 2)، **Exec** (بخش 3.6)، حریف بدل (بخش 4.4) و **Respond** (فصل 9) در درون هسته نمی‌دوند: آن‌ها همان چیزی هستند که سیستمی از هسته‌ها را می‌دواند، فراخوان‌ها را توزیع می‌کند، نگهبان‌ها را می‌سنجد، و ژتون را از یک فعال‌سازی به فعال‌سازی بعدی می‌سپارد. و تعریف اجرا خودش برنامه‌ای در زبانی نیست که اجرایش می‌کند – درست همان‌گونه که شرح خود [5] از روبه‌رویی یک بازی با یک حریف در بخش غیررسمی 3 آن‌ها می‌نشیند، بیرون دستور زبان شکل 15 – و آن یک نمادگذاری که می‌شد با نمادگذاری خود هسته اشتباه گرفت، یعنی صورت ادامه $\text{id}''.\text{FullOp}(\text{in})$ as id''' ; k را حمل می‌کند، تمام‌وکمال به همان لایه بیرونی تعلق دارد: از فصل 13 به این‌سو، حتی یک بار در درون کد خود هیچ عملی رخ نمی‌دهد. خویشاوند ساده‌ترش رخ می‌دهد، و مدام – یعنی فراخوانی معمولی به عمل کارکردی دیگر، که زیر هویت خود فراخواننده از سر گرفته می‌شود و نه با ادامه‌ای تازه – و آن یکی خودش مسئله‌ای از زبان هسته است، در زیر.

دستور زبان هسته. هسته دنباله‌ای از گزاره‌هاست، و هر عملی که این کتاب می‌نویسد دقیقاً یک هسته است. و در دامنه آنچه هسته‌های این کتاب واقعاً به کار می‌برند، و نه در دامنه کل دستور زبان $\mathcal{L}$:

$$\begin{aligned}
 \text{core} &::= \varepsilon \mid \text{stmt}; \text{core} \\
 \text{stmt} &::= \text{lv} \leftarrow e \mid \text{lv} \leftarrow_{\S} S \\
 &\quad \mid \text{if } e \text{ then core [else core]} \\
 &\quad \mid \text{for } x \in S \text{ do core} \\
 &\quad \mid \text{return } e \\
 \text{lv} &::= x \mid x[e] \\
 e &::= \dots \mid \text{call} \\
 \text{call} &::= \text{id}''.\text{Op}(e_1, \dots, e_k) \text{ as id}
 \end{aligned}$$

شکل 5: دستور زبان هسته، در دامنه کاربرد خود این کتاب. همان دستور زبان $\mathcal{L}$ [5]، شکل ۱۵ آن‌ها) با زیردستور زبان عبارت‌ها که به نمادگذاری معمولی ریاضی واگذار شده، چون هر عبارتی که این کتاب می‌نویسد از پیش روی شش نوع زیر استاندارد است؛ و call همان فراخوان خود $\mathcal{L}$ است که از محدودیت تنها-حریف آن‌ها فراتر تعمیم یافته (در زیر). (فصل 10).

شش نوع پایه، و نه پنج $\mathcal{L}$: عدد صحیح، بولی، رشته، مجموعه، جدول – که همان آرایه $\mathcal{L}$ است، که در سرتاسر به عنوان نوع یک تابع نگه داشته شده، و $\mathbb{L} : \mathbb{N} \rightarrow \{0, 1\}^n \cup \{\square\}$ درست همان‌گونه خوانده می‌شود که $A : \{0, 1\}^* \rightarrow \{0, 1\}^* \cup \{\text{undefined}\}$ آنجا، با $\square$ که undefined این کتاب است – و هویت، که افزوده خود این کتاب است، در زیر. تنها یک صورت **for** شهادت داده شده، یعنی **for** $x \in S$ **do**، و آن یک نمونه‌اش زوجی را مستقیم مقید می‌کند، **for** $(m, y) \in \mathbb{N}$ **do** (فصل 20، سطرهای 17 و 16) – که شکری آشکار است برای مقیدکردن یک متغیر و شکافتن آن. و آن صورت بازه عددی کراندار که $\mathcal{L}$ هم مجاز می‌داند در ذخیره است، تا اینجا به کارنرفته، و همان لحظه که هسته‌ای لازم داشته باشد مجاز است.

فراخواننده مستتر است. هسته هر عملی سر $\text{id}.\text{Op}(\text{in})$ **from** id' دارد، و $\text{id}' - \text{id}$ هویت فراخواننده – هر جای هسته خواندنی است، هرچند نام هیچ آرگومان Op نیست. اوراکل ساده در $\mathcal{L}$ چیزی از این دست حمل نمی‌کند؛ و هسته‌های این کتاب لازمش دارند چون بسیاری از آن‌ها با پرسیدن اینکه دقیقاً چه کسی فرا می‌خواند آغاز می‌کنند، پیش از آنکه چیز دیگری بدود، و عمل‌های $\mathcal{G}_{\text{Clock}}$ نمونه نماینده‌اند (نگه‌بان سطر 13 $\text{id}'.F$ و فساد $\text{id}.P$ را پیش از هر کار دیگری در هسته می‌سنجد).

هسته می‌تواند هسته دیگری را فرا بخواند. $\mathcal{L}$ عبارت call را تنها برای حریف رزرو می‌کند؛ و اوراکلی در درون یکی از بازی‌های آن‌ها نمی‌تواند یکی بگذارد. هر کارکرد و پروتکل و بازی این کتاب می‌گذارد – پروتکل فصل 20 $\mathcal{F}_{\text{PKI}}$ و $\mathcal{F}_{\text{Sig}}$ را در میانه عمل می‌خواند (سطر 22 و همسایه‌هایش)، و بازی پیش‌بینی‌ناپذیری خود $\mathcal{F}_{\text{Rand}}$ به درون خود $\mathcal{F}_{\text{Rand}}$ باز فرا می‌خواند (سطر 5) – چون کارکردها و پروتکل‌های این کتاب با اعلام از دیگران ساخته می‌شوند، و **U** درست نام آن چیزی را می‌برد که هسته می‌تواند فرا بخواند (فصل 1)، و پوسته یک بازی کارکردی است مثل هر کارکرد دیگر. و خود آن فراخوان همان فراخوان $\mathcal{L}$ است، بی هیچ شلی: $\text{id}''.\text{Op}(e_1, \dots) \text{ as id}$ مثل هر عبارت دیگری ارزیابی می‌شود و هسته فراخوانده را در همان سطر و زیر هویت خودش از سر می‌گیرد. آنچه اکیداً بیرون هسته می‌ماند صورت عمومی است، با ادامه مقیدنشده k

که به جای «سطر بعدی همین هسته» به نقطهٔ ازسرگیری دلخواهی می‌ایستد – و آن نمادگذاری خود مدلی اجراست برای اینکه فراخوان در حالت عمومی چه معنایی دارد، و هرگز نمادگذاری هسته نیست، و بخش 3.6 همان جایی است که تعریف می‌شود.

require شکر است. سطر `require e` خود هسته، که در سرتاسر فصل‌های مدلی اجرا و در درون بسیاری از هسته‌هایی که می‌آیند به کار رفته، شکری است برای `not if e then return ⊥` – و نه عیناً `rej`، هرچند نگاهی که جای دیگری شکست بخورد درست همان را برمی‌گرداند. بخش 3.6 `rej` را همان یک مقداری می‌کند که هیچ هسته‌ای نمی‌تواند به‌عنوان خروجی خودش برگرداند، و `rej` سطح هسته را پیش از تحویل به `⊥` می‌شوید؛ و `require` خود هسته تنها نام حالت رایج را جمع‌وجور می‌برد. عمل‌های `⌚` همان نگاهی را به‌دست هجی می‌کنند، با یک `if ... then return` معمولی (باز سطر 13): و هر دو همان یک گزارهٔ زبان هسته‌اند، شکرشده یا نه.

اندیس‌های جدول قاعدهٔ تازه‌ای لازم ندارند. `L[ctr]`، `reg[id.P]`: هیچ‌یک رشته‌اندیس نیست، و `⌚` از پیش پاسخش را دارد، یعنی افزونهٔ سوم خودش ([5]) – اندیسی که رشته نیست، نخست بی‌صدا از `encode(·)` می‌گذرد. هر جدولی که این کتاب می‌نویسد و رشته‌اندیس نیست از پیش به همین شکل خوانده می‌شود؛ و اینجا هیچ چیز برای افزودن نیست جز ارجاع به آن‌ها برای همین.

هویت نوع ششم است. یک عدول شکر نیست. تعریف 1.1 هویت را زوج `id = (pid, P)` از شناسهٔ فرایند و طرف می‌کند، و شناسهٔ فرایند خودش سه‌گانهٔ `(F, s, i)` – و هسته از هر دو سطح یک‌جا می‌خواند، با `id.F`، `id.s`، `id.P` به‌عنوان تصویرهای نام‌دار، و `id.i` که آن یک میدانی است که هیچ هسته‌ای هرگز نمی‌خواندش (فصل 1 رو می‌گویدش). هیچ‌یک از پنج نوع `⌚` تصویری حمل نمی‌کند، و `encode(·)` تنها نگه‌داشت تساوی می‌دهد، بنا بر شرح خود [5] از آن – و هویت بسیار بیش از آن از هم شکافته می‌شود که این کار را بکند. پس هویت به‌عنوان نوع پایهٔ ششم به آن فهرست می‌پیوندد، ساخت‌دار و نه اتمی، با تصویر و تساوی مؤلفه‌به‌مؤلفه به‌عنوان تنها عمل‌هایش – و همان یک جایی است که هسته‌های این کتاب از `⌚` چیزی می‌خواهند که از پیش ندارد.

خواندن کد. دستور زبان بالا همان است که هسته می‌تواند حساب کند؛ و اینکه چگونه روی صفحه نشانده می‌شود وعده‌ای جداگانه است، و وعده‌ای که پیش‌تر بیشترش نگه داشته شده. کلیدواژه‌ها سیاه‌اند، که پیش‌فرض `algorithmicx` است: `if`، `then`، `else`، `do`، `return`، و `require`. نام هر عمل، هر جا اعلام می‌شود و هر جا به آن ارجاع می‌شود پیوندی حمل می‌کند، و هر دو با حروف کوچک بزرگ‌نمای `accentsoft` نشانده می‌شوند – یک ماکرو آن اعلام را لنگر می‌کند و دو ماکروی دیگر از هر جایگاه فراخوان به آن اشاره می‌کنند. حالت در `x` زندگی می‌کند، با حروف ماشین‌تحریری، که برای میدان‌های تغییرپذیر خود هسته رزرو شده؛ و `id` و `in` و `out` ایتالیک می‌مانند، چون پارامترهای مقیدند و نه حالت، و آن دو نباید با هم آمیخته شوند – خوانندهٔ آینده‌ای که تصمیم می‌گیرد نوع رکورد خود هسته چه شکلی است، درست همین تمایز را لازم دارد و نه هیچ تمایز دیگر. توضیح‌ها با `commentint` و ایتالیک چاپ می‌شوند، در کنار، و هرگز باربر نیستند. آن پاس ویرایشی روی فصل‌هایی که پیش‌تر نوشته شده‌اند انجام شده،

و نه نمادگذاری تازه بلکه یک جاروی یک‌دستی است: نخستین ذکر نوشتاری عملی که از پیش اعلام شده در سرتاسر به اعلامش پیوند می‌خورد، و آن اتم‌های ثابت – `rej`, `ok`, `done`, `false`, `true`، به همراه هر برچسب رد و هر کلید ذخیره در جای دیگر این کتاب – `commenttint` حمل می‌کنند و نه رنگِ خودِ نامِ یک عمل، پس رنگِ خالی اکنون «این یک نام است» را از «این یک مقدار است» جدا می‌کند.

آنچه دستور زبان و عدول‌های بالا با هم می‌خرند یک آزمون تنهاست: از فصل 13 به این‌سو، سطری از ریاضیات این کتاب درست وقتی کدِ خودِ یک هسته است که در برابر جدول بالا به‌علاوه عدول‌هایی که اینجا نامشان برده شده نوع‌سنجی شود، و درست وقتی یک مشخصه است – حکم یک بازی، سور یک ویژگی، `pick` خود `San` – که نوع‌سنجی نشود. و بستن شکاف تا زبانی واقعی، `OCaml` یا هر چیز دیگر، آن‌گاه مسئله کامپایل کردنِ شش نوع و یک مشتِ صورتِ گزاره است، یک بار، و نه مسئله اینکه نخست فصل به فصل تصمیم بگیریم که یک نمایش داده‌شده از نمادها اصلاً ادعا داشته چه چیزی را حساب کند.

همسان تا خرابی

اثبات‌های کدمحور با جهش [5، 13] پیش می‌روند: دو بازی که همان یک برنامه‌اند جز آنجا که پرچمی برافراشته شده، و کرانی بر احتمال آن برافراشتن. لم بنیادی بازی بازی همان جواز کار است – اگر آن دو بازی تا bad همسان باشند، برآمدهایشان با احتمال دست‌بالا $\Pr[\text{Bad}]$ تفاوت دارند – و اثباتش یک جفت‌سازی است: هر دو بازی را روی همان سکه‌ها بدوانید و پیش از برافراشتن پرچم نمی‌توانند از هم جدا شوند. در این کتاب، بازی یک برنامه نیست. اجرا یک محیط، یک اشغال‌گر جایگاه، یک دفتر و یک سیستم را می‌دواند؛ ژتون میانشان سرگردان است؛ اشغال‌گر می‌تواند ژتون را در میانه یک عمل بگیرد و پیش از پاسخ‌دادن باقی اجرا را براند – همان آزادی‌ای که فصل 9 برای نظم‌دادنش هست – و جهش باید از همه آن جان سالم ببرد. این فصل آن لم را روی اجراها از نو می‌دواند، و یافته‌اش این است که آن سرگردانی هیچ هزینه‌ای ندارد: لم زیر اصلاً هیچ فرضی ندارد.

پرچم کجا زندگی می‌کند. bad متغیر حالت معمولی ماشینی معمولی است: در Initialize به صورت $\text{false} \leftarrow \text{bad}$ اعلام می‌شود، پس از آن هرگز مقداری جز true نمی‌گیرد، و مثل باقی حالت یک ماشین در مالکیت همان ماشینی است که اعلامش می‌کند. چارچوب از پیش هر دست دیگری را از آن دور نگه می‌دارد – تنها کاری که ماشین می‌تواند با حالتی که مالکش نیست بکند، برانگیختن فراخوان‌هایی است که آن را می‌خوانند یا گسترشش می‌دهند، و در طرف فاسد Mediate پاسخ‌ها را جایگزین می‌کند و هرگز حالت را (نکته 5.4) – پس پرچم را کد خود مالکش برمی‌افرازد یا هیچ‌کس. و اینکه آیا کسی می‌تواند بخواندش، بی‌اهمیت است، خواه از راه واسطی و خواه از راه Leak که Guard را دور می‌زند: تا پیش از برافراشتن پرچم آن دو ماشین زیر به هر خواندنی یکسان پاسخ می‌دهند، و پس از برافراشتنش آن لم از پیش بهایش را داده است. چند ماشین می‌توانند یک‌جا پرچم حمل کنند – اعضایی که نامی را شریکند هم در آن، که هر یک مالک پرچم خودش است – و Bad زیر پیشامد این است که برخی ماشین برخی پرچم را برافرازد.

تعریف 11.1 (همسان تا خرابی). کارکردهای استاندارد $\mathcal{F}$ و $\mathcal{F}'$ همسان تا خرابی‌اند اگر پنج میدان‌شان جور باشد؛ Initialize هایشان سطر به سطر جور باشند، با

اعلام‌هایی که در هر دو سو مشترکند و $\text{false} \leftarrow \text{bad}$ در میانشان؛ و باقی کدشان جور باشد جز در درون بلوک‌های خرابی‌نگهبانی‌شده، که دو شکل دارند: ادامهٔ انتساب $\text{true} \leftarrow \text{bad}$ ، از همان جا که آن انتساب تمام می‌شود تا پایان بلوک دربرگیرنده‌اش – یعنی شاخه یا حلقه یا بدنهٔ عملی که آن انتساب در آن ایستاده، به‌گونه‌ای که هر چه در آن ادامه است تنها با گذر از آن انتساب در دسترس باشد – و بدنهٔ شرطی‌ای که آزمونش bad است. پاک‌کردن بلوک‌های خرابی‌نگهبانی‌شده از هر دو سو یک کارکرد باقی می‌گذارد، عمل به‌عمل و سطر به‌سطر، با Leak هم در آن؛ و به‌ویژه خود آن انتساب‌ها کد مشترکند، که هر یک سر ادامهٔ واگرای خودش است، و هیچ‌یک از دو سو به bad مقداری جز true نمی‌دهد. کارکردهای برابر همسان تا خرابی‌اند، با هیچ چیز برای پاک‌کردن. و دو اشغال‌گر جایگاه حریف همسان تا خرابی‌اند هرگاه Initialize ها و هسته‌هایشان چنین باشند، که به همان شکل خوانده می‌شود. سیستم‌های π و π' همسان تا خرابی‌اند اگر دوسویی‌ای هر عضو یکی را با عضوی از دیگری جفت کند که با آن همسان تا خرابی است.

دو چیز در باب این شکل، که هر دو از [5] است. عضویت با بازرسی سنجیده می‌شود – مقایسهٔ کد است و نه ادعایی در باب رفتار – و همین است که حساب‌رسی جهش‌ها را ارزان نگه می‌دارد. و چنان ساخته شده که گزارهٔ واگرا نتواند زودتر بدود: تنها راه‌های ورود به بلوک خرابی‌نگهبانی‌شده سر آن است، یعنی انتسابی که پرچم را در همان گذر برمی‌افزارد، و آزمون خود پرچم، و پرچم یکنواست – پس هیچ چیز خرابی‌نگهبانی‌شده در حالی که پرچم مالکش پایین است نمی‌دود. و چون اعضای جفت‌شده میدان‌های برابر حمل می‌کنند، آن دو سیستم همان هویت‌ها را اشغال می‌کنند و همان نام‌ها را به کار می‌برند، $\langle \pi' \rangle = \langle \pi \rangle$ ، و یک ردهٔ محیط به هر دو خدمت می‌کند: $\mathbb{Z}_{\pi, \pi'} = \mathbb{Z}_{\pi, \pi} = \mathbb{Z}_{\pi', \pi'}$.

لم 11.2 (لم بنیادی، روی سیستم‌ها). بگذارید سیستم‌های π و π' همسان تا خرابی باشند، اشغال‌گران $\mathcal{X}$ و $\mathcal{X}'$ از جایگاه حریف همسان تا خرابی باشند، و $\mathcal{Z} \in \mathbb{Z}_{\pi, \pi'}$. Bad را برای این پیشامد بنویسید که برخی ماشین انتساب $\text{true} \leftarrow \text{bad}$ را اجرا می‌کند، در هر اجرایی که نامش برده شود. آن‌گاه

$$\Pr[\text{Bad in Exec}(\pi, \mathcal{Z}, \mathcal{X}, \mathcal{C})] = \Pr[\text{Bad in Exec}(\pi', \mathcal{Z}, \mathcal{X}', \mathcal{C})],$$

و، با نوشتن $\Pr[\text{Bad}]$ برای آن مقدار مشترک،

$$\begin{aligned} & \left| \Pr[\text{Exec}(\pi, \mathcal{Z}, \mathcal{X}, \mathcal{C}) = 1] \right. \\ & \left. - \Pr[\text{Exec}(\pi', \mathcal{Z}, \mathcal{X}', \mathcal{C}) = 1] \right| \leq \Pr[\text{Bad}]. \end{aligned}$$

و اگر علاوه بر آن آن دو سیستم، سیستم بازی باشند – که آن‌گاه آن جفت‌شدن بازی را با بازی جور می‌کند، چون میدان‌ها نگه داشته می‌شوند – همین در باب هر

نهایی‌سازی Fin که آن دو بازی شریکند برقرار است:

$$\begin{aligned} & \left| \Pr[\text{Win}_{\text{Fin}} \text{ in } \text{Exec}(\pi, \mathbb{Z}, \mathbb{X}, \mathbb{C})] \right. \\ & \quad \left. - \Pr[\text{Win}_{\text{Fin}} \text{ in } \text{Exec}(\pi', \mathbb{Z}, \mathbb{X}', \mathbb{C})] \right| \\ & \leq \Pr[\text{Bad}]. \end{aligned}$$

اثبات. یک جفت‌سازی، که نقطه به نقطه خوانده می‌شود. ثابت‌کردن هر نوار، هر دو اجرا را رو تعیین می‌کند، پس آن دو را می‌شود از یک نمونه گرفت و گام به گام سنجید. آن استقرا نشان می‌دهد که تا وقتی هیچ پرچمی بالا نیست آن دو اکیداً متناظرند و ژتون روی کد مشترک می‌ایستد – که درست همان چیزی است که دو شکل خرابی‌نگهبانی‌شده تعریف 11.1 می‌خزند، چون هیچ‌یک در دسترس نیست جز با گذر از انتسابی که پرچم را برمی‌افزارد یا از راه آزمونی که در پایین‌بودن پرچم شکست می‌خورد. پس نخستین برافراشتن همان یک گام در هر دو سو است، و آن همان نمایش اول است: $\Pr[\text{Bad}]$ یک عدد است و نه دو عدد که تصادفاً کرانی را شریکند. و بیرون Bad آن دو اجرا رو برابرند، پس هر واقعیت آن اجرا آنجا یک نشانگر تنها دارد، و هر اختلاف احتمالات دست‌بالا اندازه مجموعه‌ای است که نشانگرها می‌توانند در آن تفاوت داشته باشند. دو قرارداد از فصل‌های 1 و 3.6 به کار می‌روند، همان‌ها که لم جذب بازگوشان می‌کند: هر ماشین از نوار سکه خودش می‌کشد، یکنواخت و مستقل از نوار هر ماشین دیگر، که تنها وقتی کدش نمونه‌برداری می‌کند خوانده می‌شود – و رونوشت‌های میهمان یک محیط نوارهایی از خودشان نگه می‌دارند، چون دسته‌کردن چیزی را یکی نمی‌کند – و Initialize هیچ فراخوانی نمی‌گذارد. پس ثابت‌کردن هر نوار، اجرا را رو تعیین می‌کند: اینکه کدام ماشین ژتون را در دست دارد، در کدام سطر از کدام عمل، با چه حالتی، و اینکه گزاره بعدی‌اش چه می‌کند، همه تابع آن پیکربندی‌اند، و سکه‌ها تنها به‌عنوان موقعیت‌های ناخوانده بعدی نوار خود دارنده ژتون وارد می‌شوند.

آن دو اجرا را جفت کنید. آن جفت‌شدن ماشین را با ماشین جور می‌کند – عضو با عضو، اشغال‌گر با اشغال‌گر، و محیط و دفتر هر یک با خودش – و ماشین‌های جفت‌شده همان هویت‌ها را اشغال می‌کنند، پس به ماشین‌های جفت‌شده نوارهای برابر بدهید. هر اجرا جداگانه توزیع خودش را نگه می‌دارد، چون نوارها در هر دو سو یکنواخت و مستقلند، پس هر دو را می‌شود از یک نمونه نوارها خواند و نقطه به نقطه سنجید. بگویید آن دو در یک پیکربندی اکیداً متناظرند اگر (C1)–(C4) تعریف 4.17 زیر آن جفت‌شدن برقرار باشند – حالت‌های برابر، نوارهای ناخوانده برابر، ژتون در دستان جفت‌شده در همان سطر با همان مقدرهای محلی، و فراخواندگان معلق که قاب به قاب جورند – با هیچ بازپختی در هیچ کجا و هیچ چیز حذف‌شده.

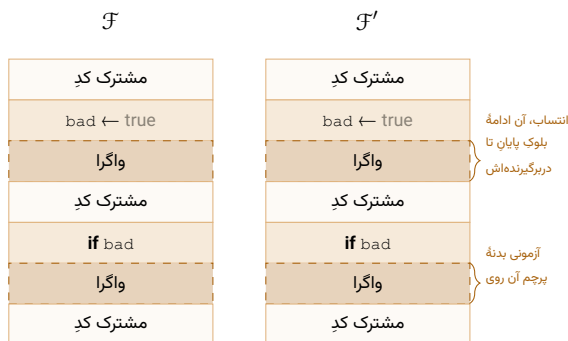
در آغاز اکیداً متناظرند. سطر 1 دفتر و هر عضو را در هر دو سو مقدار آغازین می‌دهد، به هر ترتیبی – چون Initialize هیچ فراخوانی نمی‌گذارد و به هیچ حالتی جز حالت مالک خودش دست نمی‌زند، پس آن ترتیب هیچ نشانی در پیکربندی نمی‌گذارد؛ اعضای جفت‌شده Initialize‌هایی می‌دوانند که سطر به سطر جورند، با $\text{bad} \leftarrow \text{false}$

در میانشان، که هیچ فراخوانی نمی‌گذارند و هر جا می‌کشند، همان موقعیت‌ها را از نوارهای برابر می‌کشند؛ و سطر² هر دو ژتون را به همان یک محیط در همان هویت می‌سپارد.

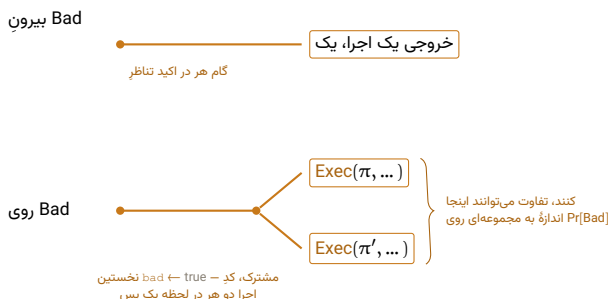
فرض کنید آن دو اکیداً متناظرند، هنوز هیچ پرچمی در هیچ سو برافراشته نشده، و بگذارید یک گزاره بدود. ژتون در ماشین‌های جفت‌شده در همان سطر ایستاده، و آن سطر کد مشترک است: گزاره خرابی‌نگهبانی‌شده تنها از راه سرش یا از راه آزمون پرچم در دسترس است، و آن سر پرچم را در همان گذر برمی‌افزاید، و آن آزمون در پایین‌بودن پرچم شکست می‌خورد، و هیچ پرچمی بالا نیست. پس آن گزاره همان یک گزاره است، که روی همان حالت و همان سکه‌های بعدی خوانده می‌شود، چون تاریخ‌ها نواربه‌نوار یکسان مصرف شده‌اند. اگر بنویسد، هر دو سو یکسان می‌نویسند؛ اگر برگردد، هر دو همان قاب را با همان مقدار از سر می‌گیرند؛ اگر فراخوانی بگذارد، هر دو همان فراخوان را می‌گذارند — همان هدف، همان بار پیام و همان هویت ادعایی — که با همان پوشانه‌ها روی میدان‌های برابر و حالت برابر دفتر روبه‌رو می‌شود، و در ماشین‌های جفت‌شده فرود می‌آید، یا در هیچ ماشینی و یکسان rej پاسخ می‌گیرد؛ و هر تحویلی که بخش 3.6 تعیین می‌کند — rej برای $require$ رده‌شده، $\perp$ آنجا که هسته‌ای rej برمی‌گرداند یا جایگاهی واسطه‌گری‌شده یکی را پاسخ می‌دهد — یک قاعده است و نه یک انتخاب. پس تناظر اکید از آن گام جان سالم می‌برد. و اگر آن گزاره انتساب $bad \leftarrow true$ باشد — که کد مشترک است و سر ادامه و اگرای خودش ایستاده — هر دو سو اجرایش می‌کنند، و آن گام یک‌جا نخستین برافراشتن در هر دو اجراست.

پس نقطه‌به‌نقطه روی فضای جفت‌شده: یا هیچ پرچمی هرگز برافراشته نمی‌شود و آن دو اجرا در هر گامی اکیداً متناظرند — پس یکی درست وقتی می‌ایستد که دیگری، با همان خروجی، چون محیط همان یک ماشین است که یکسان از سر گرفته می‌شود — یا نخستین برافراشتنی هست، هم‌زمان است، و آن دو اجرا تا آن و در طول آن توافق دارند. پس پیشامد Bad روی فضای جفت‌شده یک نشانگر دارد، که همان نمایش اول است، با تساوی. و بیرون Bad آن دو اجرا رو برابرند، پس هر واقعیتی آن اجرا آنجا یک نشانگر دارد: خروجی، و، برای سیستم‌های بازی، پیشامد Win_{Fin} که واسطه $FullFin$ ($\mathcal{G}m.pid, Z$) مقدار 1 برمی‌گرداند (تعریف 5.8). و هر اختلاف احتمال‌ها دست‌بالا اندازه مجموعه‌ای است که نشانگرهایش می‌توانند در آن تفاوت داشته باشند، که $Pr[Bad]$ است. $\square$

آن تعریف و آن اثبات هر یک تصویری دارند، و ارزش دیدن یک‌جا دارند، چون یکی درست همان چیزی است که دیگری مصرفش می‌کند. بالا آن است که تعریف 11.1 از کد می‌خواهد؛ و پایین آن است که جفت‌سازی با آن می‌کند. هیچ‌یک از دو تخته واژگان جعبه‌وسیم شکل‌های پیشین را به کار نمی‌برد، و نکته اصلی همین است: هیچ چیز اینجا از ماشینی به ماشین دیگر جابه‌جا نمی‌شود، پس چیزی نیست که میانش سیم بکشیم.



می‌ماند، کارکرد یک و کنید پاک را سایه‌دار بلوک دو آن
سطر به سطر و عمل به عمل



شکل 6: همسان تا خرابی، در دو تخته. بالا، آنچه تعریف 11.1 از کد می‌خواهد؛ پایین، آنچه جفت‌سازی با آن می‌کند. هیچ‌یک از دو تخته واژگان جعبه‌وسیم شکل‌های پیشین را به کار نمی‌برد، چون هیچ‌چیز اینجا از ماشینی به ماشین دیگر جابه‌جا نمی‌شود. (تعریف 11.1 و لم 11.2).

تخته بالا با بازرسی سنجیدنی است، و همین است که حساب‌رسی یک جهش را ارزان نگه می‌دارد: مقایسه کد است و نه ادعایی در باب رفتار. آن دو شکل سایه‌دار تنها شکل‌های مجازند، و میان خودشان همان دلیلی هستند که چرا هیچ‌چیز سایه‌دار در پایین‌بودن پرچم نمی‌تواند بود - تنها راه‌های ورود، سر است، که پرچم را در همان گذر برمی‌افزارد، و آزمون پرچم، که در پایین‌بودنش شکست می‌خورد. پس آن سایه بالا درست همان است که آن‌سوی دوشاخه پایین می‌دود، و هیچ جای دیگری.

آن دوشاخه یک نقطه تنهاست، و همان کل استدلال است. گزاره‌ای که سر هر بلوک و اگر ایستاده خودش کد مشترک است، پس هر دو اجرا آن را در همان گام از همان نمونه جفت‌شده اجرا می‌کنند - که نمایش اول آن لم را می‌دهد، یعنی اینکه $\Pr[\text{Bad}]$ در آن دو اجرا همان یک عدد است و نه دو عدد با یک کران. و بیرون Bad اصلاً دوشاخه‌ای نیست: آن دو اجرا یک اجرا هستند، پس هر واقعیت آن آنجا یک نشانگر تنها دارد - خروجی، و، برای سیستم‌های بازی، Win_{Fin} - و هر اختلاف احتمال‌ها دست‌بالا

اندازه مجموعه‌ای است که نشانگرها می‌توانند در آن تفاوت داشته باشند. آن مجموعه همان شاخه پایینی است، و اندازه‌اش $\Pr[\text{Bad}]$.

نکته 11.3 (هیچ چیز مفروض نیست). این لم هیچ از سیستم‌ها و هیچ از جایگاه نمی‌خواهد: نه خوش‌ساختی، نه تنگی، نه بی‌اعتنایی به رد، و — که ارزش چشیدن دارد — نه نبود فراخوان‌های پاسخگو. هر نتیجه‌ای که ماشینی را جابه‌جا می‌کند برای آن جابه‌جایی فرضی می‌پردازد: قضیه 4.29 و گزاره 5.9 هسته‌های پاسخگو را کنار می‌گذارند، نتیجه 5.11 آن کنارگذاری را یک‌جا در هر دو سو به ارث می‌برد، و ساعت خبردهنده و شبکه و کانال درست به همین سبب بیرون هر سه می‌نشینند. و لم بالا بی‌اعتناست — خبردهی‌های پاسخگوی آن‌ها به همان ارزانی هر چیز دیگر جهش می‌کنند — چون هیچ چیز جابه‌جا نمی‌شود. مثل بازگروه‌بندی، این هم این‌همانی‌ای در باب اجراهاست، که تنها بر قطعیت یک اجرا به محض ثابت‌شدن نوارهایش تکیه دارد، و نه مقایسه‌ای از آنچه دو اجرا به دست می‌آورند. و هیچ بودجه‌ای به همان دلیل پیدا نمی‌شود: دقت هیچ بودجه‌ای لازم ندارد، و تعریف 4.30 تنها آنجا وارد می‌شود که کاربردی به کران‌گذاشتن بر $\Pr[\text{Bad}]$ بنشیند.

نکته 11.4 (چرا نه با جذب). راه کوتاه‌تر و سوسه‌کننده‌ای هست: کل اجرا را در محیط جذب کنید — لم 4.18 برای جابه‌جا کردن ماشین‌ها هست — تا هر سو یک برنامه تنها شود، و آن‌گاه [5] را در باب آن زوج نقل کنید. هر جا کار کند کار می‌کند، و آموزنده است که ببینیم چرا لم بدتری است. فرض‌های جذب همراهش می‌آیند: اشغال‌گر جایگاهی که جابه‌جا می‌شود باید بی‌اعتنا به رد باشد، هیچ هسته‌ای نباید فراخوان پاسخگویی روی هویتی واگذار شده بگذارد (نکته 9.4)، و par محیط نباید هیچ هویت Z در بر داشته باشد — پس آن فروکاست این لم را تنها برای سیستم‌هایی اثبات می‌کند که قضیه بدل از پیش می‌پوشاندشان، و ساعت خبردهنده، برای نمونه، بیرونش می‌افتد. و جفت‌سازی مستقیم از آن فروکاست بلندتر نیست و هیچ چیز حمل نمی‌کند: ماشین‌آلات تعریف 4.17 برای یک اجرا که از نو کمانک‌گذاری شده ساخته شده بود، و دو اجرا که روی نوارهای برابر جفت شده‌اند همان استقرا است با تحلیل حالتی آسان‌تر — بی هیچ بازپختی، بی هیچ بازنویسی ادعایی، بی هیچ ردی که هموارش کنیم.

این لم عملاً به چه کار می‌آید. استدلال‌های تقلید میان اشیای ایده‌آل همسایه جهش می‌کنند: یک کارکرد و گونه تئیل‌ترش، یک شبیه‌ساز و همان شبیه‌ساز با یک آزمون حذف‌شده. هر جهش زوجی همسان تا خرابی است، با پرچم‌هایی که همان‌جا برافراشته می‌شوند که آن دو از هم جدا می‌شوند، و این لم بهای آن جهش را $\Pr[\text{Bad}]$ می‌گذارد — در همان اجرا، در برابر همان محیط، و با اشغال‌گرانی که هر جا اثباتی لازم داشته باشد در کنارش جهش می‌کنند. و آنچه می‌ماند کران‌گذاشتن بر $\Pr[\text{Bad}]$ است، و آن عموماً یک ویژگی است: نهایی‌سازی‌ای که حکمش درست همین است که آن شرط

برافراشتن رخ داده، که تعریف 5.8 بهایش را می‌گذارد و، هر جا انتقال اِعمال شود، از راه نتیجه 5.11 به ارث می‌رسد. و نمونه متعارفش در همان کارکرد امضای فصل 15 می‌نشیند. $\mathcal{F}_{\text{Sig}}$ را چنان ابزارگذاری کنید که همان جا که آزمون کلید درستکار Verify شلیک می‌کند bad را برافرازد – افزوده‌ای که هیچ فراخوانده‌ای نمی‌تواند ببیندش، چون آن پرچم نوشته می‌شود و هرگز خوانده نمی‌شود – و در کنارش آن گونه‌ای را بنشانید که آن‌گاه می‌گذارد آن جعل سر پا بماند. آن دو همسان تا خرابی‌اند، و جهش میانشان احتمال اینکه هرگز جعلی ارسال شود را هزینه می‌کند – همان کمیتی که FinUF برای پاسبانی‌اش هست.

پاک سازی

ما بسیار لازم می‌شود که خروجی‌های شبیه‌ساز را پاک‌سازی کنیم، تا حمله‌ها بر کارکردهای ایده‌آل زیادی بد اثر نگذارند. مثلاً در زمینه‌های اکثریت‌درستکار، معمولاً خروجی $\perp$ را به خروجی‌ای غیر $\perp$ پاک‌سازی می‌کنیم تا زنده بودن نگه داشته شود. چنین رویه‌ای را $\text{San}[\text{Clean}](v; c)$ می‌نویسیم، که در آن Clean محمولی است روی یک مقدار به همراه هر زمینه‌ای که مشورتش می‌کند، و خروجی شبیه‌ساز v را با $\text{San}[\text{Clean}](v; c) \leftarrow v'$ عوض می‌کنیم، که جز آنجا که خلافتش گفته شود چنان که در زیر می‌آید کار می‌کند. آن زمینه c پس از نقطه‌ویرگول، فهرست حالت کارکردی است که آن محمول می‌خواند؛ تنهاخواندن است، و محمول‌های مختلف زمینه‌های مختلف می‌گیرند.

پاک‌کننده $\text{San}[\text{Clean}]$

محمول Clean ، مقدار v ، زمینه c

$:\text{San}[\text{Clean}](v; c)$

```

1: if  $\text{Clean}(v; c)$  then
2:   return  $v$ 
3: برگزین را  $\{v' : \text{Clean}(v'; c)\}$ 
4: return  $v'$ 

```

پس پاک‌سازی تنها وقتی عمل می‌کند که $\text{Clean}(v; c)$ شکست بخورد، و آنچه آن‌گاه برمی‌گرداند Clean را برآورده می‌کند.¹ پاک‌کننده‌های ما به بیش از دسترسی تنهاخواندن به حالت کارکرد نیازی ندارند. و اینکه سطر 3 چگونه برمی‌گزیند بخشی از مشخصه است – مقدار برگزیده به فراخواننده می‌رسد، پس قاعده‌های مختلف کارکردهایی مشاهده‌پذیر متفاوتند – و ما در سرتاسر نخستین مقدار خوب به ترتیب واژه‌نگاری را تعیین می‌کنیم. آن انتخاب کم‌اهمیت‌تر از آن است که به نظر می‌رسد: آن برگزیدن

¹ مقدار خوب هر جا هست که فضایی که Clean از آن می‌کشد نامتناهی باشد و هر چه جز نبود عضویت کنار می‌گذارد، مقداری باشد که در زمینه c نامش برده شده. هر دو نیمه برای محمول‌های فصل 15 لازمند: عضویت در K یا Σ نامتناهی عطف اول هر یک است، و هر چیز دیگری که کنار می‌گذارند درایه‌ای از جدولی است که در c داده شده. و زیر تعریف 4.30 آن زمینه در هر اجرایی تکیه‌گاه متناهی دارد – چون اجرای کراندار تعداد متناهی درایه ثبت می‌کند – پس آن مجموعه کنارگذاشته‌شده در درون فضایی نامتناهی متناهی است.

تنها وقتی به سطح می‌آید که پاسخِ خودِ جایگاهِ Clean را برآورده نکند، و حریفی که مقدارِ خوبِ خاصی را می‌پسندد می‌تواند به‌سادگی همان را ارسال کند.

تصادف

13.1 کاربرد

$\mathcal{F}_{\text{Rand}}$ کوچک‌ترین کارکرد این کتاب است و درست همان است که نخست باید خوانده شود. رشته‌های یکنواخت n -بیتی بیرون می‌دهد، آنچه بیرون داده را به یاد می‌آورد، می‌گذارد طرفی درایه‌ای را فراموش کند، و آنچه هنوز به یاد دارد را نشت می‌دهد. فصل 1 نامش برد و کدی بدهکار نبود؛ و اینجا کد است.

دو چیز آن را سزاوار فصلی از خود می‌کند و نه یک پانویس. هیچ فراخوانی اصلاً نمی‌گذارد – نه روی ساعت، نه روی حریف – پس همان یک مثالی است که در آن اصلاً هیچ چیز به جایگاه واگذار نشده، و فصل 9 چیزی برای بها گذاشتن ندارد. و ویژگی‌اش، به‌خلاف هر ویژگی دیگر این کتاب، در 0 حاصل نمی‌شود: تصادف تا احتمالی که آن کران باید نامش را برد پیش‌بینی‌ناپذیر است، و همان جایی است که ε تعریف 5.8 از تزئینی بودن دست می‌کشد.

میدان‌ها. همان الگوی $\mathcal{F}_{\text{Sig}}$. pid ، P و N پارامترند، با یک نمونه به‌ازای هر نشست، و N نام پروتکلی را می‌برد که در الگوی محلی فصل 1 خدمتش می‌کند. $\text{U} := \emptyset$ ، چون هیچ فراخوانی برای گذاشتن نیست. و $\text{par} := n$ ، یعنی پهنای آن رشته‌ها، که Rnd در سطر 5 می‌خواندش.

پاک‌کردن همان است که نشت برایش هست. Leak کلی $\mathbb{L}$ را برمی‌گرداند، پس کل تاریخ کشیدن‌های یک طرف فاسد مال حریف است. و این عیب نیست بلکه نکته داشتن Erase در کنارش است: پروتکلی که تصادفی می‌کشد که نباید نگاهش دارد، آن درایه را پاک می‌کند، و آنچه پاک شده نمی‌تواند نشت کند. اگر این‌گونه خوانده شود، $\mathcal{F}_{\text{Rand}}$ به‌همان اندازه که مدلی از نمونه‌برداری است مدلی از پاک‌کردن امن هم هست، و همان جفت است که چرا انبار فصل 14 به همان شکل ساخته شده. یک پیامد ارزش‌گفتنی رو دارد، چون به‌سادگی می‌شود بد بیان‌ش کرد: تا وقتی هیچ چیز پاک نشده، شمار درایه‌های $\mathbb{L}$ همان شمار بارهایی است که Rnd فرا خوانده شده، پس آن نشت شمار را هم مثل مقدارها گزارش می‌کند؛ و پاک‌کردن درست همان است که درایه‌ای را از آن شمار بیرون می‌برد.

کارکرد $\mathcal{F}_{\text{Rand}}$	
$\text{par} := n$	$\mathcal{U} := \emptyset$
$\mathcal{N}$	$\mathcal{P}$
pid	
id' از <u>id.Erase(i)</u>	<u>:Initialize()</u>
8 $\mathcal{L}[i] \leftarrow \square$	1 $\text{ctr} \leftarrow 0$
9 return ok	2 $\mathcal{L} : \mathcal{N} \rightarrow \{0, 1\}^n \cup \{\square\}$
	3 $\mathcal{L}[*] \leftarrow \square$
id' از <u>id.Leak()</u>	id' از <u>id.Rnd()</u>
10 return $\mathcal{L}$	4 $\text{ctr} \leftarrow \text{ctr} + 1$
	5 $r \leftarrow_{\$} \{0, 1\}^n$ / کتاب این نمونه برداری تنها
	6 $\mathcal{L}[\text{ctr}] \leftarrow r$
	7 return (ctr, r) / بتواند فراخوانده تا اندیس،
	کند پاک

سه نکته در بابِ کد. عمل Rnd همراه آن رشته، اندیسی را هم که نوشته پاسخ می‌دهد. فراخوانده‌ای که قصد پاک کردن یک کشیدن را دارد باید بتواند نامش را ببرد، و ctr حالتِ کارکرد است و نه حالتِ فراخوانده، پس بی سطر 7 آن Erase کنارش را تنها فراخوانده‌ای می‌توانست به کار برد که حاضر به حدس زدن باشد – پروتکل بخش 21.2 هر سکه‌ای که می‌کشد را پاک می‌کند، و همین است که می‌گذاردش. درایه‌ها با یک شمارنده اندیس‌گذاری می‌شوند و هرگز بازنویسی نمی‌شوند، پس Erase به همتایی برای آن تمایز $\mathcal{F}_{\text{Store}}$ میان هرگز-نوشته‌نشده و پاک‌شده نیازی ندارد: اندیسی که Rnd از آن گذشته هرگز از نو به کار نمی‌رود، و $\square$ در اندیسی زیر ctr می‌گوید آن درایه پاک شده در حالی که $\square$ در اندیسی بالای آن می‌گوید شمارنده به آنجا نرسیده. و آن شمارنده بین طرف‌ها مشترک است، چنان که حالتِ هر کارکردی هست: دو طرف که از یک نمونه می‌کشند درایه‌های متمایز می‌کشند، و همین است که $\mathcal{F}_{\text{Rand}}$ را سرچشمه می‌کند و نه مولدی به‌ازای هر طرف.

13.2 ویژگی‌ها

یک ویژگی، FinUnp ، یعنی پیش‌بینی‌ناپذیری: هیچ محیطی از پیش نام مقداری را نمی‌برد که کشیدنی بعدی برمی‌گرداندش. این ویژگی خوانش جست‌وجوی تعریف 5.8 را می‌گیرد. آن بازی روی $\gamma := \{\mathcal{Gm}\} \cup \{\mathcal{F}_{\text{Rand}}\}$ بازی می‌شود با $\mathcal{F}_{\text{Rand}}.\mathcal{N} := \{\mathcal{Gm}.\text{pid}\}$ سنجاق‌شده و $\mathcal{F}_{\text{Rand}}.\mathcal{P} \subseteq \mathcal{Gm}.\mathcal{P}$ ، که باز همان استدلال تنگی بخش 15.2 است و به همان دلیل: کشیدنی که بازی نبیندش، کشیدنی است که حکم نمی‌تواند بر آن داوری کند.

آن پوسته یک واسط حمل می‌کند که بازپخش نیست. عمل Guess هیچ فراخوانی نمی‌گذارد و اصلاً به $\mathcal{F}_{\text{Rand}}$ دست نمی‌زند؛ و هست تا محیط بتواند در سابقه ثبت شود، و دنباله‌بودن آن رد همان است که می‌گذارد حکم بپرسد آیا نخست ثبت شده. و این همان به‌کاربردن ترتیب در بخش 17.2 است که برای هدفی دیگر قرض گرفته شده: پایداری ترتیب پاسخ‌ها را می‌خواند، و پیش‌بینی‌ناپذیری ترتیب یک ادعا در برابر یک پاسخ را.

پوسته بازی روی $\mathcal{F}_{\text{Rand}}$: نهایی سازی در پی می آید

$\mathcal{U} := \{(\mathcal{F}_{\text{Rand}}, \text{serve}_{\text{gm}})\}$, $\mathcal{N} := \mathbb{Z}$, $\mathcal{P} := \mathcal{F}_{\text{Rand}} \cdot \mathcal{P} \cup \{\mathbb{Z}\}$, $\text{pid} := (\text{gm}, \mathcal{F}, 0, 0)$
 $\text{par} := \perp$

```

:13 require  $\neg \text{done}$ 
:14  $\text{tr} \leftarrow \text{tr} \cdot (\text{guess}, \text{id}, \mathcal{P}, y)$  // بی فراخوان؛ بزند حدس می تواند هم ریشه
:15 return ok

:16 return  $\perp$ 

:1 Initialize()
:2  $\text{done} \leftarrow \text{false}$ 
:3  $\text{tr} \leftarrow ()$  // دنباله یک رد،
:4  $\text{id}'$  از  $\text{id}.\text{Rnd}()$ 
:5 require  $\neg \text{done}$ 
:6  $\text{id}'$  از  $\text{id}.\text{Guess}(y)$ 
:7  $\text{tr} \leftarrow \text{tr} \cdot (\text{rnd}, \text{id}, \mathcal{P}, r)$ 
:8  $\text{tr} \leftarrow \text{tr} \cdot (\text{erase}, \text{id}, \mathcal{P}, i)$ 
:9  $\text{tr} \leftarrow \text{tr} \cdot (\text{erase}, \text{id}, \mathcal{P}, i)$ 
:10  $\text{tr} \leftarrow \text{tr} \cdot (\text{erase}, \text{id}, \mathcal{P}, i)$ 
:11  $\text{tr} \leftarrow \text{tr} \cdot (\text{erase}, \text{id}, \mathcal{P}, i)$ 
:12 return ok

:3 require  $\neg \text{done}$ 
:4  $\text{require id.P} \neq \mathbb{Z}$  // می کند نهایی تنها ریشه
:5  $(i, r) \leftarrow \text{id}_{\mathcal{F}_{\text{Rand}}}.\text{FullRnd}()$  as  $\text{id}$  // ثبت اندیس
:6  $\text{tr} \leftarrow \text{tr} \cdot (\text{rnd}, \text{id}, \mathcal{P}, r)$ 
:7 return r

:8 require  $\neg \text{done}$ 
:9  $\text{require id.P} \neq \mathbb{Z}$ 
:10  $\text{id}_{\mathcal{F}_{\text{Rand}}}.\text{FullErase}(i)$  as  $\text{id}$ 
:11  $\text{tr} \leftarrow \text{tr} \cdot (\text{erase}, \text{id}, \mathcal{P}, i)$ 
:12 return ok

```

نهایی سازی gm : پوسته بازی روی $\mathcal{F}_{\text{Rand}}$ مشترک است

```

:17 require  $\neg \text{done}$ 
:18  $\text{require id.P} = \mathbb{Z}$ 
:19  $\text{done} \leftarrow \text{true}$ 
:20  $\text{tr}$  را به صورت  $(e_j)_{j=1}^m$  کن تجزیه
:21  $w \leftarrow 1$  اگر  $\exists j < k \exists y : e_j = (\text{guess}, \cdot, y) \wedge e_k = (\text{rnd}, \cdot, y)$ 
:22  $w \leftarrow 0$  وگرنه
:23 return w

```

دو چیز در باب کد. بازپخش ها ریشه را رد می کنند و Guess نمی کند، و آن ناهمسانی درست ناهمسانی بخش 17.2 است: بازپختی در ریشه طرفی را خطاب می کرد که $\mathcal{F}_{\text{Rand}}$ خدمتش نمی دهد، در سطر 1 rej پاسخ می گرفت، و $(\text{rnd}, \mathbb{Z}, \text{rej})$ را به آن رد می افزود — که آن گاه محیطی که rej را حدس زده بود رو می بُرد، بی آنکه چیزی پیش بینی کرده باشد. و چون در بازپخش رد می شود، هیچ چیز افزوده نمی شود، و هر درایه rnd رشته ای را حمل می کند که $\mathcal{F}_{\text{Rand}}$ واقعاً کشیده. عمل Guess هیچ فراخوانی نمی گذارد، پس چیزی ندارد که برایش رد شود و همه جا خدمت می شود، ریشه هم در آن. و آن حکم از هیچ کس درستکاری نمی خواهد: در طرف فاسد، پوشانه خود بازی پیش از دویدن هسته می ربايد (نکته 5.4)، پس هر درایه tr از پیش درایه هسته ای درستکار است، درست همان گونه که در بخش 17.2.

گزاره 13.1 ($\mathcal{F}_{\text{Rand}}$ پیش‌بینی‌ناپذیر است). بگذارید $\mathcal{Z}$ دست‌بالا q_g فراخوان روی Guess و دست‌بالا q_r فراخوان روی Rnd بگذارد. آنگاه به‌ازای هر اشغال‌گر $\mathcal{X}$ از جایگاه حریف،

$$\Pr[\text{Win}_{\text{FinUnp}}] \leq q_g q_r 2^{-n}.$$

پس $\{\mathcal{F}_{\text{Rand}}\}$ پیش‌بینی‌ناپذیری را در $q_g q_r 2^{-n}$ و در برابر $(\mathbb{Z}, \mathbb{X})$ دارد، برای $\mathcal{Z}$ که هر رده‌ای است که آن شماره‌ها را برآورده می‌کند و $\mathbb{X}$ که رده همه اشغال‌گران است.

اثبات. k را با $e_k = (\text{rnd}, \cdot, r)$ ثابت کنید. بنا بر تنگی، هیچ فراخواننده‌ای جز $\mathcal{G}_m$ به $\mathcal{F}_{\text{Rand}}$ نمی‌رسد، و $\mathcal{G}_m$ تنها از راه بازپخش Rnd آن پوسته (سطر 5) به آن می‌رسد، پس r همان مقداری است که Rnd خود $\mathcal{F}_{\text{Rand}}$ در سطر 5 کشید: یکنواخت روی $\{0, 1\}^n$ و مستقل از هر چه آن اجرا پیش از آن کرده بود، چون $\mathcal{F}_{\text{Rand}}$ هیچ فراخوانی نمی‌گذارد و برای کشیدنش هیچ حالتی نمی‌خواند.

روی آن اجرا تا لحظه پیش از آن کشیدن شرط بگذارید. هر درایه guess با $k < j$ با آن تاریخ تعیین می‌شود، و دست‌بالا q_g تای آن‌ها هست، پس احتمال اینکه r با هر یک از مقدارهایشان برابر باشد دست‌بالا $q_g 2^{-n}$ است. و دست‌بالا q_r انتخاب برای k هست، و حکم سطر 21 تنها وقتی 1 برمی‌گرداند که زوجی (j, k) جور باشد، پس کران اجتماعی روی k همان $q_g q_r 2^{-n}$ را می‌دهد.

دو جزء که این کران بر آن‌ها تکیه دارد. اشغال‌گر جایگاه پیدا نمی‌شود: هسته‌های $\mathcal{F}_{\text{Rand}}$ هیچ فراخوانی نمی‌گذارند، پس هیچ کاری که $\mathcal{X}$ می‌کند به آن نمونه‌برداری نمی‌رسد، و همان کران در برابر هر اشغال‌گری بی هیچ استدلال جذبی برقرار است. و حدسی که پس از یک کشیدن زده شود بنا بر ساخت بی‌ارزش است، چون آن حکم $k < j$ را می‌خواهد؛ محیطی که r را می‌خواند و سپس حدسش می‌زند چیزی پیش‌بینی نکرده، و همین است که ترتیب آن رد را محتوای این حکم می‌کند. $\square$

این کران نخستین کران این کتاب است که 0 نیست، و ارزش گفتن دارد که این چه چیزی را تغییر می‌دهد. نتیجه 5.11 در هر دو حال یکسان خوانده می‌شود: سیستم π که به جای $\{\mathcal{F}_{\text{Rand}}\}$ نشانه‌ش شود پیش‌بینی‌ناپذیری را زیر خوانش جست‌وجو در $\epsilon + q_g q_r 2^{-n}$ دارد، و کران ایده‌آل و خطای تقلید به‌سادگی جمع می‌شوند. آنچه تغییر می‌کند این است که سوی ایده‌آل دیگر مفت نیست – یک تحقق کرانی به ارث می‌برد که از پیش نانهی بود، و آن دو جمله از دو گونه‌اند: یکی مال خود آن نمونه‌برداری است، که هیچ پروتکلی بهتر از آن نمی‌کند، و دیگری هزینه همان تحقق است. و خود انتقال به ارزان‌ترین شکل ممکن است: هسته‌های $\mathcal{F}_{\text{Rand}}$ هیچ فراخوانی نمی‌گذارند، پاسخگو یا جز آن، پس هشدار نکته 9.4 هرگز پیش نمی‌آید و قضیه 4.29 دست‌نخورده است، درست همان‌گونه که برای $\mathcal{G}_{\text{PKI}}$ (بخش 16.2).

ذخیره‌سازی

14.1 کارکرد

$\mathcal{F}_{\text{Store}}$ همتای $\mathcal{F}_{\text{Rand}}$ است: آنجا که یکی مقدار تولید می‌کند و می‌گذارد فراموش شوند، دیگری مقدار می‌گیرد و نگه‌شان می‌دارد تا فراموش شوند. طرفی مقداری را در اندیسی می‌گذارد، پسش می‌گیرد، و پاکش می‌کند؛ و آن نشت گزارش می‌دهد چه چیزی هنوز نگه داشته شده. آن دو با هم همان چیزی هستند که فصل 1 از نگه‌نداشتن هیچ حالتی روی نوار ماشین به‌دست یک پروتکل مراد می‌کند – هر تصادفی از راه $\mathcal{F}_{\text{Rand}}$ ، هر پایایی از راه $\mathcal{F}_{\text{Store}}$ ، و هر یک آنچه نگه می‌دارد را از راه Leak خودش رو می‌کند.

میدان‌ها. درست همان‌های $\mathcal{F}_{\text{Rand}}$ ، جز آن پارامتر: pid ، $\mathbf{P}$ و $\mathbf{N}$ پارامترند، $\mathbf{U} := \emptyset$ ، و $\perp := \text{par}$ ، چون انبار هیچ پارامتری نمی‌خواند. و مثل حالت $\mathcal{F}_{\text{Rand}}$ ، حالتش بین طرف‌هایی که خدمتشان می‌دهد مشترک است – طرفی می‌تواند آنچه طرفی دیگر گذاشته را بگیرد، و همین است که آن را انبار می‌کند و نه نواری به‌ازای هر طرف، و همین است که فصل 1 از نکته اصلی بودن اشتراک مراد می‌کند.

سه حالت، نه دو. اندیس در یکی از سه وضع است، و کد عمداً از هم جدایشان نگه می‌دارد. اگر هرگز چیزی آنجا گذاشته نشده باشد $\square$ است؛ اگر چیزی گذاشته و پاک نشده باشد مقداری در بر دارد؛ و اگر گذاشته و سپس پاک شده باشد $\perp$ است. عمل **Put** اندیس را در هر دو وضع تهی می‌پذیرد، پس اندیس پاک‌شده باز به کار می‌آید؛ و **Get** تنها $\square$ را رد می‌کند، پس گرفتن از اندیسی پاک‌شده به‌جای رد شدن $\perp$ پاسخ می‌گیرد. آن تمایز جای خود را در تفاوت همان دو پاسخ به‌دست می‌آورد: فراخوانده می‌تواند هرگز چیزی اینجا نبوده را از چیزی اینجا بود و رفته تشخیص دهد، و همین است که پاک‌کردن را برای پروتکلی که انجامش داده مشاهده‌پذیر می‌کند و نه حفره‌ای بی‌صدا. انباری که آن تفاوت را پنهان می‌کرد هر دو را یکسان رد می‌کرد و نمی‌شد از او پرسید که آیا پاک‌کردنی رخ داده.

کارکرد $\mathcal{F}_{\text{Store}}$	
$\text{par} := \perp$	$\text{U} := \emptyset$
N	P
pid	
id' از id.Get(i) می‌کند رد نوشته نشده هرگز / باشد شده پاک اگر $\perp$:8 require $\text{L}[i] \neq \square$:9 return $\text{L}[i]$	id' از id.Put(i, x) رد اشغال شده اندیس / می‌کند :3 require $\text{L}[i] \in \{\perp, \square\}$:4 $\text{L}[i] \leftarrow x$:5 return ok
id' از id.Leak() :10 return L	id' از id.Erase(i) :6 $\text{L}[i] \leftarrow \perp$:7 return ok

دو نکته در باب کد. آن ردها به صورت **require** نوشته شده‌اند و نه به صورت بازگرداندن **rej**، که همان چیز است که به واژه‌های خود چارچوب گفته شده: بخش 3.6 به **require** رده شده **rej** پاسخ می‌دهد، پس سطرهای 3 و 8 درست همان دو ردی را می‌دهند که انبار بدهکار است، و هیچ هسته این کتاب **rej** را با نوشتنش برنمی‌گرداند. و **Erase** هیچ ردی نمی‌کند: پاک کردن اندیسی که هیچ در بر ندارد بی‌عملی است که **ok** پاسخ می‌دهد، چون پروتکلی که تدافعی پاک می‌کند نباید ناچار باشد بداند که آنجا چیزی بوده یا نه.

14.2 ویژگی‌ها

دو ویژگی، و آن دو، دو نیمه آن چیزی هستند که انبار برایش هست. **FinKeep**، یعنی پایایی: مقداری که گذاشته و پاک نشده، همان مقداری است که پس گرفته می‌شود. و **FinGone**، یعنی پاک شدن: مقداری که پاک شده پس گرفته نمی‌شود. هر دو خوانش جست‌وجو را می‌گیرند، و هر دو نهایی‌سازی از یک بازی‌اند – همان یک رد همان سه بازپخش را می‌خوانند و تنها در حکم تفاوت دارند، که همان آزمونی است که بخش 17.2 برای شریک شدن در یک پوسته می‌دهد.

این دو روی $\mathcal{F}_{\text{Store}} \cdot \text{N} := \{\text{gm.pid}\}$ بازی می‌شوند با $\gamma := \{\text{gm}\} \cup \{\mathcal{F}_{\text{Store}}\}$ با تنگی مثل پیش و به همان دلیل: گذاشتنی که سنجاق شده و $\mathcal{F}_{\text{Store}} \cdot \text{P} \subseteq \text{gm.P}$ با تنگی می‌تواند حسابش کند. و بازپخش‌ها ریشه را رد می‌کنند، به دلیل بخش 13.2: هر دو حکم مقدار گرفته شده را در برابر مقدار گذاشته شده می‌سنجند، پس **rej**ی که خود بازی ساخته باشد اگر در آن رد بیفتد به عنوان یک مقدار خوانده می‌شد و می‌توانست هر یک را ببرد.

پوسته بازی روی $\mathcal{F}_{\text{Store}}$ ؛ نهایی‌سازی‌ها در پی می‌آیند

$\mathcal{U} := \{(\mathcal{F}_{\text{Store}}, \text{serve}_{\text{gm}})\}$ $\mathcal{N} := \mathbb{Z}$ $\mathcal{P} := \mathcal{F}_{\text{Store}} \cdot \mathcal{P} \cup \{\mathbb{Z}\}$ $\text{pid} := (\text{gm.F}, 0, 0)$
 $\text{par} := \perp$

<pre> :13 require $\neg \text{done}$:14 require $\text{id.P} \neq \mathbb{Z}$:15 $y \leftarrow \text{id}_{\mathcal{F}_{\text{Store}}}.\text{FullGet}(i)$ as id :16 $\text{tr} \leftarrow \text{tr} \cdot (\text{get}, \text{id.P}, i, y)$:17 return y :18 return $\perp$ </pre>	<pre> :1 done $\leftarrow$ false :2 $\text{tr} \leftarrow ()$:3 require $\neg \text{done}$:4 require $\text{id.P} \neq \mathbb{Z}$:5 $a \leftarrow \text{id}_{\mathcal{F}_{\text{Store}}}.\text{FullPut}(i, x)$ as id :6 $\text{tr} \leftarrow \text{tr} \cdot (\text{put}, \text{id.P}, i, x, a)$:7 return a :8 require $\neg \text{done}$:9 require $\text{id.P} \neq \mathbb{Z}$:10 $\text{id}_{\mathcal{F}_{\text{Store}}}.\text{FullErase}(i)$ as id :11 $\text{tr} \leftarrow \text{tr} \cdot (\text{erase}, \text{id.P}, i)$:12 return ok </pre>
--	--

نهایی‌سازی‌های gm ؛ پوسته روی $\mathcal{F}_{\text{Store}}$ مشترک است

```

:19 require  $\neg \text{done}$ 
:20 require  $\text{id.P} = \mathbb{Z}$ 
:21 done  $\leftarrow$  true:  $\text{tr}$  را به صورت  $(e_j)_{j=1}^m$  کن تجزیه
:22  $w \leftarrow 1$  اگر  $\exists j < k \exists i, x, y: e_j = (\text{put}, \cdot, i, x, \text{ok}) \wedge e_k = (\text{get}, \cdot, i, y)$ 
:23  $\wedge y \neq x \wedge \neg \exists l \in (j, k): e_l = (\text{erase}, \cdot, i)$ ؛
:24 وگرنه  $w \leftarrow 0$ 
:25 return w

:26 require  $\neg \text{done}$ 
:27 require  $\text{id.P} = \mathbb{Z}$ 
:28 done  $\leftarrow$  true:  $\text{tr}$  را به صورت  $(e_j)_{j=1}^m$  کن تجزیه
:29  $w \leftarrow 1$  اگر  $\exists j < k < l \exists i, x: e_j = (\text{put}, \cdot, i, x, \text{ok})$ 
:30  $\wedge e_k = (\text{erase}, \cdot, i) \wedge e_l = (\text{get}, \cdot, i, x)$ 
:31  $\wedge \neg \exists l' \in (k, l): e_{l'} = (\text{put}, \cdot, i, \cdot, \text{ok})$ ؛
:32 وگرنه  $w \leftarrow 0$ 
:33 return w

```

حکم‌ها، به زبان واژه. نهایی‌سازی FinKeep مقدار 1 برمی‌گرداند اگر گرفتن چیزی جز مقداری پاسخ داده باشد که آخرین گذاشتن موفق در آن اندیس آنجا گذاشته، بی‌آنکه پاک‌کردنی در میان آمده باشد. و نهایی‌سازی FinGone مقدار 1 برمی‌گرداند اگر گرفتن درست همان مقداری را پاسخ داده باشد که پاک‌کردنی قرار بود برداردش، بی‌آنکه گذاشتنی در میانه بازش گردانده باشد. هر دو ترتیب آن رد را می‌خوانند، چنان

که پایداری بخش 17.2 می‌خواند و به همان دلیل: انبار ادعایی است در باب آنچه میان یک فراخوان و فراخوان بعدی رخ می‌دهد، و «میان» یک ترتیب است.

گزاره 14.1 ($\mathcal{F}_{\text{Store}}$ هر دو ویژگی را دارد). برای Fin که هر یک از FinKeep و FinGone باشد، به‌ازای هر $\gamma, \gamma' \in \mathbb{Z}$ و هر اشغال‌گر $\mathcal{X}$ از جایگاه حریف، $\text{Pr}[\text{WinFin}] = 0$. پس $\{\mathcal{F}_{\text{Store}}\}$ هر ویژگی را در 0 و در برابر $(\mathbb{Z}_{\gamma, \gamma'}, \mathbb{X})$ به معنای تعریف 5.8 دارد، برای $\mathbb{X}$ که ردهٔ همهٔ اشغال‌گران است.

اثبات. یک ناوردا هر دو ویژگی را حمل می‌کند: $\mathbb{L}[i]$ تنها در درون فراخوانی تغییر می‌کند که آن رد ثبتش می‌کند، و تنها به مقداری که آن فراخوان نامش می‌برد. و این از تنگی می‌آید (هیچ فراخواننده‌ای جز بازی از نگهبان نمی‌گذرد)، از ربودن پوشانهٔ خود بازی در طرف فاسد پیش از دودیدن هر هسته‌ای، و از اینکه هسته‌های $\mathcal{F}_{\text{Store}}$ هیچ فراخوانی نمی‌گذارند، پس میان فراخوان یک بازپخش و بازگشتش هیچ چیز نمی‌دود و دریاها به ترتیب اجرا افزوده می‌شوند. آن‌گاه هر دو ویژگی با همان یک حرکت از آن خوانده می‌شوند: عطف‌های آن حکم را مفروض بگیرید، و پی بگیرید که چه سطرهایی می‌توانستند $\mathbb{L}[i]$ را میان دریاها نام‌برده شده بنویسند. پایایی بر ردکردن اندیس اشغال‌شده به‌دست گذاشتن می‌گردد، پس هیچ گذاشتن بعدی نمی‌تواند ثابت در میان بیاید؛ و پاک‌شدن بر این می‌گردد که $\{0, 1\}^*$ نه $\perp$ را در بر دارد و نه $\square$ ، پس مقداری که گرفتنی پس از پاک‌کردنی برمی‌گرداند نمی‌تواند همان مقداری باشد که پیش از آن گذاشته شده بود. همه چیز بر یک ناوردا تکیه دارد: $\mathbb{L}[i]$ تنها در درون فراخوانی تغییر می‌کند که آن رد ثبتش می‌کند، و تنها به مقداری که آن فراخوان نامش می‌برد. بنا بر تنگی، هیچ فراخواننده‌ای جز $\mathcal{G}_m$ از سطر 2 Guard در $\mathcal{F}_{\text{Store}}$ نمی‌گذرد، و $\mathcal{G}_m$ تنها از راه سه بازپخش آن پوسته به آن می‌رسد (سطرهای 5، 11 و 15)؛ و در طرف فاسد، پوشانهٔ خود بازی یک گام زودتر می‌رباید و اصلاً هیچ هسته‌ای از γ نمی‌دود (نکته 5.4). هسته‌های $\mathcal{F}_{\text{Store}}$ هیچ فراخوانی نمی‌گذارند، پس هیچ ماشین دیگری از γ میان فراخوان یک بازپخش و بازگشتش نمی‌دود، و دریاها به ترتیب اجرا افزوده می‌شوند. و شمارهٔ سطرهای زیر، جز آنجا که خلافش گفته شود، نام هسته‌های خود $\mathcal{F}_{\text{Store}}$ را می‌برند.

برای پایایی: فرض کنید $e_j = (\text{put}, \cdot, i, x, \text{ok})$ و $e_k = (\text{get}, \cdot, i, y)$ با $k < j$ و بی هیچ erase در i در میانه. آن گذاشتن ok پاسخ داد، پس سطر 3 Put گذشت و سطر 4 آن $x \leftarrow \mathbb{L}[i]$ را نشاند. تنها همان سطر و سطر 6 Erase $\mathbb{L}[i]$ را می‌نویسند، و بنا بر آن ناوردا هر یک ثبت شده است: هیچ دریأ erase در i در میانه نیست، و put دیگری در i که ok پاسخ دهد هم نمی‌تواند در میانه باشد، چون سطر 3 اندیس اشغال‌شده را رد می‌کند و $\mathbb{L}[i]$ از z به این‌سو اشغال است. پس $\mathbb{L}[i] = x \neq \square$ هنوز در k برقرار است، سطر 8 Get می‌گذرد، و سطر 9 آن x را برمی‌گرداند؛ پس $y = x$ و حکم سطر 22 مقدار 0 می‌دهد.

$e_l = \text{and}$ و $e_k = (\text{erase}, \cdot, i)$ ، $e_j = (\text{put}, \cdot, i, x, \text{ok})$ suppose erasure: For l and k between i at put successful no and $j < k < l$ with $(\text{get}, \cdot, i, x)$ that line only the invariant the by and k at $\mathbb{L}[i] \leftarrow \perp$ set 6 line 'sErase answering i at put recorded a inside 4 line 'sPut is value a restore could

9 line 'sGet and $\perp$ at $\perp[i] = \perp$ So excludes. hypothesis the which ok argument. 'sPut a from 4 line by written was $x : x \neq \perp$ Now $\perp$ returns x answer not did get the Hence $\square$ nor $\perp$ neither contains $\{0, 1\}^*$ and $\square$.0 gives 29 line at verdict the and e_1 to contrary

beyond discharge to nothing with 5.11 Corollary by transfer Both the so all. at call no place cores 's $\mathcal{F}_{\text{Store}}$ conditions: standing four its complete- dummy and way cheap the met is hypothesis responsive-call functionalities three The $\mathcal{G}_{\text{PKI}}$ and $\mathcal{F}_{\text{Rand}}$ for as exactly untouched. is ness pat- the and nothing. pay that three the are nothing adversary the ask that the is clock the charges 17.2 Section what – coincidence a not is tern bill. no has notification no with functionality a and notification.

امضاهاى دیجیتال

15.1 کارکرد

own Canetti's of tradition the in signatures, digital idealizes below $\mathcal{F}_{\text{Sig}}$ not and parameter a is id process Its [6] functionality signature ideal subscript the what is that – fixed is component name The constant, a and session the but – 1 Chapter of convention naming the by records, several and session per copy a hold may system one so not, are instance Def- one and handle, to exists composition situation the session, a within The pinned, were id process whole the if outright forbid would 1.2 inition any at read be it lets 7 Chapter so component, neither reads below code global any which, 7.2 Definition of condition the meeting $\mathbf{N}$ an for – other abbreviates $\mathcal{A}(x)$ that 3.4 Section from Recall does, one local purely or ideal the in served; being party the of behalf on adversary the to call a here them of one every and calls, those answers simulator the execution use, before 12 Chapter of sense the in sanitized is

throughout, same the is style The **functionalities. ideal write we How** pin not does specification the and produced be to has value a Wherever fly, the on choice the sanitize and it choose simulator the let we down, it goal security the as freedom much as have to meant then is simulator The goal the and – theorem a not intention, design a – more no and permits code, the in buried than rather predicates the in visible stays correct- protect two first the and times, three happens this $\mathcal{F}_{\text{Sig}}$ In adversary, the from key verification the takes **Gen** operation The ness, issued already key every from distinct key, actual an it makes Clean_{vk} and generation so it, under valid recorded already signature any of free and oper- The pre-forged, arrive nor collide neither keys and succeeds always not signature actual an it makes Clean_σ and signature, the takes **Sign** ation been has key a whenever succeeds signing so invalid, recorded already

asks and that, for asks Locality verifies, produces it what and generated, elsewhere, token the passing without answer simulator the that addition in The code, the of property a than rather class simulator the on condition a outside verdict a instead: unforgeability for sanitized is **Verify** operation key a under triple fresh a on 1 of verdict any does so and, 0 becomes $\{0, 1\}$ unforgeable, strongly signatures makes which generated, party honest an rather inline written is sanitization the so unique, is value good the There so authenticated, assumed not are keys Verification, **San** through than own, its of vk a takes Verify

ad- the hands functionality, ideal an of requires 3.2 Section which, Leak recorded signatures the with together key verification party's the versary fixes Initialize operation The state, the of else nothing and it, under valid **Ver** and party served per key one holds **vk** types: their and tables two the ev- starting both signature, and message key, of triple each on verdict the recorded a from decided" yet "not separates that value the, $\perp$ at erywhere infinite : $\perp$ nor $\perp$ neither contain and infinite are Σ and $\mathcal{K}$ Both, 1 or 0 two the of clean and values, good of out run never sanitizers the that so them, refuses already membership that so markers

to restricted be must class simulator the wanted, is locality Where placing without call simulator each answering those ones: responsive the are calls simulator and once at returns token the so own, their of call any enforces 9 Chapter discipline the — calls coroutine than rather subroutine the nothing, costs this signatures stateless non-interactive, For code, in already, responsive being simulators usual

Sec- re-entrance, against guards **Verify** and **Gen** in line further One operations both and re-entered, be instance suspended a lets 3.6 tion writing and entry table a testing between call adversary an at suspend recorded value a and resumption, on entry the re-tests therefore each it: two issue could calls nested two re-test the Without wins, meanwhile con- very the — triple one on verdicts different return or party, one to keys record, to exists **Ver** sistency

$\mathcal{F}_{\text{Sig}}$ کارکرد	
$\text{par} := \perp$, $\mathcal{U} := \{(\mathcal{A}, \text{ serves})\}$, $\mathcal{N}$, $\mathcal{P}$, pid	
id' از <u>id.Verify(vk, msg, σ)</u> <pre> :20 if $\text{Ver}[\text{vk}, \text{msg}, \sigma] \neq \square$ then :21 return $\text{Ver}[\text{vk}, \text{msg}, \sigma]$:22 $b \leftarrow \mathcal{A}(\text{id}. \text{Verify}, \text{vk}, \text{msg}, \sigma)$:23 if $\text{Ver}[\text{vk}, \text{msg}, \sigma] \neq \square$ then :24 return $\text{Ver}[\text{vk}, \text{msg}, \sigma]$ / شد ثبیت تعلیق حال در :25 if $b \notin \{0, 1\}$ then :26 $b \leftarrow 0$:27 if $\exists P' \notin \mathcal{C} : \text{vk} = \text{vk}[P']$ then :28 $b \leftarrow 0$ / درستکار کلید زیر جعلی هیچ :29 $\text{Ver}[\text{vk}, \text{msg}, \sigma] \leftarrow b$:30 return b id' از <u>id.Gen()</u> :31 if $\text{vk}[\text{id}.P] = \square$ then :32 return $(\perp, \emptyset)$:33 return $(\text{vk}[\text{id}.P],$ { $(\text{msg}, \sigma) : \text{Ver}[\text{vk}[\text{id}.P], \text{msg}, \sigma] = 1$ }) :Clean_{vk}(vk, vk, Ver) :34 return $\text{vk} \in \mathcal{K} \wedge \neg \exists P : \text{vk}[P] = \text{vk}$:35 $\wedge \neg \exists \text{msg}, \sigma : \text{Ver}[\text{vk}, \text{msg}, \sigma] = 1$:Clean_{σ}(σ; msg, vk, Ver) :36 return $\sigma \in \Sigma \wedge \text{Ver}[\text{vk}, \text{msg}, \sigma] \neq 0$</pre>	<pre> :1 $\text{vk} : \mathcal{F}_{\text{Sig}}.\mathcal{P} \rightarrow \mathcal{K} \cup \{\square\}$:2 $\text{vk}[*] \leftarrow \square$:3 $\text{Ver} : \mathcal{K} \times \mathcal{M} \times \Sigma \rightarrow \{0, 1\} \cup \{\square\}$:4 $\text{Ver}[*], [*] \leftarrow \square$ id' از <u>id.Gen()</u> :5 if $\text{vk}[\text{id}.P] \neq \square$ then :6 return $\text{vk}[\text{id}.P]$:7 $\text{vk} \leftarrow \mathcal{A}(\text{id}. \text{Gen})$:8 if $\text{vk}[\text{id}.P] \neq \square$ then :9 return $\text{vk}[\text{id}.P]$ / شد ثبیت تعلیق حال در :10 $\text{vk} \leftarrow \text{San}[\text{Clean}_{\text{vk}}](\text{vk}; \text{vk}, \text{Ver})$:11 $\text{vk}[\text{id}.P] \leftarrow \text{vk}$:12 return vk id' از <u>id.Sign(msg)</u> :13 $\text{vk} \leftarrow \text{vk}[\text{id}.P]$:14 if $\text{vk} = \square$ then :15 return $\perp$:16 $\sigma \leftarrow \mathcal{A}(\text{id}. \text{Sign}, \text{msg})$:17 $\sigma \leftarrow \text{San}[\text{Clean}_{\sigma}](\sigma; \text{msg}, \text{vk}, \text{Ver})$:18 $\text{Ver}[\text{vk}, \text{msg}, \sigma] \leftarrow 1$:19 return σ</pre>

15.2 ویژگی‌ها

one give We instance. an worth is shape a and shape. a is 5.1 Definition the what that saying ,FinCor properties: two carrying $\mathcal{F}_{\text{Sig}}$ over $\mathcal{G}_m$ game else nothing that saying ,FinUF and verifies, produces interface signing which games. two than rather shell one of finalizations two are They does. read they — for ask two these what and allows now shape the what is The alone. verdict the in differ and oracles, same the of trace. same the and call. to which choosing by on judged is it which chooses environment other. the against game the closes one calling

5.8 Definition of reading search the take Both

$$\text{Adv}_{\mathcal{G}_m, \mathcal{F}_{\text{Sig}}, \mathcal{Z}, \mathcal{X}}^{\text{FinCor}} := \Pr[\text{Win}_{\text{FinCor}}],$$

$$\text{Adv}_{\mathcal{G}_m, \mathcal{F}_{\text{Sig}}, \mathcal{Z}, \mathcal{X}}^{\text{FinUF}} := \Pr[\text{Win}_{\text{FinUF}}],$$

better no merely not and impossible be to meant is either winning because is time the half succeed forgeries whose scheme signature a coin: a than

average. not broken.

with $\gamma := \{\mathcal{G}_m\} \cup \{\mathcal{F}_{\text{Sig}}\}$ system game the over played are They and – name its not pinned. id , process game's the – $\mathcal{F}_{\text{Sig}}.\mathbf{N} := \{\mathcal{G}_m.\text{pid}\}$ the at and, serves $\mathcal{F}_{\text{Sig}}$ party every at exist relays the so $\mathcal{F}_{\text{Sig}}.\mathbf{P} \subseteq \mathcal{G}_m.\mathbf{P}$ the – 1 line at rej answered are and serve not does it party a address root. interface an Inside plays. it where not finalizes. game the where being root party the at $\mathcal{F}_{\text{Sig}}$ of interface the $(\mathcal{F}_{\text{Sig}}.\text{pid}, \text{id}.P)$ for $\text{id}_{\mathcal{F}_{\text{Sig}}}$ write we id at 5.1 Definition which $\{\mathcal{G}_m\}$ at tightness is That at. called was game the game legal a γ makes It over. twice keep its earns it here and requires. 2 line passes $\mathcal{G}_m$ but caller No complete. trace the makes it and system: out- lie names fresh and -identitiesZ whose environment. the not $\mathcal{F}_{\text{Sig}}$ at not and -identity:A an is claim whose adversary. the not $\mathbf{N}$ pinned a side game's the of instance invented an at hosts environment the machine a every So refuses. name the than rather id the pinning what is which name. A recorded. is and below 8 line through passes issues ever $\mathcal{F}_{\text{Sig}}$ signature interface" signing the from obtained not was "this is verdict whose game know. to way other no has

پوسته بازی روی $\mathcal{F}_{\text{Sig}}$ ؛ نهایی سازی ها در پی می آیند

$\text{par} := \perp$, $\mathbf{U} := \{(\mathcal{F}_{\text{Sig}}, \text{serve}_{\mathcal{G}_m})\}$, $\mathbf{N} := \mathbf{Z}$, $\mathbf{P} := \mathcal{F}_{\text{Sig}}.\mathbf{P} \cup \{\mathbf{Z}\}$, $\text{pid} := (\mathcal{G}_m.F, 0, 0)$

<pre> id' از id.Verify(vk, msg, σ) :11 require ¬done :12 b ← id_{sig}.FullVerify(vk, msg, σ) asid :13 tr ← tr · (ver, id.P, vk, msg, σ, b) :14 return b id' از id.Leak() :15 return ⊥ </pre>	<pre> :1 done ← false :2 tr ← () id' از id.Gen() :3 require ¬done :4 vk ← id_{sig}.FullGen() as id :5 tr ← tr · (gen, id.P, vk) :6 return vk id' از id.Sign(msg) :7 require ¬done :8 σ ← id_{sig}.FullSign(msg) as id :9 tr ← tr · (sign, id.P, msg, σ) :10 return σ </pre>
--	---

shared is $\mathcal{F}_{\text{Sig}}$ over shell the $\mathcal{G}_{\text{m}}$ of finalizations The

```

                                (correctness)  id' from  id.FinCor()
:16 require  $\neg \text{done}$ 
:17 require  $\text{id}.P = Z$ 
:18  $\text{done} \leftarrow \text{true}$ 
:19  $\mathbf{C} \leftarrow (\mathbf{C}, Z).\text{FullStatus}()$  as id
:20  $w \leftarrow 1$  if  $\exists P, Q, vk, \text{msg}, \sigma : P \notin \mathbf{C} \wedge Q \notin \mathbf{C} \wedge \sigma \in \Sigma$ 
:21    $\wedge (\text{gen}, P, vk) \in \text{tr} \wedge (\text{sign}, P, \text{msg}, \sigma) \in \text{tr} \wedge (\text{ver}, Q, vk, \text{msg}, \sigma, 0) \in \text{tr}$ ;
:22 else  $w \leftarrow 0$ 
:23 return  $w$ 

                                (unforgeability)  id' from  id.FinUF()
:24 require  $\neg \text{done}$ 
:25 require  $\text{id}.P = Z$ 
:26  $\text{done} \leftarrow \text{true}$ 
:27  $\mathbf{C} \leftarrow (\mathbf{C}, Z).\text{FullStatus}()$  as id
:28  $w \leftarrow 1$  if  $\exists P, Q, vk, \text{msg}, \sigma : P \notin \mathbf{C} \wedge Q \notin \mathbf{C}$ 
:29    $\wedge (\text{gen}, P, vk) \in \text{tr} \wedge (\text{ver}, Q, vk, \text{msg}, \sigma, 1) \in \text{tr} \wedge (\text{sign}, P, \text{msg}, \sigma) \notin \text{tr}$ ;
:30 else  $w \leftarrow 0$ 
:31 return  $w$ 

```

oc- means $\in$ sequence. a is trace The code. the about things Four
 and tr read verdicts The order. the reads verdict no and it. in currence
 serves $\mathcal{G}_{\text{m}}$ in exemption root the why is which nothing. call and register the
 rather end the at $P \notin \mathbf{C}$ ask Both .Z serve not need $\mathcal{F}_{\text{Sig}}$ and suffices
 that makes (3.5 (Section monotonicity and signing. of time the at than
 honest was closes game the when honest party a reading: stronger the
 no where $\perp$ answers Sign because $\sigma \in \Sigma$ asks Correctness throughout.
 func- the against hold to signature no is $\perp$ and generated. been has key
 the not triple. the excludes 28 line :strong is unforgeability And tionality.
 forgery a as counts message signed a on signature second a so message.
 give. to written was sanitizer 's15 Chapter notion the is which —

گزاره 15.1 $\mathcal{F}_{\text{Sig}}$ both has $\mathcal{F}_{\text{Sig}}$.properties) both has $\mathcal{F}_{\text{Sig}}$.FinUF and FinCor of either Fin For
 slot. adversary the of $\mathcal{X}$ occupant every and $\mathcal{Z} \in \mathcal{Z}_{\gamma, \gamma}$ every for
 $(\mathcal{Z}_{\gamma, \gamma}, \mathcal{X})$ against 0 within property each has $\{\mathcal{F}_{\text{Sig}}\}$ So . $\Pr[\text{Win}_{\text{Fin}}] = 0$
 occupants. all of class the $\mathcal{X}$ for 5.8 Definition of sense the in

اثبات. One preliminary and two. The verdicts. both that is preliminary
 a at bookkeeping: not is it and honest. be must verdict a by named ties
 so adversary's. the is answer the and run not does core the party corrupt
 forgery. a on 1 a or signature good a on 0 a report could Verify mediated a

Cor- wrong. nothing done having $\mathcal{F}_{\text{Sig}}$ with fire would finalization either and Verify and $\text{ver}[\text{vk}, \text{msg}, \sigma] \leftarrow 1$ recording Sign from follows then rectness whether on splits Unforgeability line. first its at entry recorded a returning under 1 a computed have cannot it one: computed or entry an read Verify records Sign only and recorded. was entry the so key. party's honest an honest them makes Monotonicity close. the at honest parties Take one. firing 4 line mediated. is them at places game the call no so throughout. ver- Both $\{0, 1\}^*$ in lying name own game's the and party corrupt a at only both: of it need both and Q verifier the of and P signer the of this ask dicts adversary's. the is answer the and run not does core the party corrupt a at signature. good a on 0 a — likes it whatever reports Verify mediated a so having $\mathcal{F}_{\text{Sig}}$ with 1 return would finalization either and — forgery a on 1 a or wrong. nothing done

$\sigma \in \Sigma$ with $(\text{sign}, P, \text{msg}, \sigma) \in \text{tr}$ suppose correctness. For set then and Clean_σ against σ sanitized it so ran. Sign of core The never and once fixed Gen which $\text{vk} = \text{vk}[P]$ for $\text{ver}[\text{vk}, \text{msg}, \sigma] \leftarrow 1$ at it returns and set entry that finds $\text{Verify}(\text{vk}, \text{msg}, \sigma)$ later Any changes. Q every for $(\text{ver}, Q, \text{vk}, \text{msg}, \sigma, 0) \notin \text{tr}$ thus :1 returns it so line. first its .0 gives 20 line and

$\text{vk} =$ with $(\text{ver}, Q, \text{vk}, \text{msg}, \sigma, 1) \in \text{tr}$ suppose unforgeability. For computed it or entry recorded a read Verify Either .P honest an for $\text{vk}[P]$ sets ver writing before test last the :1 a computed have cannot It one. So witnesses. P which $P' \notin \mathbf{C}$ some for $\text{vk} = \text{vk}[P']$ whenever $b \leftarrow 0$ whose party the at — 1 a records Sign only and recorded. was entry the Hence twice. key no issues Clean_{vk} since P being party that .vk is key $\square$.0 gives 28 line and $(\text{sign}, P, \text{msg}, \sigma) \in \text{tr}$

ideal the says merely 0 is it that part: interesting the not is bound The functional- ideal an as properties. these have to written was functionality the and it. with does then 5.11 Corollary what is content The be. should ity $\{\mathcal{F}_{\text{Sig}}\}$ of place in put π system a useful: that makes what is reading search adversary every against ε most at probability with finalization either wins is probability that reading search the under and dummy. the only not and reading decision the Had itself. property the bounds ε so advantage. the which $2 \Pr[\text{Win}_{\text{Fin}}] - 1$ on 2ε read would conclusion same the taken been suc- forger a — all at constraint no is unwinnable be to meant game a for ap- 5 Chapter of rule calibration the is That it. meets time the half ceeding each displays: both carries 5.10 Proposition why and bites. it where plied

inherit two these and fixes, reading own its scale the at inherits finalization
 .ε at

by discharged is none and corollary the with come conditions Four
 well-formed: be must π by $\{\mathcal{F}_{\text{Sig}}\}$ of replacement The alone. emulation
 pin to members 's π asks which system, game a be again must $\{\mathcal{G}_m\} \cup \pi$
 simu- the :5.1 Definition of part is tightness since did, $\mathcal{F}_{\text{Sig}}$ where $\mathcal{G}_m$.pid
 environments the and refusal-oblivious: be must adversary the and lator
 argued separately never thus is realization A -identity.Z no silence must
 emulation one the from both inherits it — unforgeable or correct be to
 above finalizations two the and checks, four those with together proof,
 of. is inheritance the what are

5.11 Corollary bites. it where is this and it. with travels caveat One
 $\mathcal{F}_{\text{Sig}}$ and call. responsive no place system game the of cores the that asks
 Should applies. corollary the so „ $\mathcal{A}(\cdot)$ through slot the reaches written as
 which — Verify and Sign .Gen in $\mathcal{A}^1$ write and advice 's9 Chapter adopt one
 reason the for instead, directly argued be must transfer the — locality buys
 priced stand, things as are, inheritance and Locality gives. 9.4 Remark
 other. each against

زیرساختِ کلید عمومی

16.1 کارکرد

Verify — out hands it key the authenticate not does it that plainly said $\mathcal{F}_{\text{Sig}}$ keys verification because caller. any to unconnected own. its of vk a takes be- $\mathcal{G}_{\text{PKI}}$ functionality The .(15.1) (Section authenticated assumed not are cer- own Canetti's of shape the in gap. the closes that setup the is low the to key a binding directory shared single. a :[6] functionality tification because trusted be can Verify to handed vk a that so it. claims that party believed. simply was it because than rather here from came it only and key. one register may party Each said. quickly is does it What one conjures which .**Gen** unlike — input as key the takes **Register** one: first the keeps and — factory a not and directory a is this slot. the from two no that so file on already key every against sanitized given. is it key anyone answers **Retrieve** operation The one. same the hold ever parties has one no if nothing or name. they party the for file on is whatever with registered.

id process The reason. same the for and 's: $\mathcal{G}_{\text{Clock}}$ are They **fields. The** copy: one at system the caps 1.2 Definition so .(GPKI, 0, 0) constant. a is key. which owns who of notion second a be would directory second a global($\mathcal{G}_{\text{PKI}}$) makes $\mathbf{N} := \mathbf{Std} \cup \mathbf{A} \cup \mathbf{Z}$ out. rule to exists PKI a what exactly the universe the again is $\mathbf{P}$ and clock's. the as disjunct same the by hold in state. is registration — it used has who than rather covers directory The . $\mathbf{U} := \emptyset$ clock's: the than simpler is field One membership. not . $\mathbf{VK}$ with arriving key the all. at call no place Retrieve and Register operations on depends $\mathcal{G}_{\text{PKI}}$ nothing is there so slot. the from than rather request the has. already functionality every machinery ambient the beyond

ex- two the of each from line one takes Register **guards. borrowed Two** for acting Z or A named caller a 's: $\mathcal{G}_{\text{Clock}}$ is first The it. before amplex

nothing, changes and key own party's that answered is party honest an honest an on tick to allowed than rather time the answered is it as exactly goes what decide not file, on is what ask may outsider an – behalf party's the – proceeds call the so fire, not does check the party corrupt a At there. and one, ticks it as freely as behalf party's corrupt a on registers adversary letting mediation show, will directory the what is submits it key whatever sends corrupt 's $\mathcal{F}_{AC}$ for does it as unmediated through call this exactly later wins, succeed to call first the 's: $\mathcal{F}_{Sig}$ is second The .(19.1) (Section as exactly it, overwriting than rather file on already key the returning ones from differs here Sanitizing exists, one once key a re-issues never Gen a fix each $Clean_\sigma$ and $Clean_{vk}$ repairs: it what in predicates signature both the is which supplied, caller the one fixes $Clean_{reg}$ proposed: slot the value already presumably is key own party's honest an – position exposed more one the and in, wrote adversary the whatever is one's corrupt a but good, the key a claiming party second a is through let cannot directory a thing reason same the for here exists always value good A holds, already first those exactly are excluded keys the and infinite is $\mathcal{K} : Clean_{vk}$ for does it .(12 Chapter of (footnote many finitely recorded, have can run bounded a

کارکرد $\mathcal{G}_{PKI}$	
$par := \perp$	$\mathcal{U} := \emptyset$
$\mathcal{N} := Std \cup A \cup Z$	$\mathcal{P}$
$pid := (GPKI, 0, 0)$	
$id' \text{ از } id.Retrieve(P)$:10 return $vk[P]$	:Initialize()
$id' \text{ از } id.Leak()$:11 return vk	:1 $vk : \mathcal{G}_{PKI}.P \rightarrow \mathcal{K} \cup \{\square\}$:2 $vk[*] \leftarrow \square$
$:Clean_{reg}(vk; vk)$:12 return $vk \in \mathcal{K} \wedge \neg \exists P : vk[P] = vk$	$id' \text{ از } id.Register(vk)$:3 if $id'.F \in \{A, Z\} \wedge id.P \notin \mathcal{C}$ then :4 return $vk[id.P]$ // نوشتن یک نه نگاه، یک :5 if $vk[id.P] \neq \square$ then :6 return $vk[id.P]$:7 $vk \leftarrow San[Clean_{reg}](vk; vk)$:8 $vk[id.P] \leftarrow vk$:9 return vk

party, the against recorded is Registration code. the on remarks Three the tied already has Guard are: ticks reason same the for caller, the not reg- P when token the holds machine whichever so, 2 line by id.P to call none and, Deregister no is There entry, own 'sP on lands key the isters, the is bound once key a but revocation, support can PKI real a added: is for left addition genuine a is revocation and is, there directory simplest the with answers Leak And over, papered gap a than rather elsewhere at entry one out gives already Retrieve than more no is which table, whole

this only not interfaces. its of one every at public is directory the — time a times. enough asked ,Retrieve nothing adversary the tells Leak so one. not. would

No either. than less needs $\mathcal{G}_{PKI}$ it. before examples both against Set key the — Gen in as nothing. of out choose to simulator's the is value than rather input caller's a repairs here sanitizing so — call the with arrives no And it. before **San** every from differs this place one the output. slot's a about responsive be to notification no is there so all. at core the leaves call as transfer below properties two the settle: to **9** Chapter for price no and can. property a as cheaply

16.2 ویژگی‌ها

local a — promise of kinds two having for properties two carried $\mathcal{G}_{Clock}$ moves it that one. global a and coded. as moves counter the that one. same the splits $\mathcal{G}_{PKI}$ directory The agreed. has honest everyone when only regis- a says fixity. ,**FinFix** game. one of finalizations two into and way. entry. own party's one about promise. local the — changes never key tered global the — key same the hold ever parties two no says binding. ,**FinBind** of reading search the take Both once. at $\mathcal{G}_{PKI}$. **P** of all over quantified one. **5.8** Definition

$\mathcal{G}_{PKI}$ with , $\gamma := \{\mathcal{G}_m\} \cup \{\mathcal{G}_{PKI}\}$ system game the over played are Both Re- — pinned , **N** := $\{\mathcal{G}_m.pid\}$ field: one save **16.1** Section in as taken iden- the for and , **(17.2)** (Section clock the for as exactly forcing. 's **6.5** mark adversary an against property a is $\mathcal{F}$ global a of property a reason: tical there key every about claims both are binding and fixity and it. shares that is. there all is trace game's the where only certifiable is. session one for asks **3** line so global. longer no is directory the Pinned. passes $\mathcal{G}_m$ but caller No it. meets 0 session own experiment's the and adversary's the not and hosts. it machine a not environment. the not **:2** line ever $\mathcal{G}_{PKI}$ key every So apply. even could **4** line before refused -identities. A whose game a — recorded is and below relay the through enters holds an Inside it. know to way other no has key" a share parties "two is verdict $.(\mathcal{G}_{PKI}.pid, id.P)$ for $id_{\mathcal{G}_{PKI}}$ write we id at interface

پوسته بازی روی $\mathcal{G}_{PKI}$ ؛ نهایی سازی ها در پی می آیند	
$\text{par} := \perp$, $\mathcal{U} := \{(\mathcal{G}_{PKI}, \text{serve}_{\mathcal{G}_m})\}$, $\mathcal{N} := \mathcal{Z}$, $\mathcal{P} := \mathcal{G}_{PKI} \cdot \mathcal{P} \cup \{\mathcal{Z}\}$, $\text{pid} := (\mathcal{G}_m.F, 0, 0)$	
id' از id.Retrieve(P) :9 require $\neg \text{done}$:10 require $\text{id}.P \neq \mathcal{Z}$:11 $\text{vk}' \leftarrow \text{id}_{\mathcal{G}_{PKI}}.\text{FullRetrieve}(P)$ as id :12 return vk'	:Initialize() :1 $\text{done} \leftarrow \text{false}$:2 $\text{tr} \leftarrow ()$ / دنباله یک رد, id' از id.Register(vk) :3 require $\neg \text{done}$:4 require $\text{id}.P \neq \mathcal{Z}$:5 $\text{vk}' \leftarrow \text{id}_{\mathcal{G}_{PKI}}.\text{FullRegister}(vk)$ as id :6 $\text{tr} \leftarrow \text{tr} \cdot (\text{reg}, \text{id}.P, \text{vk}')$:7 return vk' id' از id.Leak() :8 return $\perp$

shared is $\mathcal{G}_{PKI}$ over shell the $\mathcal{G}_m$ of finalizations The	
(fixity) id' from id.FinFix()	
:13 require $\neg \text{done}$:14 require $\text{id}.P = \mathcal{Z}$:15 $\text{done} \leftarrow \text{true}$:16 $\mathcal{C} \leftarrow (\mathcal{C}, \mathcal{Z}).\text{FullStatus}()$ as id :17 $w \leftarrow 1$ if $\exists P, \text{vk}_1, \text{vk}_2 : P \notin \mathcal{C} \wedge \text{vk}_1 \neq \text{vk}_2$:18 $\wedge (\text{reg}, P, \text{vk}_1) \in \text{tr} \wedge (\text{reg}, P, \text{vk}_2) \in \text{tr}$; :19 else $w \leftarrow 0$:20 return w	
(binding) id' from id.FinBind()	
:21 require $\neg \text{done}$:22 require $\text{id}.P = \mathcal{Z}$:23 $\text{done} \leftarrow \text{true}$:24 $\mathcal{C} \leftarrow (\mathcal{C}, \mathcal{Z}).\text{FullStatus}()$ as id :25 $w \leftarrow 1$ if $\exists P, Q, \text{vk} : P \notin \mathcal{C} \wedge Q \notin \mathcal{C} \wedge P \neq Q$:26 $\wedge (\text{reg}, P, \text{vk}) \in \text{tr} \wedge (\text{reg}, Q, \text{vk}) \in \text{tr}$; :27 else $w \leftarrow 0$:28 return w	

and relayed is Retrieve operation The code. the about things Two look not could that game a trace: its reads verdict neither though logged inter- the leaving and all. at $\mathcal{G}_{PKI}$ exercising be not would up key anyone's games for Read relayed shell's $\mathcal{G}_{Clock}$ as exactly nothing, costs whole face and vk_1 need not does verdict's FinFix And mutators. the only read that means idempotence trace: its of reading steadiness's unlike ordered. vk_2 the write. the be can them produced that calls two the of one most at a identically, game the loses order either and it. of read later a only other

first. came one whichever key changed a being key changed

گزاره 16.1 $\mathcal{X}$ occupant every and $\mathcal{Z} \in \mathbb{Z}_{\gamma, \gamma}$ every For .fixity) has $\mathcal{G}_{PKI}$ 0 within fixity has $\{\mathcal{G}_{PKI}\}$ So . $\Pr[\text{Win}_{\text{FinFix}}] = 0$ slot, adversary the of all of class the $\mathcal{X}$ for .5.8 Definition of sense the in $(\mathbb{Z}_{\gamma, \gamma}, \mathcal{X})$ against occupants.

ثبات. Suppose $(\text{reg}, P, vk_1), (\text{reg}, P, vk_2) \in \text{tr}$ with $P \notin \mathbf{C}$. By tightness P is at core own's $\mathcal{G}_{PKI}$. Register running each line 5 at logged were both intercepts never line 4 so ,Z or A not 's. $\mathcal{G}_m$ is name own relay's the :P once and ,vk[P] = $\square$ while only reached ,8 line is write only core's The it. call later every – good for value sanitized some to vk[P] fixes it reached unchanged. one. same the value. '6 line returns and vk[P] $\neq \square$ finds P at wrote first ran whichever agree: vk₂ and vk₁ producing calls two the So gives line 17 at verdict The back. it read only other the and value. the $\square$.0

گزاره 16.2 $\mathcal{X}$ occupant every and $\mathcal{Z} \in \mathbb{Z}_{\gamma, \gamma}$ every For .binding) has $\mathcal{G}_{PKI}$ within binding has $\{\mathcal{G}_{PKI}\}$ So . $\Pr[\text{Win}_{\text{FinBind}}] = 0$ slot, adversary the of all of class the $\mathcal{X}$ for .5.8 Definition of sense the in $(\mathbb{Z}_{\gamma, \gamma}, \mathcal{X})$ against 0 occupants.

ثبات. Suppose $(\text{reg}, P, vk), (\text{reg}, Q, vk) \in \text{tr}$ and $P, Q \notin \mathbf{C}$. By moment the from $vk = vk[Q]$ and $vk = vk[P]$ argument's 16.1 Proposition 'sP case the second, came write 'sQ Say onward. written first was each $vk \leftarrow \text{San}[\text{Clean}_{\text{reg}}](\cdot; vk)$ ran line 8 'sRegister moment that at symmetric: output its that is (12 Chapter guarantee 'sSan and set, already vk[P] with freshness and $\mathcal{K}$ in membership – input the whatever $\text{Clean}_{\text{reg}}$ satisfies cannot write 'sQ So them. among 'sP file. on already key every against line 25 at verdict The .vk = vk[P] = vk[Q] contradicting ,vk[P] equal $\square$.0 gives

hypo- The argue. to extra nothing with 5.11 Corollary by transfer Both is – call responsive no place cores system's game the that – asks it esis or responsive kind, any of call no place cores 's $\mathcal{G}_{PKI}$ way: cheap the met (Sec- completeness dummy and arises never caveat 's9.4 Remark so not. at locality bought notification whose . $\mathcal{G}_{\text{Clock}}$ Unlike untouched. is (4.4 tion priced calls slot whose , $\mathcal{F}_{AC}$ and $\mathcal{F}_{\text{Net}}$ unlike and expense, inheritance's $\{\mathcal{G}_{PKI}\}$ of place in put π system a nothing: pays $\mathcal{G}_{PKI}$ transfer, own their

's15.2 Section of conditions four the directly, binding and fixity inherits that directory A discharge. to left ones only the being paragraph closing spend. to nothing has nothing adversary the asks

ساعتِ جهانی

17.1 کارکرد

con- it about said was what of most and randomized, and local is $\mathcal{F}_{\text{Sig}}$ counts. both on opposite its is example second The simulator. the cerned and Tackmann Maurer, Katz, by introduced was functionality clock A Maurer, Badertscher, by recast [11] UC in synchrony model to Zikas it [3], counter monotone a around setup shared a as Zikas and Tschudi composable the one the functionality, global canonical the become has be- $\mathcal{G}_{\text{Clock}}$ clock The by, time keep successors its and Bitcoin of analyses paper. this of language the in functionality that is low only and 0 at starts `now` counter The said, quickly is does it What enter Parties it, read may serves clock the Anyone forward, steps ever the and, **Deregister** and Register through structure round the leave and **Update** to: it asked has party registered honest every when steps counter registered honest the once and party, calling the against tick a records them, clears and counter the advances 17 line on test the in, all are ticks the and done, is it in party honest last the when exactly ends round A code the how – early it close nor round the stall neither can adversary adversary the to reported is tick recorded Each below, is each refuses the with and (9 (Chapter responsively, 20 line on returns, Update before at settled is costs it what and buys notification the what discarded, answer stands it as counter the answers operation Every .17.2 Section of end the ticks the and time new the learns round a closes that tick the so return, on again, reads stands it where know must that party a old: the it before a was 's $\mathcal{F}_{\text{Sig}}$ where $(\text{GClock}, 0, 0)$ constant, a is id process The **fields. The** which system, any in copy one most at puts then 1.2 Definition parameter, would clock second a and shared, instance, one means global point: the is first its by hold global($\mathcal{G}_{\text{Clock}}$) makes $\mathbf{N} := \mathbf{Std} \cup \mathbf{A} \cup \mathbf{Z}$ time, second a be

session every and check session the waives **Guard** of 3 line so disjunct, component session the separation very the – instance one the reaches time keeps that protocol A here. made not deliberately make. to exists **1.3** Definition of disjunct global the and ,U its in $(\mathcal{G}_{\text{Clock}}, \text{serves})$ declares copy. shared the at entry the meet session every of members lets what is notification the places. code the call one the for $U := \{(A, \text{serves})\}$ And the than rather universe the as but parameter a stays **P** And .20 line of in is who while for, exists interface an parties the fixes it membership: through left and entered state, is moment given a at structure round the table, more one is membership dynamic – [3] in as Deregister and Register field. of kind new a not

operation, an of not functionality, the of field a is **N ticking. and Reading** :**Read** for one right the is above set the and core, every governs set one so and both, lets [3] – time the ask may environment the and adversary the global a makes what of much is clock shared the reading environment the free adversary an where mutators, three the for wide too is It global. setup it drop or finished, not had party the round a close could party a for act to of line first the as sits refinement The altogether. structure round the from answered is party honest an for acting Z or A named caller a core: each the – passes it party corrupt a at while nothing, changes and time the ,[3] in as owns, it parties the for ticks and deregisters registers, adversary corrupt a reading never 17 line matters, it where inert are ticks those and is party honest no while naming: deserves consequence One flag, party's a register may adversary the so vacuous, is 17 line of test the registered is time registration, honest first the until – freely it tick and party corrupt .[3] in as too there spend, to adversary's the

is it and party", registered honest "every reads 17 line of quantifier The stan- a party corrupt a At itself. enforces $\mathcal{G}_{\text{Clock}}$ it of little how seeing worth inter- full the of 4 line by adversary the to handed is Update caller's dard party same the at call own adversary's the and runs, core the before face adversary's the are ticks party's corrupt a way either :13 line by admitted is conven- the by register the reads which – $\forall$ the and withhold, or give to therefore parties Corrupt regardless, them ignores – 3.1 Section of tion both party honest an At one. close help nor open round a hold neither ticks observers' the turns 13 line and fire, not does **Mediate** shut: routes tick- after corrupted party a piece: last the adds Monotonicity reads, into on, then from quantifier the outside is it but behind, flag stale a leaves ing it. clears advance next the and

کارکرد $\mathcal{G}_{\text{Clock}}$	
$\text{par} := \perp$, $\mathcal{U} := \{(\mathcal{A}, \text{ serves})\}$, $\mathcal{N} := \text{Std} \cup \mathcal{A} \cup \mathcal{Z}$, $\mathcal{P}$, $\text{pid} := (\mathcal{G}_{\text{Clock}}, 0, 0)$	
id' از id.Update() :12 if $\text{id}'.F \in \{\mathcal{A}, \mathcal{Z}\} \wedge \text{id}.P \notin \mathcal{C}$ then :13 return now // تیک زدن نه خواندن، :14 if $\text{reg}[\text{id}.P] = 0$ then :15 return now // نیست دور ساختار در :16 $\text{tick}[\text{id}.P] \leftarrow 1$:17 if $\forall P \in \mathcal{G}_{\text{Clock}}.\mathcal{P} \setminus \mathcal{C} : \text{reg}[P] = 1 \Rightarrow$ $\text{tick}[P] = 1$ then :18 $\text{now} \leftarrow \text{now} + 1$:19 $\text{tick}[*] \leftarrow 0$ // شد تمام دور / :20 $\mathcal{A}'(\text{id.Update}, \text{now})$ // ریخته پاسخ بده؛ خبر / می شود :21 return now id' از id.Read() :22 return now id' از id.Leak() :23 return (now, reg, tick)	:Initialize() :1 now $\leftarrow 0$:2 reg, tick : $\mathcal{G}_{\text{Clock}}.\mathcal{P} \rightarrow \{0, 1\}$:3 $\text{reg}[*] \leftarrow 0$; $\text{tick}[*] \leftarrow 0$ id' از id.Register() :4 if $\text{id}'.F \in \{\mathcal{A}, \mathcal{Z}\} \wedge \text{id}.P \notin \mathcal{C}$ then :5 return now :6 $\text{reg}[\text{id}.P] \leftarrow 1$; $\text{tick}[\text{id}.P] \leftarrow 0$ // تازه عضو / است بدهکار تیک یک :7 return now id' از id.Deregister() :8 if $\text{id}'.F \in \{\mathcal{A}, \mathcal{Z}\} \wedge \text{id}.P \notin \mathcal{C}$ then :9 return now :10 $\text{reg}[\text{id}.P] \leftarrow 0$:11 return now

recorded are ticks and Registrations code. the on remarks Three register may P for acting machine any caller: the not party, the against in- [3] .2 line on party the to call the tied already having Guard ,P tick or before separately tick functionality and party registered every has stead identity by indexed tables is wanted. if discipline, that closes: round the 'Update is notification The changes. else nothing and party, than rather and too, departures and registrations about adversary the tells [3] alone: And mechanical. is mutators two other the to pattern 's20 line extending is counter the secrets: no keeping clock a state. whole the returns Leak and round the in is who only say tables the and anyway, read to anyone's it. with done is who

still is needed example signature the what of most, $\mathcal{F}_{\text{Sig}}$ against Set to simulator's the is value No code. the in legible absence each absent. to nothing is there so – discarded is answer notification's the – choose its re-tested and $\mathcal{A}(\cdot)$ through slot the reached $\mathcal{F}_{\text{Sig}}$ where And sanitize. re- is call one clock's the hung, it while happen could what against tables and .20 line precedes Update of mutation every construction: by sponsive re-entered ever is instance no so leaving, from token the keeps Respond oc- every against enforced – re-test to nothing is there and mid-operation class. simulator a advise only could 15 Chapter where slot. the of cupant plays 17.2 Section bites: it where recorded is costs notification the What there. it prices and clock the over properties two

17.2 ویژگی‌ها

second, a worth is clock the once: 5.1 Definition instanced 15.2 Section is order whose paper this in thing one the is time because least not properties: two carrying $\mathcal{G}_{\text{Clock}}$ over $\mathcal{G}_m$ game one give We content. the promises code the as moves counter the that saying steadiness. **FinStead** saying unanimity. **FinUna** and — pushed when only and one. by forward. — Both pushed. has party registered honest every when only moves it that be to meant being either winning. 5.8 Definition of reading search the take hard. than rather impossible

the with $\gamma := \{\mathcal{G}_m\} \cup \{\mathcal{G}_{\text{Clock}}\}$ system game the over played are They The pinned. $\mathbf{N} := \{\mathcal{G}_m.\text{pid}\}$ field: one save 17 Chapter in as taken clock asks requires. 5.1 Definition which $\{\mathcal{G}_m\}$ at Tightness forced. is change is **Std** bare a and ids. process whole admit to test under members the operational in why says 5.7 Definition refuses: 6.5 Remark what exactly any- as it touching functionality. global a shares environment an — terms adversary an against property a is $\mathcal{F}$ global a of property a and may. one the but nuisance a not is sharing properties two these For it. shares that all about assertions are unanimity and steadiness claim: the of negation all is trace its where only them certify can game a and are. there ticks the are certified properties the and privately. played is clock the So is. there 15.2 Section of situation the — change not does sharing which code. its of under $\mathcal{F}_{\text{Sig}}$ the and protocol its names ships that $\mathcal{F}_{\text{Sig}}$ the where exactly. game. the names test

one for asks **Guard** of 3 line so global. longer no is clock the Pinned. carries id process clock's the and 0 session in sits experiment the session: at 2 line passes now $\mathcal{G}_m$ but caller no And passes. check the so $s = 0$ lie claims whose hosts. it machine a or environment the not clock: the refused are -identitiesA whose adversary. the not and $\mathbf{N}$ pinned a outside changes counter the So time. the them answer even could 13 line before Register the inside only membership the and below 15 line inside only game a — recorded is change every and it. beside relays Deregister and way other no has pushed" one no when moved time "the is verdict whose and $(\mathcal{G}_{\text{Clock}}, \text{pid}, \text{id}, P)$ for $\text{id}_{\mathcal{G}_{\text{Clock}}}$ write we id at interface an Inside know. to gave. clock the τ answer the carries entry trace every

root. the refuse relays the :15.2 Section in analogue no has line One were and serve not does $\mathcal{F}_{\text{Sig}}$ party a addressed root the at relays the There well-shaped over existential verdicts. the which .1 line at **rej** answered game's the of **rej** a so entry. every reads Steadiness read. never tuples.

nothing relay, the at refused trace; the into allowed be cannot making own gave. actually clock the answer an is τr in entry every and appended. is honest an answering π a rightly: and bites. this replacement a Against that clock a spot, the on steadiness loses number a but anything read moves. it however broken being time the say cannot prop- single a that for asks shell shared a thing first the is line That like existential. is verdict its — it need not does Unanimity not. would erty steady- is refusal the so — did those as rej a past read would and 's: $\mathcal{F}_{Sig}$ finalizations both because it carries shell the and alone, demand ness's serves. it verdicts the of strictest the to built is shell A shell. the off hang trace a on played is unanimity others: the by paid is that of cost the and thing one the is it because naming, worth cost a is It needs. it than cleaner by weakened be would property a Where away. take can shell a sharing second a is answer the over-served. merely than rather demand another's finalization. second a not and game

پوسته بازی روی $\mathcal{G}_{Clock}$; نهایی سازی ها در پی می آیند

$par := .U := \{\{\mathcal{G}_{Clock}, serves_{gm}\}\} \quad .N := Z \quad .P := \mathcal{G}_{Clock}.PU\{Z\} \quad .pid := (gm.F, 0, 0)$

$\perp$

<pre> :18 require $\neg done$:19 require $id.P \neq Z$:20 $\tau \leftarrow id_{\mathcal{G}_{Clock}}.FullRead()$ as id :21 $\tau r \leftarrow \tau r \cdot (read, id.P, \tau)$:22 return τ :23 return $\perp$ </pre>	<pre> id' از <u>id.Read()</u> :1 done $\leftarrow$ false :2 $\tau r \leftarrow ()$:3 require $\neg done$:4 require $id.P \neq Z$ // می کند نهایی تنها ریشه :5 $\tau \leftarrow id_{\mathcal{G}_{Clock}}.FullRegister()$ as id :6 $\tau r \leftarrow \tau r \cdot (reg, id.P, \tau)$:7 return τ id' از <u>id.Leak()</u> :8 require $\neg done$:9 require $id.P \neq Z$:10 $\tau \leftarrow id_{\mathcal{G}_{Clock}}.FullDeregister()$ as id :11 $\tau r \leftarrow \tau r \cdot (dereg, id.P, \tau)$:12 return τ id' از <u>id.Tick()</u> :13 require $\neg done$:14 require $id.P \neq Z$:15 $\tau \leftarrow id_{\mathcal{G}_{Clock}}.FullUpdate()$ as id :16 $\tau r \leftarrow \tau r \cdot (tick, id.P, \tau)$:17 return τ </pre>	<pre> :Initialize() // دنباله یک رد, id' از <u>id.Register()</u> id' از <u>id.Deregister()</u> </pre>
--	--	---

shared is $\mathcal{G}_{\text{Clock}}$ over shell the $\mathcal{G}_{\text{m}}$ of finalizations The

(steadiness) id' from [id.FinStead\(\)](#)

```

:24 require  $\neg \text{done}$ 
:25 require  $\text{id}.P = Z$ 
:26  $\text{done} \leftarrow \text{true}$ 
:27 parse  $\text{tr}$  as  $((\text{op}_j, P_j, \tau_j))_{j=1}^n$ ;  $\tau_0 := 0$ 
:28  $w \leftarrow 1$  if  $\exists j \in \{1, \dots, n\} : \tau_j \notin \{\tau_{j-1}, \tau_{j-1} + 1\}$ 
:29  $\vee (\tau_j = \tau_{j-1} + 1 \wedge \text{op}_j \neq \text{tick})$ ;
:30 else  $w \leftarrow 0$ 
:31 return  $w$ 

```

(unanimity) id' from [id.FinUna\(\)](#)

```

:32 require  $\neg \text{done}$ 
:33 require  $\text{id}.P = Z$ 
:34  $\text{done} \leftarrow \text{true}$ 
:35  $\mathbf{C} \leftarrow (C, Z).\text{FullStatus}()$  as  $\text{id}$ 
:36  $w \leftarrow 1$  if  $\exists t \geq 0 \exists P \in \mathcal{G}_{\text{Clock}}.P \setminus \mathbf{C} :$ 
:37  $(\exists \tau_r \leq t : (\text{reg}, P, \tau_r) \in \text{tr}) \wedge (\text{dereg}, P, \cdot) \notin \text{tr}$ 
:38  $\wedge (\exists \tau > t : (\cdot, \cdot, \tau) \in \text{tr}) \wedge (\forall \tau' \in \{t, t+1\} : (\text{tick}, P, \tau') \notin \text{tr})$ ;
:39 else  $w \leftarrow 0$ 
:40 return  $w$ 

```

an- an on 1 returns FinStead finalization The words. in verdicts. The an or it. above one than more answer an predecessor. its below swer must departures and registrations reads. tick: a at but anywhere increase is $\tau_{j-1} + 1$ answered tick A it. left answer last the where time the find 'sInitialize is $\tau_0 := 0$ and – round the closed that tick the is it – allowed already is 1 answered read first a so observation. first the as taken word re- FinUna finalization The pushed. anyone before moved that counter a the at honest – party some although t some passed counter the if 1 turns t answered tick no has – released never and t round by registered close. covering $t + 1$ answer the is. that t round during placed tick no $:t + 1$ or the not trace. the read conjuncts registration The it. closed that one the did. itself game the what only name too they so tables. clock's was trace the There instructive. both .15.2 Section with contrasts Two else. little reads FinStead here read: verdict no order whose sequence a had verdicts both there And besides. much not and order an being time any- of nothing asks FinStead named: they party every of $P \notin \mathbf{C}$ ask to be- intercepts wrapper own game's the party corrupt a at because one. an is tr in entry every so .(5.4 (Remark records and runs core the fore is honesty name must that verdict one The already. answer core's honest it names it and call. not did that parties concerns claim whose 's.FinUna

stronger the that making again (3.5 (Section monotonicity – close the at not; other the register, the reading one – so differ two the That reading. for: is shell shared a what is – membership other the order. reading one them made that verdicts, the not common, in have they oracles the is it game. one

property The .parties) across read steadiness is (Agreement 17.1 نکته
hold parties honest :agreement is for wanted usually is clock shared a
corol- a but game third a not is it Here time. the of view common a
the over quantifies 28 Line design. by and above, verdict the of lary
en- tick consecutive two between party: by party not interleaved, trace
two so asked, whoever predecessor, its equal must answer every tries
the answered are ticks two same the between reading parties honest
own party's each – party per Stated won. is game the or value same
agree- and weaker, strictly be would verdict the – steady subsequence
5 with P answer then could replacement a property: separate a ment
What property. neither fail and activations adjacent in 3 with Q and
to- single a model: execution the is claims both carry verdict one lets
and exist not do reads" "simultaneous so trace, the orders totally ken
Two entries. adjacent of consistency the to reduces view common a
a only, parties honest among is Agreement naming. worth are limits
says it and :(5.4 (Remark adversary's the being answers party's corrupt
while stale grow may view whose ask, not does that party a of nothing
the of disagreement not party, the of staleness – ahead runs clock the
never that caller a promise could functionality a nothing and clock,
syn- loosely the , Δ some within agreeing views – variant drift A calls.
,28 line of weakening genuine a be would – [11] of clocks chronized
third a than rather finalization third a is it shape present the under and
the in differs and relays same the of trace same the reads it game:
shell. this off it hanging for test the exactly is which alone, verdict

and FinStead of either Fin For .properties) both has $\mathcal{G}_{\text{Clock}}$ گزاره 17.2
slot. adversary the of $\mathcal{X}$ occupant every and $\mathcal{Z} \in \mathbb{Z}_{\gamma, \gamma}$ every for .FinUna
($\mathbb{Z}_{\gamma, \gamma}, \mathbb{X}$) against 0 within property each has $\{\mathcal{G}_{\text{Clock}}\}$ So . $\Pr[\text{Win}_{\text{Fin}}] = 0$
occupants. all of class the $\mathbb{X}$ for ,5.8 Definition of sense the in

in- only changes counter the properties: both carries invariant One اثبات.
only membership and one. exactly by and records trace the tick a side

first the is it Establishing departures. and registrations recorded inside caller no so tightness. – things three on turns it and work. paragraph's only them reaching game the cores: clock's the reaches game the but handing γ of core no and mediated: is answer no so parties. honest at cores clock's the call single the one. run could that anything to token the each order. execution in append therefore Entries responsive. being place com- by follows then Steadiness returned. call its as counter the carrying Unanimity runs. nothing them between since entries. consecutive paring de- and verdict the of conjuncts four the assumes it half: longer the is corruption of monotonicity using invariant. the from contradiction a rives invariant: one on rests Everything throughout. honest party the keep to exactly by and records. trace the tick a inside only changes counter the departures. and registrations recorded inside only membership the one: at counter the – 17 Chapter of cores the in only written are tables Both `reg` passes. 17 line of test the when one by stepping .Update of line one passing calls on only run cores those and – Deregister and Register in only calls `gm` and .2 line passes `gm` but caller no pinned **N** with :Guard wrapper own its one corrupt a at – parties honest at relays. the through mediated is call no so – ran γ of core no and earlier step a intercepted γ of core no Moreover recorded. answer the is answer core's each and clock's the call one the one: run could that anything to token the hands Respond – responsive is which .20 line of notification the is place cores slot. the occupies whatever place. would answerer its call every refuses relays' the and – case any in it precedes Update of mutation every and γ of machine other no with returning chain that clock. the on is call one each order. execution in append therefore Entries between. in run having returned. call its as counter the carrying

.0 = τ_0 'sInitialize is counter the entry first the before steadiness: For entry at counter the so runs. γ of core no entries consecutive Between own 'sj entry inside taken steps the by $j - 1$ entry at that from differs j tick. a for one or none departure. a or registration a read. a for none call: tick. a at only increment the with throughout. $\tau_j \in \{\tau_{j-1}, \tau_{j-1} + 1\}$ Hence .0 gives 28 line and

some of hold 36 line of conjuncts four the suppose unanimity: For mono- by throughout. honest – close the at $P \in \mathcal{G}_{\text{Clock}} \cdot \mathbf{P} \setminus \mathbf{C}$ some and t the only $\text{reg}[P] \leftarrow 1$ set $\tau_r \leq t$ answered registration recorded A tonicity. Deregister no placed game the .`reg` write Deregister and Register of cores honest an reach cannot observers the and – entry `dereg` no is there – P at $\tau_r > t$ answers entry Some on. moment that from $\text{reg}[P] = 1$ so party.

stepped it so took, counter the values are answers the invariant the by and ev- of flags the read 17 line its tick, recorded some inside $t + 1$ to t from them, among P – then registered and moment that at honest party ery flag That $\text{tick}[P] = 1$ so $-\tau_r \leq t$ and growing ever only register the the of step every at cleared and P at Update of core the by only written is stood counter the while set was it so re-registration, every at and counter itself was it if $t + 1$ or t answered tick that P at game the of tick a by t at $(\text{tick}, P, \tau') \in \text{tr}$ So recorded. was it and round, the closed that one the 36 line and conjunct, fourth the contradicting $\tau' \in \{t, t + 1\}$ some for $\square$.0 gives

was specification the that only says 0 being bound the $\mathcal{F}_{\text{Sig}}$ with As no- the here and transfer, the is content the properties: its have to written cores the that asks 5.11 Corollary price. its collects 20 line of tification the so one, is line that and call, responsive no place system game the of for directly, argued be must $\{\mathcal{G}_{\text{Clock}}\}$ of place in put π system a to transfer Theo- guards hypothesis same the and – gives 9.4 Remark reason the com- dummy forgoes clock notifying the over built system a so ,4.29 rem and 20 line delete – notification the without clock A it, with pleteness them inherits and proof same the by properties both has – entry **U** the discharge, to nothing with corollary the through notification A whether, not price, which only is note, choice, The to token the hand would took, $\mathcal{F}_{\text{Sig}}$ route the $\mathcal{A}(\cdot)$ ordinary the through would answers answering, before ticks nested drive to free adversary an ideal the of fail would steadiness and order, trace of out append then because only $\mathcal{G}_{\text{Clock}}$ of properties are above verdicts two the – itself clock chose $\mathcal{F}_{\text{Sig}}$ where So returns, Update before on acted be cannot 20 line locality choose must clock notifying a locality, with paid and inheritance tells that one the is neither pays that clock the inheritance, with pay and nothing, adversary the

Network - Delayed Δ

18.1 کارکرد

Communications network. a is reader first the and read, be to exists clock The measured delays form, diffusion the]'s:11] is delay bounded a with tion analyses blockchain composability the network the is clock, shared a on language the in functionality that is below $\mathcal{F}_{\text{Net}}$ network The .[3] over run it. be than rather clock the use to example first the and – paper this of is $\mathcal{F}_{\text{Net}}$ code: no owed and $\mathcal{F}_{\text{Diffuse}}$ channel diffusion a named 1 Chapter attached. time with like looks channel a such what

mes- the timestamps **Send** operation The said. quickly is does it What inter- clock's the $\text{id}_{\mathcal{G}_{\text{Clock}}} := (\mathcal{G}_{\text{Clock}}, \text{pid}, \text{id.P})$ through – clock the by sage tells it. stores – 17.2 Section of notation the served, being party the at face The id. message a answers and – public is network the – adversary the ev- answers then early, reveal to what adversary the asks **Fetch** operation The old. Δ everything with together party fetching the to released erything and Δ within when chooses adversary the point: the is power of division with- is do cannot it what own: its of traffic add may and first, whom to last 'sFetch in sits bound that and time, its past message honest an hold is τ time at sent message A it. reaches slot the of answer no where line. first. favoured adversary the whoever on, $\tau + \Delta$ time from fetch every in

are **N** and **P**.pid clock's. the not pattern, 's $\mathcal{F}_{\text{Sig}}$ follow They **fields. The** naming **N** its session, per instance one local, is network a parameters: in- every and – 1 Chapter of pattern local the in serves it protocol the com- " τ " time makes what is which clock, shared one the reads stance en- first the $\text{U} := \{(\mathcal{G}_{\text{Clock}}, \text{serves}), (\mathcal{A}, \text{serves})\}$ sessions. across parable met. is and does network the party every serve to clock the requires try global the by copy shared the at in, sits network the session whatever only well-formed is $\mathcal{F}_{\text{Net}}$ containing system a – 1.3 Definition of disjoint

be- calls slot two the covers second the and – alongside clock the with
is $n \cdot s\mathcal{F}_{\text{Rand}}$ as reads. actually code of line a parameter a, $\text{par} := \Delta$ low.
(13 Chapter)

mes- a deliver adversary the lets [3] **called. not answered. is Scheduling**
lit- Transcribed accord. own its of network the on acting by early sage
A containing **N** local a so calls. adversary the interface an is that erally
shape one the other. no in and 0 session in slot the from reachable –
asked is slot the So carry. cannot relocation and excludes 7.2 Definition
(S, I) pair a back takes and time the adversary the hands Fetch instead:
in- sanitized – traffic fresh of injections ids. stored the among releases –
are calls slot Both 15 Chapter in as canonical. being repair good the line.
it before mutates Send operation The discipline. clock's the responsive.
responsive- and answer. the reading mutations its cannot. Fetch notifies:
straight comes token the immaterial: difference the makes what is ness
re-tests re-entrance $s\mathcal{F}_{\text{Sig}}$ and call. either inside runs else nothing back.
too. here unnecessary stay

are Fetch and Send party's corrupt A **code. of line a without Corruption.**
buffer the so run. cores the before interface full the of 4 line by intercepted
– mediated never are reads clock cores' the and only. sends honest holds
"sends", party corrupt a What word. own clock's the always is timestamp a
of injections the instead: side receiving the from supplies adversary the
the to released ∞ timestamp and $\perp$ sender with buffer the enter 16 line
guarantee. 's20 line into ripening no so and age. no – alone party fetching
traffic. own adversary's the conscripts never and honest the for is which

کارکرد $\mathcal{F}_{\text{Net}}$

$\text{par} := \Delta \quad \mathcal{U} := \{(\mathcal{G}_{\text{Clock}}, \text{serves}), (\mathcal{A}, \text{serves})\} \quad \mathcal{N} \quad \mathcal{P} \quad \text{pid}$

<pre> id' from <u>id.Fetch()</u> :11 $\tau \leftarrow \text{id}_{\mathcal{G}_{\text{Clock}}}.\text{FullRead}()$ as id :12 $(S, I) \leftarrow \mathcal{A}'(\text{id.Fetch}, \tau)$ / a non-pair reads as (ϕ, ϕ) :13 $S \leftarrow S \cap \{m : (m, \cdot, \cdot, \cdot) \in \text{buf}\}$; :14 $I \leftarrow I \cap \mathcal{M}$ / sanitized inline :15 $\text{rel}[m, \text{id.P}] \leftarrow 1$ for each $m \in S$:16 for each $\text{msg} \in I$: :17 $\text{buf} \leftarrow \text{buf} \cup \{(\text{ctr}, \perp, \text{msg}, \infty)\}$; :18 $\text{rel}[\text{ctr}, \text{id.P}] \leftarrow 1$; $\text{ctr} \leftarrow \text{ctr} + 1$:19 return $\{(m, \text{msg}) : (m, P', \text{msg}, \tau_s) \in \text{buf}$:20 $\wedge (\text{rel}[m, \text{id.P}] = 1 \vee \tau_s + \Delta \leq \tau)\}$</pre>	<pre> :Initialize() :1 $\text{ctr} \leftarrow 0$:2 $\text{buf} \leftarrow \phi$ // درباره (m, P, msg, τ) :3 $\text{rel} : \mathbb{N} \times \mathcal{F}_{\text{Net}}.\mathcal{P} \rightarrow \{0, 1\}$:4 $\text{rel}[*] \leftarrow 0$ / شده رها کسی چه به چیزی چه id' از <u>id.Send(msg)</u> :5 $\tau \leftarrow \text{id}_{\mathcal{G}_{\text{Clock}}}.\text{FullRead}()$ as id :6 $m \leftarrow \text{ctr}$; $\text{ctr} \leftarrow \text{ctr} + 1$:7 $\text{buf} \leftarrow \text{buf} \cup \{(m, \text{id.P}, \text{msg}, \tau)\}$:8 $\mathcal{A}'(\text{id.Send}, m, \text{msg})$ / است عمومی شبکه :9 return m id' از <u>id.Leak()</u> :10 return (buf, rel)</pre>
--	--

net- the sender: no carry pairs Fetched code. the on remarks Three what are injections the and]'s.3] diffusion. unauthenticated an is work sent. one no messages contain may fetch a — means unauthenticated one is diffusion authenticated An write. to adversary's the being wire the drop keeps. already buffer the sender the return — away line one and field dif- a and step further a is addressing with Authentication injections. the fetch one what sticky: is `rel` table The it. gives 19 Chapter object: ferent grows only network the of view party's a so hide. may fetch later no reveals Δ past silence adversary's the so deadline. the or `rel` reads 20 line and — state. whole the returns Leak And punished. than rather ignored simply is adversary the to told were contents its secrets: no keeping network the doing. own slot's the is table release the and arrived. they as 8 line at

18.2 ویژگی‌ها

monotone Δ by delivery unformulated: properties three left 18.1 Section all: at game no needs last The sending. before delivery no and views. `buf` with intersecting 20 line name. can S nothing is `buf` in yet not id an release ever can slot the of occupant no so runs. else anything before unformulated. leave still we views Monotone sent. been yet not has what formalize we What timing. to orthogonal and alone `rel` of property a being by ripened message a :liveness. **FinLive** name the under first. the is here property a formalize we And fetch. honest later no from missing is Δ an message a :authentication. **FinAuth** have. to claimed never was $\mathcal{F}_{\text{Net}}$ reading search the take Both someone. by sent was fetches party honest .5.8 Definition of

shape that where is this and game. one of finalizations two are They unau- an being and. outright first the has $\mathcal{F}_{\text{Net}}$ network The most. the earns injec- the on remark own 's18.1 Section — design by diffusion thenticated oracles. of set one shell. one second: the lacks provably — 16 line of tions the and unwinnable is one which of it off read verdicts two and trace. one be would that games separate two Under .1 probability with won is other twice. measured object one is it here objects: two about statements two en- an flag: single 's5.3 Remark is apart measurements the keeps What no so. **FinLive** against game the closed has **FinAuth** calls that vironment conjunction. a is pair the about nothing and both. on judged is run with $\gamma := \{\mathcal{G}_m\} \cup \{\mathcal{F}_{\text{Net}}, \mathcal{G}_{\text{Clock}}\}$ system game the over played are Both tight- 's15.2 Section — $\mathcal{F}_{\text{Net}}.\mathbf{P} \subseteq \mathcal{G}_m.\mathbf{P}$ and pinned $\mathcal{F}_{\text{Net}}.\mathbf{N} := \{\mathcal{G}_m.\text{pid}\}$ follow already fields own 's $\mathcal{F}_{\text{Net}}$ since whole. transplanted argument ness

be- box interface The .(18.1 (Section clock's the than rather pattern 's $\mathcal{F}_{\text{Sig}}$ the not : $\mathcal{F}_{\text{Net}}$ at **Guard** passes $\mathcal{G}_m$ but caller No full. in $\mathcal{G}_m$. **U** states low in- invented an at hosted machine a not adversary, the not environment, stores, ever core 's $\mathcal{F}_{\text{Net}}$ message every So name. game's the of stance be- relays the through passes .Fetch a to returns ever it answer every and someone" by "sent names verdict whose game a — recorded is and low and clock the on dependence own 's $\mathcal{F}_{\text{Net}}$ it. know to way other no has $\mathcal{F}_{\text{Net}}$ for was it as exactly met is and this of any by unaffected is slot the .1.3 Definition of disjunct global the by copy shared the at clock the itself, for $\text{id}_{\mathcal{G}_{\text{Clock}}}$ and $(\mathcal{F}_{\text{Net}}.\text{pid}, \text{id}.P)$ for $\text{id}_{\mathcal{F}_{\text{Net}}}$ write we id at interface an Inside .17.2 Section of notation the second the , $(\mathcal{G}_{\text{Clock}}.\text{pid}, \text{id}.P)$

پوسته بازی روی $\mathcal{F}_{\text{Net}}$ و $\mathcal{G}_{\text{Clock}}$ ؛ نهایی سازی ها در پی می آیند

$U := .N := Z .P := \mathcal{F}_{\text{Net}}.P \cup \{Z\}$, $\text{pid} := (\mathcal{G}_m.F, 0, 0)$
 $\text{par} := \perp . \{(\mathcal{F}_{\text{Net}}, \text{serve}_{\mathcal{G}_m}), (\mathcal{G}_{\text{Clock}}, \text{serve})\}$

<pre> :8 require ¬done :9 $\tau \leftarrow \text{id}_{\mathcal{G}_{\text{Clock}}}.\text{FullRead}()$ as id :10 $M \leftarrow \text{id}_{\mathcal{F}_{\text{Net}}}.\text{FullFetch}()$ as id :11 $\text{tr} \leftarrow \text{tr} \cdot (\text{fetch}, \text{id}.P, \tau, M)$:12 return M :13 return $\perp$ </pre>	<pre> :1 done $\leftarrow$ false :2 $\text{tr} \leftarrow ()$:3 require ¬done :4 $\tau \leftarrow \text{id}_{\mathcal{G}_{\text{Clock}}}.\text{FullRead}()$ as id :5 $m \leftarrow \text{id}_{\mathcal{F}_{\text{Net}}}.\text{FullSend}(\text{msg})$ as id :6 $\text{tr} \leftarrow \text{tr} \cdot (\text{send}, \text{id}.P, m, \tau, \text{msg})$:7 return m </pre>
--	---

/ دنباله یک رد،

shared is $\mathcal{G}_{\text{Clock}}$ and $\mathcal{F}_{\text{Net}}$ over shell the $\mathcal{G}_m$ of finalizations The

(liveness) id' from $\text{id.FinLive}()$

```
:14 require  $\neg \text{done}$ 
:15 require  $\text{id.P} = Z$ 
:16  $\text{done} \leftarrow \text{true}$ 
:17  $\mathbf{C} \leftarrow (\mathbf{C}, Z).\text{FullStatus}()$  as  $\text{id}$ 
:18  $w \leftarrow 1$  if  $\exists P, Q, m, \tau_s, \text{msg}, \tau, M : P \notin \mathbf{C} \wedge Q \notin \mathbf{C}$ 
:19  $\wedge (\text{send}, P, m, \tau_s, \text{msg}) \in \text{tr} \wedge (\text{fetch}, Q, \tau, M) \in \text{tr}$ 
:20  $\wedge \tau \geq \tau_s + \Delta \wedge (m, \text{msg}) \notin M$ 
:21 else  $w \leftarrow 0$ 
:22 return  $w$ 
```

(authentication) id' from $\text{id.FinAuth}()$

```
:23 require  $\neg \text{done}$ 
:24 require  $\text{id.P} = Z$ 
:25  $\text{done} \leftarrow \text{true}$ 
:26  $\mathbf{C} \leftarrow (\mathbf{C}, Z).\text{FullStatus}()$  as  $\text{id}$ 
:27  $w \leftarrow 1$  if  $\exists Q, \tau, M, m, \text{msg} :$ 
:28  $Q \notin \mathbf{C} \wedge (\text{fetch}, Q, \tau, M) \in \text{tr} \wedge (m, \text{msg}) \in M$ 
:29  $\wedge \neg \exists P, \tau_s : (\text{send}, P, m, \tau_s, \text{msg}) \in \text{tr}$ 
:30 else  $w \leftarrow 0$ 
:31 return  $w$ 
```

'sFetch or 'sSend reads relay Neither code. the about things Three read both so – read to none is there – timestamp a for value return own readings two the $\mathcal{F}_{\text{Net}}$ to call the above line one themselves. clock the guard own 's $\mathcal{F}_{\text{Net}}$ not them. between runs γ of core no because coincide relay the honest. id.P finds and $\mathbf{C}$ reads only which check. corruption and .(17.1 (Section call no places which itself. Read not and all. at run having follow fields 's $\mathcal{F}_{\text{Net}}$'s:17.2 Section unlike $\text{id.P} = Z$ excludes relay Neither worse the any reads verdict neither and clock's. the not and pattern 's $\mathcal{F}_{\text{Sig}}$ demand the spared is shell the so – too fetching or sending root the for they shell the get finalizations both and clock's. the of made steadiness of nothing asks verdict authentication's And alone. for asked have would is pair fetched the – about ask to none returns Fetch because sender. a exactly is whom. by sent. ever was m some whether and alone. (m, msg) whole the what is which recover. otherwise can $\mathcal{F}_{\text{Net}}$ of caller no fact the testing. is game

occupant every and $\mathcal{Z} \in \mathcal{Z}_{\gamma, \gamma}$ every For .liveness) has $\mathcal{F}_{\text{Net}}$ 18.1 گزاره
liveness has $\{\mathcal{F}_{\text{Net}}, \mathcal{G}_{\text{Clock}}\}$ So . $\text{Pr}[\text{Win}_{\text{FinLive}}] = 0$ slot. adversary the of $\mathcal{X}$
class the $\mathbb{X}$ for .5.8 Definition of sense the in $(\mathcal{Z}_{\gamma, \gamma}, \mathbb{X})$ against 0 within

occupants. all of

اثبات. Suppose $(\text{send}, P, m, \tau_s, \text{msg}) \in \text{tr}$. (Sec- one internal 's $\mathcal{F}_{\text{Net}}$.Send and read clock own Its relay. Send the core the so given, just argument the by value. same the answer (18.1 tion
 buf in $(m, P, \text{msg}, \tau_s)$ exactly stores
 's $\mathcal{F}_{\text{Net}}$.Fetch. $\tau \geq \tau_s + \Delta$ with $(\text{fetch}, Q, \tau, M) \in \text{tr}$ also suppose Now
 and $(m', P', \text{msg}', \tau_{s'}) \in \text{buf}$ with (m', msg') pair every returns core
 clock the of reading own its τ . (20 (line $\text{rel}[m', Q] = 1 \vee \tau_{s'} + \Delta \leq \tau$
 $(m, P, \text{msg}, \tau_s)$ is entry buffer the : $m' = m$ Taking relay's. the again, and.
 what- met is disjunct second the so hypothesis, by holds $\tau_s + \Delta \leq \tau$ and
 settles which, 12 line at answered slot the whatever and is $\text{rel}[m, Q]$ ever
 gives 18 line and $(m, \text{msg}) \in M$ Hence disjunct. neither and I and S only
 $\square$.0

گزاره 18.2 authentication) have not does $\mathcal{F}_{\text{Net}}$ are There $\mathcal{Z}^* \in \mathbb{Z}_{\gamma, \gamma}$
 $\varepsilon < \text{no} : \Pr[\text{Win}_{\text{FinAuth}}] = 1$ with slot adversary the of $\mathcal{X}^*$ occupant an
 containing class any against authentication 's $\{\mathcal{F}_{\text{Net}}, \mathcal{G}_{\text{Clock}}\}$ bounds 1
 them.

اثبات. The machines. two in counterexample A. answers $\mathcal{X}^*$ occupant
 ex- is which – call no places and injection one with query fetch 's $\mathcal{F}_{\text{Net}}$
 the to relay to is answer to way only whose dummy, the not is it why actly
 point whole the is difference That refuses. Respond call a environment.
 the than rather slot the of occupant every over properties these stating of
 and satisfied, out come would FinAuth dummy, the at read alone: dummy
 on finalizes and once fetches then $\mathcal{Z}^*$ environment The vacuously. only
 the reaches sanitizing, survives injection The else. nothing and FinAuth
 .1 advantage with fires verdict the so ran, ever relay Send no and trace.
 $\text{id}.A(\text{id}''.\text{Fetch}, \tau)$ query the On call. no place but fields 's $\mathcal{A}$ have $\mathcal{X}^*$ Let
 for $(\emptyset, \{\text{msg}^*\})$ returns it – 12 line of shape the – $\text{id}''.\text{F} = \mathcal{F}_{\text{Net}}.\text{pid}$ with
 is it call. no Placing. $\perp$ returns it else anything on : $\text{msg}^* \in \mathcal{M}$ fixed one
 dummy, the unlike calls. slot 's $\mathcal{F}_{\text{Net}}$ of either at Respond by refused never
 does Respond call a .(5 (line $\mathcal{Z}$ to relay to is answer to way only whose
 exactly and , $\mathcal{D}$ from differs $\mathcal{X}^*$ where exactly is This .(9.4 (Remark refuse
 of occupant every over stated are 18.1 and 17.2 ,15.1 Propositions why
 .5.8 Definition of case dummy's the – alone dummy the not and slot the
 and satisfied, as FinAuth read would subscript. the from dropped $\mathcal{X}$ with
 all. at inject to unable being $\mathcal{D}$ vacuously, only

claim then once. $\text{Fetch}()$ call, $Q \in \mathcal{F}_{\text{Net}}$. $\mathbf{P}$ for $(\mathcal{G}_m.\text{pid}, Q)$ claim $\mathcal{Z}^*$ Let the is which other. no and FinAuth and $-\text{FinAuth}()$ call and $(\mathcal{G}_m.\text{pid}, Z)$ beside one the not and property this on judged be to takes it what of whole 's $\mathcal{G}_m$ of identities only claims and nothing. hosts one. no corrupts It it.

$\mathcal{Z}^* \in \mathbb{Z}_{\gamma, \gamma}$ so name. own

of loop the so. $(I \leftarrow \text{INM})$ sanitizing survives $I = \{\text{msg}^*\}$ fetch. that At $\text{rel}[m, Q] \leftarrow 1$ sets and buf to $(m, \perp, \text{msg}^*, \infty)$ fresh some adds 16 line No $(m, \text{msg}^*) \in M$ with $(\text{fetch}, Q, \tau, M) \in \text{tr}$ and returned. is (m, msg^*) indeed $-\text{(send, } \cdot, m, \cdot, \text{msg}^*)$ entry no carries tr so ran. ever relay Send $Q \notin \mathbf{C}$ So injection. the to fresh being m all. at m naming entry send no .1 gives 27 line and throughout.

$\mathcal{Z}^*$ environment The .18.1 Proposition touches run that about Nothing $\text{Win}_{\text{FinLive}}$ and answered never was FinLive so. FinAuth at game the closed same the and .5.3 Remark by disjoint are events two the occur: not did always it one beside keeps always it property a host therefore can shell $\square$ other. each about anything saying two the without loses

of cores the if only 5.11 Corollary by transfers liveness. $\mathcal{G}_{\text{Clock}}$ with As 8 lines do. 's $\mathcal{F}_{\text{Net}}$ of both and call. responsive no place system game the transfer the have then must $\{\mathcal{F}_{\text{Net}}, \mathcal{G}_{\text{Clock}}\}$ of place in put π system a :12 and clock's the as exactly gives. 9.4 Remark reason the for directly. argued no did note. Tightness. thing. same the 17.2 Section cost notification own unlike system: game a as γ licensing beyond above proof either in work a out rule to needs here verdict neither unforgeability. and correctness what off read to only sees. never $\mathcal{G}_m$ route some by buf entering message a survive would propositions both so $-\text{placed itself } \mathcal{G}_m$ what to happens it. for asks above nothing though wanted. one were dependence. weaker Proposi- transfer: to nothing having nothing. transfers Authentication exactly specification. the about fact a but inherit to bound a not is 18.2 tion than rather now proved $-\text{prose in asserted already 18.1 Section fact the its ignores that one than stranger no adversary an by and said. merely does What it. on improve to asked is } \{\mathcal{F}_{\text{Net}}, \mathcal{G}_{\text{Clock}}\}$ of replacement No mail. traffic sender's corrupt a : $\mathcal{F}_{\text{AC}}$'s 19 Chapter is property analogous the have through than rather identity. own its under itself. Send through enters there adver- the is "Authentication. 19 (Chapter answer slot unaccountable an verdict a for sender a carries pair fetched the so interface)". own sary's addressed. fetched. $-\text{condition win transplanted the there and name. to met. not is } -\text{sent never and}$

Authenticated -Delayed Δ Channel

19.1 کارکرد

one the is object complementary The signs. nobody and diffuses $\mathcal{F}_{\text{Net}}$ arrives none and attached sender its with arrives message every where and Δ same the given. [7] of channel authenticated the – nobody from two the and functionality. that is below $\mathcal{F}_{\text{AC}}$ channel The clock. same the the than more deal good a in differ they side: by side reading worth are lives. actually authentication where is difference the and field. sender as recipient a takes Send operation The said. quickly is does it What fresh a under it files clock. the by pair the stamps message. a as well this and secrecy. not is authentication – adversary the tells and index. ad- everything gathers Fetch operation The all. at nothing hides channel adversary the asks age. of come has that party fetching the to dressed the empties and union. the answers early. release to rest the of which sender. record: whole the is gets receiver the What answered. it entries ad- the – ' $\mathcal{F}_{\text{Net}}$ ' is power of division The message. time. send recipient. bound: the is so and – order what in and Δ within when chooses versary slot the whatever names. it party the of fetch next the in is old Δ entry an answers.

pa- are **N** and **P**, pid reasons. ' $\mathcal{F}_{\text{Net}}$ ' for ' $\mathcal{F}_{\text{Net}}$ ' are They **fields. The** served protocol the naming **N** with session. per instance one rameters. is field **U** The .par := Δ and .1 Chapter of pattern local the in the by copy shared the at met entry clock its. $\{(\mathcal{G}_{\text{Clock}}, \text{serves}), (\mathcal{A}, \text{serves})\}$ and needed are serves of conjuncts Both .1.3 Definition of disjunct global chan- the party every serve to clock the asks first the formality: a is neither be not would name own channel's the second the without and does. nel

.4 line on read every refuse would **Guard** so clock, the at admitted

be- 16 line needs $\mathcal{F}_{\text{Net}}$ **interface. own adversary's the is Authentication**
 under: enter to name no traffic adversarial gives channel diffusion a cause
 it on write slot the letting by modelled is forgery so anonymous, is wire the
 At it. delivers already interface full the and name, a is there Here directly.
 A admit 4 and 3 lines — core the reaches adversary the sender corrupt a
 test- both calls, its on fires hook mediation neither and party corrupt a at
 sender's the under Send through enters traffic corrupt so — $\text{id}' \cdot F \neq A$ ing
 $\mathcal{F}_{\text{AC}}$ channel The anyone's, like clock the by timestamped identity, own
 an of field sender the and none, needs and injections no has therefore
 the interface which about fact a but wire the about report a not is entry
 is forgery is: channel authenticated an what is That through, came call
 from, forge to nowhere is there because impossible
 adversary the message A ripens, traffic adversarial that is price The
 12 line so timestamp, real a with list the in is party corrupt a from sends
 $\mathcal{F}_{\text{Net}}$ network The it, retract can answer later no and Δ by it deliver will
 the for is guarantee the so ∞ at injections filing deliberately, this refuses
 con- a is difference The traffic, own slot's the conscripts never and honest
 message a it: against made choice a than rather addressing of sequence
 is sender the message a is recipient named a and sender named a with
 accepted it deadline the to adversary the holding and for, record the on
 field sender the keeps that reading the is party corrupt a as spoke it when
 says, it what meaning

dis- it answering so recipient, one names entry Each **empties. Fetching**
 them between slot the and 12 line what clears 16 line and it, charges
 owed is message diffused a — this do cannot $\mathcal{F}_{\text{Net}}$ network The selected.
 and instead table `rel` sticky the carries it why is which — everyone to
 differ- are guarantees two The answered, already has it what re-answers
 only view party's a that is 's $\mathcal{F}_{\text{Net}}$ other: the implies neither and kind in ent
 over protocol A once, exactly delivered is message a that 's $\mathcal{F}_{\text{AC}}$ grows.
 fetched, it what lose not must $\mathcal{F}_{\text{AC}}$ over one and deduplicate must $\mathcal{F}_{\text{Net}}$
 absence, other's the is obligation each and

release to indices of set a is answer slot's The **release. the Sanitizing**
 are that entries naming set a is be can it wrong thing only the and early,
 procedure 's12 Chapter all, at entries not or — party's fetching the not
 it through answer the runs 14 line so fault, of kind this exactly repairs

under

$$\text{Clean}_{\text{addr}}(S; \text{list}, P) : \Longleftrightarrow S \subseteq \{i : \text{list}[i] = (\cdot, P, \cdot, \cdot)\},$$

things Two context. read-only the as party fetching the and list the with right the **San** makes what are they because saying, worth are that follow so exists, always value good A assertion. an than rather here instrument satisfies $\emptyset$ case: this in prove to nothing has 12 Chapter of footnote the lex- the also is it set empty the being and holds, list the whatever $\text{Clean}_{\text{addr}}$ release early no to repaired is answer failing a so such, first icographically had, have not could it nothing adversary the costs repair that And all, at may released subset good particular some wanting adversary an since 's**San** in assertion An untouched, it returned be and subset that submit of denial a slot the hand and answers good on same the read would place dies fetch party's honest the and index stray one – ones bad on service out, rule to exists 12 Chapter failure the is which –

load- is it here and discipline, clock's The **responsive. are calls slot Both** time the reads Fetch operation The out, spelling worth way a in bearing responsive not call that Were .13 line on slot the asks and 10 line on clock the drive could and between in token the hold would adversary the used 12 line time the by stale be then would τ answering: before forward deadline its past message a and $\tau_i + \Delta \leq \tau$ clear would entries fewer it, and read the makes what is Responsiveness withheld, lawfully be would and notifies it before mutates Send operation The moment, one test the nothing reason: 's $\mathcal{F}_{\text{Net}}$ for too, responsive is but argument, such no needs call, either inside run may else

$\mathcal{F}_{AC}$ کارکرد

$\text{par} := \Delta$
 $\mathcal{U} := \{(\mathcal{G}_{\text{Clock}}, \text{serves}), (\mathcal{A}, \text{serves})\}$
 $\mathcal{N}$
 $\mathcal{P}$
 pid

id' from id.Fetch()

```

:10  $\tau \leftarrow \text{id}_{\mathcal{G}_{\text{Clock}}}.\text{FullRead}()$  as id
:11  $R \leftarrow \{m : \text{list}[m] = (\cdot, \text{id}.P, \tau_m, \cdot)\}$ 
:12  $\wedge \tau_m + \Delta \leq \tau$  / come of age
:13  $S \leftarrow \mathcal{A}^i(\text{id}.Fetch, \tau)$  / released early
:14  $S \leftarrow \text{San}[\text{Clean}_{\text{addr}}](S; \text{list}, \text{id}.P)$ 
:15  $M \leftarrow \{\text{list}[m] : m \in R \cup S\}$ 
:16  $\text{list}[m] \leftarrow \perp$  for each  $m \in R \cup S$ 
:17 return M

```

:Initialize()

```

:1 ctr  $\leftarrow 0$ 
:2  $\text{list} : \mathbb{N} \rightarrow (\mathcal{F}_{AC}.P^2 \times \mathbb{N} \times \mathcal{M}) \cup \{\perp\}$ 
:3  $\text{list}[*] \leftarrow \perp$  / درایه ها (P, Q,  $\tau$ , msg)

id' از id.Send(Q, msg)

:4  $\tau \leftarrow \text{id}_{\mathcal{G}_{\text{Clock}}}.\text{FullRead}()$  as id
:5 ctr  $\leftarrow \text{ctr} + 1$ 
:6  $\text{list}[\text{ctr}] \leftarrow (\text{id}.P, Q, \tau, \text{msg})$ 
:7  $\mathcal{A}^i(\text{id}.Send, \text{ctr}, Q, \tau, \text{msg})$ 
:8 return ok

id' از id.Leak()

:9 return list

```

P against checked not is recipient The code. the on remarks Three and filed is parties channel's the outside addressed message a :Send on fetch to declines who party a silence same the is which fetched, never entries about statement a is below guarantee the and produce, would receiver a so message, the with travels time send The fetches, someone re- fetch 's $\mathcal{F}_{\text{Net}}$ — itself deadline the check and receives it what date can auditable locally is Δ 's $\mathcal{F}_{\text{AC}}$ that is difference the and thing, such no turns the returns **Leak** And functionality, the about fact a only is 's $\mathcal{F}_{\text{Net}}$ where the to told were contents its secrets: no keeping channel the entire, list is drained been since has what and arrived, they as 7 line at adversary leave, watched itself slot the what

19.2 ویژگی‌ها

$\mathcal{F}_{\text{Net}}$ one the and Δ by delivery claims: two named above paragraph The A names, it party the by sent, was fetched message every — make cannot in yet not index an game: no needs again sending, before delivery no third, nothing and $\emptyset$ to S repairs **San** so, $\text{Clean}_{\text{addr}}$ fails drained, already or, list for point same the made 18.2 (Section either early released is real yet not the formalize below FinAuth and FinLive finalizations The buffer). 's $\mathcal{F}_{\text{Net}}$ the — over starting than rather pair 's18.2 Section transplanting two, other channel diffusion a between change not does finalize log, relay, of shell same the being these names, the do neither and one, addressed an and reading, search the take Both channel, different a of asked properties two the and time, this both, keep: can $\mathcal{F}_{\text{AC}}$ two the of which is changes What for name is transplant the That design, whole the of point the is second channels two the between difference the put to way sharpest the is name differently, answered, $\mathcal{F}_{\text{AC}}$ of and $\mathcal{F}_{\text{Net}}$ of asked game, one — with $\gamma := \{\mathcal{G}_{\text{m}}\} \cup \{\mathcal{F}_{\text{AC}}, \mathcal{G}_{\text{Clock}}\}$ system game the over played are Both ar- tightness same the — $\mathcal{F}_{\text{AC}}.\mathbf{P} \subseteq \mathcal{G}_{\text{m}}.\mathbf{P}$ and pinned $\mathcal{F}_{\text{AC}}.\mathbf{N} := \{\mathcal{G}_{\text{m}}.\text{pid}\}$ already fields 's $\mathcal{F}_{\text{AC}}$ reason, same the for and, 18.2 Section as gument **Guard** passes $\mathcal{G}_{\text{m}}$ but caller No .(19.1 (Section pattern 's $\mathcal{F}_{\text{Net}}$ following Fetch record every and, list in places ever Send entry every so, $\mathcal{F}_{\text{AC}}$ at inter- The recorded, is and below relays the through passes returns, ever for $\text{id}_{\mathcal{F}_{\text{AC}}}$ write we id at interface an Inside full, in $\mathcal{G}_{\text{m}}.\mathbf{U}$ states box face of notation the second the, $(\mathcal{G}_{\text{Clock}}.\text{pid}, \text{id}, \text{P})$ for $\text{id}_{\mathcal{G}_{\text{Clock}}}$ and $(\mathcal{F}_{\text{AC}}.\text{pid}, \text{id}, \text{P})$.17.2 Section

پوسته بازی روی $\mathcal{F}_{AC}$ و $\mathcal{G}_{Clock}$ ؛ نهایی سازی ها در پی می آیند

$U := .N := Z .P := \mathcal{F}_{AC}.P \cup \{Z\}$, $pid := (\mathcal{G}_m.F, 0, 0)$
 $par := \perp$, $\{(\mathcal{F}_{AC}, serves_{\mathcal{G}_m}), (\mathcal{G}_{Clock}, serves)\}$

<pre> :8 require $\neg done$:9 $\tau \leftarrow id_{\mathcal{G}_{Clock}}.FullRead()$ as id :10 $M \leftarrow id_{\mathcal{F}_{AC}}.FullFetch()$ as id :11 $tr \leftarrow tr \cdot (fetch, id.P, \tau, M)$:12 return M id' از <u>id.Fetch()</u> </pre>	<pre> :1 done $\leftarrow false$:2 $tr \leftarrow ()$ / دنباله یک رد، id' از <u>id.Send(Q, msg)</u> :3 require $\neg done$:4 $\tau \leftarrow id_{\mathcal{G}_{Clock}}.FullRead()$ as id :5 $r \leftarrow id_{\mathcal{F}_{AC}}.FullSend(Q, msg)$ as id :6 $tr \leftarrow tr \cdot (send, id.P, Q, \tau, msg)$:7 return r </pre>
<pre> :13 return $\perp$ id' از <u>id.Leak()</u> </pre>	

shared is $\mathcal{G}_{Clock}$ and $\mathcal{F}_{AC}$ over shell the $\mathcal{G}_m$ of finalizations The

```

(liveness) id' from id.FinLive()

:14 require  $\neg done$ 
:15 require  $id.P = Z$ 
:16 done  $\leftarrow true$ 
:17  $C \leftarrow (C, Z).FullStatus()$  as id
:18  $w \leftarrow 1$  if  $\exists P, Q, \tau_s, msg, \tau, M : P \notin C \wedge Q \notin C$ 
:19  $\wedge (send, P, Q, \tau_s, msg) \in tr \wedge (fetch, Q, \tau, M) \in tr$ 
:20  $\wedge \tau \geq \tau_s + \Delta \wedge \neg \exists \tau', M' :$ 
:21  $(fetch, Q, \tau', M') \in tr \wedge (P, Q, \tau_s, msg) \in M'$ 
:22 else  $w \leftarrow 0$ 
:23 return w

(authentication) id' from id.FinAuth()

:24 require  $\neg done$ 
:25 require  $id.P = Z$ 
:26 done  $\leftarrow true$ 
:27  $C \leftarrow (C, Z).FullStatus()$  as id
:28  $w \leftarrow 1$  if  $\exists Q, \tau, M, P, \tau_s, msg :$ 
:29  $Q \notin C \wedge (fetch, Q, \tau, M) \in tr \wedge (P, Q, \tau_s, msg) \in M$ 
:30  $\wedge (send, P, Q, \tau_s, msg) \notin tr$ 
:31 else  $w \leftarrow 0$ 
:32 return w

```

the read relays Both familiar. them of two code. the about things Three readings two the 's:18.2 Section as reason same the for themselves clock call no placing Read them. between runs γ of core no because coincide nothing having .ok return. own 's $\mathcal{F}_{AC}$.Send keeps relay 'sSend own. its of and back pass to id message a had which 's $\mathcal{F}_{Net}$ unlike — report to else over search no needs verdict authentication's And out. hands never $\mathcal{F}_{AC}$ names already record fetched a namesake's: its unlike times. or senders

was tuple exact the that check direct a is condition win the so , τ_s and P the — it produced have might what over one existential an not sent, never the at does it what exactly verdict, the in doing, is carries $\mathcal{F}_{AC}$ address interface.

occupant every and $\mathcal{Z} \in \mathbb{Z}_{\gamma, \gamma}$ every For .liveness) has $\mathcal{F}_{AC}$ 19.1 گزاره
liveness has $\{\mathcal{F}_{AC}, \mathcal{G}_{Clock}\}$ So . $\Pr[\text{Win}_{FinLive}] = 0$ slot, adversary the of $\mathbb{X}$
class the $\mathbb{X}$ for ,5.8 Definition of sense the in $(\mathbb{Z}_{\gamma, \gamma}, \mathbb{X})$ against 0 within
occupants. all of

of activation some at written , $(\text{send}, P, Q, \tau_s, \text{msg}) \in \text{tr}$ Suppose اثبات.
one internal 's $\mathcal{F}_{AC}$.Send and read clock own Its .(5 (line relay Send the
between runs γ of core no :18.2 Section of argument the by agree, (4 (line
relay the since honest id.P finding check corruption and guard 's $\mathcal{F}_{AC}$ them,
fresh some wrote core the So call. no placing itself Read and all, at ran
moment. that at $\text{list}[m] = (P, Q, \tau_s, \text{msg})$ with m
 $\text{list}[m]$ If . $\tau \geq \tau_s + \Delta$ with $(\text{fetch}, Q, \tau, M) \in \text{tr}$ too Suppose
by m $\in R$ then runs, core fetch's that when $(P, Q, \tau_s, \text{msg})$ holds still
its τ for hypothesis, by $\tau_s + \Delta \leq \tau$ and ,Q names entry the — 12 line
— relay's the again, argument same the by and. clock the of reading own
whatever and 13 line at answers slot the whatever $(P, Q, \tau_s, \text{msg}) \in M$ so
been already had $\text{list}[m]$ instead If .R touching neither it. with does San
the — Q by fetch earlier some of 16 line at only happened that cleared.
in placed already indices on only and — entry an clears ever that line one
.M own fetch's earlier that in present already hence ,RUS own fetch's that
verdict the and 's.Q of fetch some in appears $(P, Q, \tau_s, \text{msg})$ way Either
 $\square$.0 gives 18 line at

oc- every and $\mathcal{Z} \in \mathbb{Z}_{\gamma, \gamma}$ every For .authentication) has $\mathcal{F}_{AC}$ 19.2 گزاره
has $\{\mathcal{F}_{AC}, \mathcal{G}_{Clock}\}$ So . $\Pr[\text{Win}_{FinAuth}] = 0$ slot, adversary the of $\mathbb{X}$ cupant
,5.8 Definition of sense the in $(\mathbb{Z}_{\gamma, \gamma}, \mathbb{X})$ against 0 within authentication
 $\mathcal{F}_{Net}$ proved 18.2 Section property the — occupants all of class the $\mathbb{X}$ for
instead. here held lacks.

$(P, Q, \tau_s, \text{msg}) \in$ and $Q \notin \mathbf{C}$ with $(\text{fetch}, Q, \tau, M) \in \text{tr}$ Suppose اثبات.
index both and , $M = \{\text{list}[m] : m \in R \cup S\}$ code, 's $\mathcal{F}_{AC}$.Fetch By .M
match pattern own its by R stands: already it as list on only draw sets
with i indices only admits Clean_{addr} since sanitizing, after S and ,(12 (line
values among picks (12 (Chapter 3 line and already $\text{list}[i] = (\cdot, Q, \cdot, \cdot)$

not does list index an manufactures neither – predicate that meeting some for $\text{list}[m] = (P, Q, \tau_s, \text{msg})$ if only $(P, Q, \tau_s, \text{msg}) \in M$ So hold. point. some at m

is $\perp$ than other value a to $\text{list}[m]$ writes ever that line only The that – pinned $\mathcal{F}_{AC}.\mathbf{N} = \{\mathcal{G}m.\text{pid}\}$ – tightness by and core. own 'sSend the not environment. the not places: relay 's $\mathcal{G}m$ calls on only runs core game's the of instance invented an at hosted machine a not adversary. activa- some at happened $(P, Q, \tau_s, \text{msg})$ producing write the So name. pre- the of argument coinciding-read the by and relay. Send the of tion Hence $.(send, P, Q, \tau_s, \text{msg})$ exactly as there logged was it proof vious $\square$.0 gives 28 line at verdict the and $.(send, P, Q, \tau_s, \text{msg}) \in \text{tr}$

only 5.11 Corollary by transfer properties both $\mathcal{F}_{Net}$ and $\mathcal{G}_{Clock}$ with As there. as here. and – call responsive no place cores system's game the if 'sSend covers responsive" are calls slot "Both 's19.1 Section does: neither $\{\mathcal{F}_{AC}, \mathcal{G}_{Clock}\}$ of place in put π system A alike. query 'sFetch and notification 9.4 Remark reason the for directly. argued transfers both have then must thing. same the 18.2 Section cost notification own 's $\mathcal{F}_{Net}$ as exactly gives. re- negative a as only standing left is here nothing network. the Unlike from transplanted 18.2 Section condition win the is 19.2 Proposition sult. laid proofs. two the and – instead here closed close. not could and $\mathcal{F}_{Net}$ tightness whether fact: one in difference whole the locate side. by side it $\mathcal{F}_{AC}$ For it. produced that Send a to back record fetched a trace can cannot. it $\mathcal{F}_{Net}$ for entry: an gain to way other no having list can. always trans- game one chapters. Two in. way other the exactly being 16 line them. between difference entire the is address the and twice. planted comparison the makes what is names the keeps transplant the That resem- that properties two not is FinAuth finalization The all. at sayable – sent never addressed. fetched. – condition win one but other each ble Under $\mathcal{F}_{AC}$ for 0 and $\mathcal{F}_{Net}$ for 1 is answer the and channels. two of asked game. own its carried property each where replaced. 5 Chapter shape the would sameness the and $\mathcal{G}m'_{Auth}$ and $\mathcal{G}m_{Auth}$ been have would two the notation. the is it here remark: a been have

تحقق کانالِ اصالت دار

proto- a it: earns chapter This postulated. was 19 Chapter of channel The unauthen- the directory, public-key a — subroutines ideal three over ρ col that — 15 Chapter of signatures the and 18 Chapter of network ticated ev- 0 within exactly, so does and channel. authenticated an UC-realizes information-theoretic. reduction whole the and ideal being subroutine ery the exercises that one the and paper. the in construction largest the is It sanitization. discipline. responsive the composition. once: at it of most do- each appear, all 11 Chapter of lemma fundamental the and tightness. could. other no job a ing

from key signing a draws time first the for sending party A **does. ρ What** the reads it Q to msg send To $\mathcal{F}_{PKI}$ directory the in it registers and $\mathcal{F}_{Sig}$ signs s number sequence per-party a increments τ timestamp a for clock over signature its with tuple the diffuses and $(\mathbb{I}, Q, \tau, s, msg)$ tuple the item diffused a keeps and delivers. $\mathcal{F}_{Net}$ what collects it fetch. To $\mathcal{F}_{Net}$ un- verifies that signature a carries fetcher, the to addressed is it if only seen not pair (sender, s) a has and key. directory sender's claimed the der carries Diffusion $(\mathbb{I}, Q, \tau, msg)$ as up hands it survives What before. and second the supplies signature the authenticity: no and addresses no a trusts receiver a that so first, the tuple signed the inside address the sig- own sender's the under is name that because exactly name sender's nature.

and obstruction. the is sender corrupt A **$\mathcal{F}_{AC}$ not $\mathcal{F}_{AC}^\circ$ is target the Why** ad- the network diffusion a Over incidental. than rather instructive is it — fetch a during particular in likes. it whenever wire the on writes versary the of window responsive the inside chosen are (16 (line injections ' $s\mathcal{F}_{Net}$ the key a under signed injected. so message A them. carries that fetch and receiver the by accepted is party, corrupt a for registered adversary only delivers which $\mathcal{F}_{AC}$ so it, placed ever Send prior no but delivered:

call can simulator no and it, produce cannot stored. Send own its what Worse. gap. the repair to window responsive fetch's the inside $\mathcal{F}_{AC}$. Send val- signature the – choose to own its is timestamp sender's corrupt the time true the stamps $\mathcal{F}_{AC}$ where – carries payload the τ whatever idates au- an what are both ρ in defect a is gap Neither witnessed. it call a of 20 Chapter and party, corrupt a gives unavoidably diffusion thenticated does subroutines these over protocol no that argument the with closes traffic sender's corrupt a with $\mathcal{F}_{AC}$: $\mathcal{F}_{AC}^\circ$ is object realized the So better. exactly is which timestamped, freely and unstored fetch, the at admitted honest- The less. no and channel authenticated]'s7] of level guarantee the party's honest an that authenticity, and Δ by delivery – guarantees sender survive – names it party honest the by sent genuinely was record fetched objects two the one no corrupts that adversary an against And whole. a for $\Pr[\text{Bad}]$ at difference the prices lemma fundamental the coincide: exactly, itself, $\mathcal{F}_{AC}$ realizes ρ so raise, can sender corrupt a only that bad .(20.2 (Corollary corrupt is nobody whenever

re- the are signatures The **shelf. the off quite not are subroutines Two** and – **Verify** and **Sign**, Gen in $\mathcal{A}^1$ – 9 Chapter of $\mathcal{F}_{Sig}^1$ variant sponive respon- is call slot one its atomic: is fetch 's $\mathcal{F}_{AC}$ token. the is reason the answer. its and call the between execution the drives occupant no so sive, would verification a $\mathcal{A}(\cdot)$ ordinary the through $\mathcal{F}_{Sig}$ reach to fetch 's ρ Were re- before elsewhere act it let and mid-fetch token the adversary the hand simulator no and show cannot fetch atomic 's $\mathcal{F}_{AC}^\circ$ interleaving an turning, 's $\mathcal{F}_{Net}$ escape: the removes $\mathcal{F}_{Sig}^1$ variant responsive The reproduce. could is fetch every and send every so responsive, already are calls 's $\mathcal{F}_{AC}^\circ$ and of $\mathcal{G}_{PKI}$ global the not local, is directory the And sides. both on atomic real the in ρ . Send honest an by written is directory shared a :16 Chapter where, **Retrieve** through environment the by back straight read and world peek the only holds simulator the and it writes nothing world ideal the in commit- the difference, unsimulatable an – parties honest at grants $\mathcal{G}_{PKI}$, $\mathcal{F}_{PKI}$ local A coordinates. paper's this in setup global a of problem ment out kept is Z it: closes hybrid, any like session the to private identities its a being environment the by retrieval direct a reason. same the for **N** its of serve. to machine no has world ideal the call

key registered a sanitizes $\mathcal{F}_{PKI}$ **purpose. on bindings forgets directory The** fresh- cross-party 's $\mathcal{G}_{PKI}$ dropping slot, per – more nothing and key a be to ad- the weapon: a be would freshness The. (Clean_{reg}, 16 (Chapter ness $\mathcal{F}_{Sig}$. Gen value the being it key, signing party's honest an chooses versary

the have and slot corrupt a at key that pre-register could it so returns. direc- leaving one. different a to repaired registration own party's honest rejected message every party's the and disagreeing key signing and tory Per-slot unconditionally. promises $\mathcal{F}_{AC}^\circ$ liveness the of denial a – ever for sender's the binding: on rested never attribution because enough is fixity and up. name that looks receiver the tuple, signed the inside travels name different a naming tuple a on key party's honest an under valid signature a costs and here nothing buys Binding suppresses. $\mathcal{F}_{Sig}$ forgery a is sender goes. it so liveness.

دفترچه محلی $\mathcal{F}_{PKI}$	
$pid, .P, .N, .U := \emptyset$	$par := \perp$
$id' \text{ از } id.Retrieve(P)$	$:Initialize()$
:10 return $vk[P]$	:1 $vk : \mathcal{F}_{PKI}.P \rightarrow \mathcal{K} \cup \{\square\}$
$id' \text{ از } id.Leak()$	:2 $vk[*] \leftarrow \square$
:11 return vk	$id' \text{ از } id.Register(vk)$
$:Cleankey(vk)$	:3 if $id'.F \in \{A, Z\} \wedge id.P \notin C$ then
:12 return $vk \in \mathcal{K}$	:4 return $vk[id.P]$ / نوشتن یک نه نگاه، یک
	:5 if $vk[id.P] \neq \square$ then
	:6 return $vk[id.P]$
	:7 $vk \leftarrow San[Cleankey](vk)$
	:8 $vk[id.P] \leftarrow vk$
	:9 return vk

(S, I) pair a back takes Fetch changed: operation one with $\mathcal{F}_{AC}$ is $\mathcal{F}_{AC}^\circ$ of set a I before. as ids stored among releases early the S slot, the from injection each pins Sanitizing unstored. delivers slot the records injected ever only can injection an so party. fetching the to and sender corrupt a to .Send own 's $\mathcal{F}_{AC}$ it: for asked that fetch the into word party's corrupt a put unchanged. are Initialize and Leak

replaced, $\mathcal{F}_{AC}$ of fetch the $\mathcal{F}_{AC}^{\circ}$ Functionality(19.1) (Section $\mathcal{F}_{AC}$ as fields

id' from id.Fetch()

```

:1  $\tau \leftarrow \text{id}_{\mathcal{G}_{Clock}}.\text{FullRead}()$  as id
:2  $\mathbf{C} \leftarrow (\mathbf{C}, \text{id.P}).\text{FullStatus}()$  as id
:3  $\mathbf{R} \leftarrow \{m : \text{list}[m] = (\cdot, \text{id.P}, \tau_m, \cdot) \wedge \tau_m + \Delta \leq \tau\}$ 
:4  $(S, I) \leftarrow \mathcal{A}^I(\text{id.Fetch}, \tau)$  / releases and injections
:5  $S \leftarrow \text{San}[\text{Clean}_{addr}](S; \text{list}, \text{id.P})$ 
:6  $I \leftarrow \text{San}[\text{Clean}_{inj}](I; \mathbf{C}, \text{id.P})$ 
:7  $M \leftarrow \{\text{list}[m] : m \in R \cup S\} \cup I$ 
:8  $\text{list}[m] \leftarrow \perp$  for each  $m \in R \cup S$ 
:9 return M

:1 return  $I \subseteq \{(P, Q, \tau, \text{msg}) : P \in \mathbf{C} \wedge Q = P_f\}$ 

```

:Clean_{inj}(I; C, P_f)

the at interfaces the through subroutines four its reads protocol The
U Its .18 Chapter of $\text{id}_{\mathcal{F}} := (\mathcal{F}.\text{pid}, \text{id.P})$ notation the served, being party
 payloads own its stamps ρ since them, among clock the four, all names
 returns. never $\mathcal{F}_{Net}$ which stamp, 's $\mathcal{F}_{Net}$ recover not does and

{ $\mathcal{F}_{PKI}, \mathcal{F}_{Net}, \mathcal{F}_{Sig}^I$ } پروتکل ρ روی

$\mathbf{U} := \{(\mathcal{F}_{PKI}, \text{serve}), (\mathcal{F}_{Net}, \text{serve}), (\mathcal{F}_{Sig}^I, \text{serve}), (\mathcal{G}_{Clock}, \text{serve})\}$ $\mathbf{N}$ $\mathbf{P}$ id
 $\text{par} := \perp$

id' from id.Fetch()

```

:15  $\mathbf{N} \leftarrow \text{id}_{\mathcal{F}_{Net}}.\text{FullFetch}()$  as id
:16  $M \leftarrow \emptyset$ 
:17 for  $(m, y) \in \mathbf{N}$  do
:18   parse  $y$  as  $(x, \sigma)$ .  $x = (P, Q, \tau, s, \text{msg})$  /
      else skip
:19   if  $Q \neq \text{id.P}$  then skip
:20    $\text{vk} \leftarrow \text{id}_{\mathcal{F}_{PKI}}.\text{FullRetrieve}(P)$  as id
:21   if  $\text{vk} = \square$  then skip
:22    $b \leftarrow \text{id}_{\mathcal{F}_{Sig}}.\text{FullVerify}(\text{vk}, x, \sigma)$ 
      asid
:23   if  $b \neq 1$  then skip
:24   if  $(P, s) \in \text{seen}[\text{id.P}]$  then skip
:25    $\text{seen}[\text{id.P}] \leftarrow \text{seen}[\text{id.P}] \cup \{(P, s)\}$ 
:26    $M \leftarrow M \cup \{(P, Q, \tau, \text{msg})\}$ 
:27 end for
:28 return M

:1 gen :  $\mathbf{P} \rightarrow \{0, 1\}$ ;  $\text{gen}[*] \leftarrow 0$ 
:2 seq :  $\mathbf{P} \rightarrow \mathbf{N}$ ;  $\text{seq}[*] \leftarrow 0$ 
:3 seen :  $\mathbf{P} \rightarrow \mathcal{P}(\mathbf{P} \times \mathbf{N})$ 
:4  $\text{seen}[*] \leftarrow \emptyset$ 

      id' از id.Send(Q, msg)
:5 if  $\text{gen}[\text{id.P}] = 0$  then
:6    $\text{vk} \leftarrow \text{id}_{\mathcal{F}_{Sig}}.\text{FullGen}()$  as id
:7    $\text{id}_{\mathcal{F}_{PKI}}.\text{FullRegister}(\text{vk})$  as id
:8    $\text{gen}[\text{id.P}] \leftarrow 1$ 
:9  $\tau \leftarrow \text{id}_{\mathcal{G}_{Clock}}.\text{FullRead}()$  as id
:10  $\text{seq}[\text{id.P}] \leftarrow \text{seq}[\text{id.P}] + 1$ 
:11  $x \leftarrow (\text{id.P}, Q, \tau, \text{seq}[\text{id.P}], \text{msg})$ 
:12  $\sigma \leftarrow \text{id}_{\mathcal{F}_{Sig}}.\text{FullSign}(x)$  as id
:13  $\text{id}_{\mathcal{F}_{Net}}.\text{FullSend}((x, \sigma))$  as id
:14 return ok

```

:Initialize()

world real the In slot. adversary the of $\mathcal{A}$ occupant an Fix **simulator. The**
 must $\mathcal{S}_{\mathcal{A}}$ and gone, are they world ideal the in hybrids: three the faces $\mathcal{A}$
 .4.26 (Definition absorption an is It itself. inside from faces their present
 each copy internal one with $\mathcal{A}$ bundles $\mathcal{S}_{\mathcal{A}}$ side): adversary the on read
 execution real the as exactly them against $\mathcal{A}$ runs, $\mathcal{F}_{Sig}^I$ and $\mathcal{F}_{Net}$, $\mathcal{F}_{PKI}$ of

no- Send honest an On it. tells $\mathcal{F}_{AC}^\circ$ what from them drives and would. $\mathbb{I}$ for seq internal the advances it: $(\mathbb{I}, Q, \tau, \text{msg})$ it hands $\mathcal{F}_{AC}^\circ$ tification internal the into steps own 'sp replays and $x = (\mathbb{I}, Q, \tau, s, \text{msg})$ forms answers. $\mathcal{A}$ hosted the query $\mathcal{A}^!$ whose $\mathcal{F}_{Sig}^!$.Sign internal an – hybrids hosted the so – delivers it notification whose $\mathcal{F}_{Net}$.Send internal an and makes $\mathcal{F}_{AC}^\circ$ When place. would world real the queries the precisely sees $\mathcal{A}$'s $\mathcal{F}_{Net}$ internal the over filter 'sp.Fetch runs $\mathcal{S}_{\mathcal{A}}$ query. fetch responsive its hand would ρ records which exactly learns $\mathcal{A}$ hosted the against delivery their by entries honest early-released or ripe the: (S, I) answers and up. key a under verifying those – records corrupt-sender the and S in ids $\mathcal{F}_{AC}^\circ$ through. straight passes it Corruption I in – slot corrupt a at registered does. 's3.2 Remark as bundle the surviving Corrupt

passes above paragraph the state of pieces two explicit: Made chosen. than rather it above just code the by forced both silently. over with slot the notifies (19.1 Section $\mathcal{F}_{AC}$ from (inherited Send own 's $\mathcal{F}_{AC}^\circ$ the alone. $(\mathbb{I}, Q, \tau, \text{msg})$ not – index list own its m . (Send, $m, Q, \tau, \text{msg})$ The (9.2 (Definition at arrives query the identity the off read $\mathbb{I}$ sender it later: entry the name to below. fid index. that keep must $\mathcal{S}_{\mathcal{A}}$ simulator And itself. $\mathcal{F}_{AC}^\circ$.list to access no having time. fetch at it recover cannot reads test freshness its seen the and seq .5 line at gen – tables own 'sp world ideal the of part no being ρ host. not does $\mathcal{S}_{\mathcal{A}}$ tables are – 24 line at shad- needs filter its reproducing so here. internal subroutines its only and reading by than rather construction by 'sp to equal kept own. its of ows them.

whether decide both and open. leaves prose the questions routing Two $\mathcal{A}$ hosted the call a of becomes what is first The all. at works simulator the neither and party corrupt a at $\mathcal{A}$ admits 3 Line identity. own 'sp on places reaches call a such world real the in so calls. its on fires hook mediation mediated is places then core that call subroutine every and – core 'sp so corrupt). party the and ρ being caller the 4 (line $\mathcal{A}$ to back straight instead would it outward Relayed buffer. 's $\mathcal{F}_{Net}$ reaches ever it of nothing Δ by delivered is and ripens that record a files Send whose $\mathcal{F}_{AC}^\circ$ reach real-world no with delivery a – (19.1 (Section answers slot the whatever rout- explicit 's4.13 Definition relayed: not are calls these So counterpart. against core 'sp runs below handler last the where inside. them keeps ing declared is set outward The really. does Mediate as exactly $\mathcal{A}$ hosted the accordingly.

handlers Both reaches. answer responsive a far how is second The call every refuses (9.1 (Definition Respond and call. responsive a answer

placed not are machines hosted two between calls :places responder the nothing but reachable. are copies internal the so , (4.13 Definition all at reached. be otherwise would it outside things Two is. bundle the outside each. avoids code $'s\mathcal{S}_{\mathcal{A}}$ and register. corruption the reading by opens interface full copy's hosted A computed is answer responsive a — outright refuses 9.2 Definition which copies the drive handlers the So knows. already responder the what from nothing and above. FullGen than rather Gen written cores. their through before mediated is fetch the or send the at corrupt party a it: by lost is and behalf its on issued ever is query slot no so runs. core own $'s\mathcal{F}_{AC}^{\circ}$ adds interface full the party honest an at and one. serves ever handler no opens $\mathcal{F}_{Net}$ And fires. hook neither and passes. gate the — silencer its only hosted. than rather shared is which. $\mathcal{G}_{Clock}$ reading by Fetch and Send both carries. already query or notification the τ the with answered is read That it send the on one: convenient a merely not and value right the is which the on and share. reads own $'s\mathcal{F}_{Net}$ and $'sp$ shows proof the stamp the is past advanced have can call responsive no which read. $'s\mathcal{F}_{AC}^{\circ}$ is it fetch .(9.3 (Proposition

$\mathcal{S}_A$ Simulator

4.26 (Definition $\mathcal{F}_{\text{Sig}}^1, \mathcal{F}_{\text{Net}}, \mathcal{F}_{\text{PKI}}$ of each copy internal one and $\mathcal{A}$ hosts $!IDs(\mathcal{A})$ occupies serves. still execution ideal the identities the set outward side): adversary the on read
(4.13 (Definition hand by routes handler last the which own. 'sp less

Q at (τ) query Fetch responsive 's $\mathcal{F}_{AC}^*$ on :Initialize()

```

:14 N ← internal id $\mathcal{F}_{\text{Net}}$ .Fetch() / clock
    answered  $\tau$ :  $\mathcal{A}$  answers the release query
:15 S, I ←  $\emptyset, \emptyset$ 
:16 for (m, y) ∈ N do
:17   parse y as (x,  $\sigma$ ): x = ( $\mathbb{Q}$ , Q',  $\tau_x$ , s, msg)
    / else skip
:18   if Q' ≠ Q then skip
:19   vk ← internal id $\mathcal{F}_{\text{PKI}}$ .Retrieve( $\mathbb{Q}$ )
:20   if vk =  $\square$  then skip
:21   b ← internal id $\mathcal{F}_{\text{Sig}}$ .Verify(vk, x,  $\sigma$ )
:22   if b ≠ 1 then skip
:23   if ( $\mathbb{Q}$ , s) ∈ seen[Q] then skip
:24   seen[Q] ← seen[Q] ∪ {( $\mathbb{Q}$ , s)}
:25   if fid[ $\mathbb{Q}$ , s] ≠  $\square$ 
:26     then S ← S ∪ {fid[ $\mathbb{Q}$ , s]}
:27     else I ← I ∪ {( $\mathbb{Q}$ , Q',  $\tau_x$ , msg)}
:28 end for
:29 return (S, I)

```

$\mathbb{Q}$ at (m, Q, τ , msg) note Send 's $\mathcal{F}_{AC}^*$ on

```

:1 gen: P → {0, 1}: gen[*] ← 0
:2 seq: P → N: seq[*] ← 0
:3 seen: P → P(P × N): seen[*] ←  $\emptyset$ 
:4 fid: P × N → N ∪ { $\square$ }: fid[*] ←  $\square$  / ثبت
    اندیسی  $\mathcal{F}_{AC}^*$  خودی
:5 if gen[ $\mathbb{Q}$ ] = 0 then
:6   vk ← internal id $\mathcal{F}_{\text{Sig}}$ .Gen()
:7   internal id $\mathcal{F}_{\text{PKI}}$ .Register(vk)
:8   gen[ $\mathbb{Q}$ ] ← 1
:9   seq[ $\mathbb{Q}$ ] ← seq[ $\mathbb{Q}$ ] + 1: s ← seq[ $\mathbb{Q}$ ]
:10 x ← ( $\mathbb{Q}$ , Q,  $\tau$ , s, msg)
:11  $\sigma$  ← internal id $\mathcal{F}_{\text{Sig}}$ .Sign(x) / hosted  $\mathcal{A}$ 
    answers the query
:12 internal id $\mathcal{F}_{\text{Net}}$ .Send((x,  $\sigma$ )) / its clock read
    answered  $\tau$ 
:13 fid[ $\mathbb{Q}$ , s] ← m

```

$\mathbb{Q}$ corrupt a at Fetch or Send 's $\mathcal{A}$ on

```

:30 run p's own core for that operation (the pro-
    tocol box above) on gen, seq, seen, with
    every subroutine call it places — to  $\mathcal{F}_{\text{Sig}}^1$ ,
     $\mathcal{F}_{\text{PKI}}$ ,  $\mathcal{F}_{\text{Net}}$  or  $\mathcal{G}_{\text{Clock}}$  — handed to the hosted
     $\mathcal{A}$  rather than placed, as the full interface's
    line 4 hands it really: return the core's own
    value
:31 never relayed outward: no  $\mathcal{F}_{AC}^*$ .list entry,
    and none drained

```

Corrupt and Routing.

```

:32 any other call between the hosted  $\mathcal{A}$  and an
    internal hybrid: served inside, the bundle's
    ordinary routing (Definition 4.13.) A Corrupt
    leaves under the hosted  $\mathcal{A}$ 's own claim, the
    bundle occupying  $IDs(\mathcal{A})$ , so Corrupt's own
    gate id'.F = A (its line 3) is met without
    rewriting

```

read to easy but proof the by used all code. 's $\mathcal{S}_A$ about things Three
entry list undrained “an exactly is 26 line at fid[$\mathbb{Q}$, s] ≠ $\square$ First. past.
record a recognizes $\mathcal{S}_A$ words: own proof's the in record” the matches
is which it. produced that Send honest the relayed once it when precisely
all. at fid into gets entry an way only the
in- the of reading a not and flag own 'sp of shadow a is gen Second.
Register 's $\mathcal{F}_{\text{PKI}}$ — agree two the party honest an at though, $\sqrt{VK}$'s $\mathcal{F}_{\text{PKI}}$ ternal

ρ only so ,4 line its at peek a into there call -caller'sZ or -A an turns is which one. corrupt a at company part They entry. that writes ever before away mediated is Register 'sp there taken: not is shortcut the why regardless. $\text{gen}[\mathbb{Q}] \leftarrow 1$ sets ρ while unset $\text{vk}[\mathbb{Q}]$ leaving runs. core 's $\mathcal{F}_{\text{PKI}}$ re- would vk testing simulator A answer. the reading not 5 line 'sp.Send hosted the show and send later every party's the on Register and Gen run sees. never $\mathcal{A}$ real the queries of pair a $\mathcal{A}$ in- ones ripe ,S into record honest matching every puts 26 line Third, own. its on R through ones ripe the taken have would $\mathcal{F}_{\text{AC}}^\circ$ where cluded. entry undrained any admits $\text{Clean}_{\text{addr}}$ nothing: costs and deliberate is This their drains and 7 line its at S with R unions $\mathcal{F}_{\text{AC}}^\circ$.Fetch and ,Q to addressed it Naming once. it drains and delivers twice entry ripe a naming so union. with query the inside own its of read clock no having do. can $\mathcal{S}_{\mathcal{A}}$ all also is early. from ripe tell to which

$\pi := \text{Let .channel})$ authenticated the realizes ρ) 20.1 **قضيه**
well- is $\pi \cup \{\mathcal{G}_{\text{Clock}}\}$ that so matched parameters with $\{\rho, \mathcal{F}_{\text{PKI}}, \mathcal{F}_{\text{Net}}, \mathcal{F}_{\text{Sig}}^1\}$
UC- π Then $\mathcal{F}_{\text{Net}}.\text{par} = \mathcal{F}_{\text{AC}}^\circ.\text{par} = \Delta$ and $\rho.\mathbf{N} = \mathcal{F}_{\text{AC}}^\circ.\mathbf{N}$ formed.
slot adversary the of $\mathcal{A}$ occupant every for :0 within $\{\mathcal{F}_{\text{AC}}^\circ\}$ emulates
 $\mathcal{Z} \in \mathbb{Z}_{\pi, \{\mathcal{F}_{\text{AC}}^\circ\}}$ every for gives. above $\mathcal{S}_{\mathcal{A}}$ simulator the

$$\text{Exec}(\pi, \mathcal{Z}, \mathcal{A}, \mathcal{C}) \equiv \text{Exec}(\{\mathcal{F}_{\text{AC}}^\circ\}, \mathcal{Z}, \mathcal{S}_{\mathcal{A}}, \mathcal{C}),$$

the not adversary. every over is quantifier The distributions. as equal
Theo- so responsive. is world either in call slot every alone: dummy
against exhibited is simulator the and this reach not does 4.29 rem
directly. $\mathcal{A}$ each

parts. three in invariant an carrying activations. on induction One اثبات.
routing. the so ,4.18 Lemma in as machine to machine pairs coupling The
state the is new is what lemma's: that are arguments refusal and claims
the of whole the is it on invariant the and cut. $\mathcal{F}_{\text{AC}}^\circ/\rho$ the across carried
hold hybrids internal the parts: three the states invariant The argument.
 $\mathcal{F}_{\text{AC}}^\circ.\text{list}$ and holds. ρ what hold shadows 's $\mathcal{S}_{\mathcal{A}}$ hold. ones real the what
Send it. preserve cases Three sends. honest undrained the exactly holds
replaying that and time. same the at send a stamp worlds two the shows
carries Fetch equal. them keeps copies internal the into steps own 'sp
an whether on but status current sender's the on not splits it weight: the
unmatched the that shows and record. the matches entry list undrained

and here, buys unforgeability what is which — sender corrupt a forces case activation one the covers traffic Corrupt-party trips. never $\text{Clean}_{\text{inj}}$ why what is it withholding that shows and withholds, set outward declared the paired tapes, equal on executions two the Couple needs, invariant the to each $\mathcal{G}_{\text{Clock}}$ and $\mathcal{C}$, $\mathcal{Z}$: 4.18 Lemma in as machine paired to machine the and $\mathcal{A}$ real the to $\mathcal{S}_{\mathcal{A}}$ of hybrids internal the and $\mathcal{A}$ hosted the itself. Everything $\mathcal{F}_{\text{AC}}^{\circ}$ to hold hybrids the what with together ρ and hybrids, real the : 4.17 Definition of shape the in activations, on induction one is below routing its and verbatim, 's 4.18 Lemma are paragraphs refusal and claims that calls the and bundle, a is $\mathcal{S}_{\mathcal{A}}$ — one but call every of holds paragraph those save relays, inward and outward ordinary the are boundary its cross set outward declared the which identities, own 'sp on places $\mathcal{A}$ hosted the carried state the only so — inside serves handler last the and withholds the of whole the is it on invariant the and new, is cut $\mathcal{F}_{\text{AC}}^{\circ}/\rho$ the across argument.

the (i) configurations: of pair corresponding every At invariant. The hybrids, real the as tables same the hold $\mathcal{S}_{\mathcal{A}}$ of $\mathcal{F}_{\text{Sig}}^1$ and $\mathcal{F}_{\text{Net}}, \mathcal{F}_{\text{PKI}}$ internal or- same the in calls same the by driven and alike initialised been having holds $\mathcal{F}_{\text{AC}}^{\circ}.\text{list}$ (iii) and 's: ρ equal seen and seq, gen shadow 's $\mathcal{S}_{\mathcal{A}}$ (ii) der: party's a through made ρ sends the of (P, Q, τ, msg) records the exactly at $\rho.\text{Send}$ such per entry one — honest was party that while Send own others, no and drained, since has fetch no that — read it timestamp the being corruption entry, the disturb not does sender the of corruption Later paired because hold (ii) and (i) placed, already record the and monotone an at driving: same the under tapes equal on code same the run copies their drive handlers 's $\mathcal{S}_{\mathcal{A}}$ and hybrids real the drives core 'sp party honest mediate sides both one corrupt a at and steps, same the through copies third the as runs, core callee's any before away calls subroutine same the preserve, cases three the what is (iii) checks, below case $\mathcal{F}_{\text{AC}}^{\circ}.\text{Send}$ side ideal the On $\mathbb{I}$ honest an at $\text{Send}(Q, \text{msg})$ calls $\mathcal{Z}.\text{Send}$, respon- $\mathcal{S}_{\mathcal{A}}$ notifies and list to $(\mathbb{I}, Q, \tau, \text{msg})$ appends clock, the reads $\mathcal{F}_{\text{Net}}.\text{Send}$ internal its and clock, the reads $\rho.\text{Send}$ side real the On sively, Sec- of argument the by coincide, reads clock two the again: it reads $\mathcal{G}_{\text{Clock}}.\text{Read}$ them, between runs execution either of core no — 18.2 tion notified, $\mathcal{S}_{\mathcal{A}}$ sides, both on τ stamped is send the so — call no placing that copies into runs, ρ steps Send/Sign/Register/Gen the exactly replays with answered read clock own 's $\mathcal{F}_{\text{Net}}$ internal the — 'sp to equal keeps (i) respon- the since stamp, same that is which carried, notification the τ the answers $\mathcal{A}$ hosted the and — it make to own its of call no place may der

one ok return Both tapes. equal over does $\mathcal{A}$ real the as query $\mathcal{A}^1$ each holds. (iii) send: one the matches entry list new deliv- worlds two the on turns Everything. Q at $\text{Fetch}()$ calls $\mathcal{Z}$ Fetch . $\text{Clean}_{\text{inj}}$ within it name to able being $\mathcal{S}_{\mathcal{A}}$ on and $\mathcal{M}$ set same the ering record delivered a internally: mirrors $\mathcal{S}_{\mathcal{A}}$ which filter. 'sp.Fetch Run is $\mathcal{x} = (P, Q, \tau, s, \text{msg})$. $(\mathcal{x}, \sigma)$ tuple diffused whose one is (P, Q, τ, msg) fresh is and $\text{VK}_{\mathcal{F}_{\text{PKI}}}[P]$ under verifying signature a carries. Q to addressed undrained an whether on but status current 'sP on not Split. $\text{seen}[Q]$ in case injected the and case honest the – record the matches entry list two. these exactly being honest. was P while P for made ρ send a is it (iii) by matches: one If id its $\mathcal{S}$ through and ripe. if R through – store from it delivers $\mathcal{F}_{\text{AC}}^\circ$ and match to early it releases $\mathcal{S}_{\mathcal{A}}$ if Q to addressed as $\text{Clean}_{\text{addr}}$ by admitted the now. by corrupt is sender the not or Whether release. early 's $\mathcal{F}_{\text{Net}}$ delivered is none so injected. never and list from delivered is record twice.

un- where is this and fetch. the at corrupt is P then matches: none If honest be would it fetch the at honest P Were place. its earns forgeability – key $\mathcal{F}_{\text{Sig}}$ own 'sP be would $\text{VK}_{\mathcal{F}_{\text{PKI}}}[P]$ so monotonicity. by throughout. under verifying signature a and – returned Gen what exactly registers ρ (Sec- test honest-key 'sVerify forgery: a never is key party's honest an $b = 1$ so key. a such under triple fresh any on $b \leftarrow 0$ sets (15.1 tion by which one. records Sign honest an only and recorded. triple the forces the $P \in \mathbf{C}$ So case. the against – entry list matching a leave would (iii) signature a or injection $\mathcal{F}_{\text{Net}}$ an produced. have to adversary's the is record $\text{Clean}_{\text{inj}}$ which $\mathcal{I}$ in (P, Q, τ, msg) places $\mathcal{S}_{\mathcal{A}}$ and party. corrupt a as made it delivers $\mathcal{F}_{\text{AC}}^\circ$ channel The fetcher. the Q and corrupt being P admits. record no buys: unforgeability what is This up. it hands ρ as unstored. the so name. honest an wear ever can it behind send genuine a lacking never $\text{Clean}_{\text{inj}}$ and senders. corrupt for asked ever only is channel injection trips.

a matches: draining and (ii). by sides both on matches Freshness re- seen marked and ideally list from drained is delivered record stored and really seen by filtered is tuple same the of re-delivery 's $\mathcal{F}_{\text{Net}}$ so ally. the So time. second a case neither into falling ideally. entry list no finds configurations: corresponding to step and $\mathcal{M}$ same the return esFetch two lockstep. in drained entries stored its preserved. is (iii) own 'sp on $\mathcal{I}$ corrupt a at Fetch or Send calls $\mathcal{A}$ traffic. Corrupt-party the case the and withholds. set outward the activation one the – identity

being ¶ 3 line at it admits interface full The reach. not do above two core 'sp side real the on so -caller. A an on fires hook neither and corrupt. a at name 'sp carries places then core that call subroutine Every runs. the before A to each hands interface same that of 4 line so party, corrupt 's $\mathcal{F}_{PKI}$ buffer. 's $\mathcal{F}_{Net}$ reaches activation the of nothing runs: core callee's 's $\mathcal{S}_A$ supply. to 'sA is timestamp very its and tables. 's $\mathcal{F}_{Sig}$ or directory so shadows. same the on A hosted the against core same that runs 30 line untouched. hybrids the leave and alike *seen* and *seq*, *gen* move sides both the because ideally – *list* touches side neither and (ii); and (i) preserving is (iii) so – touch to $\mathcal{F}_{AC}^\circ$ no is there because really, $\mathcal{S}_A$ leaves never call check. to nothing with preserved

why saying worth is it and needs. (iii) what is calls these Withholding reach would call the outward Relayed routing. the to it leaving than rather in- $\mathcal{F}_{AC}^\circ$ test ripeness the time: true the at record a files which, $\mathcal{F}_{AC}^\circ$.Send whatever Δ within Q to it deliver then would 12 line 's $\mathcal{F}_{AC}$.Fetch at herits A unless all at nothing delivers world real the where answered. slot the was it moment the (iii) break would entry The account. own its on injects com- to 's $\mathcal{F}_{AC}$ being R – it retract could 's $\mathcal{S}_A$ of answer later no and filed. sender corrupt a to charges 19.1 Section that ripening the is This pute. speak parties corrupt whose protocol a and, $\mathcal{F}_{AC}$.Send through speaking it. pay not does instead $\mathcal{F}_{Net}$ through

and $\mathcal{C}$, $\mathcal{G}_{Clock}$, $\mathcal{Z}$ among call a cut: the touches activation other No on identically claimed and routed is identities 'sp of none targets that A Cor- governs. invariant the state no reads and, 4.18 Lemma by sides both executions The output. one 's $\mathcal{Z}$ return configurations halting responding an but bound a not – 0 is advantage the and distributions. as equal are forgery no and fixed are tapes the once deterministic step every identity. □ refuse. would sanitizer the injection an force to through slipping ever

was weaker nothing why on clear being worth is it and .0 is bound The so ideal. is subroutine Every needed. was stronger nothing and available identity an is reduction the for: charge to gap computational no is there does note. Transfer, is. theorem's composition the as distributions. of and call. responsive no placing cores wants 5.11 Corollary here: run not $\mathcal{F}_{AC}^\circ$ of property a so responsively, all clock the and $\mathcal{F}_{Net}$, $\mathcal{F}_{Sig}^!$ reach 'sp as exactly, π over re-proved be must it – free for π to descend not does the What clock. notifying the over built anything for warned 17.2 Section bound a not realization: a for thing stronger the is instead gives theorem adversary. every against equality an but dummy the against

In- .itself) $\mathcal{F}_{AC}$ realizes ρ classes. non-corrupting (Against 20.2 نتیجه
 $\mathcal{F}_{AC}^b$ let and $I \neq \emptyset$ when 7 line at $bad \leftarrow \text{true}$ set to $\mathcal{F}_{AC}^o$ strument
 (S, I) taking 's. $\mathcal{F}_{AC}$ fetch its – I ignores that $\mathcal{F}_{AC}$ pair-shaped the be
 (Defini- bad until identical are $\mathcal{F}_{AC}^b$ and $\mathcal{F}_{AC}^o$ Now alone. S using and
 occu- for So difference. bad-guarded one the injection the. (11.1 tion
 $I = \emptyset$ forces Clean_{inj} nobody, corrupt that environments and pants
 indistinguish- are two the 11.2 Lemma by whence, $\Pr[\text{Bad}] = 0$ and
 non- the against 0 within $\{\mathcal{F}_{AC}\}$ hence $\{\mathcal{F}_{AC}^b\}$ UC-emulates π and able
 less: no 's. $\mathcal{F}_{AC}^o$ is guarantee the and party a Corrupt classes. corrupting
 -Send its transfers. (19.2 (Section FinAuth of half honest-sender the
 FinAuth full the while records, honest only reading argument tracing
 reason. same the for and $\mathcal{F}_{Net}$ of does it as exactly $\mathcal{F}_{AC}^o$ of fails

$\{\mathcal{F}_{PKI}, \mathcal{F}_{Net}, \mathcal{F}_{Sig}^I\}$ over protocol No .forced) is weakening (The 20.3 نکته
 target. true the but convenience a not is $\mathcal{F}_{AC}^o$ so shipped, as $\mathcal{F}_{AC}$ realizes
 responsive fetch's a inside diffuses, party corrupt A did. one Suppose
 exactly grants $\mathcal{F}_{Net}$ – accepts receiver its item well-formed a window,
 carrying – (18.1 (Section write to adversary's the being wire the this.
 ac- the side ideal the On choosing. adversary's the of τ^* timestamp a
 $\mathcal{F}_{AC}$. Send called have must simulator the so atomic. is fetch cepting
 the call that stamps $\mathcal{F}_{AC}$ but entry; matching a place to earlier strictly
 de- the off τ^* reads receiver the and τ^* not made, was it time true
 its has and time one at sends that environment An record. livered
 The certainty. with distinguishes another claim confederate corrupt
 since it, removes check receiver-side no and give to 's $\mathcal{F}_{Net}$ is freedom
 weak- only carries; payload the time whatever validates signature the
 gap. the closes – $\mathcal{F}_{AC}^o$ is which – it admit to object ideal the ening
 's $\mathcal{F}_{Net}$ forces that impossibility the of analogue diffusion the is This
 guarantee. delivery its outside sit to injections own

run- of point the and construction, single longest paper's the is This
 load-bearing. up shows framework the of layer every that is full in it ning
 responsive the hybrids; its and ρ between cut the supplies Composition
 can schedules the so atomic fetches both makes 9 Chapter of discipline
 senders; corrupt to slot corrupt a confines sanitization all: at coupled be
 its and descend. to needs $\mathcal{F}_{AC}^o$ of property a what be would tightness
 directly proved is realization the why is cores responsive under absence

what is 11 Chapter of lemma fundamental the and inherited: than rather realization exact the into object weaker the of realization exact the turns authenticated The play. in is corruption no wherever one stronger the of theorem. a now is ,19 Chapter in postulated channel.

Dive Deeper A : $\mathcal{F}_{\text{Sig}}$

of properties two proved and functionality signature a gave 15 Chapter a does What open. leaves that questions three the asks chapter This. properties those inherit to it on built protocol a for satisfy to have scheme own framework's this in drawn like. look protocol the does what and — they says 3.2 Section where held state its and randomness its with terms. char- to enough properties two the are hold: converse the Does be? must realizes them satisfying interfaces its with anything that so. $\mathcal{F}_{\text{Sig}}$ acterize it of realization a does or shape. its of object strongest the $\mathcal{F}_{\text{Sig}}$ is And it? not? does it something satisfy no. bound: the as security own scheme's a with yes are: answers The which stronger. strictly be can realization a — no and so: instructively and general. in functionalities ideal about knowing worth is

21.1 طرح‌های امضا و بازی‌هایشان

a over $\text{DS} = (\text{Gen}, \text{Sign}, \text{Ver})$ algorithms of triple a is scheme signature A space coin a and Σ space signature a, $\mathcal{M}$ space message a, $\mathcal{K}$ space key explicitly: coins their taking Sign and Gen with $\{0, 1\}^n$

$$\begin{aligned}(\text{vk}, \text{sk}) &\leftarrow \text{Gen}(r), & \sigma &\leftarrow \text{Sign}(\text{sk}, \text{msg}; r), \\ b &\leftarrow \text{Ver}(\text{vk}, \text{msg}, \sigma),\end{aligned}$$

tape implicit an than rather arguments are coins The deterministic. Ver ran- whose scheme a and $\mathcal{F}_{\text{Rand}}$ through them draws below π_{Sig} because function- randomness a to wired be not could it inside hidden is domness all. at ality

property-based ordinary the in games are notions security three Its framework. no — alone DS against $\mathcal{B}$ adversary an by played each sense. un- anyway. down write would cryptographer a notions the execution. no One. [10] them among chief attack chosen-message under forgeability

pa- the idiom same the in three all by shared oracle, signing one shell.
(Def- finalizations several across shell a share systems game own per's
calling by halts then likes, it as Sign queries ,vk receives $\mathcal{B}$: (5.1 intion
wins. it whether decides which below, procedures three the of one exactly

بازی برای طرح امضای DS: پوسنه

```

:1 (vk, sk) ← DS.Gen(r)                                     :Initialize()
:2 return vk                                                  / r یکنواخت

:3 σ ← DS.Sign(sk, msg; r)                                    :Sign(msg)
:4 tx ← tx · (msg, σ)                                         / r یکنواخت
:5 return σ

```

shared is above shell the notion: per one game. the of Finalizations

```

:6 σ ← Sign(msg)                                              (correctness) Cor(msg)
:7 w ← 1 if DS.Ver(vk, msg, σ) ≠ 1: else w ← 0                / a fresh call. on the game's own oracle
:8 return w

:9 w ← 1 if DS.Ver(vk, msg, σ) = 1 ∧ (msg, σ) ∉ tx: else w ← 0 (unforgeability) Uf(msg, σ)
:10 return w

:11 w ← 1 if msg ≠ msg' ∧ DS.Ver(vk, msg, σ) = DS.Ver(vk, msg', σ) = 1: else
    w ← 0                                                       resistance) (collision Col(msg, msg', σ)
:12 return w

```

and Uf, Cor that probabilities the for $\epsilon_{\text{col}}(\mathcal{B})$ and $\epsilon_{\text{uf}}(\mathcal{B})$, $\epsilon_{\text{cor}}(\mathcal{B})$ Write
have schemes most which, $\epsilon_{\text{cor}} = 0$ is correctness Perfect. 1 return Col
ex- 9 line :[1] strong is Unforgeability needs. follows what of none and
match- alone, message the than rather returned oracle the pair the cludes
standard not is resistance Collision .15.2 Section in reading 'sFinUF ing
place. its earns it where is 21.4 Section two: other the by implied not is and

π_{Sig} protocol The 21.2

every own: its of nothing holds and interfaces three 's $\mathcal{F}_{\text{Sig}}$ carries π_{Sig}
invocation an outlives that value every and $\mathcal{F}_{\text{Rand}}$ through drawn is coin
the and shape leakage-well-formed 's3.2 Section is which, $\mathcal{F}_{\text{Store}}$ in sits
Leak- decoration. not is That it. to written paper this in protocol first

Leak functionality's a but (4 (line corrupt being party the on gated is age would parties across shared $\mathcal{F}_{\text{Rand}}$ single a so entire, state its over hands corrupted, was party one any moment the coins party's every surrender The it, survive could unforgeability no – key signing every them with and $\mathcal{F}_{\text{Rand}}^{\text{P}}$ party, per each of instance one addresses therefore π_{Sig} protocol guard's the and allows already 5.5 Remark which, $\mathbf{P} = \{\text{P}\}$ with $\mathcal{F}_{\text{Store}}^{\text{P}}$ and each erases it And reaches, already number instance the to blindness put was Erase 's $\mathcal{F}_{\text{Rand}}$ what is which used, been has it moment the coin corrupt a so leak, cannot erased been has what :13.1 (Section for there own its even not – else nothing and key signing own its surrenders party coins, past

$\{\mathcal{F}_{\text{Rand}}^{\text{P}}, \mathcal{F}_{\text{Store}}^{\text{P}}\}_{\text{P} \in \mathbf{P}}$ روی π_{Sig} پروتکل $\text{par} := \text{DS} \quad \mathbf{U} := \{(\mathcal{F}_{\text{Rand}}, \text{serve}), (\mathcal{F}_{\text{Store}}, \text{serve})\} \quad \mathbf{N}, \mathbf{P}, \text{pid}$	
id' از $\text{id}.\text{Sign}(\text{msg})$ 11 $s_k \leftarrow \text{id}.\mathcal{F}_{\text{Store}}.\text{FullGet}(s_k)$ as id 12 if $s_k = \text{rej}$ then 13 return $\perp$ / نیست کلیدی هنوز 14 $(i, r) \leftarrow \text{id}.\mathcal{F}_{\text{Rand}}.\text{FullRnd}()$ as id 15 $\sigma \leftarrow \text{DS}.\text{Sign}(s_k, \text{msg}; r)$ 16 $\text{id}.\mathcal{F}_{\text{Rand}}.\text{FullErase}(i)$ as id / به محض می شود پاک استفاده 17 return σ id' از $\text{id}.\text{Leak}()$ 18 return $\perp$ / کند نشد که ندارد چیزی	id' از $\text{id}.\text{Gen}()$ 1 $\text{vk} \leftarrow \text{id}.\mathcal{F}_{\text{Store}}.\text{FullGet}(\text{vk})$ as id 2 if $\text{vk} \neq \text{rej}$ then 3 return vk / طرف هر به ازای کلید یک 4 $(i, r) \leftarrow \text{id}.\mathcal{F}_{\text{Rand}}.\text{FullRnd}()$ as id 5 $(\text{vk}, s_k) \leftarrow \text{DS}.\text{Gen}(r)$ 6 $\text{id}.\mathcal{F}_{\text{Rand}}.\text{FullErase}(i)$ as id / نمی توانند سکه ها / کنند نشد 7 $\text{id}.\mathcal{F}_{\text{Store}}.\text{FullPut}(\text{vk}, \text{vk})$ as id 8 $\text{id}.\mathcal{F}_{\text{Store}}.\text{FullPut}(s_k, s_k)$ as id 9 return vk id' از $\text{id}.\text{Verify}(\text{vk}, \text{msg}, \sigma)$ 10 return $\text{DS}.\text{Ver}(\text{vk}, \text{msg}, \sigma)$

the with answers Rnd 's $\mathcal{F}_{\text{Rand}}$ because pairs as back come draws The the name π_{Sig} lets what is which, (7 (line string the beside wrote it index index the to indifferent is and strings the reads FinUnp erases; then it entry them, beside travelling

state no has it because $\perp$ returns **Leak** own 's π_{Sig} remarks. Three there, it reads adversary party's corrupt a and 's $\mathcal{F}_{\text{Store}}^{\text{P}}$ is key the give: to where is leak the – asks leakage-well-formedness what precisely is which all, at call no places **Verify** operation The else, nowhere and is, state the nor draws neither and local is verification a so deterministic, being **Ver** be cannot **Verify** whose here protocol one the is π_{Sig} why is that stores: fetched is key the And movement, token's the by 's $\mathcal{F}_{\text{Sig}}$ from distinguished key "one makes what is which, π_{Sig} in cached than rather store the from rather (**FinKeep**, 14.2 (Section fixity per-slot 's $\mathcal{F}_{\text{Store}}$ about fact a party" per itself, about makes π_{Sig} promise a than

گزاره 21.1 $\gamma := \{\mathcal{G}_m\} \cup \pi_{\text{Sig}} \cup \text{Let}.$ properties) two the inherits π_{Sig} with 5.1 Definition of sense the in system game a be $\{\mathcal{F}_{\text{Rand}}^P, \mathcal{F}_{\text{Store}}^P\}_P$ q most at placing $\mathcal{Z} \in \mathbb{Z}_{\gamma, \gamma}$ every For 15.2 Section of shell the $\mathcal{G}_m$ there slot adversary the of $\mathcal{X}$ occupant every and relays the on calls once $\mathcal{X}$ and $\mathcal{Z}$ running each. DS against $\mathcal{B}_{\text{uf}}$ and $\mathcal{B}_{\text{cor}}$ adversaries are that such overhead, $O(q)$ with

$$\Pr[\text{Win}_{\text{FinCor}}] \leq q \cdot \varepsilon_{\text{cor}}(\mathcal{B}_{\text{cor}}),$$

$$\Pr[\text{Win}_{\text{FinUF}}] \leq q \cdot \varepsilon_{\text{uf}}(\mathcal{B}_{\text{uf}}) + q^2 2^{-n}.$$

اثبات. One preliminary paragraph and two. The reductions preliminary. The col- trace every makes tightness give: already subroutines the what lects 14.1 Proposition party, honest an at gave core own 's π_{Sig} answer an try good, for party per issued is key one so faithful and fixed store the makes unread- erased, being and, uniform coins the makes 13.1 Proposition and same the by game own scheme's the to reduces then property Each able. failure a as names it triple the identify fires, verdict the assume move: at the of which advance in guess reduction the have and outright, DS of from, comes q factor the where is which — one failing the is calls q most, gen every so, π_{Sig} reaches $\mathcal{G}_m$ but caller no tightness By bounds, both in honest an at gave core own 's π_{Sig} answer an is τ_r of entry ver and sign noth- records and intercepts wrapper game's the one corrupt a at — party the faithful: and fixed is store the 14.1 Proposition By (5.4 Remark ing 1 line on guard 'sGen and gets, later it what are puts party a s_k and v_k coins the 13.1 Proposition By good, for party per key one issues therefore are, Sign in counterpart its and 6 line at erased being and, uniform are alone, P serves $\mathcal{F}_{\text{Rand}}^P$ and them of empty is leak 's $\mathcal{F}_{\text{Rand}}^P$ nobody: by read them, reaches party another of corruption no so a, (gen, P, vk) a, Q, P honest fires: 20 line suppose correctness, For verification The (ver, Q, vk, msg, σ , 0) a and, $\sigma \in \Sigma$ with (sign, P, msg, σ) DS.Sign(s_k , msg; τ) is signature the and, 10 line by DS.Ver(vk, msg, σ) is the by uniform coins on, 5 line by vk very that with paired s_k the under The outright, DS of failure correctness a is triple the So above, paragraph at the of which advance in guesses execution, the runs $\mathcal{B}_{\text{cor}}$ adversary wins it message: its outputs and one, failing the is calls signing q most, q factor the is which right, is guess the and does game the whenever

.(gen, P, vk) ,Q ,P honest fires: 28 line suppose unforgeability, For is vk Either cases. Two .(sign, P, msg, σ) no and (ver, Q, vk, msg, σ , 1) re- (msg, σ) with $\text{Ver}(\text{vk}, \text{msg}, \sigma) = 1$ then and ,P to issued Gen key the planting by harvests $\mathcal{B}_{\text{uf}}$ which forgery. strong a – P at Sign no by turned parties generating q most at the of one guessed a at key challenge its party other every oracle. its from calls signing party's that answering and 's.P equal to happens that key party's honest another is vk or itself: runs it collided. coins -bitn uniform on draws $\text{Gen}(\tau)$ independent two then and era- the where Note . $q^2 2^{-n}$ most at is generations q most at over which of coins the $\mathcal{X}$ hand would party any corrupting it. without spent: is sure does it oracle an with answer not could $\mathcal{B}_{\text{uf}}$ and .Sign and Gen own 'sP $\square$ for. coins the have not

information- nor 0 neither is that paper the in bound first the is This $\mathcal{F}_{\text{Sig}}$ functionality The changed. has what saying worth is it and theoretic. has π_{Sig} :(15.1 (Proposition to written was it because properties its has a of price ordinary the are -factorsq two the and does. DS because them legitimate exchange the makes what is 5.11 Corollary reduction. guessing of realization any to descends $\mathcal{F}_{\text{Sig}}$ of property a – direction other the in re- the from approached statement same the is proposition this and – it by $\mathcal{F}_{\text{Sig}}$ from obtained not is π_{Sig} because directly proved side, alization's scheme. a from built but emulation

21.3 عکس قضیه، و آنچه ویژگی‌ها به آن نمی‌رسند

converse The properties. the to scheme a from runs 21.1 Proposition charac- properties two the that further: much and way other the run would satisfying and interfaces its carrying functionality any that so , $\mathcal{F}_{\text{Sig}}$ terize of ask we what are properties – conjecture natural a is It it. realizes them the as good as be to ought them satisfying object an so object. ideal an false. is it and – of them asked we one

a is There .($\mathcal{F}_{\text{Sig}}$ characterize not do properties two (The 21.2 گزاره $\mathcal{F}'$ functionality standard both having interfaces. and fields 's $\mathcal{F}_{\text{Sig}}$ with $\mathcal{F}'$ functionality standard occu- every and environment every against 0 within FinUF and FinCor any within $\{\mathcal{F}_{\text{Sig}}\}$ UC-emulate not does $\{\mathcal{F}'\}$ that such slot. the of pant $\epsilon < \frac{1}{2}$

اثبات. Let $\mathcal{F}'$ be $\mathcal{F}_{\text{Sig}}$ with one change: in Verify , when honest party's no when $P' \notin \mathbf{C}$ every for $vk \neq vk[P']$ when is, that vk queried the equals key query later a that so nowhere, recorded bit, uniform fresh a is answer the is $\mathcal{F}'$ triple other every On independently, answered is triple same the on

included, memoization exactly, $\mathcal{F}_{\text{Sig}}$ existential is 28 and 20 lines of Each. 0 within properties both has $\mathcal{F}'$ honest no is vk whose triple a so, $P \notin \mathbf{C}$ with entry (gen, P, vk) a over $\mathcal{F}_{\text{Sig}}$ as answers $\mathcal{F}'$ triples other all on and neither, witnesses key party's 15.1 Proposition by 0 are verdicts whose does.

generated vk^* with σ, msg, vk^* any pick $\mathcal{Z}$ Let emulate, not does It honest an at twice $\text{Verify}(vk^*, \text{msg}, \sigma)$ call and one, no corrupt nobody, by in- are answers the $\mathcal{F}'$ Against differ, answers two the if 1 returning party, the $\mathcal{F}_{\text{Sig}}$ Against $\frac{1}{2}$ probability with 1 returns it so bits, uniform dependent answers the so it, returns second the and $\text{ver}[vk^*, \text{msg}, \sigma]$ fixes call first does, simulator the whatever — 0 probability with 1 returns it and agree to able slot's the of value no and code own's $\mathcal{F}_{\text{Sig}}$ being memoization the $\frac{1}{2}$ is advantage The it, unfix $\square$

pins Emulation counterexample, the than useful more is diagnosis The things two pin properties two the behaviour, observable whole object's an un- forgeries that and verify, signatures produced honestly that — it about gap the and reach, their of out is else Everything not, do keys honest der honest, P with (gen, P, vk) over existential are verdicts both shape: a has and generated, party honest no keys about said is whatever nothing so in- verdict either all being answer one consistency, about said is nothing two but unforgeability of notion stronger a not is missing is What spect's, once obvious makes code's $\mathcal{F}_{\text{Sig}}$ ones the are they and properties, further party, per key one issues Gen its and memoizes, Verify its for: looked

15.2 Section of shell the of finalizations further Two

(consistency) id' from [id.FinCons\(\)](#)

```

:1 require  $\neg \text{done}$ ;
:2 require  $\text{id}.P = Z$ :  $\text{done} \leftarrow \text{true}$ 
:3  $w \leftarrow 1$  if  $\exists Q, Q', \text{vk}, \text{msg}, \sigma : (\text{ver}, Q, \text{vk}, \text{msg}, \sigma, 0) \in \text{tr}$ 
:4    $\wedge (\text{ver}, Q', \text{vk}, \text{msg}, \sigma, 1) \in \text{tr}$ : else  $w \leftarrow 0$ 
:5 return  $w$ 

party) per key (one  $\text{id}'$  from id.FinOne\(\))

:6 require  $\neg \text{done}$ ;
:7 require  $\text{id}.P = Z$ :  $\text{done} \leftarrow \text{true}$ 
:8  $C \leftarrow (C, Z).\text{FullStatus}()$  as  $\text{id}$ 
:9  $w \leftarrow 1$  if  $\exists P \notin C, \text{vk} \neq \text{vk}' : (\text{gen}, P, \text{vk}) \in \text{tr} \wedge (\text{gen}, P, \text{vk}') \in \text{tr}$ ;
:10 else  $w \leftarrow 0$ 
:11 return  $w$ 

```

FinCons — $\mathcal{F}_{\text{Sig}}$ by 0 at attained are both reading, search the take Both because **FinOne** thereafter, read and triple per once written is ver because first-write- same the by each thereafter, read and once written is $\text{vk}[P]$ are both and — directory the for makes 16.1 Proposition argument wins respec- FinKeep 's $\mathcal{F}_{\text{Store}}$ and determinism 's ver by π_{Sig} by 0 at attained party no names **FinCons** party: corrupt a of anything asks Neither tively. out answers corrupt kept already having wrapper own game's the all. at .tr of

stan- a be $\mathcal{F}'$ Let .properties) four the from converse. (The 21.3 قضيه $\mathcal{F}'$ returns Sign whose interfaces, and fields 's $\mathcal{F}_{\text{Sig}}$ with functionality dard let and party, calling the at generated been has key no when exactly $\perp$ and $\mathbb{Z}_{\gamma, \gamma}$ against 0 within **FinOne** and **FinCons**. **FinUF**, **FinCor** have $\mathcal{F}'$ with 15.2 Section of system game the γ for slot, the of occupant every every for :0 within $\{\mathcal{F}_{\text{Sig}}\}$ UC-emulates $\{\mathcal{F}'\}$ Then . $\mathcal{F}_{\text{Sig}}$ of place in $\mathcal{F}'$ with $\mathcal{S}_{\mathcal{A}}$ simulator a is there $\mathcal{A}$ occupant

$$\text{Exec}(\{\mathcal{F}'\}, \mathcal{Z}, \mathcal{A}, \mathcal{C}) \equiv \text{Exec}(\{\mathcal{F}_{\text{Sig}}\}, \mathcal{Z}, \mathcal{S}_{\mathcal{A}}, \mathcal{C})$$

$$\mathcal{Z} \in \mathbb{Z}_{\{\mathcal{F}'\}, \{\mathcal{F}_{\text{Sig}}\}} \text{ every for}$$

copy internal an with bundled $\mathcal{A}$ — one obvious the is simulator The اثبات. same the putting by queries slot three 's $\mathcal{F}_{\text{Sig}}$ of each answering, $\mathcal{F}'$ of sides two the that is argument the of whole the so — copy that to call an- slot's the is answer 's $\mathcal{F}_{\text{Sig}}$ Since value. returned a on disagree never override no showing to reduces that overrides, code own its unless swer

sanitizer, 'sGen memoization, 'sGen four: exactly are there and fires, ever
 ex- and turn in taken is Each test. honest-key 'sVerify and sanitizer, 'sSign
 cases sanitizer the .0 at properties hypothesised four the of one by cluded
 where is That make. would sanitizer the repair which by further splitting
 them. exactly needs theorem the why is it and spent, are properties four all
 the on it runs and $\mathcal{F}'$ of copy internal an with $\mathcal{A}$ bundles $\mathcal{S}_{\mathcal{A}}$ simulator The
 three the exactly for slot the asks $\mathcal{F}_{\text{Sig}}$ functionality The reports. $\mathcal{F}_{\text{Sig}}$ calls
 at bit the .Sign at signature the .Gen at key the — pin not does it values
 $\mathcal{F}'$ internal its to call same the putting by each answers $\mathcal{S}_{\mathcal{A}}$ and — Verify
 hosted the letting back, comes what forwarding and identity same the at
 Leakage slot. a to put would code own 's $\mathcal{F}'$ whatever answer, and see, $\mathcal{A}$
 note likewise: copy internal the from serves it instructions corruption and
 environment the all, at 's $\mathcal{F}_{\text{Sig}}$ resemble not need **Leak** 's $\mathcal{F}'$ why is this that
 occupies. $\mathcal{S}_{\mathcal{A}}$ which slot, the through only it reaching

Every- .11.2 Lemma in as tapes equal on executions two the Couple
 $\mathcal{F}'$ internal the what returns $\mathcal{F}_{\text{Sig}}$ call, every at claim: one to reduces thing
 sees. environment the value the on agree sides two the that so returned,
 it overrides. code own its unless answer slot's the is answer 's $\mathcal{F}_{\text{Sig}}$ Since
 four, exactly are there and fires, ever override no that enough is

$\mathcal{F}_{\text{Sig}}$ set already is $\text{vk}[\text{id.P}]$ If sanitizer, 'sGen and memoization 'sGen
 that at key same the returns $\mathcal{F}'$ internal the says 0 at FinOne it: returns
 offered the replaces **San**[Clean_{vk}] Otherwise agree, two the so too, party
 al- signature a has or issued, already one duplicates or key, a not is it if vk
 non- a with Gen answered $\mathcal{F}'$ then key: a Not it, under valid recorded ready
 (ver, ·, vk, ·, ·, 0) with $\sigma \in \Sigma$ carry would party that at Sign later a and key,
 environ- the once 0 at FinCor by excluded — environment the to available
 then parties honest two Duplicate: may, it which verifies, and signs ment
 ex- .FinUF wins other the at verifying and one at signing and key, a share
 issued, is vk before stands $\text{ver}[\text{vk}, \text{msg}, \sigma] = 1$ some Pre-signed: cluded,
 key a under signed party honest an so .Sign at only 1 a records $\mathcal{F}_{\text{Sig}}$ and
 the which and, $\perp$ returning refuses, Sign 's $\mathcal{F}_{\text{Sig}}$ which — generated yet not
 idle, is sanitizer the So too, refuses Sign 's $\mathcal{F}'$ on hypothesis

sig- a not is it if σ replaces **San**[Clean _{σ}] sanitizer The sanitizer, 'sSign
 $\sigma \in \Sigma$ 'sFinCor signature: a Not already, $\text{ver}[\text{vk}, \text{msg}, \sigma] = 0$ if or nature
 envi- the excluded, be case this lets that reading the exactly is conjunct
 :0 Already .0 a forbidding FinCor and value returned the verifying ronment
 $\mathcal{F}'$ internal the — now 1 verifies and earlier 0 verified triple same the then
 excluded, .FinCons is which — accept must it signature a produced having
 idle, is too it So .0 at

re- first The suppression. honest-key its and memoization 'sVerify the says 0 at FinCons and exists. one where verdict recorded the turns is this agree: they so way, same the triple same the answers $\mathcal{F}'$ internal is it and exploited. 21.2 Proposition of counterexample the override the triple fresh a on $b \leftarrow 0$ sets second The closes. FinCons what exactly answers $\mathcal{F}'$ internal the says 0 at FinUF and key. party's honest an under agree. both So condition. win its being 1 a – too there 0 's $\mathcal{F}'$ internal the is sees environment the value every fires. override No in as exactly halt. same the to step in proceed runs coupled the and own. advantage the and equal are distributions The case. no-flag 's11.2 Lemma $\square$.0 is

prop- Two point. the is hold to took it what and holds. converse the So not are added two the and are. four characterization: a not were erties table a – commits code 's $\mathcal{F}_{\text{Sig}}$ places two the but signing about facts deep properties functionality's A over. twice thereafter. read and once written read- and commitment. such every cover they when exactly it characterize missing the find to way better a is commitments its for specification a ing true. be to like would one else what asking than property

lacks $\mathcal{F}_{\text{Sig}}$ that π_{Sig} of property A 21.4

is It shape. its of object strongest the is $\mathcal{F}_{\text{Sig}}$ whether is question last The costs. sanitization what is it oversight: an not is gap the and not.

resistance collision finalization: further A

```

resistance) (collision id' from id.FinColl())
:1 require  $\neg \text{done}$ :
:2 require  $\text{id.P} = \text{Z}$ :  $\text{done} \leftarrow \text{true}$ 
:3  $\mathbf{C} \leftarrow (\text{C}, \text{Z}).\text{FullStatus}()$  as  $\text{id}$ 
:4  $w \leftarrow 1$  if  $\exists P \notin \mathbf{C}, Q, Q' \notin \mathbf{C}, \text{vk}, \text{msg} \neq \text{msg}', \sigma$  :
:5    $(\text{gen}, P, \text{vk}) \in \text{tr} \wedge (\text{ver}, Q, \text{vk}, \text{msg}, \sigma, 1) \in \text{tr}$ 
:6    $\wedge (\text{ver}, Q', \text{vk}, \text{msg}', \sigma, 1) \in \text{tr}$ : else  $w \leftarrow 0$ 
:7 return  $w$ 
```

than rather hard be to meant being winning reading, search the takes It guess. a than better no

$q \cdot \varepsilon_{\text{col}}(\mathcal{B}_{\text{col}})$ within FinColl has π_{Sig} .not) does $\mathcal{F}_{\text{Sig}}$ it. has π_{Sig}) 21.4 گزاره
func- The .21.1 Proposition in as built DS against $\mathcal{B}_{\text{col}}$ adversary an for

an and $\mathcal{Z}^*$ are there $:\varepsilon < 1$ any within it have not does $\mathcal{F}_{\text{Sig}}$ tionality
 $\text{Pr}[\text{Win}_{\text{FinColl}}] = 1$ with $\mathcal{X}^*$ occupant

اثبات. For π_{Sig} honest an under verifying σ one exhibits trace winning a $:\pi_{\text{Sig}}$ For
 10 line by DS.Ver is verification and messages. distinct two on vk party's
 The coins. uniform on drawn key a under DS for collision a is pair the so
 q most at the of one guessed a at key challenge its plants $\mathcal{B}_{\text{col}}$ adversary
 runs and oracle its from signing party's that answers parties. generating
 signature. the and messages two the outputs and itself. rest the
 Verify every key. fresh a with query Gen every answer $\mathcal{X}^*$ let $:\mathcal{F}_{\text{Sig}}$ For
 cor- $\mathcal{Z}^*$ Let $.\sigma^* \in \Sigma$ fixed one with queries Sign both and $.\emptyset$ with query
 $\text{msg} \neq \text{msg}'$ for $\text{Sign}(\text{msg}')$ and $\text{Sign}(\text{msg})$ then $\text{Gen}()$ call nobody. rupt
 sig- first The finalize. then triples. both on Verify then party. honest an at
 that and signature a be σ^* that only asking Clean_σ accepted. is nature
 $.\text{Ver}[\text{vk}, \text{msg}, \sigma^*] \leftarrow 1$ unset: being it holds. which $.\text{Ver}[\text{vk}, \text{msg}, \sigma^*] \neq 0$
 message. other the at reason same the for accepted is second The
 Both $.\text{Ver}[\text{vk}, \text{msg}', \sigma^*] \leftarrow 1$ and well. as unset being $\text{Ver}[\text{vk}, \text{msg}', \sigma^*]$
 1 gives 4 line at verdict the and $.\text{1}$ recorded the return then verifications
 $\square$ certainty. with

patched. be can it rather. or — patched be to defect a not is attack The
 other no for valid recorded be σ that conjunct the Clean_σ to adding by
 Sanitiza- cost. would that what in is interest the and $.\text{vk}$ under message
 slot the on depend not do guarantees functionality's a that so exists tion
 a is predicate Clean a of conjunct each $:(12$ (Chapter helpfully answering
 longer no may simulator the value a also is each and bought. guarantee
 unforge- and correctness buys written as $\mathcal{F}_{\text{Sig}}$ functionality The choose.
 read- defensible a is which resistance. collision buy to declines and ability
 made consequence. the and — for is functionality signature a what of ing
 stronger strictly be may realization a that is $.\text{21.4}$ Proposition by precise
 realizes. it object ideal the than

An intuition. usual the inverts it because with. sitting worth is That
 re- with behaviour. possible best the as read often is functionality ideal
 $\mathcal{F}_{\text{Sig}}$ property a satisfies π_{Sig} here below: from it approximating alizations
 Corol- — downward it transfers emulation of amount no and fails. provably
 never realization. the to object ideal the from properties carries 5.11 lary
 may protocol a what is fixes functionality ideal an What way. other the
 realization a anything achieve: to happens it what not for. on relied be

goes that argument every to invisible is and real, is that beyond achieves
object. ideal the through

Issues Pending

it. raises body the places the from gathered open, leaves paper this What
and Presentation cover. not does and does review its what with together
listed. not are typesetting

hypothe- 4.29 Theorem **calls. responsive at stops completeness Dummy**
relay dummy's The call. responsive no place cores whose systems sises
adversary real a where $\perp$ answers and call such first the at refused is
com- Recovering fails. regrouping the so substance. with answer would
do we and side. environment's the on notion matching a needs pleteness
one. give not

in- results property-transfer three All **theory. responsive no have Games**
reaches $\mathcal{F}$ whose system game A .4.29 Theorem from condition that herit
dummy-only its have must and three all outside sits $\mathcal{A}^1$ through slot the
functionalities own paper's this of Four directly. established hypothesis
by transfers their argue and reason this exactly for hypothesis the decline
hand.

simulator the fixes 4.11 Definition **converse. no has order quantifier The**
sim- the which in reading weaker the implies and environment, the before
the Whether prove. not do we converse The both. on depend may ulator
open. is separate notions two

compar- blocked a keeps Tightness **characterized. not are pairs Blocked**
.I of member no constrains it but members, private the for faithful ison
a by reachable is name used bare a admitting member interface an and
the where $\mathbf{Z} \cup \mathbf{A}$ only admitting or those. Pinning instance. fake hosted
general no give We it. close would directly, them reaches environment
characterization. no and condition sufficient

clo- name whole the withholds Admissibility **cost. would spawning What**
simulator a Letting needs. strictly comparison a than more is which sure.

only leave would occupy not does φ identities at functionalities spawn
 relax- that What pairing. usual the for empty – withhold to $\text{IDs}(\varphi) \setminus \text{IDs}(\pi)$
 out. worked not is composition in costs ation

assem- is entry trace Every **consistency. temporal not is atomicity Trace**
 an Whether checked. was paper the in append every and atomically, bled
 dif- a is time in agree it alongside fetched set the and timestamp entry's
 The atomicity. by than rather responsiveness by secured property, ferent
 one in them separates nowhere paper the and conflate to easy are two
 place.

in examined were units audit twenty-one of Six **covers. review the What**
 the than rather pass single a in and criticality, by leaves-first chosen full.
 clustered found defect Every for. asks protocol the ones independent two
 may it whether and outward, places bundle a what – mechanism one on
 were all and – call responsive a answering while members hosted its use
 the for: look not did pass one what in therefore is exposure The fixed.
 should pass further a where is channel authenticated the of realization
 defects. contained actually that unit the being go.

and signature joint of security the On Rabin. T. and Dodis. Y. An. H. J. [1]
volume ,2002 EUROCRYPT – Cryptology in Advances In encryption.
<https://eprint.iacr.org/2002/046> .2002 Springer. ,107–83 pages LNCS. of 2332

of Proceedings In provable-security. Practice-oriented Bellare. M. [2]
,(97' (ISW Security Information on Workshop International First the
<https://cseweb.ucsd.edu/~mihir/papers/isw-pops.pdf> .1998 Springer. ,231–221 pages LNCS. of 1396 volume

a as Bitcoin Zikas. V. and Tschudi. D. Maurer. U. Badertscher. C. [3]
Cryp- in Advances In treatment. composable A ledger: transaction
.356–324 pages LNCS. of 10401 volume ,2017 CRYPTO – tology
<https://eprint.iacr.org/2017/149> .2017 Springer.

and effects algebraic with Programming Pretnar. M. and Bauer A. [4]
Programming, in Methods Algebraic and Logical of Journal handlers.
<https://arxiv.org/abs/1203.1539> .2015 ,123–108:(1)84

and encryption triple of security The Rogaway. P. and Bellare M. [5]
in Advances In proofs. game-playing code-based for framework a
–409 pages LNCS. of 4004 volume ,2006 EUROCRYPT – Cryptology
<https://eprint.iacr.org/2004/331> .2006 Springer. ,426

au- and certification, signature. composable Universally Canetti. R. [6]
Workshop Foundations Security Computer IEEE 17th In thentication.
<https://eprint.iacr.org/2004/233> .2004 IEEE. ,233–219 pages .(2004 (CSFW
[org/2003/239](https://eprint.iacr.org/2003/239)

ACM, the of Journal security. composable Universally Canetti. R. [7]
<https://eprint.iacr.org/2000/067> .2020 ,28:94–28:1:(5)67

- compos- Universally Walfish, S. and Pass, R. Dodis, Y. Canetti, R. [8]
TCC – Cryptography of Theory In setup. global with security able
https://eprint.iacr.org/2006/432 .2007 Springer, .85–61 pages LNCS, of 4392 volume ,2007
//eprint.iacr.org/2006/432
- Wadler, P. and Kohlweiss, M. Knispel, A. Karvonen, M. Farshim, P. [9]
preprint arXiv proofs. diagrammatic Rigorous categorically: UC,
https://arxiv.org/abs/2608.04521 .2026 arXiv:2608.04521,
- scheme signature digital A Rivest, R. and Micali, S. Goldwasser, S. [10]
Jour- SIAM attacks. chosen-message adaptive against secure
https://people.csail.mit.edu/silvio/Selected%20Scientific%20Papers/
Digital%20Signatures/A_Digital_Signature_Scheme_
Secure_Against_Adaptive_Chosen-Message_Attack.pdf
- com- Universally Zikas, V. and Tackmann, B. Maurer, U. Katz, J. [11]
– Cryptography of Theory In computation. synchronous posable
.2013 Springer, .498–477 pages LNCS, of 7785 volume ,2013 TCC
https://eprint.iacr.org/2011/310
- 18th In effects. algebraic of Handlers Pretnar, M. and Plotkin G. [12]
5502 volume ,(2009 (ESOP Programming on Symposium European
https://homepages.inf.ed.ac.uk/gdp/publications/Effect_Handlers.pdf
.2009 Springer, .94–80 pages LNCS, of
- in complexity taming for tool A games: of Sequences Shoup, V. [13]
.2004/332 Report Archive, ePrint Cryptology IACR proofs. security
https://eprint.iacr.org/2004/332 .2004